Getting started with mdatools for R

Sergey Kucheryavskiy

June 26, 2026

Contents

Introduction **5**

What is new **7**

Overview **9**
What mdatools can do? . . . 10
How to install and update . . . 10

Datasets and plots **11**
Attributes and factors . . . 11
Excluding rows and columns . . . 14
Simple plots . . . 16
Plots for groups of objects . . . 33
Working with images . . . 40

Preprocessing **45**
Centering and scaling . . . 45
Correction of spectral baseline . . . 47
Normalization . . . 52
Removing cosmic spikes . . . 53
Smoothing and derivatives . . . 55
Variable selection as preprocessing method . . . 57
Combining methods together . . . 59
Interoperability with web-applications . . . 64
Additional methods and features . . . 67
Deprecated methods . . . 71

Principal component analysis **73**
Models and results . . . 74
Plotting methods . . . 76
Distances and critical limits . . . 87
Model validation . . . 91
Preprocessing as part of the model . . . 97
Model complexity . . . 100
Randomized PCA algorithms . . . 103
Interoperability with web-applications . . . 106

DD-SIMCA classification **111**
Calibration and validation . . . 111
Making predictions . . . 122
Using preprocessing . . . 132
Interoperability with web-application . . . 133

DD-SIMCA for 3-way data **139**
Prepare datasets . . . 139
PARAFAC DD-SIMCA . . . 141
Tucker DD-SIMCA . . . 150
Imposing non-negativity . . . 152

Partial least squares regression **155**
Models and results . . . 155
Validation . . . 157
Regression coefficients . . . 167
Plotting methods . . . 169
Variable selection . . . 174
Distances and outlier detection . . . 180
Randomization test . . . 188
Preprocessing as part of the model . . . 190
Interoperability with web-applications . . . 193

PLS Discriminant Analysis **197**
Calibration of PLS-DA model . . . 197
Classification plots . . . 200
Performance plots . . . 202
Predictions for new data . . . 204

Interval PLS **209**

Multivariate Curve Resolution **219**
Purity based . . . 220
Alternating least squares . . . 230

SIMCA classification **247**
Calibration and validation . . . 247
Multiclass classification . . . 255

Introduction

This is a user guide for *mdatools* — an R package for preprocessing, exploring and analysis of multivariate data. The package provides methods common in Chemometrics. The general idea of the package is to collect the popular chemometric methods and give a similar "user interface" for applying the methods to different datasets. So, if a user knows how to make a model and visualize results for one method, they can easily do this for the other methods as well. Usually I update the tutorial when a new version of the package is released, you can track main changes here.

The current version of the tutorial is compatible with version **0.16.0** of the package. You can also download a PDF version of the tutorial.

All methods implemented in the package were tested using well-known datasets. However, there could still be some bugs, in this case please report them to svkucheryavski@gmail.com or use Issues tool at GitHub. You are also very welcome to share your comments and suggestions about the package functionality.

If you want to cite the package, please use the following:

Sergey Kucheryavskiy, *mdatools - R package for chemometrics*, Chemometrics and Intelligent Laboratory Systems, Volume 198, 2020 (DOI: 10.1016/j.chemolab.2020.103937).

What is new

26/06/2026

New release, **0.16.0** is out and the tutorial text has been revised accordingly. This release contains minor improvements, bug fixes, and two new methods.

- `ddsimca.parafac()` PARAFAC based version of DD-SIMCA for 3-way data.
- `ddsimca.tucker()` Tucker based version of DD-SIMCA for 3-way data.

Both methods are described in a new chapter DD-SIMCA for 3-way data.

5/03/2026

New release, **0.15.0** is out and the tutorial text has been revised accordingly.

This release contains numerous improvements, bug fixes, and new methods. The main highlights are a standalone DD-SIMCA implementation, overhauled preprocessing pipelines (with some breaking changes), and JSON/CSV interoperability with the mda.tools web-applications.

New dedicated methods for DD-SIMCA

In the previous version, DD-SIMCA was implemented via more versatile method `simca`, which also lets you use other SIMCA implementations. While versatility is generally good, it limited the DD-SIMCA possibilities and it was decided to implement it separately.

Method `ddsimca` can now be used for training, testing, and applying Data Driven SIMCA models. It matches functionality of the corresponding web-application, including all plots and figures of merit (for example estimation of beta, selectivity, etc.). It also lets you change decision boundary parameters without rebuilding the main model.

The original method `simca` is still available (and always will) for compatibility.

Improvements to preprocessing methods

There are several new methods for preprocessing, including:

- `prep.spikes()` for cosmic spikes removal from Raman spectra.
- `prep.center()` for centering of data columns.
- `prep.scale()` for scaling of data columns.
- `prep.emsc()` for extended multiplicative scatter correction.

The following methods are considered *deprecated*, you can still use them (they will be kept for compatibility), but for new code it is recommended to use the alternatives:

- `prep.autoscale()` — use `prep.center()` and `prep.scale()` instead.
- `prep.snv()` — use `prep.norm()` with parameter `type = "snv"` as an alternative.
- `prep.msc()` — use `prep.emsc()` with parameter `degree = 0`.
- `employ.prep()` — use `prep.apply()` instead.

In addition to that, the ability to combine the preprocessing methods together into a preprocessing chain (we will call it *preprocessing model*) has been improved. However, these improvements **cause breaking changes**, so if you used this feature before, check the text below and the updated user guides very carefully.

First of all, the syntax for creating preprocessing items for combining them into the chain has changed — parameters are now passed as named arguments instead of a list:

- Old: `prep("savgol", list(width = 7, porder = 2, dorder = 2))`
- New: `prep("savgol", width = 7, porder = 2, dorder = 2)`

Also, the option to add user defined preprocessing methods into a preprocessing model has been removed as it caused issues and unstable behavior in some cases. From this version, only selected methods can be combined into a preprocessing model. You can see a full list of the currently supported methods by running `prep.list()`. This list will be extended eventually. And you can still use user defined methods and methods which are not in the list separately.

Second, a preprocessing model can now be integrated directly into `pca`, `pls`, `simca`, `ddsimca`, and `plsda` via the `prep` parameter — the model will train the preprocessing pipeline and apply it automatically during calibration and prediction.

Finally, preprocessing methods can also be *trained* independently using `prep.fit()`, which pre-computes parameters that depend on the training set (e.g. values for centering, scaling, or the reference spectrum for EMSC). Applying the trained preprocessing model to new data is done with `prep.apply()`.

Overview

The first version of this package was created in 2012 for an introductory PhD course on Chemometrics given at the Department of Chemistry and Bioscience, Aalborg University. Quickly I found out that using R for this course (with all the advantages it gives) requires a lot of routine work from students, since most of them were also beginners in R. Of course, it is very good for understanding when students get to know e.g. how to calculate explained variance or residuals in PCA manually or make corresponding plots and so on, but for the introductory course these things (as well as numerous typos and small mistakes in the code) take too much time, which can be spent on explaining methods and proper interpretation of results.

This is actually also true for everyday use of these methods, most of the routines can be written once and simply reused with various options. So it was decided to write a package where most widely used chemometric methods for multivariate data analysis are implemented and which also gives a quick and easy-to-use access to results, produced by these methods. First of all via numerous plots.

Here is how it works. Say, we need to make a PCA model for data matrix `x` with autoscaling. Then make an overview of most important plots and investigate scores and loadings for first three components. The `mdatools` solution will be:

```
# make a model for autoscaled data with maximum possible number of components
m = pca(x, scale = TRUE)

# show explained variance plot
plotVariance(m)

# select optimal number of components (say, 4) for correct calculation of residuals
m = selectCompNum(m, 4)

# show plots for model overview
plot(m)

# show scores plot for PC1 and PC3
plotScores(m, c(1, 3))

# show loadings plot for the same components
plotLoadings(m, c(1, 3))

# show the loadings as a set of bar plots
plotLoadings(m, c(1, 3), type = "h")
```

Fairly simple, isn't it? The other "routine", which has been taken into account, is validation — any model can be cross-validated or validated with a test set. The model object will contain the validation results, which will also appear on all model plots, etc. See the next chapters for details.

What mdatools can do?

The package includes classes and functions for analysis, preprocessing, and plotting of data and results. So far the following methods for analysis are implemented:

- Principal Component Analysis (PCA)
- Soft Independent Modelling of Class Analogy (SIMCA), including data driven approach (DD-SIMCA).
- DD-SIMCA classification for 3-way data based on PARAFAC and Tucker decompositions.
- Partial Least Squares regression (PLS) with calculation of VIP scores and Selectivity ratio.
- Partial Least Squares Discriminant Analysis (PLS-DA).
- Randomization test for PLS regression models.
- Interval PLS for variable selection.
- Multivariate curve resolution using the purity approach.
- Multivariate curve resolution using the constrained alternating least squares.

Preprocessing methods include:

- Centering and scaling
- Savitzky-Golay filter for smoothing and derivatives
- Extended Multiplicative Scatter Correction
- Normalization of spectra to unit area, length, sum, internal standard peak, and standard normal variate (SNV).
- Baseline correction with asymmetric least squares
- Kubelka-Munk transformation
- Element wise transformations (`log`, `sqrt`, power, etc.)
- Correction of cosmic spikes
- Combination of preprocessing methods into a preprocessing chain/model

Besides that, some extensions for the basic R plotting functionality have also been implemented and allow you to do the following:

- Color grouping of objects with automatic color legend bar.
- Plot for several groups of objects with automatically calculated axes limits and plot legend.
- Three built-in color schemes — one is based on Colorbrewer and the other two are jet and grayscale.
- Easy-to-use option to apply any user defined color scheme.
- Option to show horizontal and vertical lines on the plot with automatically adjusted axes limits.
- Option to extend plotting functionality by using some attributes for datasets.

See `?mdatools` and next chapters for more details.

How to install and update

The package is available on CRAN, to install it just use:

```
install.packages("mdatools")
```

This is the recommended way to install the package. If you have installed it already and just want to update to the newest version, use:

```
update.packages("mdatools")
```

If you want to install it directly from GitHub, the easiest way is to install the `devtools` package first and then run the following command in R:

```
devtools::install_github("svkucheryavski/mdatools")
```

Datasets and plots

The package uses standard representation of the data in R: data frames, matrices and vectors. However, there are several additional methods and attributes, which make the use of datasets a bit more efficient. There is also support for images. But if you are used to simple datasets and standard procedures and do not want any complications, you can simply skip this chapter.

The package also uses its own set of plotting tools, which is a sort of add-on for the R basic plotting system, extending its possibilities. From this point of view, learning how these tools work will simplify understanding of model plots a lot. The main improvements compared to the basic plotting system are:

1. Much easier way to make plots with groups of objects (points, lines, bars, etc.)
2. Much easier way of adding a legend to the group plots.
3. Much easier way of adding labels to data points, bars, etc.
4. Automatic axis limits when a plot contains several groups of objects.
5. Possibility to color points and lines according to values of a specific numerical variable or a factor.
6. Several built-in color palettes and an easy way to use a user-specific set of colors.
7. Much more!

This chapter explains most of the details.

Attributes and factors

This section tells how to extend the functionality of the package by using attributes assigned to datasets and how methods implemented in the package deal with factors.

Attributes for plots

The plot attributes will be explained very briefly here, you can find much more details in the next two sections. The idea is to provide some names and values to the data, which can be used later e.g. for making labels and titles on the plots. When a dataset is used to create a model (e.g. PCA) all results representing objects (e.g. scores, distances, etc.) will inherit the row specific attributes and all results related to variables (e.g. loadings) will inherit column specific attributes.

The main attributes for plots are as follows:

Attribute	Meaning
`name`	name of a dataset (used for plot parameter `main`).
`xaxis.name`	if variables/data columns represent the same property, e.g. wavelength, the property name can be defined by this attribute (used for plot axes labels).
`yaxis.name`	if objects/data rows represent the same property, e.g. reaction time, the property name can be defined by this attribute (used for plot axes labels).
`xaxis.values`	a vector of values, which correspond to the columns (e.g. for spectroscopic data it can be a vector with wavelength or wavenumbers).

Attribute	Meaning
`yaxis.values`	a vector of values, which correspond to the rows (e.g. for kinetic data it can be a vector with time or temperature values).

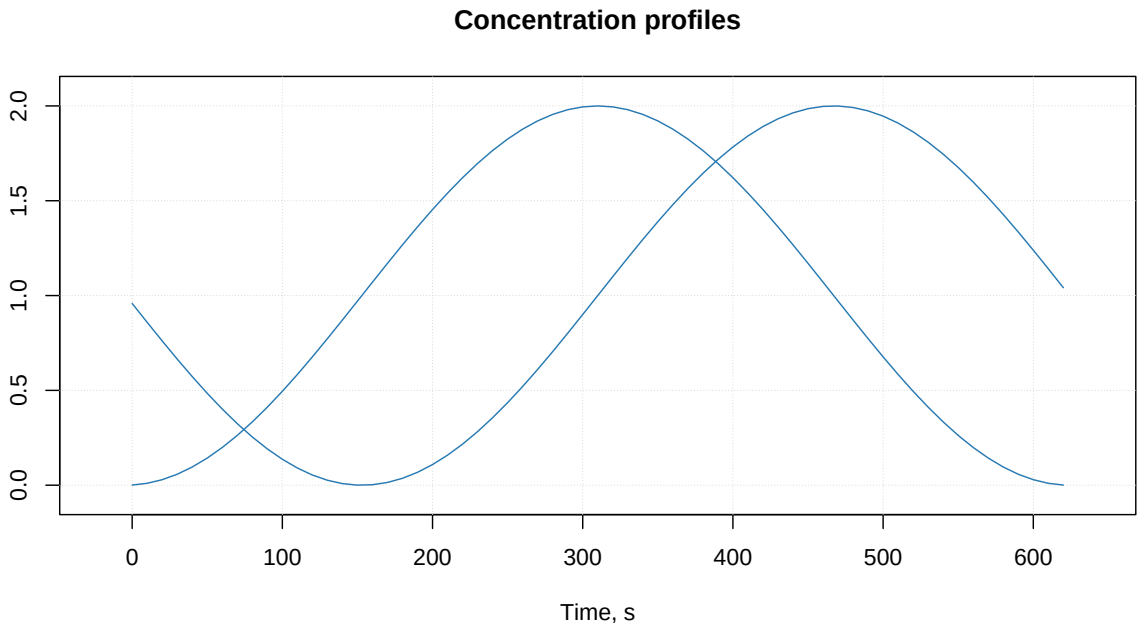
Here is a very simple example.

```
# generate data as matrix with two rows and 63 columns
t = -31:31
d = rbind(sin(t/10), cos(t/10)) + 1

# add name of the dataset
attr(d, "name") = "Concentration profiles"

# add name and values for a property representing the columns (time of reaction)
# we make values for x-axis to span from 0 to 620 seconds
attr(d, "xaxis.name") = "Time, s"
attr(d, "xaxis.values") = (t + 31) * 10

# make a line plot for the data
mdaplot(d, type = "l")
```



As you can notice, both the label and the ticks for x-axis correspond to the attributes we defined. As well as the main title of the plot. As we mentioned above these attributes will be inherited and e.g. PCA loadings will have the same x-axis if you decide to make a line plot for the loadings.

See more details in the section about plots.

Special methods for data transformations

Since data objects in R lose all user-specified attributes when e.g. we transpose them or take a subset, it was decided to write several methods, which would process attributes correctly. They also adjust indices of excluded rows and columns when a user takes a subset or merges two data objects together. When a data

matrix is transposed the corresponding method will swap the x- and y-attributes. All methods with a brief description are listed in the table below.

Method	Description
`mda.show(data)`	Show data object without excluded elements
`mda.t(data)`	Transpose data object
`mda.cbind(data1, data2, ...)`	Merge several datasets by columns
`mda.rbind(data1, data2, ...)`	Merge several datasets by rows
`mda.subset(data1, subset, select)`	Take a subset of data object (`subset` is numeric indices, names or logical values for rows, `select` — the same for columns)
`attrs = mda.getattr(data)`	Return all user specific attributes from an object
`data = mda.setattr(data, attrs)`	Assign user specific attributes to an object

To avoid any problems with arguments, use these functions instead of the traditional ones, e.g. `mda.subset()` instead of `subset()`, when available.

Data frames with factors

All methods, implemented in the package, work with matrices, therefore, if a user provides data values as a data frame, it is converted to matrix. It is also possible to provide data frames with one or several factor columns. In this case all factors will be converted to dummy variables with values 0 and 1. You can also do it manually, by using function `mda.df2mat()` as this is shown in an example below.

Let us first create a simple dataset with a factor column.

```
h = c(180, 175, 165, 190, 188)
c = as.factor(c("Gray", "Green", "Gray", "Green", "Blue"))
d = data.frame(Height = h, Eye.color = c)
show(d)
```

```
##   Height Eye.color
## 1    180      Gray
## 2    175     Green
## 3    165      Gray
## 4    190     Green
## 5    188      Blue
```

And this is the result of converting it to a matrix.

```
d.mat = mda.df2mat(d)
show(d.mat)
```

```
##      Height Blue Gray
## [1,]    180    0    1
## [2,]    175    0    0
## [3,]    165    0    1
## [4,]    190    0    0
## [5,]    188    1    0
```

The number of dummy variables by default is the number of levels minus one. You can change this by using argument `full = TRUE` as it is shown in the example below.

```
d.mat = mda.df2mat(d, full = TRUE)
show(d.mat)
```

```
##      Height Blue Gray Green
## [1,]    180    0    1     0
```

```
## [2,]    175    0    0     1
## [3,]    165    0    1     0
## [4,]    190    0    0     1
## [5,]    188    1    0     0
```

It is important to have level labels in all factor columns of the same data frame unique, as they are used for names of the dummy variables (e.g. you should not have two factors with the same level name). If a factor is hidden it will just be converted to numeric values and remain excluded from modelling.

Excluding rows and columns

Sometimes it is necessary to hide or to exclude a particular row or a particular column from a dataset, so they will not be shown on plots and will not be taken into account in modelling, but without removing them physically. So the excluded rows and columns are still there but are not treated in a usual way by the *mdatools* methods. In *mdatools* it is possible by using the following functions:

Function	Description
`mda.exclrows(x, ind)`	Exclude (hide) all rows specified by variable `ind`, which can be a vector with rows indices, names or a logical vector.
`mda.exclcols(x, ind)`	Exclude (hide) all columns specified by variable `ind`, which can be a vector with columns indices, names or a logical vector.

The mechanism is very simple, the indices of the rows or columns, which must be excluded, are saved into special attributes, which are then recognized by all methods implemented in `mdatools`. Standard R functions will ignore the attributes.

Here is a simple example. Let's create a dataset first (it can be either matrix or a data frame)

```
Height = c(180, 175, 165, 190, 188)
Weight = c(78, 79, 60, 99, 80)
Shoesize = c(44, 39, 35, 45, 44)
d = cbind(Height, Weight, Shoesize)
rownames(d) = paste0("O", 1:5)
show(d)
```

```
##    Height Weight Shoesize
## O1    180     78       44
## O2    175     79       39
## O3    165     60       35
## O4    190     99       45
## O5    188     80       44
```

Now let's exclude rows 3 and 4 and then column with name "Weight" from the data.

```
d = mda.exclrows(d, 3:4)
d = mda.exclcols(d, "Weight")
```

If you show data using `mda.show()` these values will be hidden:

```
mda.show(d)
```

```
##    Height Shoesize
## O1    180       44
## O2    175       39
## O5    188       44
```

But if you use conventional `show()` (or `print()`) method you will see them all:

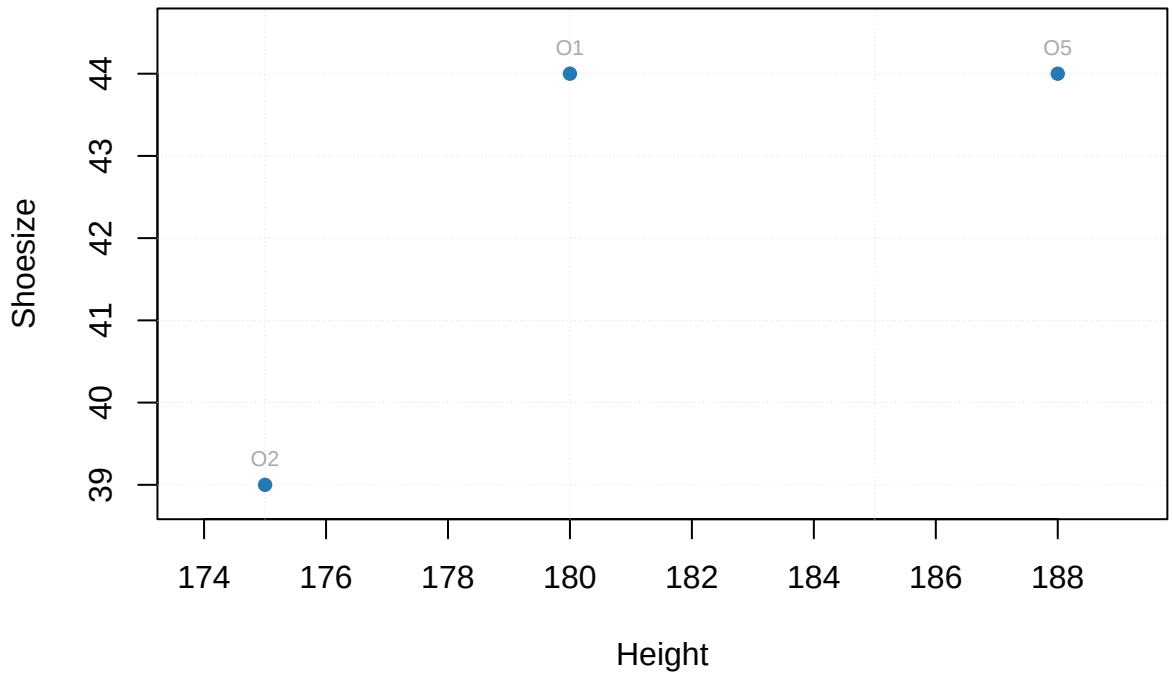
```
show(d)
```

```
##    Height Weight Shoesize
## O1    180     78       44
## O2    175     79       39
## O3    165     60       35
## O4    190     99       45
## O5    188     80       44
## attr(,"exclrows")
## [1] 3 4
## attr(,"exclcols")
## [1] 2
```

As you can see, all the values are still there, but there are two new attributes, `exclcols` and `exclrows`. To avoid any issues do not change the values of the attributes manually, always use the functions above.

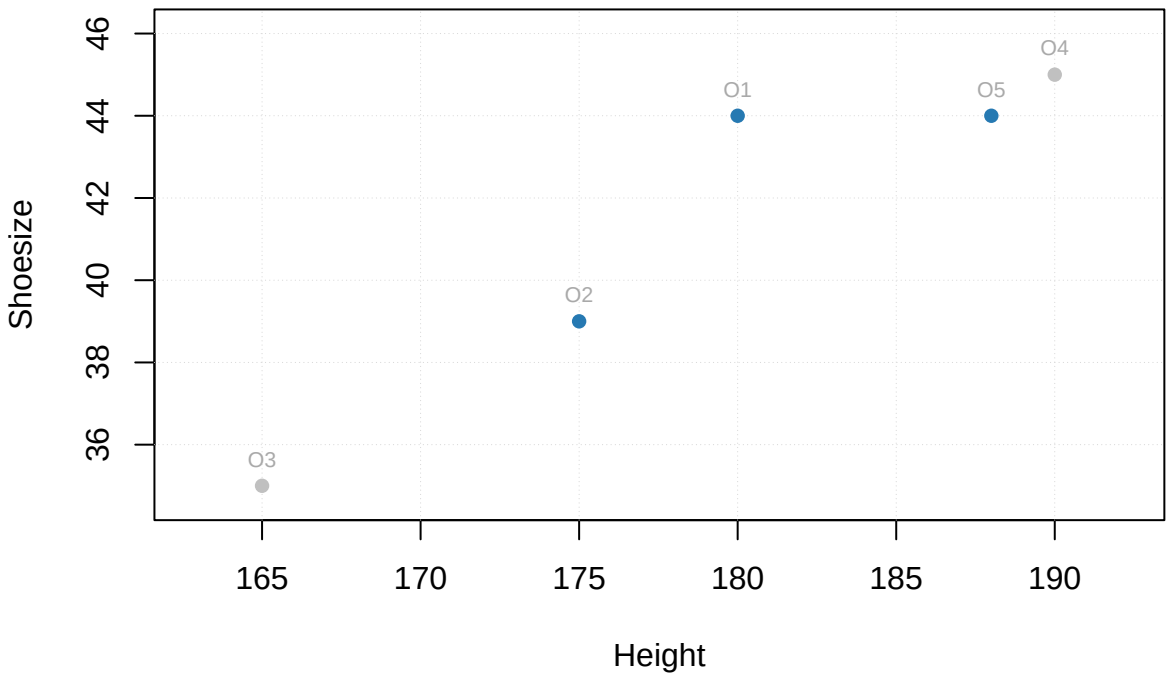
Now if you make a plot using function `mdaplot()` you will see only three points by default, because the other two are hidden. Also the plot will be made for columns 1 and 3, because the second column (Weight) was also hidden. See the code and the result below:

```
mdaplot(d, show.labels = TRUE)
```



However `mdaplot()` method has parameter `show.excluded` which reveals the hidden data points (rows):

```
mdaplot(d, show.labels = TRUE, show.excluded = TRUE)
```



You can include the hidden columns and rows back by using `mda.inclcols()` and `mda.inclrows()`.

You can also remove all excluded data forever, in this case use `mda.purge()` method:

```
d_new = mda.purge(d)
show(d_new)
```

```
##    Height Shoesize
## O1    180       44
## O2    175       39
## O5    188       44
```

Simple plots

As already mentioned, *mdatools* has its own functions for plotting with several extra options not available in basic plot tools. These functions are used to make all plots in the models and results (e.g. scores, loadings, predictions, etc.) therefore it can be useful to spend some time and learn the new features (e.g. coloring data points with a vector of values or using manual ticks for axes). But if you are going to make all plots manually (e.g. using *ggplot2*) you can skip this and the next sections.

In this section we will look at how to make simple plots from your data objects. Simple plots are scatter (`type = "p"`), density-scatter (`type = "d"`), line (`type = "l"`), line-scatter (`type = "b"` or `type="o"`), bar (`type = "h"`) or errorbar (`type = "e"`) plots made for one set of objects. All plots can be created using the same method `mdaplot()` by providing a whole dataset as a main argument. Depending on the plot type, the method "treats" the data values differently.

This table below contains a list of parameters for `mdaplot()`, which are not available for traditional R plots. In this section we will describe most of the details using simple examples.

Parameter	Description
`cgroup`	a vector of values (same as number of rows in data) used to colorize plot objects with a color gradient.
`colmap`	color map for the color gradient (possible values are `'default'`, `'gray'` or a vector with colors).
`show.colorbar`	when color grouping is used, `mdaplot()` shows a color bar legend, this parameter allows you to turn it off.
`show.labels`	logical parameter showing labels beside plot objects (points, lines, etc). Size and color of labels can be adjusted using parameters `lab.cex` and `lab.col`.
`labels`	parameter telling what to use as labels (by default row names, but can also be indices or manual values).
`lab.col`	color for the labels.
`lab.cex`	font size for the labels (as a scale factor).
`xticks`	vector with numeric values to show the x-axis ticks at.
`yticks`	vector with numeric values to show the y-axis ticks at.
`xticklabels`	vector with labels (numbers or text) for the x-ticks.
`yticklabels`	vector with labels (numbers or text) for the y-ticks.
`xlas`	an integer between 0 and 3 telling at which angle the x-tick labels have to be shown.
`ylas`	an integer between 0 and 3 telling at which angle the y-tick labels have to be shown.
`show.axes`	logical, if `TRUE`, function will make a new plot, if `FALSE`, add the plot objects to a previous one.
`show.lines`	a vector with two numbers — position of horizontal and vertical lines on a plot (e.g. coordinate axes).
`show.grid`	logical, show or not a grid. It places grid behind the plot object in contrast to conventional `grid()` method. Use `grid.lwd` and `grid.col` parameters to adjust the grid look.
`show.excluded`	logical, show or not points or lines corresponding to the excluded rows.
`opacity`	opacity of colors in range 0...1 (applied to all colors of current plot).

Scatter plots

We will use `people` dataset to illustrate how scatter plots work (see `?people` for details).

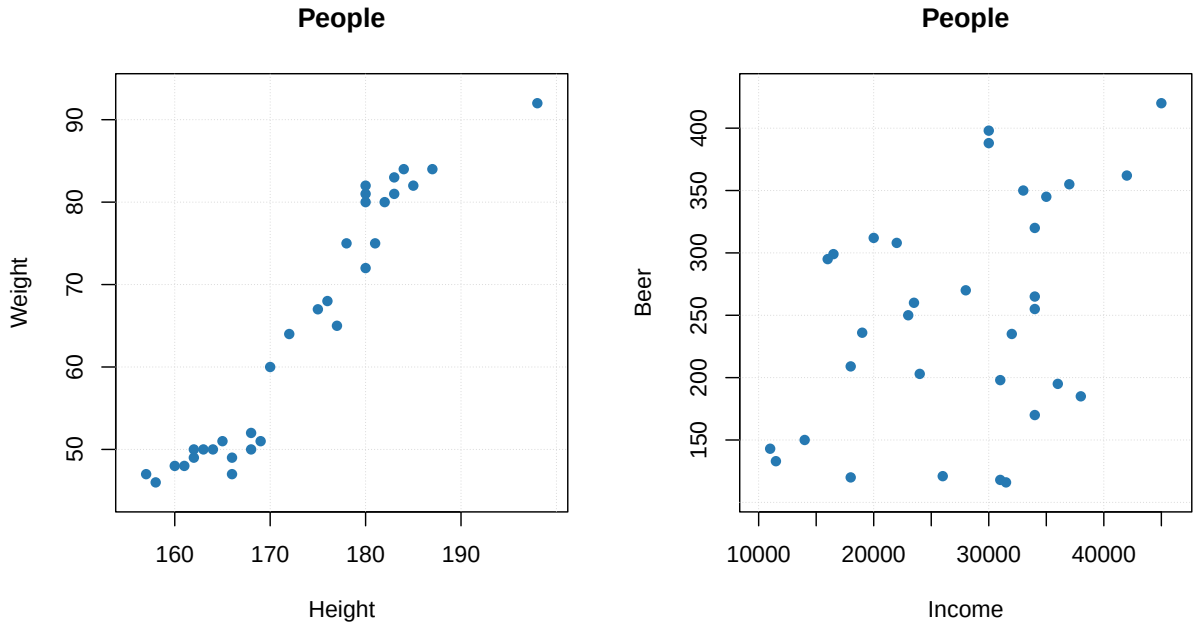
```
data(people)
attr(people, "name") = "People"
attr(people, "xaxis.name") = "Variables"
attr(people, "yaxis.name") = "Persons"
```

For scatter plots the method takes first two columns of a dataset as x and y vectors. If only one column is available `mdaplot()` uses it for y-values and generates x-values as an index for each value.

```
par(mfrow = c(1, 2))

# show plot for the whole dataset (columns 1 and 2 will be taken)
mdaplot(people, type = "p")

# subset the dataset and keep only columns 6 and 7 and then make a plot
mdaplot(mda.subset(people, select = c(6, 7)), type = "p")
```



All parameters available for the standard `points()` method will work with `mdaplot()` as well. Besides that, you can colorize points according to some values using a color gradient. By default, the gradient is generated using one of the diverging color schemes from colorbrewer2.org, but this can be changed using parameter `colmap` as it is shown below.

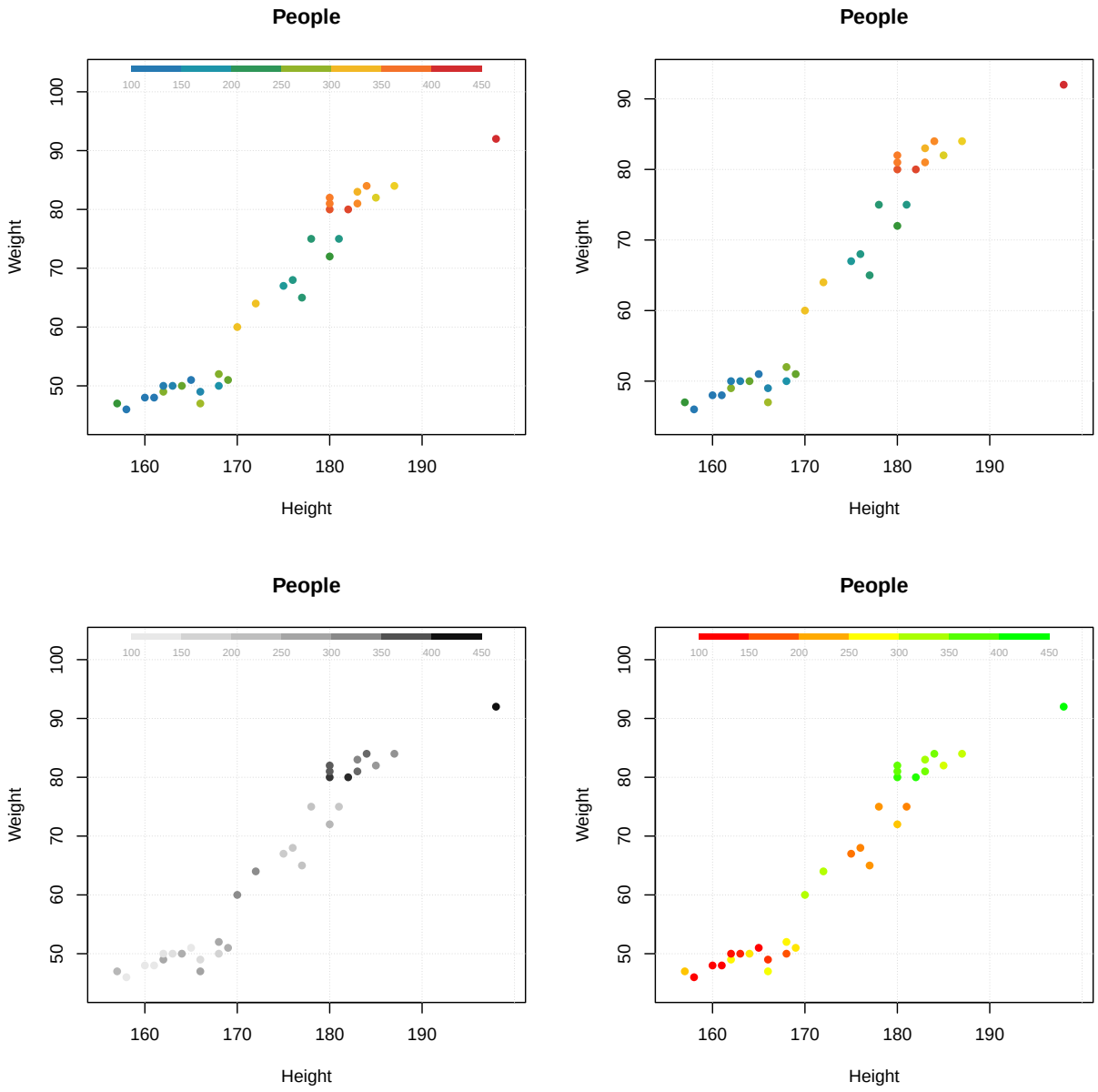
```
par(mfrow = c(2, 2))

# show Height vs Weight and color points by the Beer consumption
mdaplot(people, type = "p", cgroup = people[, "Beer"])

# do the same but do not show colorbar
mdaplot(people, type = "p", cgroup = people[, "Beer"], show.colorbar = FALSE)

# do the same but use grayscale color map
mdaplot(people, type = "p", cgroup = people[, "Beer"], colmap = "gray")

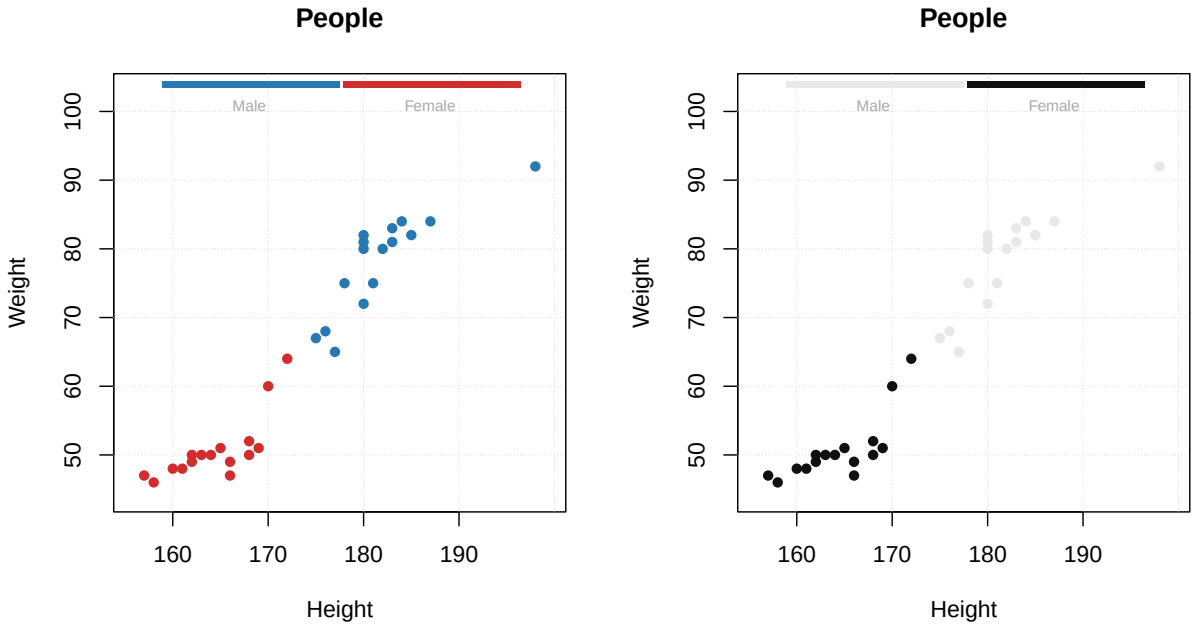
# do the same but using colormap with gradients between red, yellow and green colors
mdaplot(people, type = "p", cgroup = people[, "Beer"], colmap = c("red", "yellow", "green"))
```



If the vector with values for color grouping is a factor, level labels will be shown on a colorbar legend and there will be a small margin between bars.

```
# make a factor using values of variable Sex and define labels for the factor levels
g = factor(people[, "Sex"], labels = c("Male", "Female"))

par(mfrow = c(1, 2))
mdaplot(people, type = "p", cgroup = g)
mdaplot(people, type = "p", cgroup = g, colmap = "gray")
```

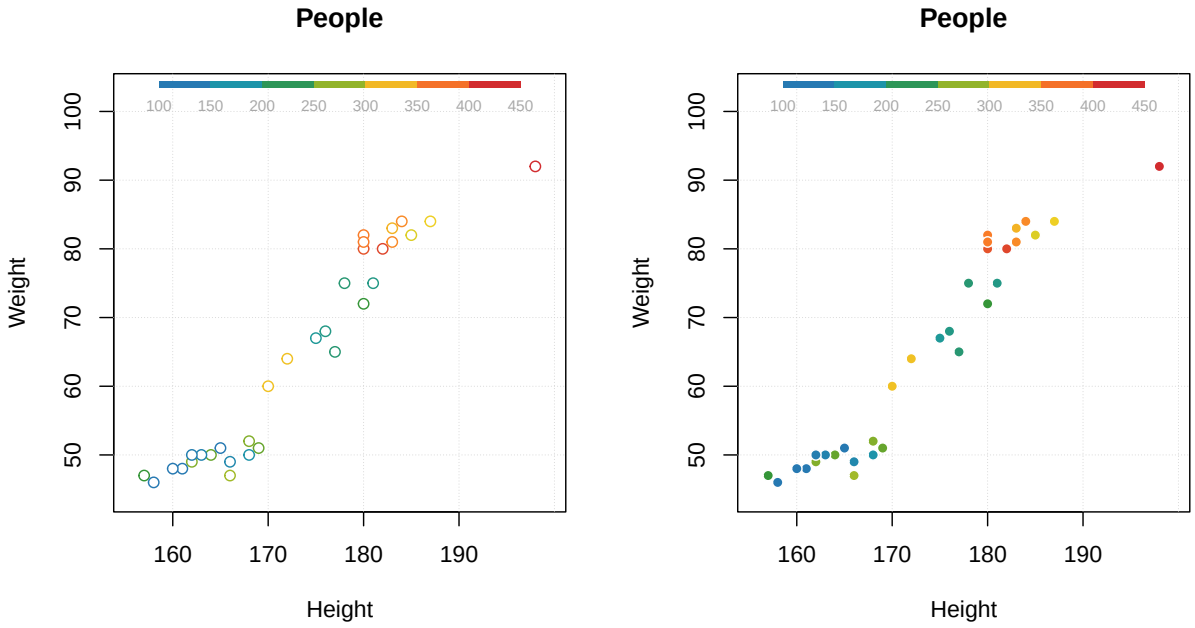


If you use point characters from 21 to 25 (the ones which allow you to specify both color of border and background of the marker symbol), then the `cgroup` is applied to color of the borders of the symbols by default. If you want to apply it to background color, use logical parameter `pch.colinv` for that as shown below:

```
par(mfrow = c(1, 2))

# default way - color grouping is used for borders and "bg" for background
mdaplot(people, type = "p", cgroup = people[, "Beer"], pch = 21, bg = "white")

# inverse - color grouping is used for background and "bg" for border
mdaplot(people, type = "p", cgroup = people[, "Beer"], pch = 21, bg = "white", pch.colinv = TRUE)
```



Another useful option is adding labels to the data points. By default, row names will be taken for the labels but you can specify a parameter `labels`, which can be either a text (`"names"` or `"indices"`) or a vector with values to show as labels. Color and size of the labels can be adjusted.

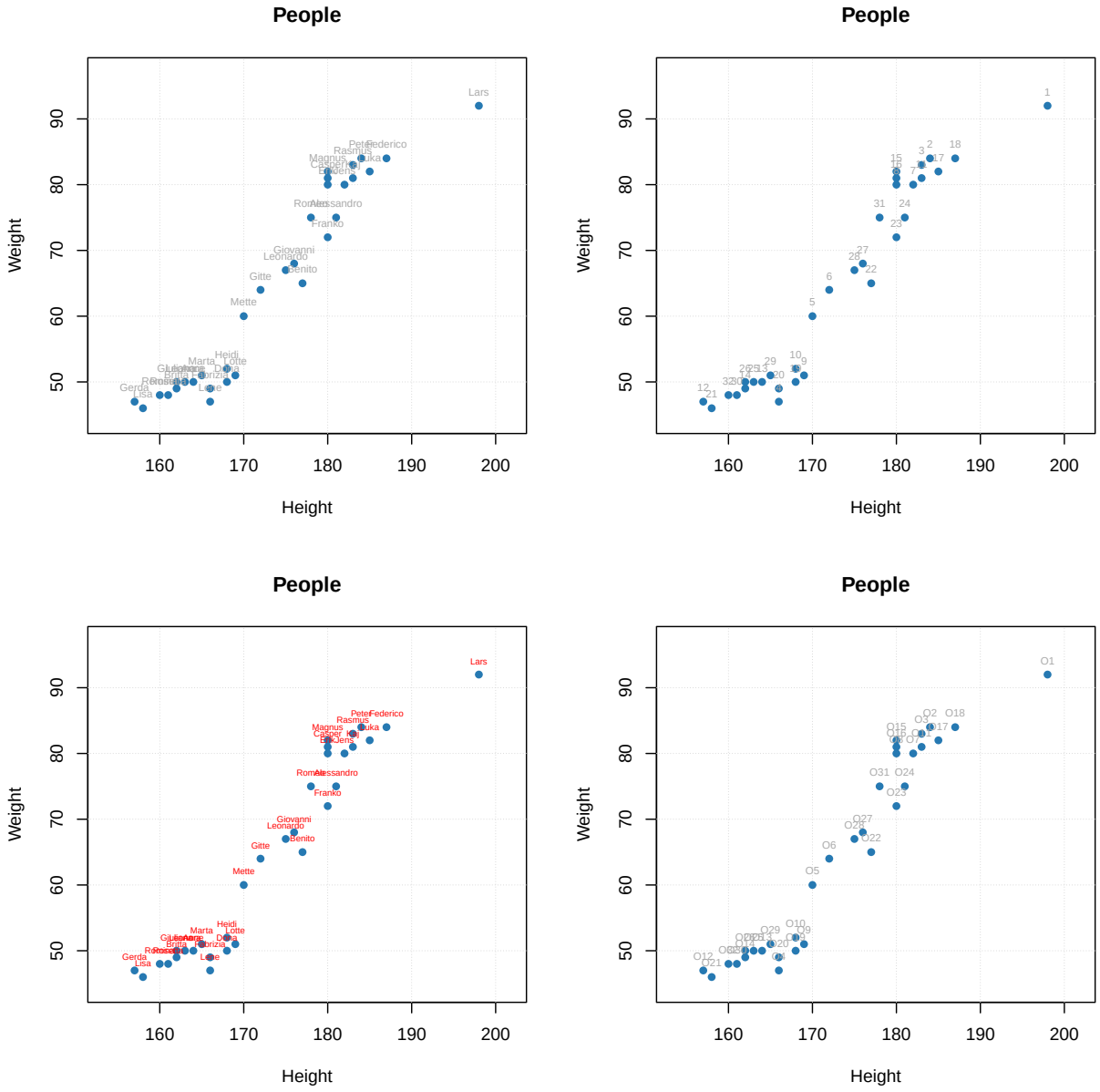
```
par(mfrow = c(2, 2))

# by default row names will be used as labels
mdaplot(people, type = "p", show.labels = TRUE)

# here we tell to use indices as labels instead
mdaplot(people, type = "p", show.labels = TRUE, labels = "indices")

# here we use names again but change color and size of the labels
mdaplot(people, type = "p", show.labels = TRUE, labels = "names", lab.col = "red", lab.cex = 0.5)

# finally we provide a vector with manual values to be used as the labels
mdaplot(people, type = "p", show.labels = TRUE, labels = paste0("O", seq_len(nrow(people))))
```



You can also manually specify axis ticks and tick labels. The labels can be rotated using parameters `xlas` and `ylas`, see the examples below. It is important though, that if you provide manual values for the tick labels you must also provide a vector of values with the positions the labels should be shown at. And, of course, the two vectors must have the same number of values.

```
par(mfrow = c(2, 2))

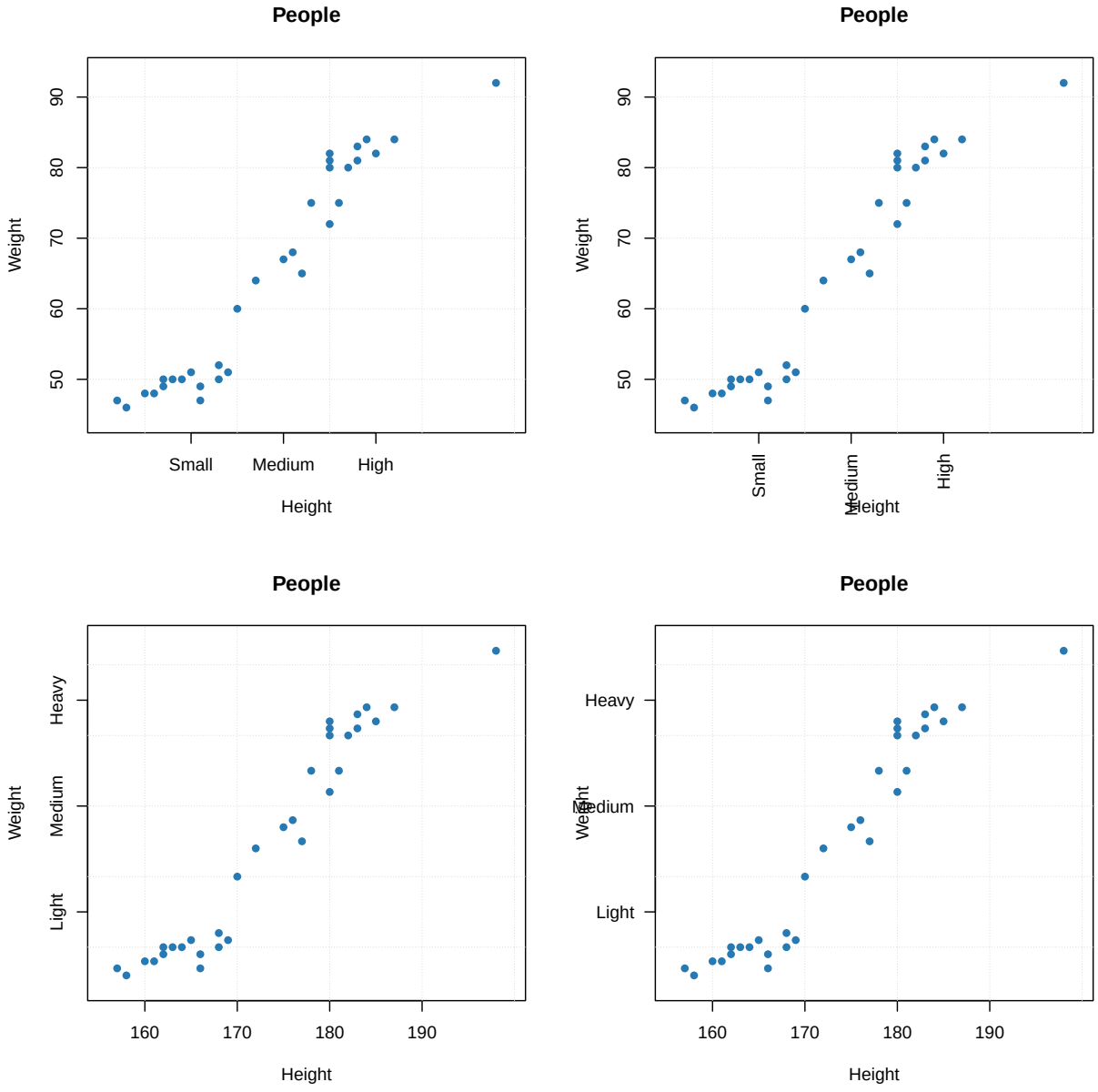
# manual values and tick labels for the x-axis
mdaplot(people, xticks = c(165, 175, 185), xticklabels = c("Small", "Medium", "High"))

# same but with rotation of the tick labels
mdaplot(people, xticks = c(165, 175, 185), xticklabels = c("Small", "Medium", "High"), xlas = 2)

# manual values and tick labels for the y-axis
```

```
mdaplot(people, yticks = c(55, 70, 85), yticklabels = c("Light", "Medium", "Heavy"))

# same but with rotation of the tick labels
mdaplot(people, yticks = c(55, 70, 85), yticklabels = c("Light", "Medium", "Heavy"), ylas = 2)
```



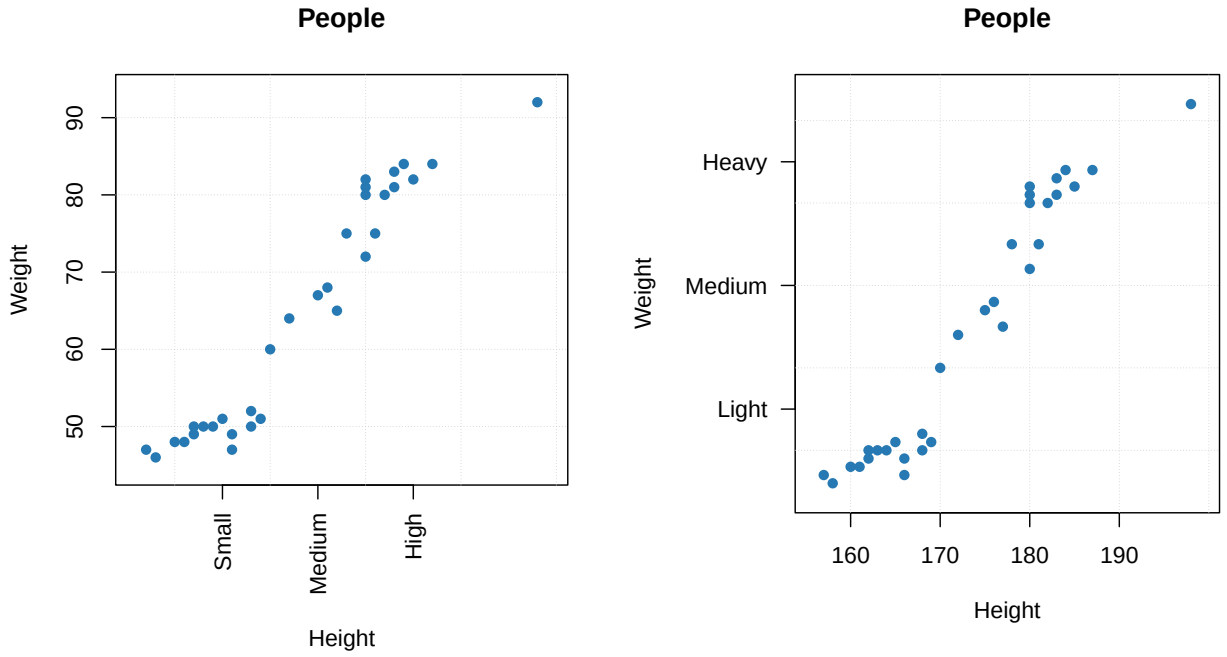
If both axis labels and rotated axis ticks have to be shown, you can adjust plot margins and position of the label using `par()` function and `mtext()` for positioning axis label manually.

```
par(mfrow = c(1, 2))

# change margin for bottom part
par(mar = c(6, 4, 4, 2) + 0.1)
mdaplot(people, xticks = c(165, 175, 185), xticklabels = c("Small", "Medium", "High"),
        xlas = 2, xlab = "")
```

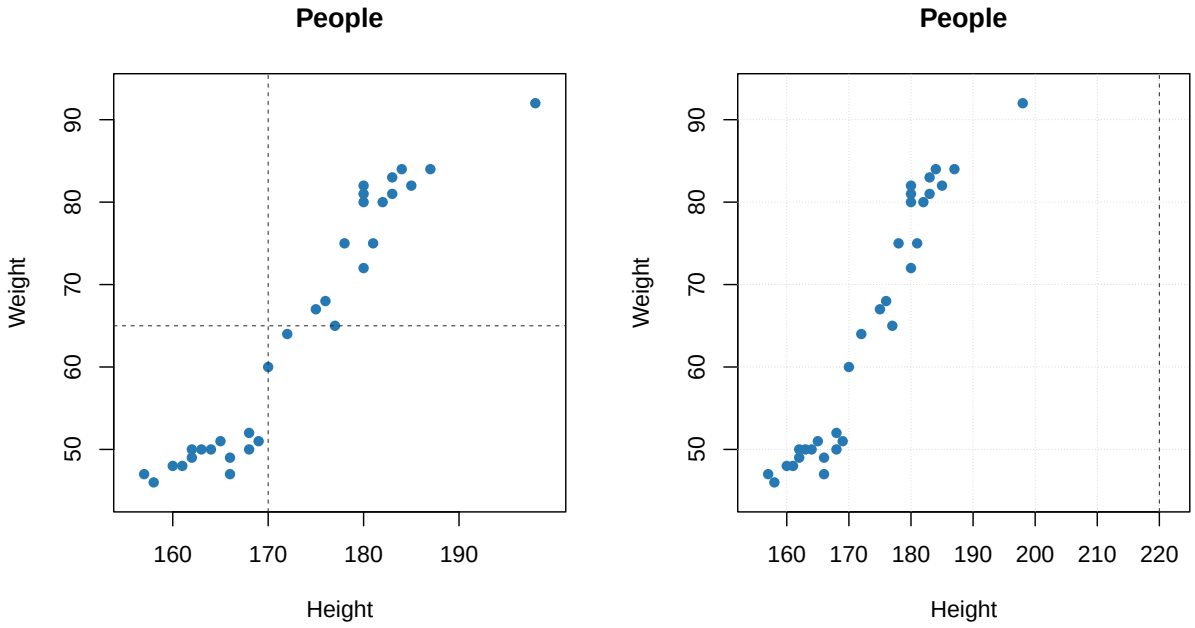
```
mtext("Height", side = 1, line = 5)

# change margin for left part
par(mar = c(5, 6, 4, 1) + 0.1)
mdaplot(people, yticks = c(55, 70, 85), yticklabels = c("Light", "Medium", "Heavy"),
        ylas = 2, ylab = "")
mtext("Weight", side = 2, line = 5)
```



There are also a couple of other parameters, allowing to show/hide grid as well as show horizontal and vertical lines on the plot (axes limits will be adjusted correspondingly).

```
par(mfrow = c(1, 2))
mdaplot(people, show.grid = FALSE, show.lines = c(170, 65))
mdaplot(people, show.lines = c(220, NA))
```



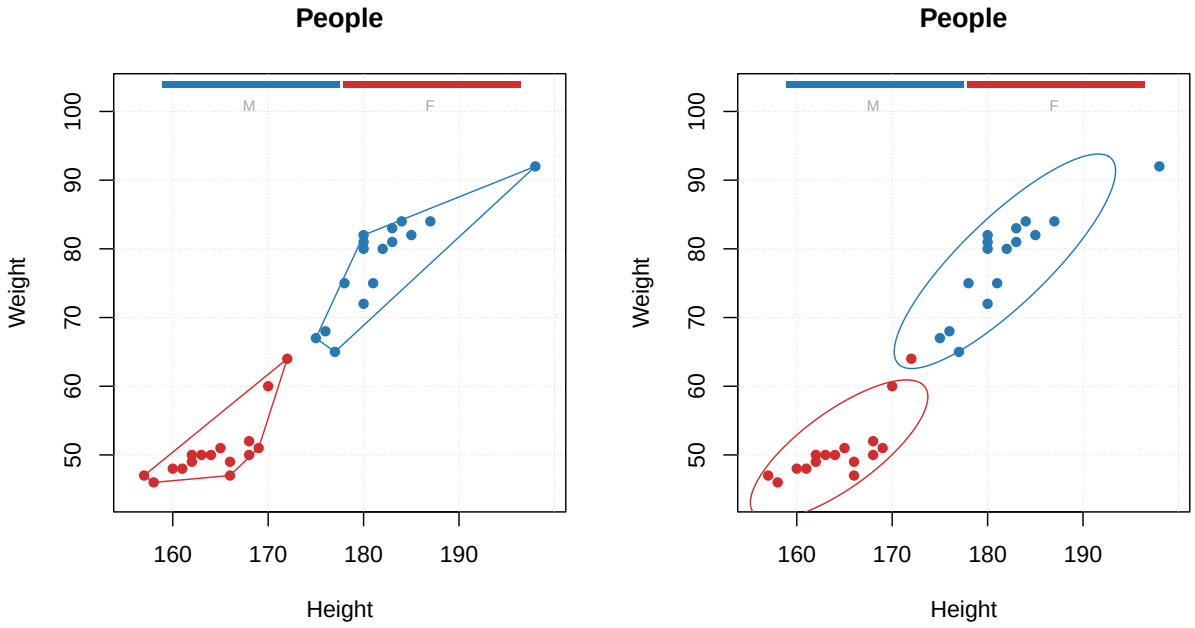
Function `mdaplot()` can also return plot series data, which can be used for extra options. For example, in the case of a scatter plot you can add a confidence ellipse or convex hull for data points. To do this, points must be color grouped by a factor as shown below. For confidence ellipse you can specify the confidence level (default 0.95).

```
# define a factor using values of variable Sex and simple labels
g = factor(people[, "Sex"], labels = c("M", "F"))

par(mfrow = c(1, 2))

# make a scatter plot grouping points by the factor and then show convex hull for each group
p = mdaplot(people, cgroup = g)
plotConvexHull(p)

# make a scatter plot grouping points by the factor and then show 90% confidence intervals
p = mdaplot(people, cgroup = g)
plotConfidenceEllipse(p, conf.level = 0.90)
```

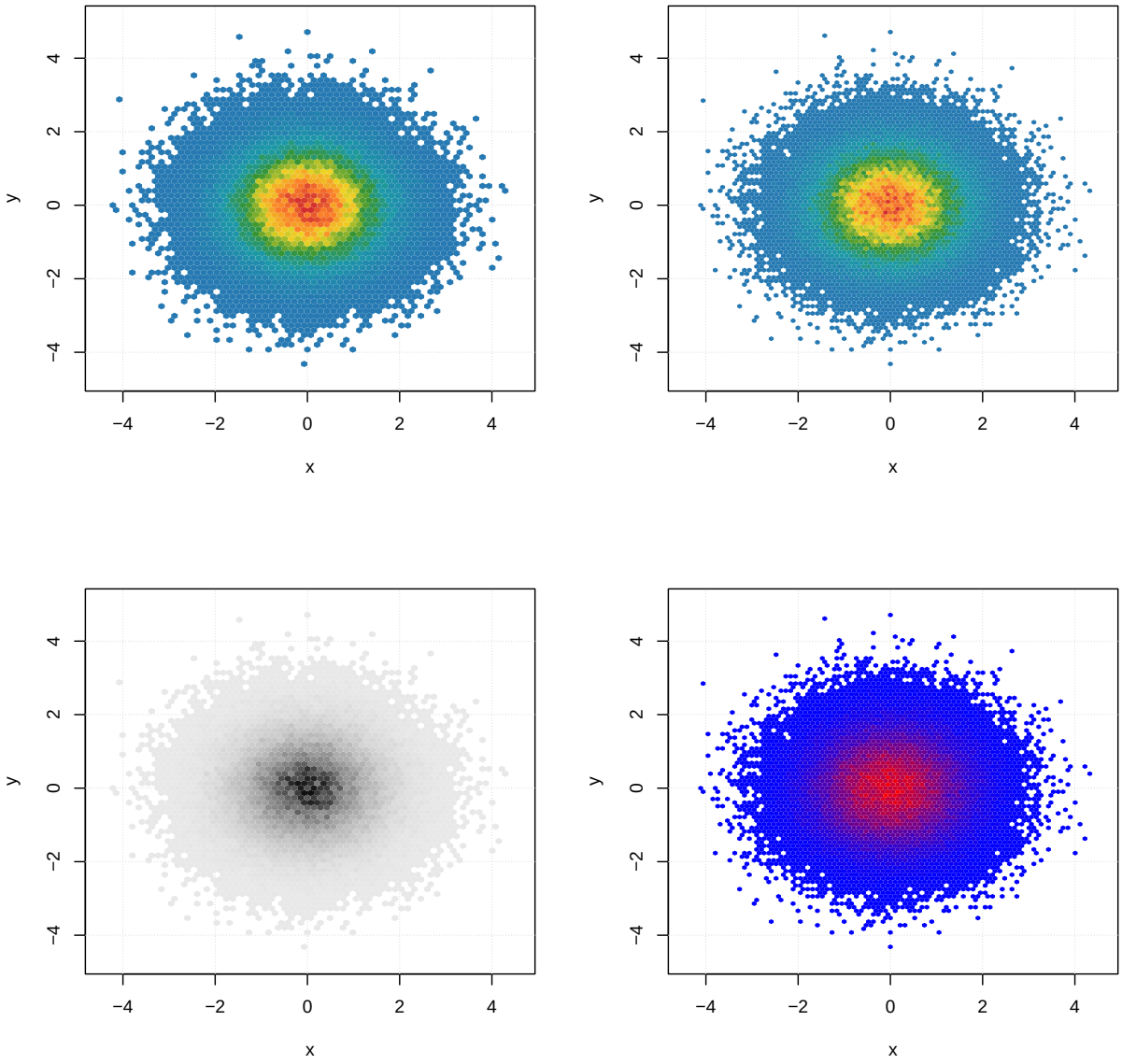


In case when number of data points is large (e.g. when dealing with images, where every pixel is a data point), using a density plot is a good alternative to conventional scatter plots. The plot does not show all data points but instead splits the whole plot space into small hexagonal regions and uses a color gradient for illustrating the density of the points in each region. This approach is known as *hexagonal binning*. To create a density plot simply use `type="d"`. You can also specify color map and number of bins along each axes (`nbins`).

The code below shows an example of using density plots for 100000 data points with x and y values taken from normally distributed population.

```
x = rnorm(100000)
y = rnorm(100000)
d = cbind(x, y)

par(mfrow = c(2, 2))
mdaplot(d, type = "d")
mdaplot(d, type = "d", nbins = 80)
mdaplot(d, type = "d", colmap = "gray")
mdaplot(d, type = "d", nbins = 80, colmap = c("blue", "red"))
```



Line plots

When a line plot is created, `mdaplot()` shows a line plot for every row of the provided dataset. So if the dataset has more than one row, the plot will show a bunch of lines having the same properties (color, type, etc). This is particularly useful when working with signals and spectroscopic data. In this subsection we will use simulated UV/Vis spectra from `simdata`. See `?simdata` for more details about this set.

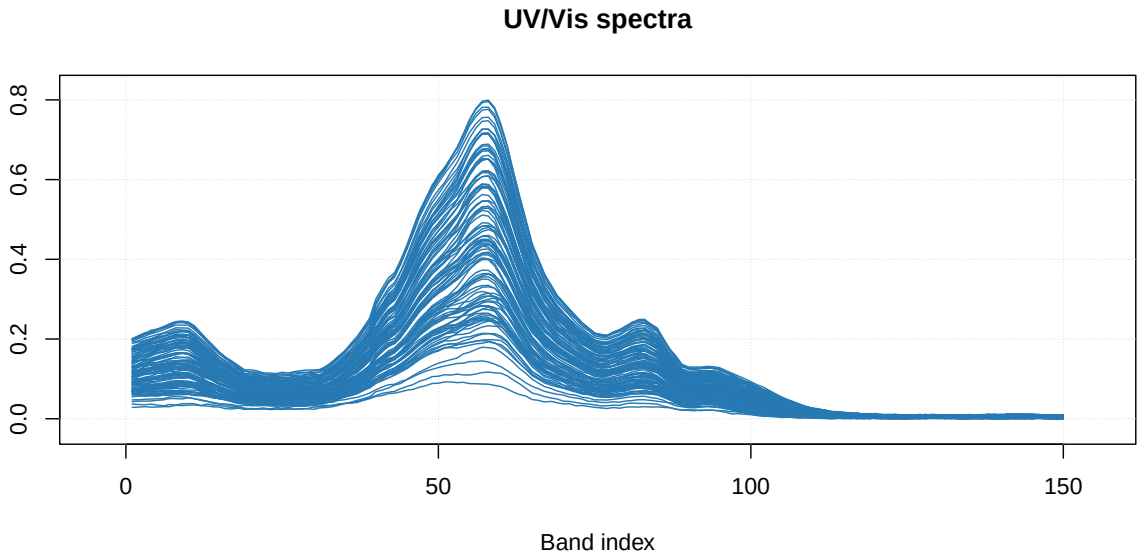
```
data(simdata)

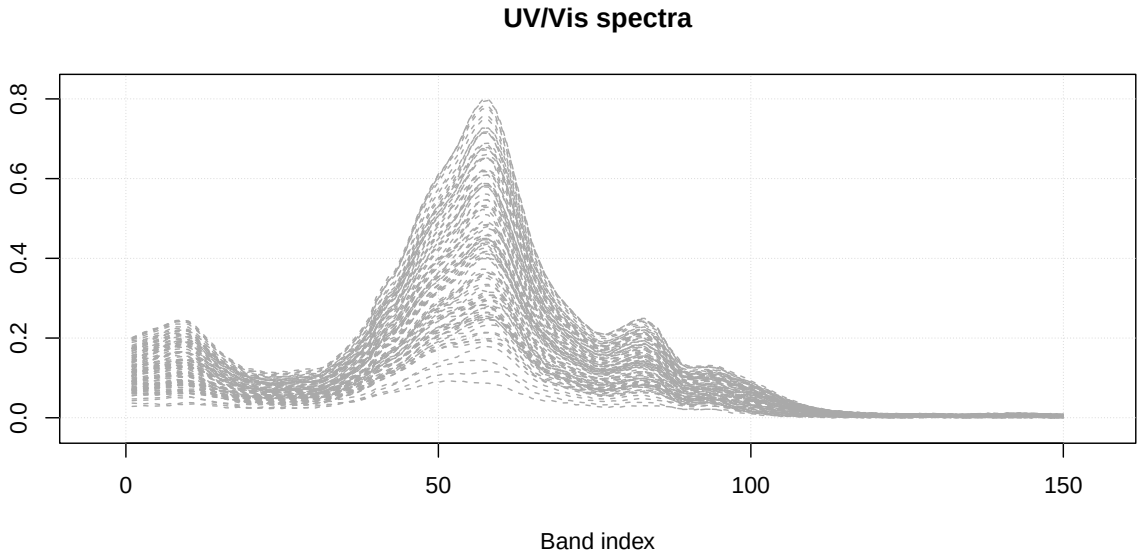
# get the calibration spectra, wavelength and
# concentration of first component as separate variables
spectra = simdata$spectra.c
wavelength = simdata$wavelength
conc = simdata$conc.c[, 1]
```

```
# add names as attributes
attr(spectra, "name") = "UV/Vis spectra"
attr(spectra, "xaxis.name") = "Band index"
```

Here are simple examples of how to make the line plots.

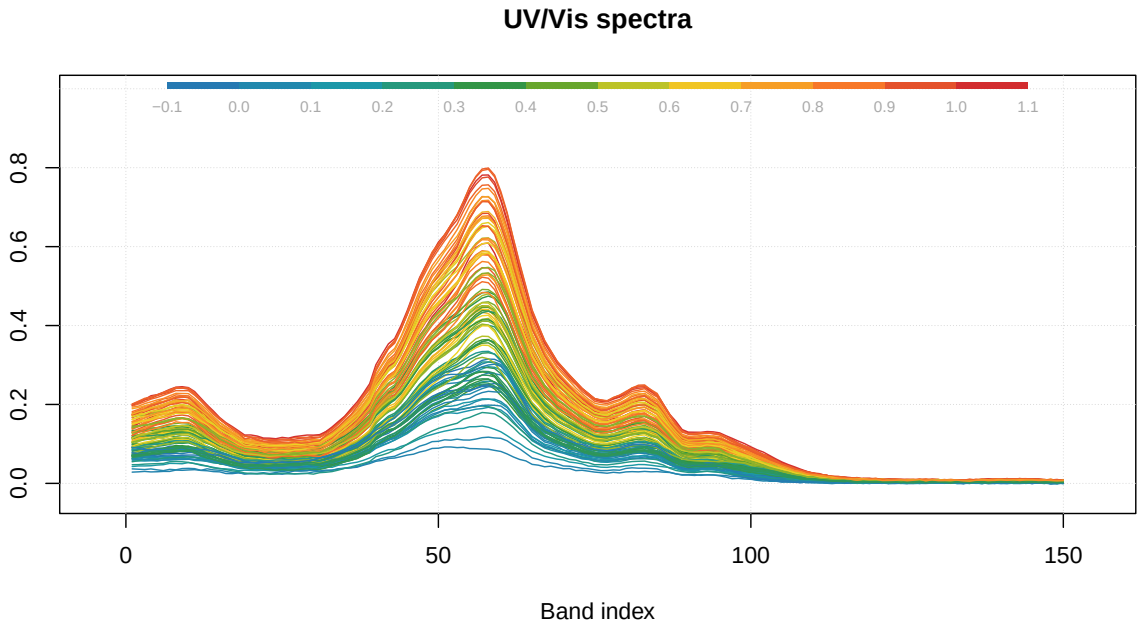
```
par(mfrow = c(2, 1))
mdaplot(spectra, type = "l")
mdaplot(spectra, type = "l", col = "darkgray", lty = 2)
```





Most of the parameters described for scatter plots will work for the line plots as well. For example, you can colorize the lines by using a vector with some values (in the example below I use concentration of one of the chemical components).

```
par(mfrow = c(1, 1))
mdaplot(spectra, type = "l", cgroup = conc)
```

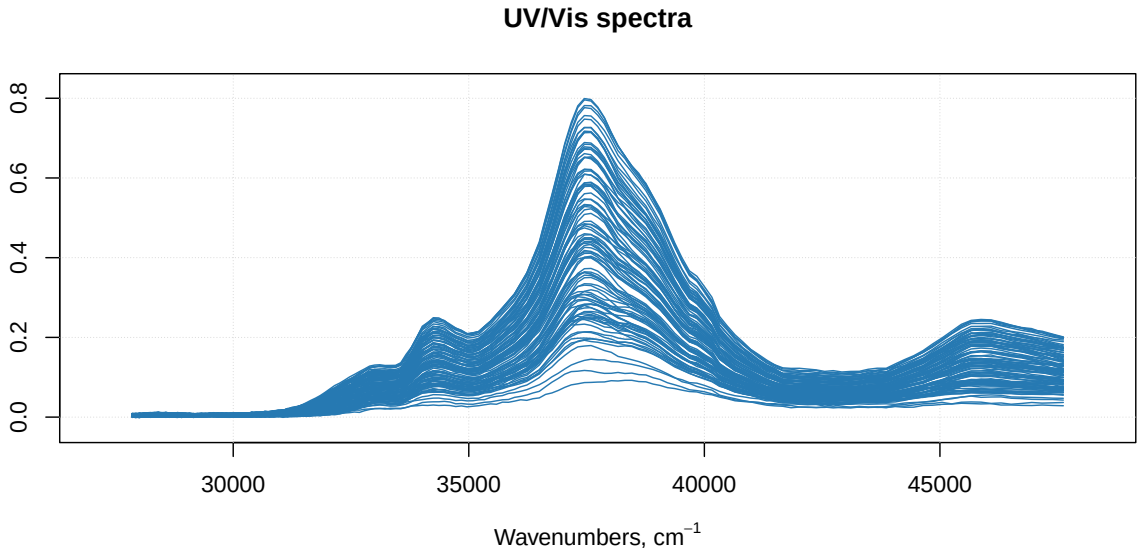


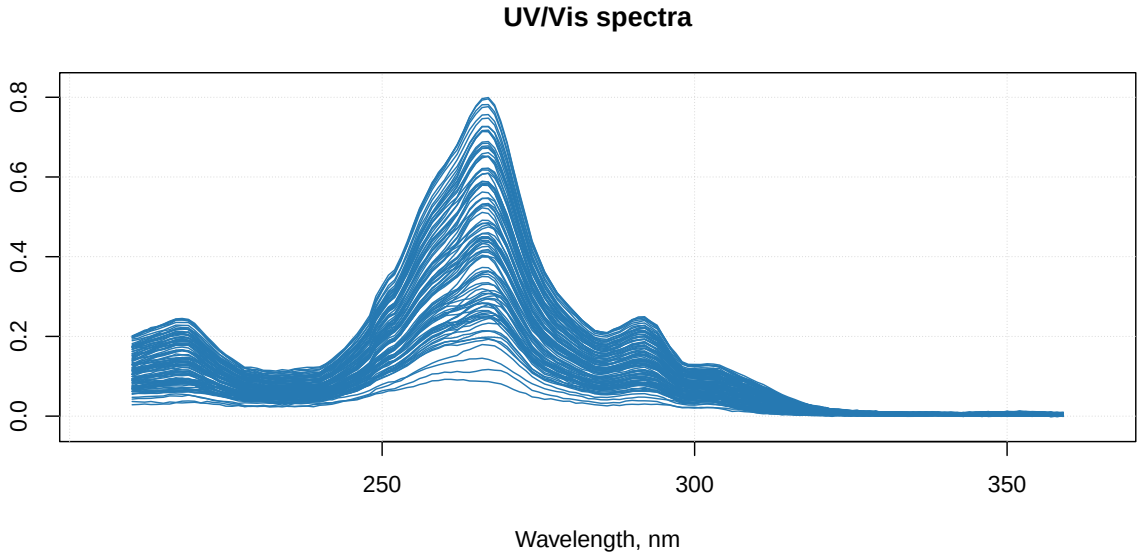
And of course you can use the attributes, allowing to provide manual x-values — `'xaxis.values'` (similar parameter for y-values is `'yaxis.values'`). In the example below we show the spectra using wavelength in nm and wavenumbers in inverse cm.

```
par(mfrow = c(2, 1))

attr(spectra, "xaxis.name") = expression("Wavenumbers, cm"^-1)
attr(spectra, "xaxis.values") = 10^7/wavelength
mdaplot(spectra, type = "l")

attr(spectra, "xaxis.name") = "Wavelength, nm"
attr(spectra, "xaxis.values") = wavelength
mdaplot(spectra, type = "l")
```





When you provide such data to any model methods (e.g. PCA, PLS, etc), then all variable related results (loadings, regression coefficients, etc.) will inherit this attribute and use it for making line plots.

Bar and errorbar plots

The bar plot is perhaps the simplest, as it shows values for the first row of the data as bars. Let us start with a simple dataset, where we have a matrix with explained variance of data from e.g. PCA decomposition.

```
# make a simple two rows matrix with values
d = rbind(
   c(20, 50, 60, 90),
   c(14, 45, 59, 88)
)

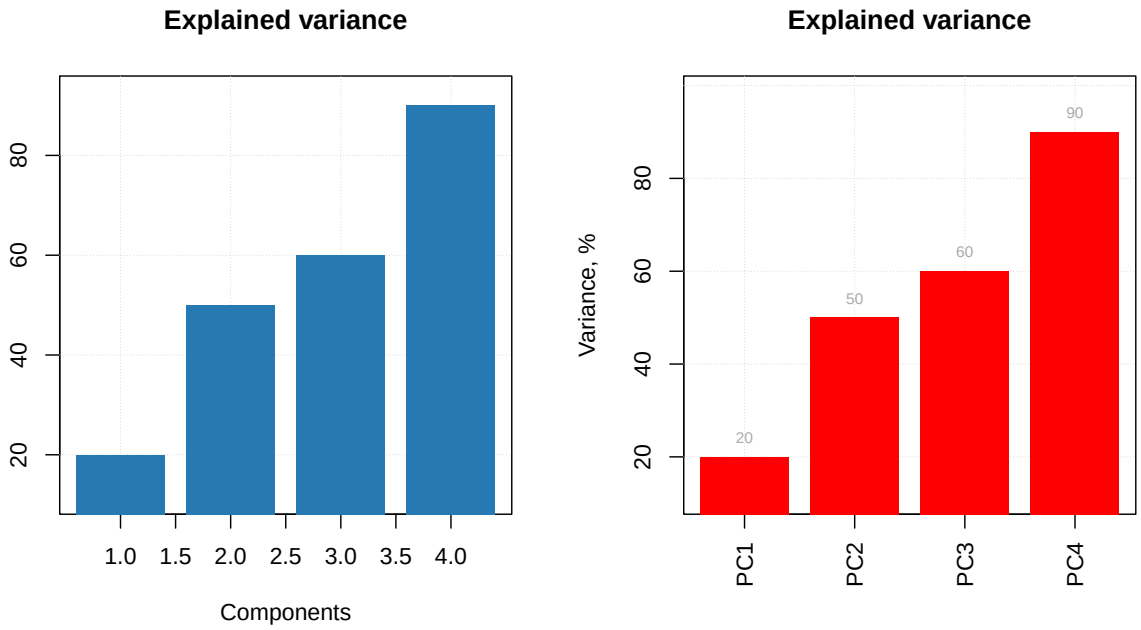
# add some names and attributes
```

```
colnames(d) = paste0("PC", 1:4)
rownames(d) = c("Cal", "CV")
attr(d, "xaxis.name") = "Components"
attr(d, "name") = "Explained variance"

par(mfrow = c(1, 2))

# make a default bar plot
mdaplot(d, type = "h")

# make a bar plot with manual xtick labels, color and labels for data values
mdaplot(d, type = "h", xticks = seq_len(ncol(d)), xticklabels = colnames(d), col = "red",
   show.labels = TRUE, labels = "values", xlas = 2, xlab = "", ylab = "Variance, %")
```



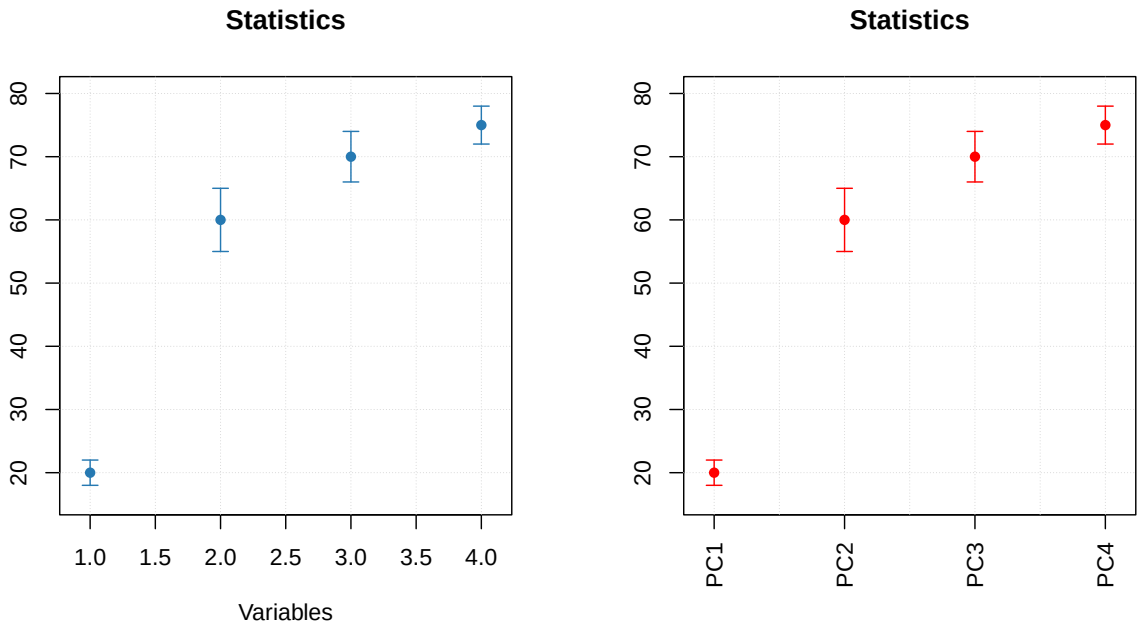
As you can notice, the values from the second row were ignored, as bar plot always takes the first row.

Errorbar plot, in contrast, always expects data to have two or three rows. The first row is the origin points of the error bars, the second row is the size of the bottom error bar and the third row is the size of the top error bar. If data has only two rows both parts will be symmetric relative to the origin.

```
# generate some mean and standard deviation values and combine them as rows of a matrix
d = rbind(
   c(20, 60, 70, 75),
   c(2, 5, 4, 3)
)

# add names and attributes
rownames(d) = c("Mean", "Std")
colnames(d) = paste0("PC", 1:4)
attr(d, 'name') = "Statistics"
```

```
# show the plots
par(mfrow = c(1, 2))
mdaplot(d, type = "e")
mdaplot(d, type = "e", xticks = seq_len(ncol(d)),
   xticklabels = colnames(d), col = "red", xlas = 2, xlab = "")
```

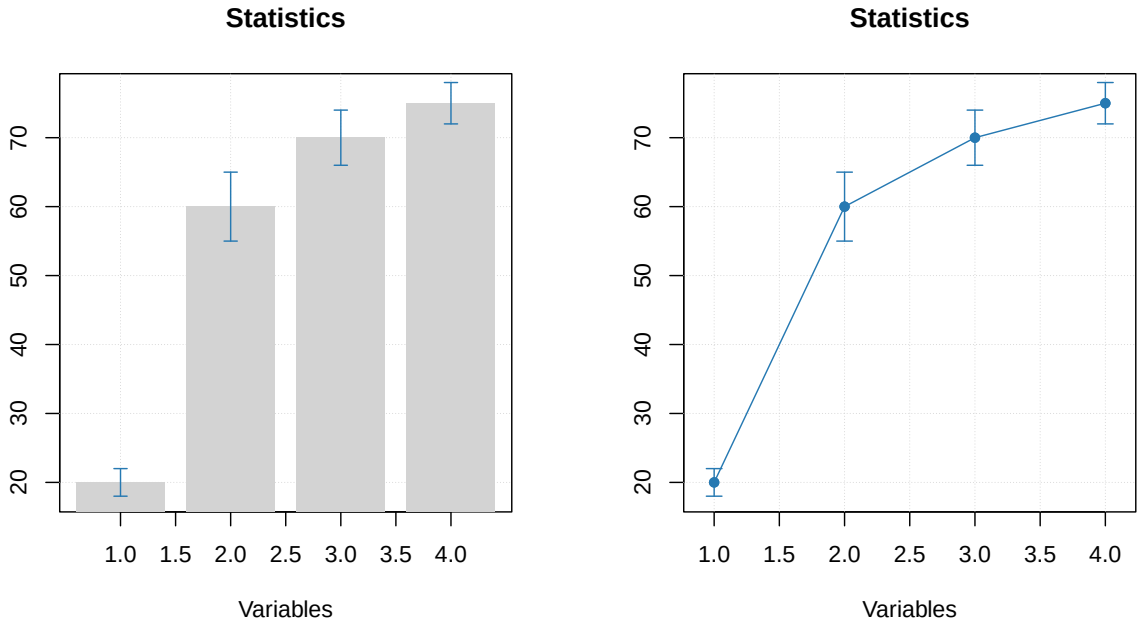


All simple plots can be combined together on the same axes. In this case, the first plot is created as usual and all other plots have to be created with option `show.axes = FALSE` as it is shown below. It must be noted that in this case axes limits have to be set manually when creating the first plot.

```
par(mfrow = c(1, 2))

mdaplot(mda.subset(d, 1), type = "h", col = "lightgray")
mdaplot(d, type = "e", show.axes = FALSE, pch = NA)

mdaplot(mda.subset(d, 1), type = "o")
mdaplot(d, type = "e", show.axes = FALSE)
```



In the next section we will discuss plots for several groups of objects (rows).

Plots for groups of objects

The package has another method for creating plots, `mdaplotg()`, which aims at making plots for groups of objects. It can be several groups of points, lines or bars, where every group has its own attributes, such as color, marker, line type and width, etc. There is a simple criterion to distinguish between the simple and group plots: group plots usually need a legend and simple plots — not. The `mdaplotg()` method allows one to do a lot of things (e.g. split data into groups, add a legend and labels, etc) much easier and this section will show most of the details.

There are three ways to provide data sets for making the group plots. Let's discuss them first and then talk about some extra features.

One matrix or data frame

If the dataset is a matrix or a data frame, `mdaplotg()` will make a line, scatter-line or a bar plot, considering every row as a separate group. This can be useful, when, for example, you want to show how explained variance depends on a number of components for calibration and test set, or how loadings for the first two components look like.

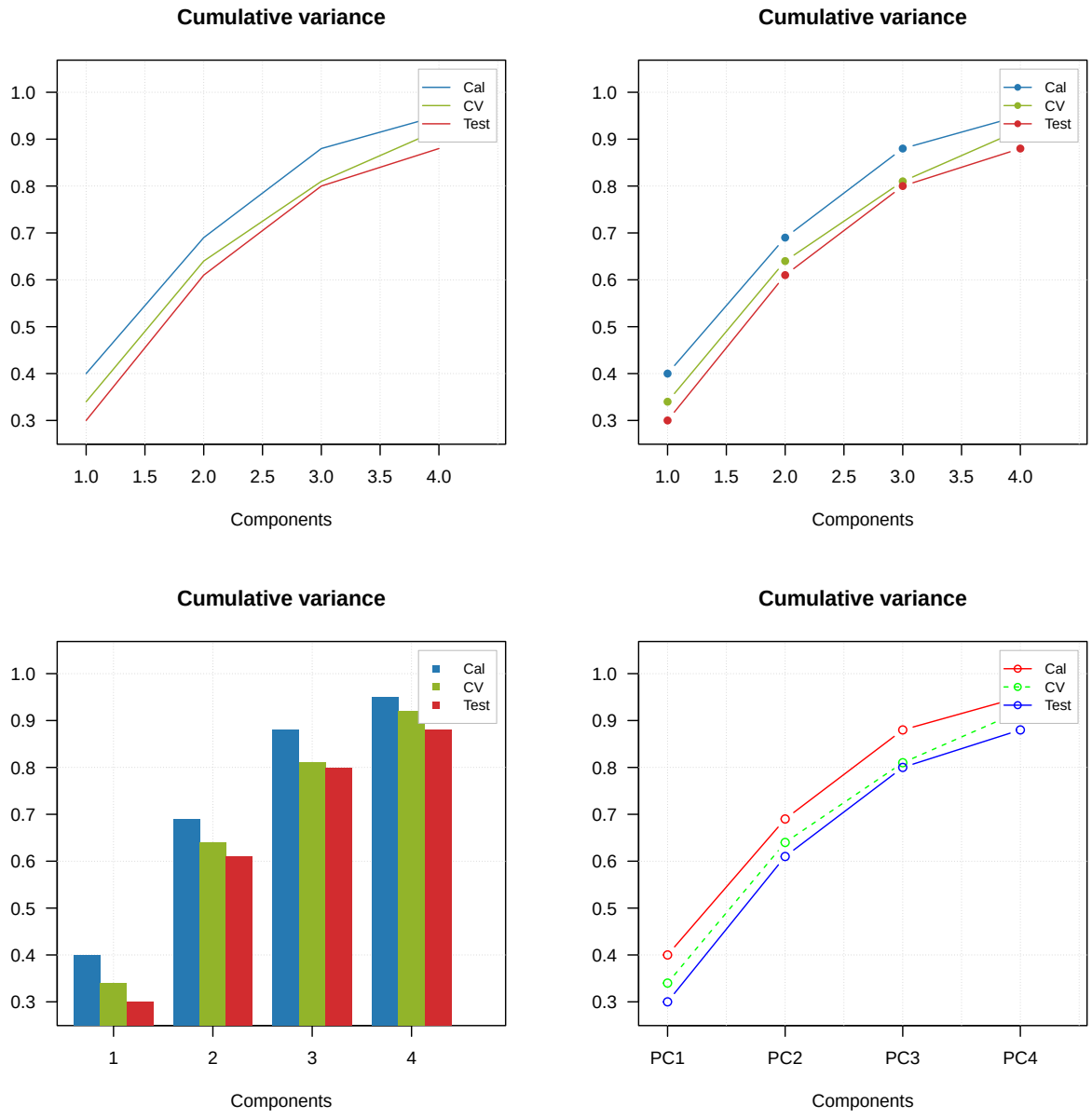
If you want to change any parameters, like `pch`, `lty`, `lwd`, `col` or similar you need to provide either a vector with values for each group or one value for all groups. Axis limits, ticks, ticklabels, etc. can be defined similarly to the simple plots. Here are some examples.

```
# let's create a simple dataset with 3 rows
p = rbind(
   c(0.40, 0.69, 0.88, 0.95),
   c(0.34, 0.64, 0.81, 0.92),
   c(0.30, 0.61, 0.80, 0.88)
)

# add some names and attributes
```

```
rownames(p) = c("Cal", "CV", "Test")
colnames(p) = paste0("PC", 1:4)
attr(p, "name") = "Cumulative variance"
attr(p, "xaxis.name") = "Components"

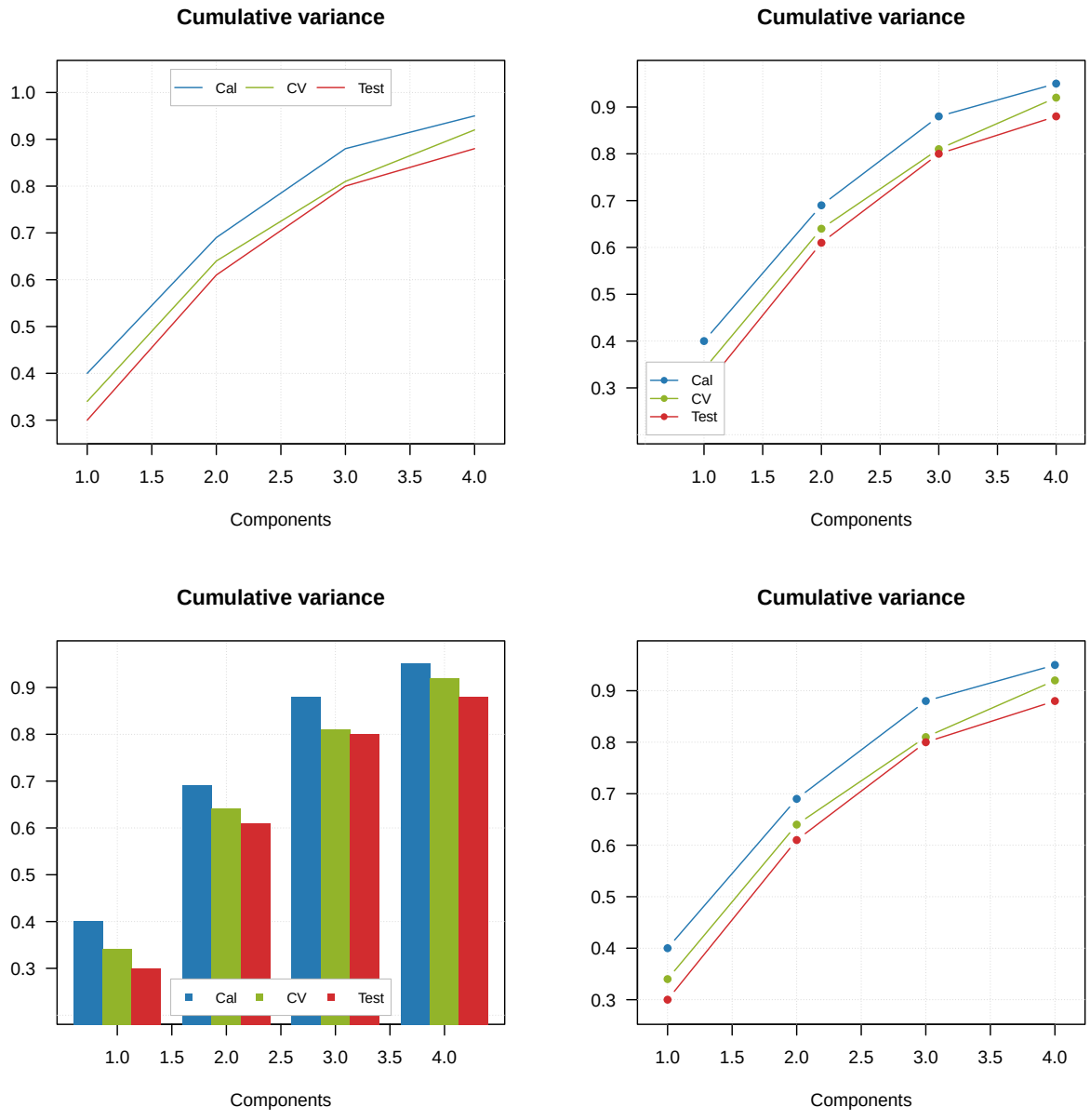
# and make group plots of different types
par(mfrow = c(2, 2))
mdaplotg(p, type = "l")
mdaplotg(p, type = "b")
mdaplotg(p, type = "h", xticks = 1:4)
mdaplotg(p, type = "b", lty = c(1, 2, 1), col = c("red", "green", "blue"), pch = 1,
         xticks = 1:4, xticklabels = colnames(p))
```



As you can see, `mdaplotg()` automatically created the legend and set colors, line parameters, etc. correctly.

You can change position of the legend using same names as for basic `legend()` command from R, or hide it using parameter `show.legend = FALSE`, as it is shown below.

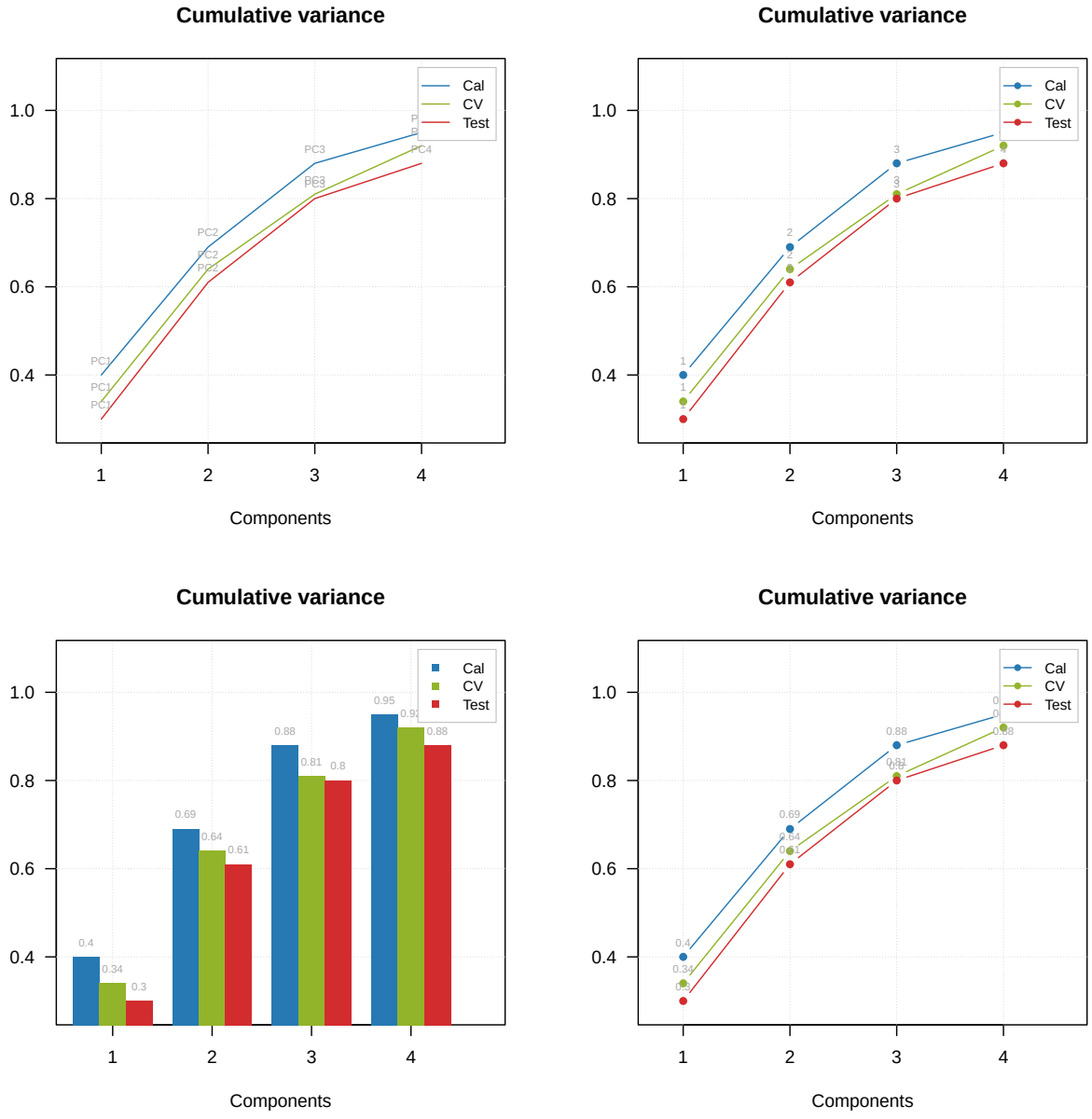
```
par(mfrow = c(2, 2))
mdaplotg(p, type = "l", legend.position = "top")
mdaplotg(p, type = "b", legend.position = "bottomleft")
mdaplotg(p, type = "h", legend.position = "bottom")
mdaplotg(p, type = "b", show.legend = FALSE)
```



Group plots also allow you to show labels, in this case they can be either values, names or indices of the columns.

```
par(mfrow = c(2, 2))
mdaplotg(p, type = "l", show.labels = TRUE)
mdaplotg(p, type = "b", show.labels = TRUE, labels = "indices")
```

```
mdaplotg(p, type = "h", show.labels = TRUE, labels = "values")
mdaplotg(p, type = "b", show.labels = TRUE, labels = "values")
```



List with matrices or data frames

In this case every element of the list will be treated as a separate group. This way allows you to make scatter plots as well as line plots with several lines in each group. Bar plot can also be made but in this case the first row from each dataset will be used. If you use names when creating the list, the names will be taken as legend labels, otherwise the method will look at the attribute `"name"` for each dataset.

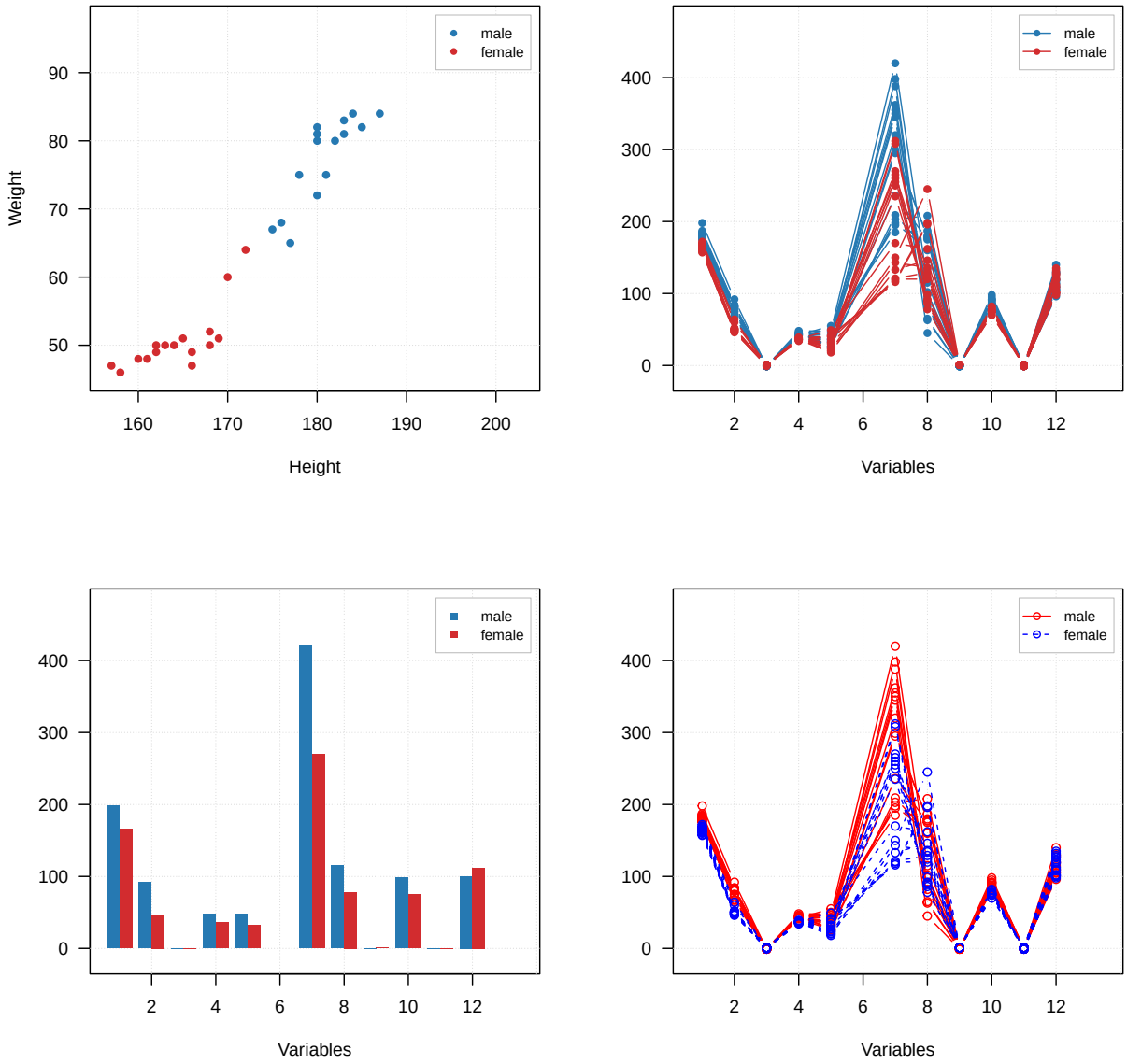
In the example below we split *People* data to males and females and show the group plots.

```
# load data and exclude column with income
data(people)
people = mda.exclcols(people, "Income")
```

```
# use values of sex variable to split data into two subsets
sex = people[, "Sex"]
m = mda.subset(people, subset = sex == -1)
f = mda.subset(people, subset = sex ==  1)

# combine the two subsets into a named list
d = list(male = m, female = f)

# make plots for the list
par(mfrow = c(2, 2))
mdaplotg(d, type = "p")
mdaplotg(d, type = "b")
mdaplotg(d, type = "h")
mdaplotg(d, type = "b", lty = c(1, 2), col = c("red", "blue"), pch = 1)
```



Most of the things described in the previous subsection will work similarly for this case. We will just add a bit more details on how labels and excluded rows are processed for the scatter plots. By default labels are row names or indices. In `mdaplotg()` you cannot provide a vector with manual values, so the best way to change them is to assign them as the row names. Indices are unique within each group, so if you have, e.g. three groups of points, there will be three points with index "1", three with "2", etc.

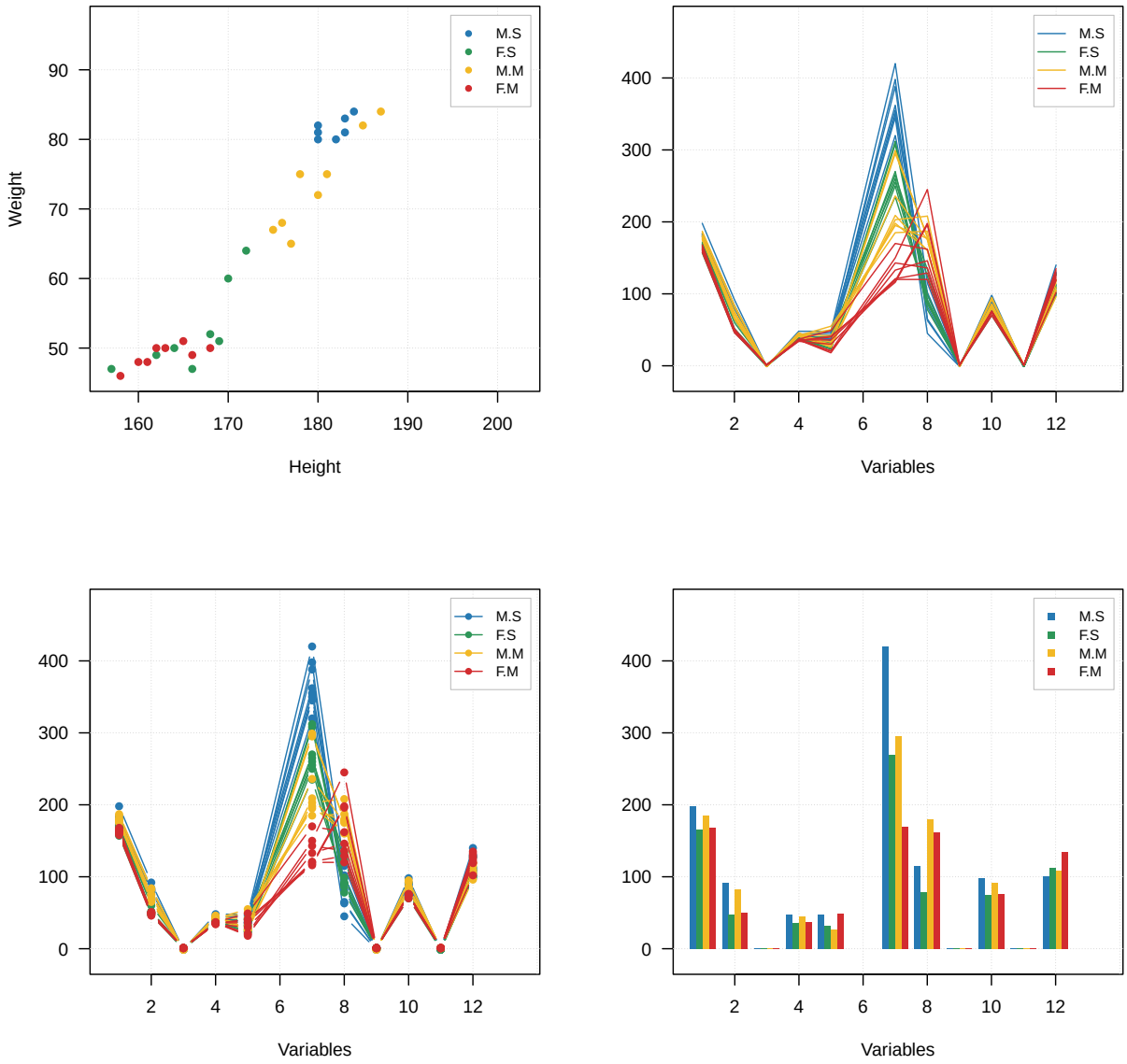
Use factors to split a dataset into groups

One more way to split a dataset into groups is to provide one or several factor columns using argument `groupby`. In this case `mdaplotg()` will find all combinations of the factor levels and split rows of the dataset into the corresponding groups. In the example below we use variables *Region* and *Sex* to make plots for four groups.

It is assumed that you already loaded the People data (from previous examples) and excluded the Income column.

```
sex = factor(people[, "Sex"], labels = c("M", "F"))
reg = factor(people[, "Region"], labels = c("S", "M"))
groups = data.frame(sex, reg)

par(mfrow = c(2, 2))
mdaplotg(people, type = "p", groupby = groups)
mdaplotg(people, type = "l", groupby = groups)
mdaplotg(people, type = "b", groupby = groups)
mdaplotg(people, type = "h", groupby = groups)
```



All parameters, described before, will work the same way in this case.

Working with images

The package also supports images, including hyperspectral images, however they have to be transformed into datasets. The idea is very simple, we keep information about image pixels in an unfolded form, as a matrix, and use attributes `width` and `height` to reshape the data when we need to show it as an image.

There are three methods that make this procedure easier: `mda.im2data()`, `mda.data2im()` and `imshow()`. The first converts an image (represented as 3-way array) to a data set, the second does the opposite, and the third takes a dataset and shows it as an image. In the code chunk below you will see several examples of how the methods work.

We will use a dataset `pellets` available in the package. It is a 3-way array of numbers, if you want to work with e.g. JPEG, PNG or other standard image files you can load them using specific packages (jpeg, png).

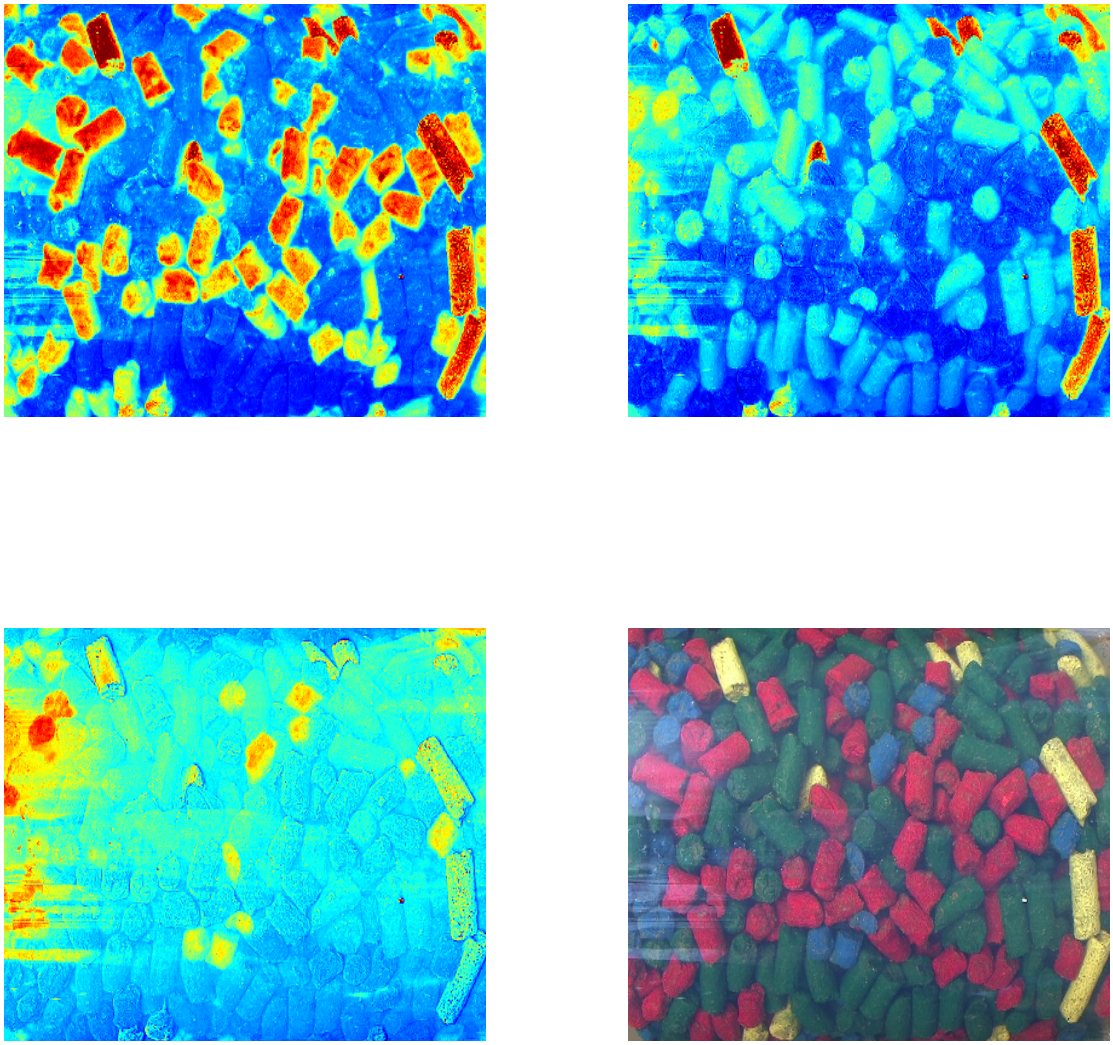
```
data(pellets)

# convert image to a data matrix and add some attributes
d = mda.im2data(pellets)
colnames(d) = c("Red", "Green", "Blue")
attr(d, "name") = "Image"

# show data values
mda.show(d, 10)
```

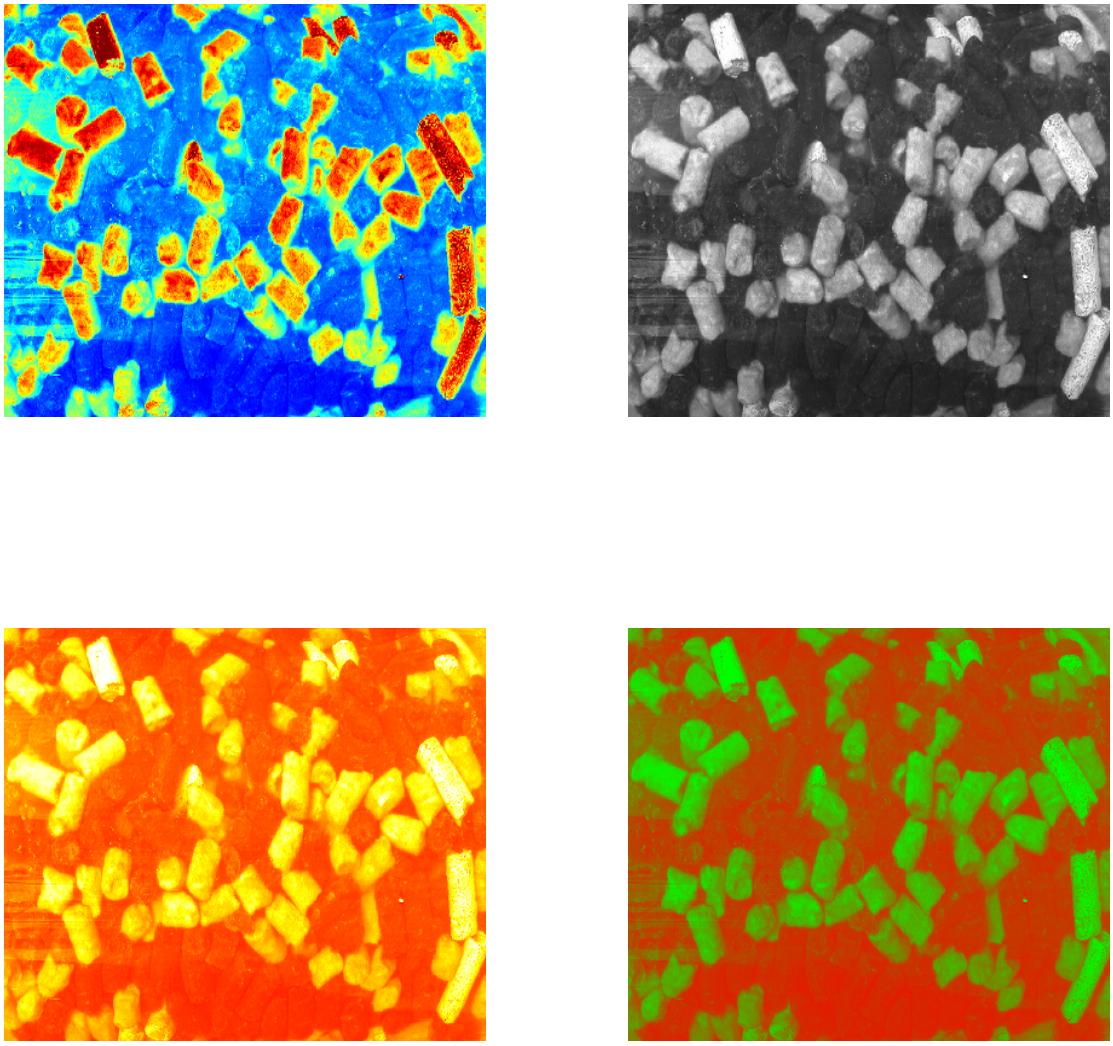
```
## Image
## -----
##       Red Green Blue
##  [1,] 183    80  101
##  [2,] 191    76  105
##  [3,] 187    73   99
##  [4,] 199    81  113
##  [5,] 198    81  110
##  [6,] 197    84  114
##  [7,] 191    83  109
##  [8,] 193    83  110
##  [9,] 188    83  114
## [10,] 172    86  115
```

```
# show separate channels and the whole image in plots
par(mfrow = c(2, 2))
imshow(d, 1)
imshow(d, 2)
imshow(d, 3)
imshow(d, 1:3)
```



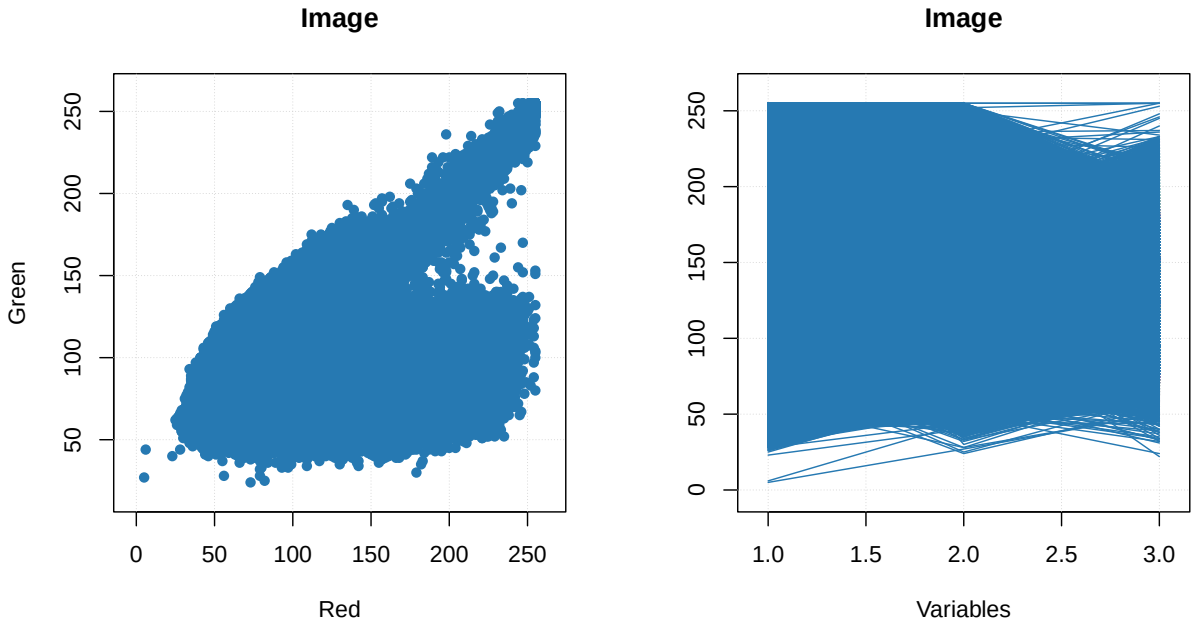
By default, an image for one channel is shown using the jet color palette for intensities, but you can also use gray colors, palette from *colorbrewer2* as well as your own.

```
par(mfrow = c(2, 2))
imshow(d, 1)
imshow(d, 1, colmap = "gray")
imshow(d, 1, colmap = heat.colors(256))
imshow(d, 1, colmap = colorRampPalette(c("red", "green"))(256))
```



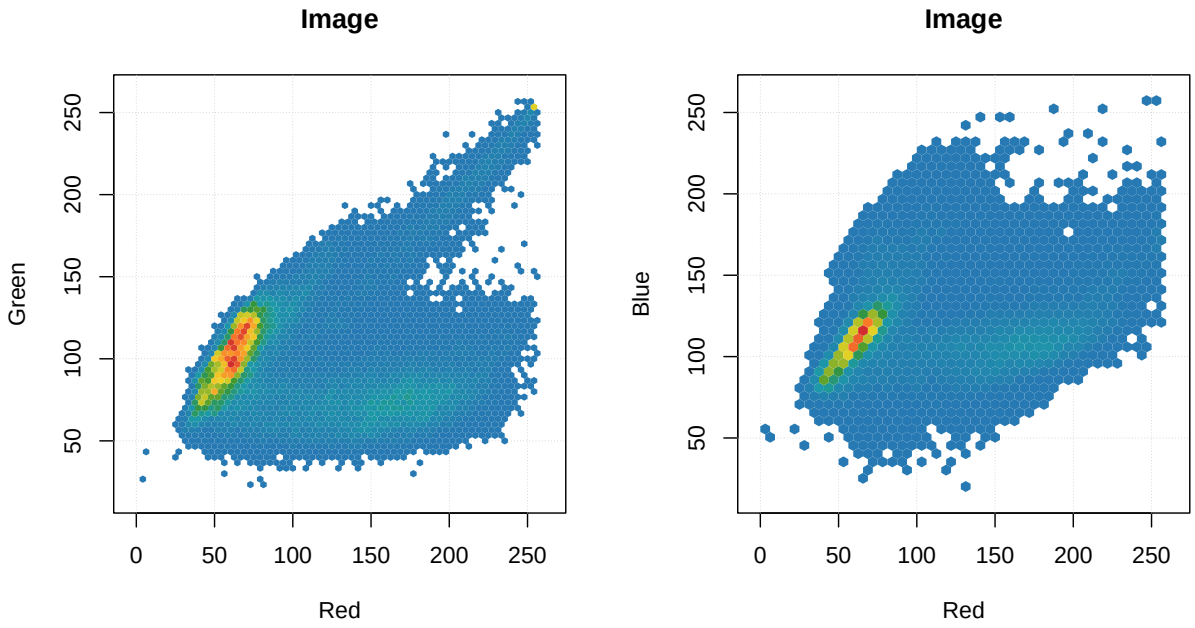
You can work with the image values as a normal dataset and show scatter, line plots, calculate statistics, etc.

```
par(mfrow = c(1, 2))
mdaplot(d, type = "p")
mdaplot(d, type = "l")
```



However, it will take some time to show these plots as this image has several hundred thousand pixels, a faster alternative can be the use of a density plot based on hexagonal binning. Use `type = "d"` for this as shown below. Number of bins can be adjusted by using parameter `nbins`.

```
par(mfrow = c(1, 2))
mdaplot(d, type = "d")
mdaplot(mda.subset(d, select = c("Red", "Blue")), type = "d", nbins = 40)
```



Another useful thing is to set some of the pixels as background. The background pixels are physically removed from the image dataset, there is no way to get them back (in contrast to excluded rows/pixels). It can be

particularly useful when working with e.g. geocorrected hyperspectral images, where, often, many pixels have NA values and there is no need to keep them in memory. To set pixels as background you need to use the method `mda.setimbg()` with either pixel indices or a vector with logical values, as shown below.

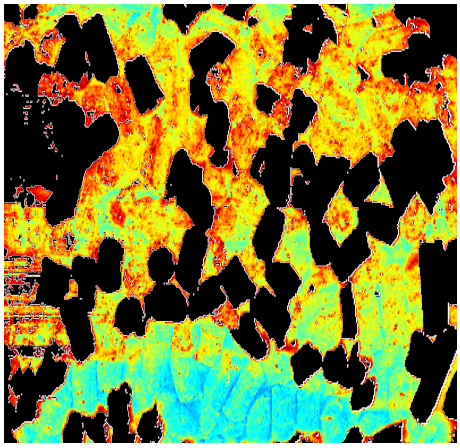
```
# original size
show(dim(d))
```

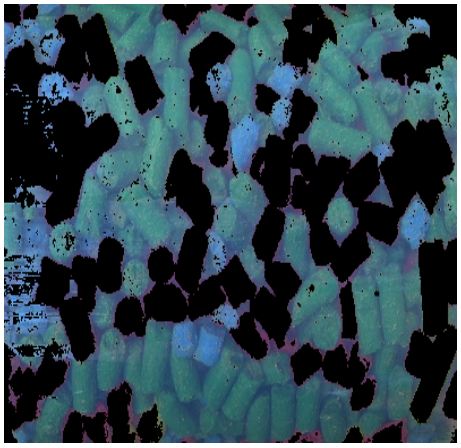
```
## [1] 114000      3
```

```
# set red pixels as background and show new size
d = mda.setimbg(d, d[, "Red"] > 100)
show(dim(d))
```

```
## [1] 66471     3
```

```
# show image with background pixels
par(mfrow = c(1, 2))
imshow(d, 1)
imshow(d, 1:3)
```





All image-related attributes are inherited by all object/row-related results, e.g. scores, residuals, predicted values and classes, etc. This means if you provide an image to any modelling method, you can visualise the corresponding results also as an image. Some examples will be shown in chapter about PCA.

Preprocessing

The package has several preprocessing methods implemented, mostly for different kinds of spectral data (but many of the methods will work with other datasets as well). All functions for preprocessing start with prefix `prep.`, which makes them easier to find by using code completion.

Selected preprocessing methods can be combined into a sequence (*preprocessing chain* or *preprocessing model*), which can be applied to any dataset as a whole. It can also be used as part of a data model (PCA, PLS, DD-SIMCA, etc). In this case the model will apply the preprocessing methods automatically every time you use the model for prediction.

This chapter provides a brief description of the methods with several examples.

Centering and scaling

Centering

Centering makes the center of each variable (column of the dataset) equal to zero, hence moving the center of the data cloud in the variable space to the origin. Usually, the center of values is estimated by computing their arithmetical mean (average) but in some cases the median can be used instead. Mathematically, it is done by subtracting mean/median from the data values separately for every column/variable.

In *mdatools* the centering can be done by using method `prep.center()`. The code chunk below shows how to apply centering to People data using both mean and median as the statistics.

```
library(mdatools)

# load People data
data(people)

# remove several columns which are very different from the others
data0 = people[, -c(6, 7, 8, 9, 11)]

# mean centering
data1 = prep.center(data0)

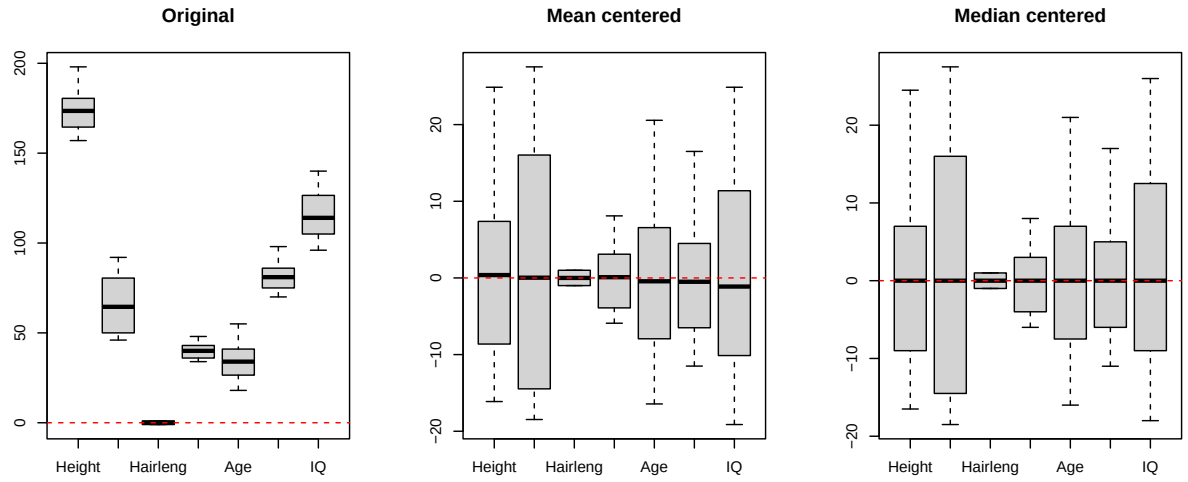
# median centering
data2 = prep.center(data0, type = "median")


# show boxplots
par(mfrow = c(1, 3))

boxplot(data0, main = "Original")
abline(h = 0, lty = 2, col = "red")
```

```
boxplot(data1, main = "Mean centered")
abline(h = 0, lty = 2, col = "red")

boxplot(data2, main = "Median centered")
abline(h = 0, lty = 2, col = "red")
```



Scaling

Scaling is used to unify the spread of the data values. It is done by dividing the data values from each variable by a statistic representing the spread. The most common choice is to use standard deviation as a scaler, in this case scaling is also called *standardization*, it results in variables having standard deviation equal to one (unit). Alternatively, you can also use inter-quartile range (IQR = Q3 - Q1), the full range (max - min), or Pareto scaling when variables are divided by the square root of the standard deviation.

The code below shows how to implement them all (it assumes that you ran the previous code block already and People data is in the current R environment). Please pay attention that the People data is mean centered beforehand, otherwise it will be difficult to see the difference on the plots:

```
# mean center the data for better visualization
data0 = prep.center(people)

# standardization (default option)
data1 = prep.scale(data0)

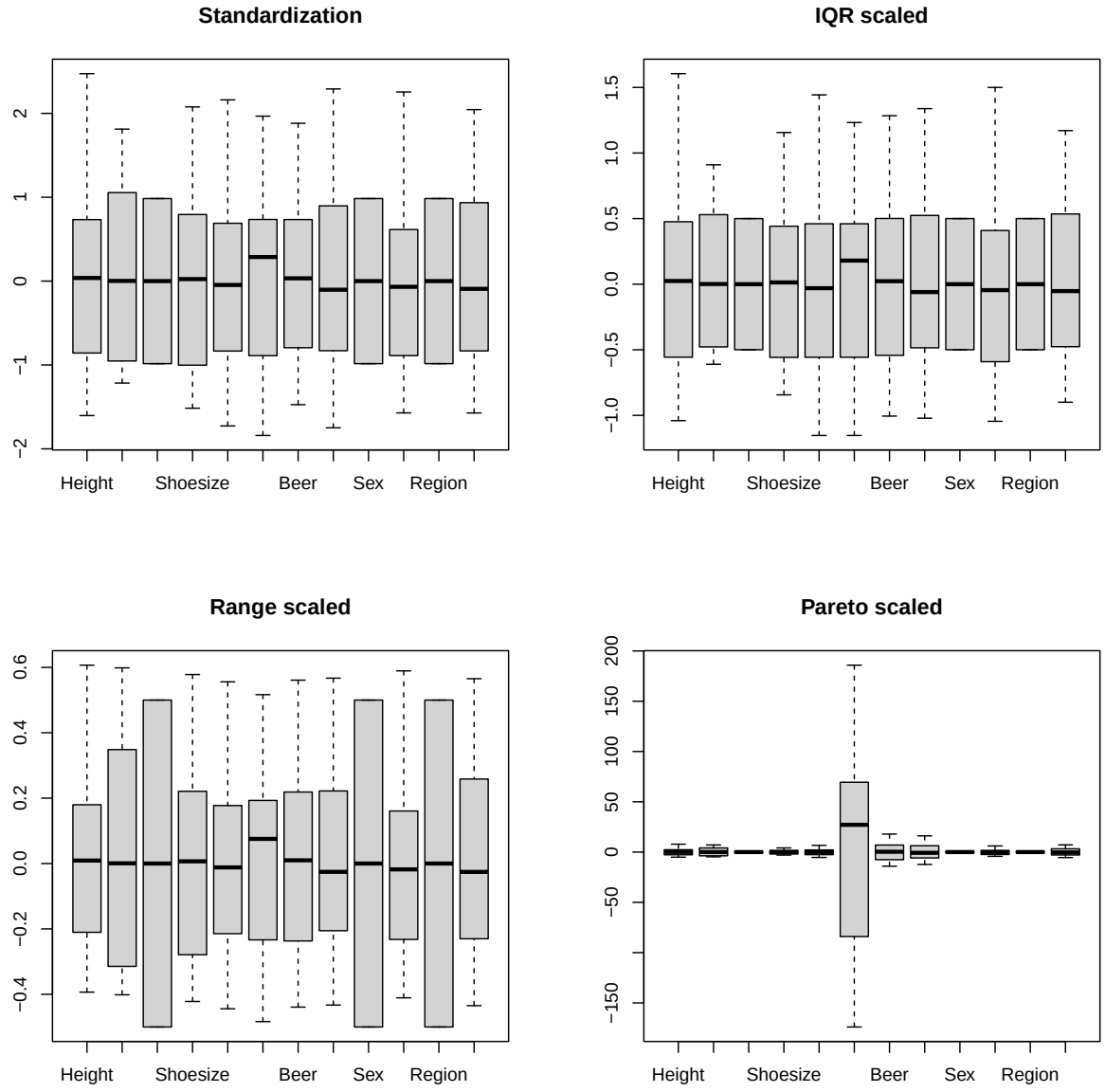
# IQR scaling
data2 = prep.scale(data0, type = "iqr")

# range scaling
data3 = prep.scale(data0, type = "range")

# Pareto scaling
data4 = prep.scale(data0, type = "pareto")


# show boxplots for each case
par(mfrow = c(2, 2))
```

```
boxplot(data1, main = "Standardization")
boxplot(data2, main = "IQR scaled")
boxplot(data3, main = "Range scaled")
boxplot(data4, main = "Pareto scaled")
```



The `prep.scale()` method has also an additional parameter `max.cov`, which helps to avoid scaling of variables with zero or very low variation. The parameter defines a limit for the coefficient of variation in percent `sd(x) / m(x) * 100` and the method will not scale variables with coefficient of variation below this limit. Default value for the parameter is 0 which will prevent scaling of constant variables (which leads to `Inf` or `NA` values).

Correction of spectral baseline

Baseline correction methods include Extended Multiplicative Scatter Correction (EMSC) and correction of

baseline with Asymmetric Least Squares (ALS). Another common method for baseline correction, Standard Normal Variate (SNV), is a part of normalization methods described in the next section.

Extended Multiplicative Scatter Correction

Multiplicative Scatter Correction is a simple procedure aiming first of all at removing additive and multiplicative scatter effects from Vis/NIR spectra. The Extended MSC does this plus corrects non-linear effects of the baseline. The theoretical foundation of the method can be found in this paper. In mdatools the method is implemented as function `prep.emsc()`.

The main parameter of the function is `degree` that defines the polynomial degree used to fit the baseline. If `degree=0`, the EMSC is simplified to the conventional MSC method. Both MSC and EMSC rely on a reference spectrum. Usually a mean spectrum of a training set is used for this purpose. If you do not provide the reference spectrum manually (via parameter `mspectrum`), it will be calculated automatically for all rows of the provided data matrix.

Here are some examples:

```
# set random generator to get reproducible results
set.seed(42)

# load UV/Vis spectra from Simdata
data(simdata)

# add attributes to make plots more informative
w = simdata$wavelength
spectra0 = simdata$spectra.c
attr(spectra0, "xaxis.values") = w
attr(spectra0, "xaxis.name") = "Wavelength, nm"

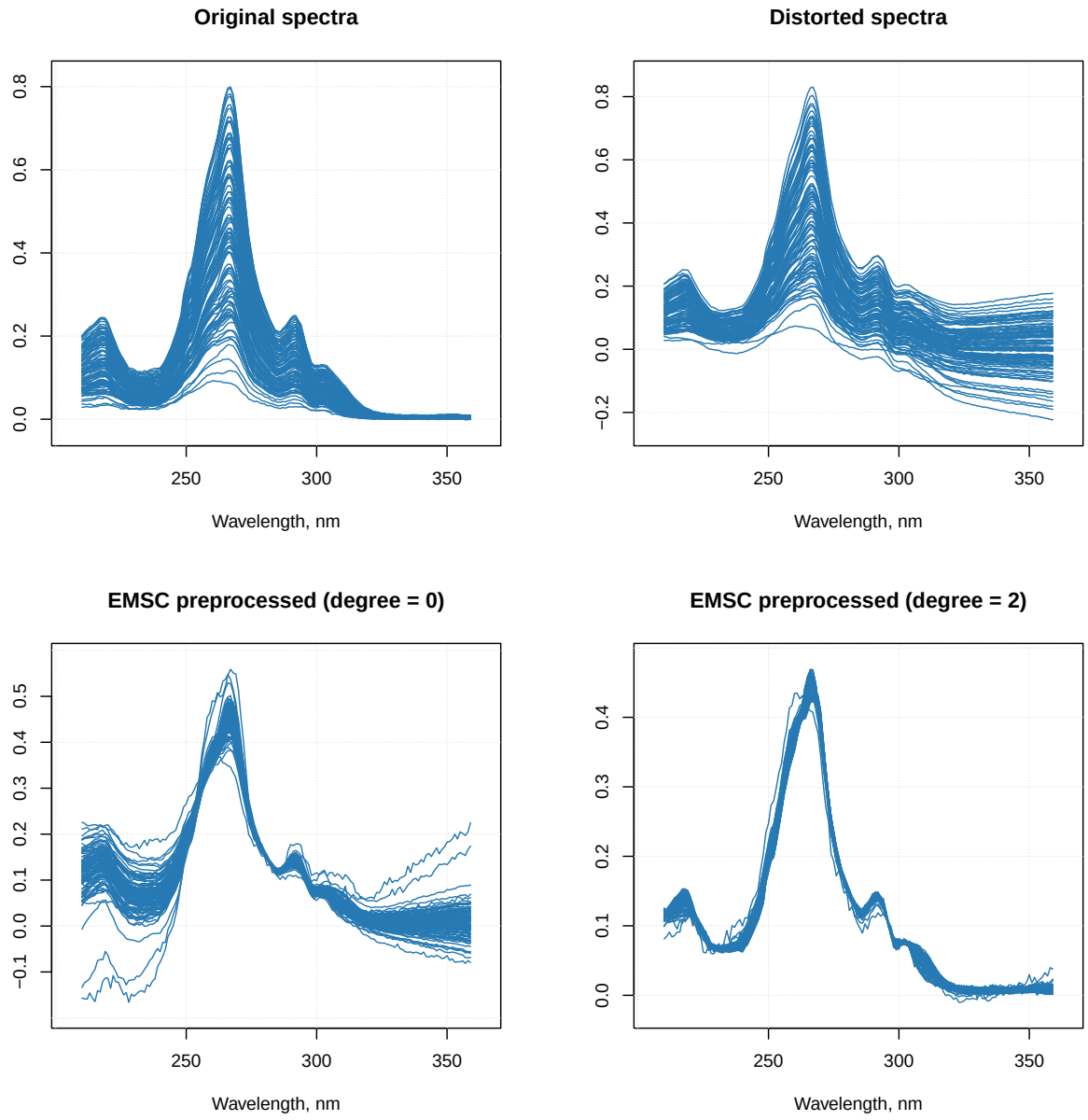
# save number of spectra (rows) and number of wavelength (columns)
n = nrow(spectra0)
m = ncol(spectra0)

# add random additive and multiplicative effects
A = matrix(rnorm(n, 0, 0.0005), n, m)          # multiplicative effect
B = matrix(rnorm(n, 0, 0.01), n, m)            # additive effect
X = matrix(seq_len(m), n, m, byrow = TRUE)     # the baseline
spectra1 = spectra0  + A * X + B

# add attributes to the distorted spectra
attr(spectra1, "xaxis.values") = w
attr(spectra1, "xaxis.name") = "Wavelength, nm"

# correct the spectra using MSC and EMSC with quadratic polynomial
spectra2 = prep.emsc(spectra1)
spectra3 = prep.emsc(spectra1, degree = 2)

# show the original, distorted and the preprocessed spectra
par(mfrow = c(2, 2))
mdaplot(spectra0, type = "l", main = "Original spectra")
mdaplot(spectra1, type = "l", main = "Distorted spectra")
mdaplot(spectra2, type = "l", main = "EMSC preprocessed (degree = 0)")
mdaplot(spectra3, type = "l", main = "EMSC preprocessed (degree = 2)")
```



Asymmetric least squares

Asymmetric least squares (ALS) baseline correction allows correcting baseline issues, which have a wider shape compared to the characteristic peaks. It can be used for example to correct the fluorescence effect in Raman spectra.

The method is based on Whittaker smoother and was proposed in this paper. It is implemented as a function `prep.alsbasecorr()`, which has two main parameters — power of a penalty parameter (`plambda`, usually varies between 2 and 9) and the ratio of asymmetry (`p`, usually between 0.1 and 0.001). For example, if `plambda = 5`, the penalty parameter λ, described in the paper will be equal to 10^5.

The choice of the parameters depends on how broad the disturbances of the baseline are and how narrow the original spectral peaks are. In the example below we took original spectra from the `carbs` dataset, added baseline disturbance using broad Gaussian peaks and then tried to remove the disturbance by applying the `prep.alsbasecorr()`. The result is shown in the form of plots.

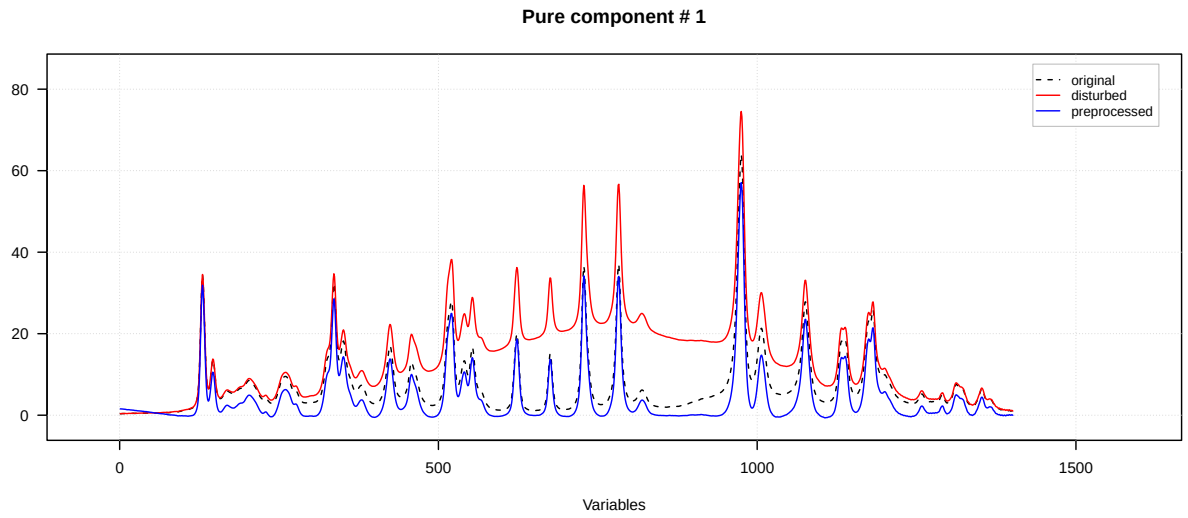
```r
library(mdatools)
data(carbs)

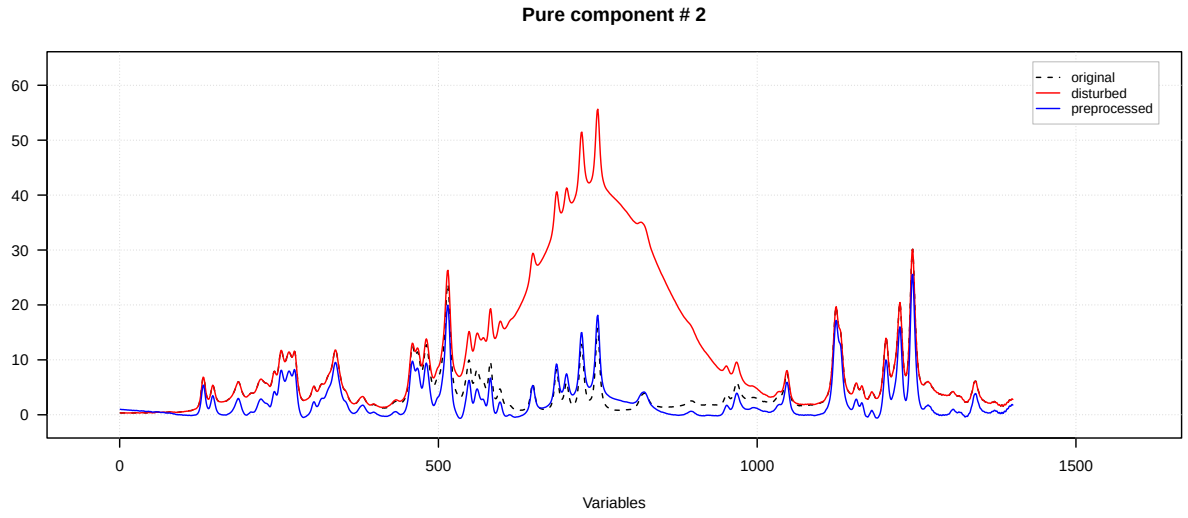
# take original spectra from carbs dataset
x <- t(carbs$S)

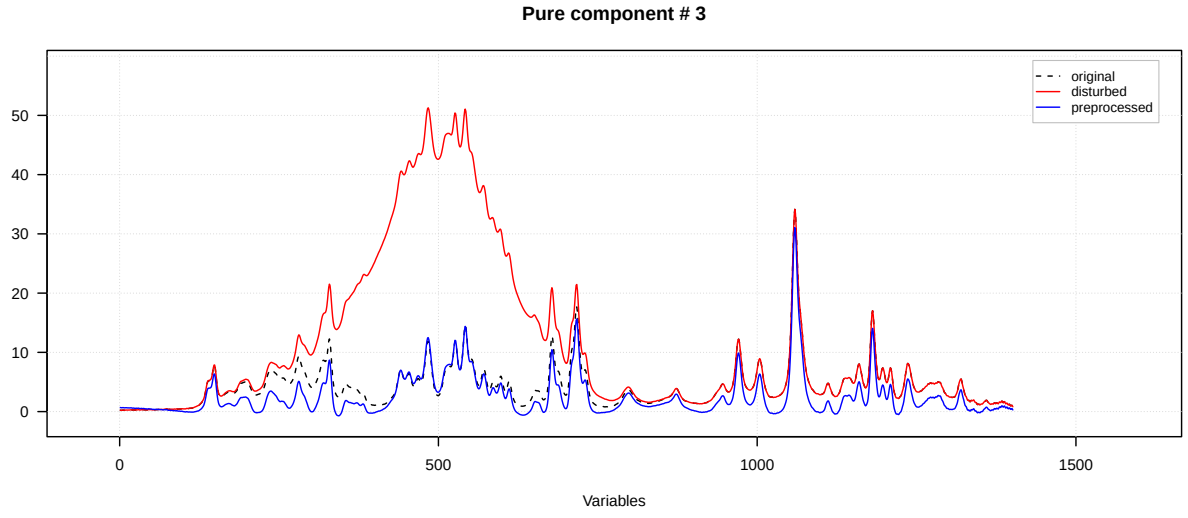
# add disturbance to the baseline by using broad Gaussian peaks
y <- x + rbind(
   dnorm(1:ncol(x), 750, 200) * 10000,
   dnorm(1:ncol(x), 750, 100) * 10000,
   dnorm(1:ncol(x), 500, 100) * 10000
)

# preprocess the disturbed spectra using ALS baseline correction
y.new <- prep.alsbasecorr(y, plambda = 5, p = 0.01)

# show the original, disturbed and the preprocessed spectra separately for each component
par(mfrow = c(3, 1))
for (i in 1:3) {
   mdaplotg(list(
      original = x[i, , drop = FALSE],
      disturbed = y[i, , drop = FALSE],
      preprocessed = y.new[i, , drop = FALSE]
      ), type = "l", lty = c(2, 1, 1), col = c("black", "red", "blue"),
      main = paste("Pure component #", i)
   )
}
```







As one can notice, the blue curves with corrected spectra are pretty similar to the original spectra shown as

dashed black curves.

Normalization

Normalization is a preprocessing method which is applied to rows of the dataset (e.g. individual spectra or abundance values for individual measurements) in order to make all rows meet the same requirement. Depending on the requirement, different normalization types exist. In *mdatools* the following methods are implemented:

- `"area"` — normalize every row to unit area under measurement points. The area is computed as a sum of absolute values from each row.
- `"sum"` — makes all values of each row sum up to one (similar to `"area"` but it takes sum of the original values, not the absolute ones).
- `"length"` — normalize every row, so if a row is represented as a vector in variable space, this vector will have a unit Euclidean length.
- `"snv"` — Standard Normal Variate, makes all values from the same row have zero mean and unit standard deviation (is also described in the previous section as this normalization is often used for correction of baseline in spectral data).
- `"is"` — Internal standard normalization. This is common for spectroscopic data, the values from each row will be normalized so the value for a given variable (or sum of values for several variables) are equal to one. Usually the variable corresponds to characteristic peak of an internal standard.
- `"pqn"` — Probabilistic Quotient Normalization, a method described in this paper.

All methods are implemented in function `prep.norm`; you just need to provide the name of the method as a second argument (or named argument `type`). Internal standard normalization also requires an additional argument, which specifies index of column (or several columns), corresponding to the internal standard. The Probabilistic Quotient Normalization requires reference spectrum as an additional argument.

An example below shows how to make normalization for several selected types using the Simdata:

```
# get spectral data and wavelength
data(simdata)
w = simdata$wavelength

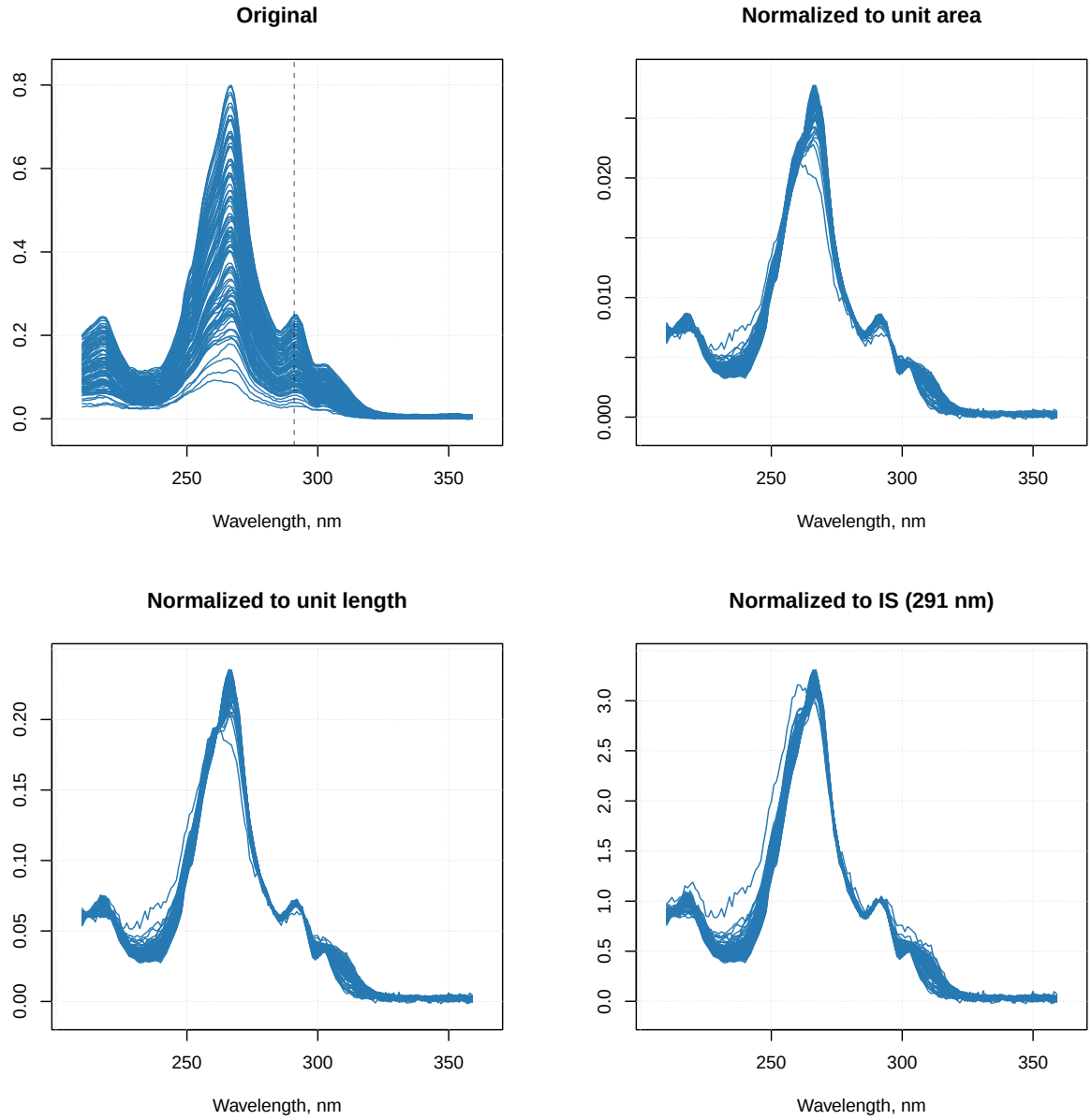
X1 = simdata$spectra.c
attr(X1, "xaxis.values") = w
attr(X1, "xaxis.name") = "Wavelength, nm"

# unit area normalization
X2 = prep.norm(X1, "area")
# unit length normalization
X3 = prep.norm(X1, "length")
# internal standard normalization for variable (column) with wavelength 291 nm
X4 = prep.norm(X1, "is", col.ind = match(291, w))

# show the original and preprocessed spectra
par(mfrow = c(2, 2))

mdaplot(X1, type = "l", main = "Original")
abline(v = 291, lty = 2, col = "#00000080")

mdaplot(X2, type = "l", main = "Normalized to unit area")
mdaplot(X3, type = "l", main = "Normalized to unit length")
mdaplot(X4, type = "l", main = "Normalized to IS (291 nm)")
```



The vertical dashed line on the first plot shows position of the peak, which we use as internal standard for preprocessing shown on the last plot.

Removing cosmic spikes

Cosmic spikes are intense, spurious noise artifacts caused by high-energy cosmic ray particles impacting the CCD detector. They appear as random, single-pixel wide features, not related to the sample, and are especially common in long-duration measurements in Raman spectroscopy, where the main signal is weak and hence a very sensitive CCD detector is employed.

These spikes can be removed using various algorithms. In *mdatools* a simple algorithm based on median absolute deviation is implemented. The theory can be found in this paper. In short, it has two parameters, `width` — width of the filter, an odd number, and `threshold`, which defines how sensitive the method is (how big a z-score should be in order to consider it as a spike).

The method is implemented as function `prep.spikes()`, the code below shows how to use it.

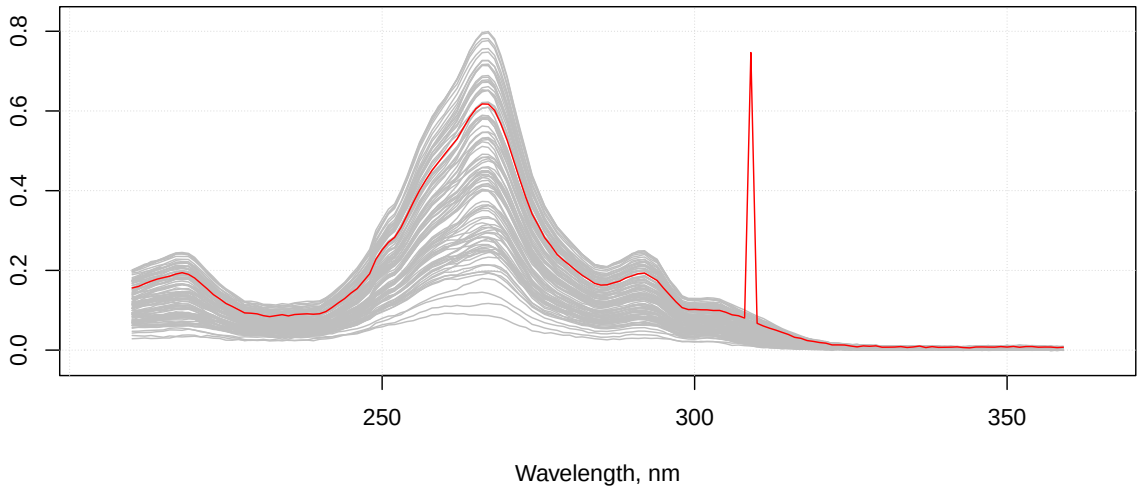
```
# load spectra from the Simdata and add some attributes
data(simdata)
spectra0 <- simdata$spectra.c
attr(spectra0, "xaxis.values") <- simdata$wavelength
attr(spectra0, "xaxis.name") <- "Wavelength, nm"

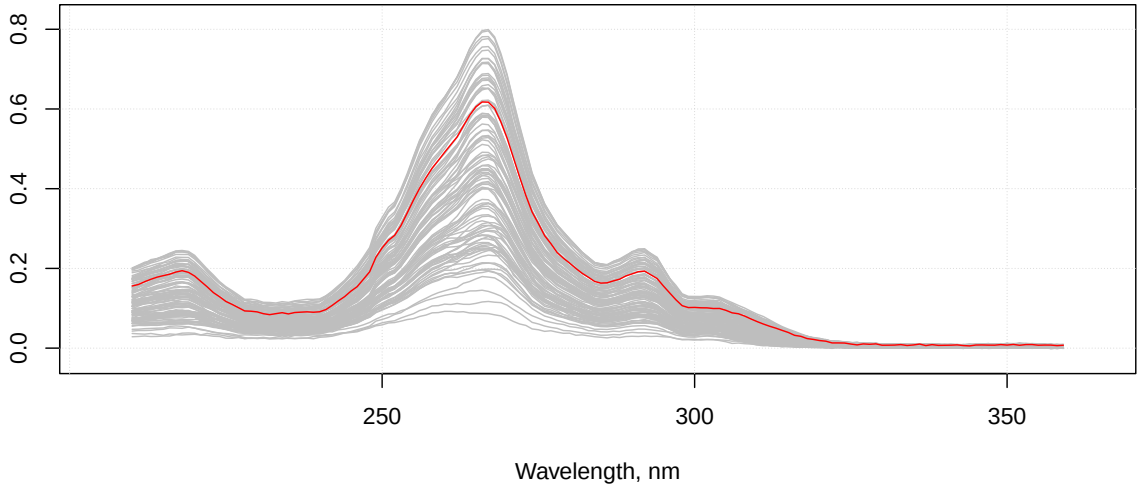
# make a copy and add artificial "spike"
spectra1 <- spectra0
spectra1[100, 100] <- spectra1[100, 100] * 10

# apply spike removal algorithm
spectra2 <- prep.spikes(spectra1, width = 11, threshold = 8)


# define colors for plots
col = rep("gray", nrow(spectra1))
col[100] <- "red"

par(mfrow = c(2, 1))
mdaplot(spectra1, type = "l", col = col)
mdaplot(spectra2, type = "l", col = col)
```





Smoothing and derivatives

The Savitzky-Golay filter is used to smooth signals and calculate derivatives. The filter has three arguments: a width of the filter (`width`), a polynomial order (`porder`) and the derivative order (`dorder`). If the derivative order is zero (default value) only smoothing will be performed.

The next chunk of code takes the spectra from *Simdata*, adds additional random noise using random number generator for normal distribution and then applies the SG filter with different settings. The results are shown as plots under the chunk.

```
# load UV/Vis spectra from Simdata
data(simdata)
ospectra = simdata$spectra.c
attr(ospectra, "xaxis.values") = simdata$wavelength
```

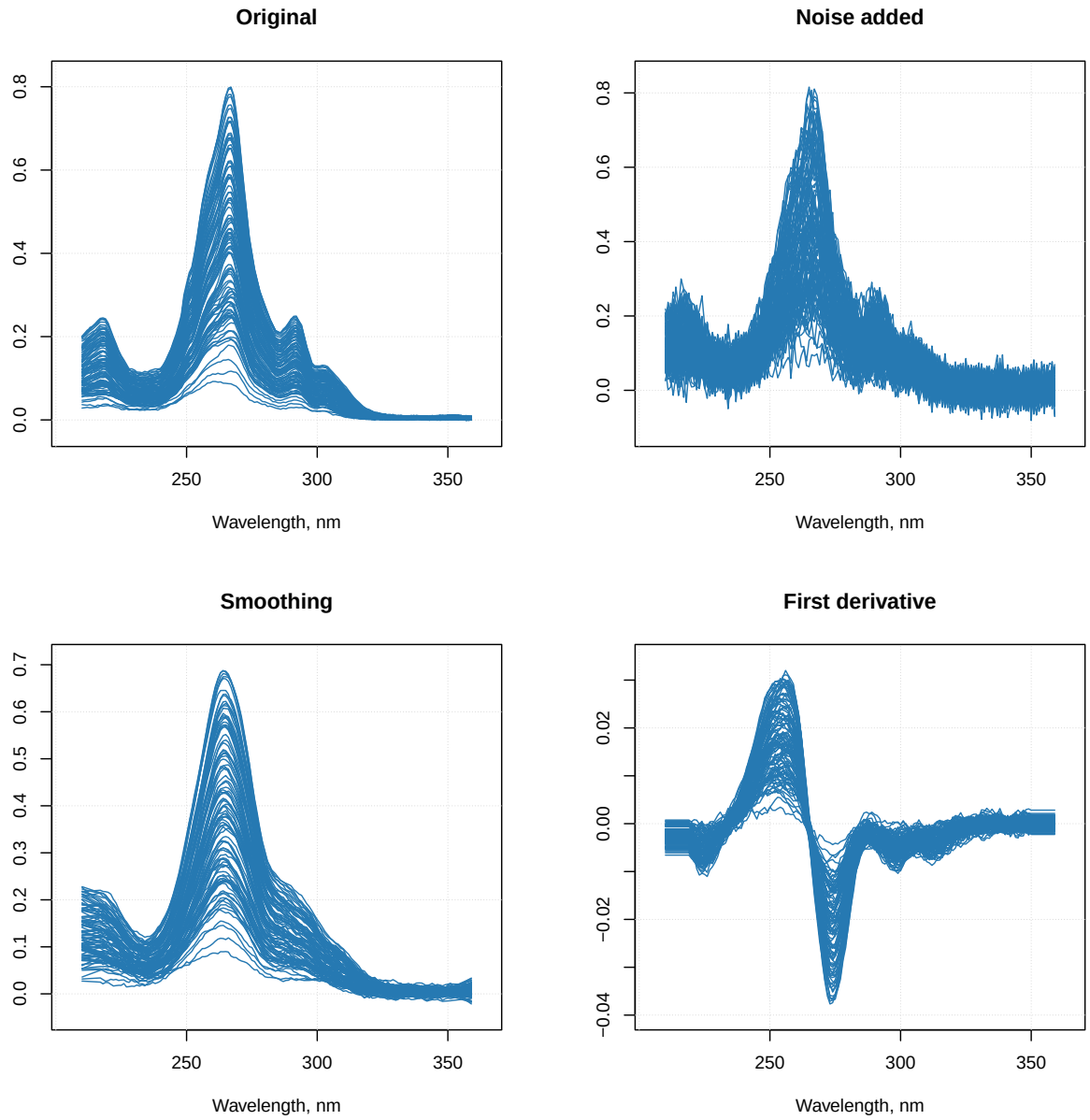
```
attr(ospectra, "xaxis.name") = "Wavelength, nm"

# add random noise to the spectra
nspectra = ospectra + 0.025 * matrix(rnorm(length(ospectra)), dim(ospectra))

# apply SG filter for smoothing
pspectra = prep.savgol(nspectra, width = 15, porder = 1)

# apply SG filter for smoothing and take a first derivative
dpspectra = prep.savgol(nspectra, width = 19, porder = 1, dorder = 1)

# show results
par(mfrow = c(2, 2))
mdaplot(ospectra, type = "l", main = "Original")
mdaplot(nspectra, type = "l", main = "Noise added")
mdaplot(pspectra, type = "l", main = "Smoothing")
mdaplot(dpspectra, type = "l", main = "First derivative")
```



Starting from *v.0.12.0* the algorithm has been modified in order to treat the end points better (for example, when a derivative is taken, the end points may look a bit weird and have to be truncated). The implemented algorithm is based on the method described in this article.

Variable selection as preprocessing method

Variable selection can be done by using `mda.exclcols()`, which simply hides variables/columns, which must not be taken into account in calculations, or by `mda.subset()` which selects only desired columns and removes the rest. Both methods preserve all additional attributes assigned to the data.

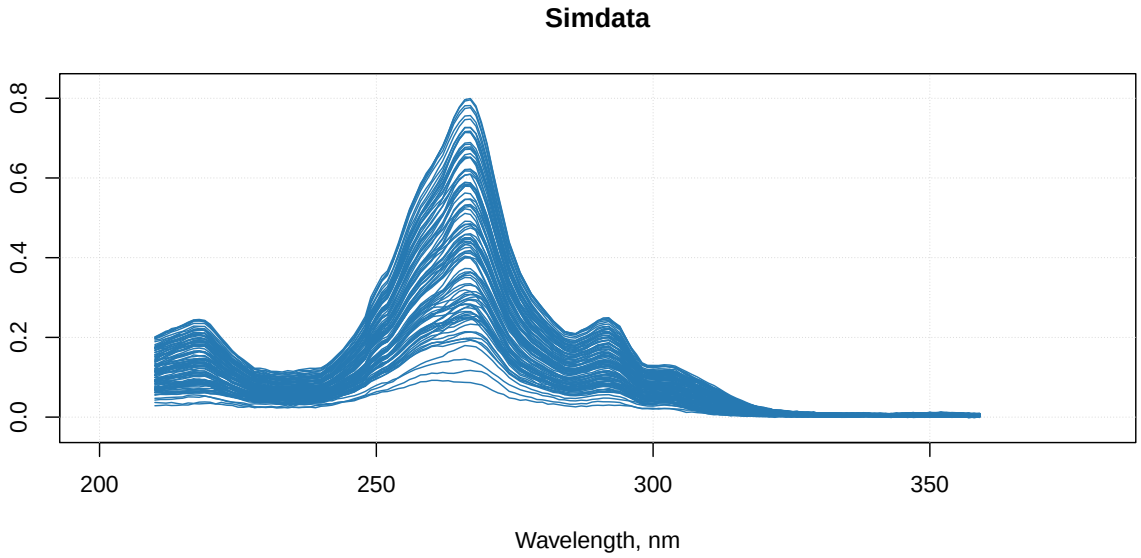
The method `prep.varsel()` is simply a wrapper, which allows selection of only desired variables (similar to `mda.subset()`) but can also be incorporated into preprocessing workflow (see next section for details). In the example below it is used to trim tails of spectra.

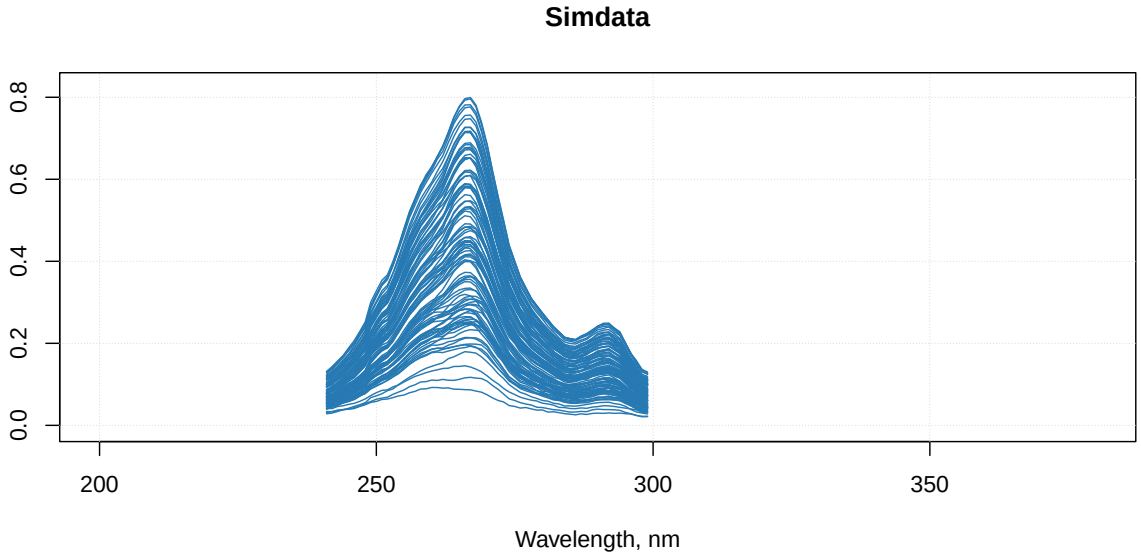
```r
# load spectra from the Simdata and add some attributes
data(simdata)
X <- simdata$spectra.c
w <- simdata$wavelength

attr(X, "xaxis.values") <- w
attr(X, "xaxis.name") <- "Wavelength, nm"
attr(X, "name") <- "Simdata"

# apply variable selection as preprocessing
col.ind = w > 240 & w < 300
Y <- prep.varsel(X, col.ind)

# show both original and preprocessed spectra
par(mfrow = c(2, 1))
mdaplot(X, type = "l", xlim = c(200, 380))
mdaplot(Y, type = "l", xlim = c(200, 380))
```





Combining methods together

All methods mentioned above (there are more, which are discussed later in this section) can be combined into a preprocessing chain, which is just an R object (list). After that this chain can be "trained" (for example if methods have parameters that depend on training set, like centering, scaling or EMSC) and be applied as a whole to any dataset. You can also provide such a list of preprocessing methods as parameters for most of the models.

It can be handy if you need to apply the same chain of methods to different datasets, so you can combine them once, train their parameters and use it as many times as needed. In this section we discuss how to do this.

First of all, you can see the list of the preprocessing methods available for this feature as well as all necessary information about them if you run `prep.list()` as shown below:

```
prep.list()
```

```
##
##
## List of available preprocessing methods:
##
##
##  Select user-defined variables (columns of dataset).
##  ---------------
##  name: 'varsel'
##  parameters:
##   'var.ind': indices of variables (columns) to select.
##
##
##  Normalization.
##  ---------------
##  name: 'norm'
##  parameters:
##   'type': type of normalization ('area', 'sum', 'length', 'is', 'snv', 'pqn').
##   'col.ind': indices of columns (variables) for normalization to internal standard peak.
##   'ref.spectrum': reference spectrum for PQN normalization.
##
##
##  Savitzky-Golay filter.
##  ---------------
##  name: 'savgol'
##  parameters:
##   'width': width of the filter.
##   'porder': polynomial order.
##   'dorder': derivative order.
##
##
##  Asymmetric least squares baseline correction.
##  ---------------
##  name: 'alsbasecorr'
##  parameters:
##   'plambda': power of the penalty parameter (e.g. if plambda = 5, lambda = 10^5)
##   'p': asymmetry ratio (should be between 0 and 1)
##   'max.niter': maximum number of iterations
##
##
##  Removes cosmic spikes.
##  ---------------
##  name: 'spikes'
##  parameters:
##   'width': width of the moving median filter
##   'threshold': threshold for spike detection
##
##
##  Center dataset columns.
##  ---------------
##  name: 'center'
##  parameters:
##   'type': what to use for centering ('mean', or 'median')
```

```
##
##
##  Scale dataset columns.
##  ----------------
##  name: 'scale'
##  parameters:
##   'type': what to use for scaling ('sd', 'iqr', 'range', or 'pareto')
##
##
##  Multiplicative scatter correction.
##  ----------------
##  name: 'emsc'
##  parameters:
##   'degree': polynomial degree (0 for MSC)
##   'mspectrum': reference spectrum (if NULL mean spectrum will be used).
```

What you need to know is the name of the method and which parameters you can or want to provide (if you do not specify any parameters, default values will be used instead).

Let's start with a simple example, where we want to take spectra from *Simdata*, and, first of all, apply Savitzky-Golay with filter width = 5, polynomial degree = 2 and derivative order = 1. Then we want to normalize them using SNV and get rid of all spectral values from 300 nm and above. Here is how to make a preprocessing object for this sequence:

```
data(simdata)
w <- simdata$wavelength

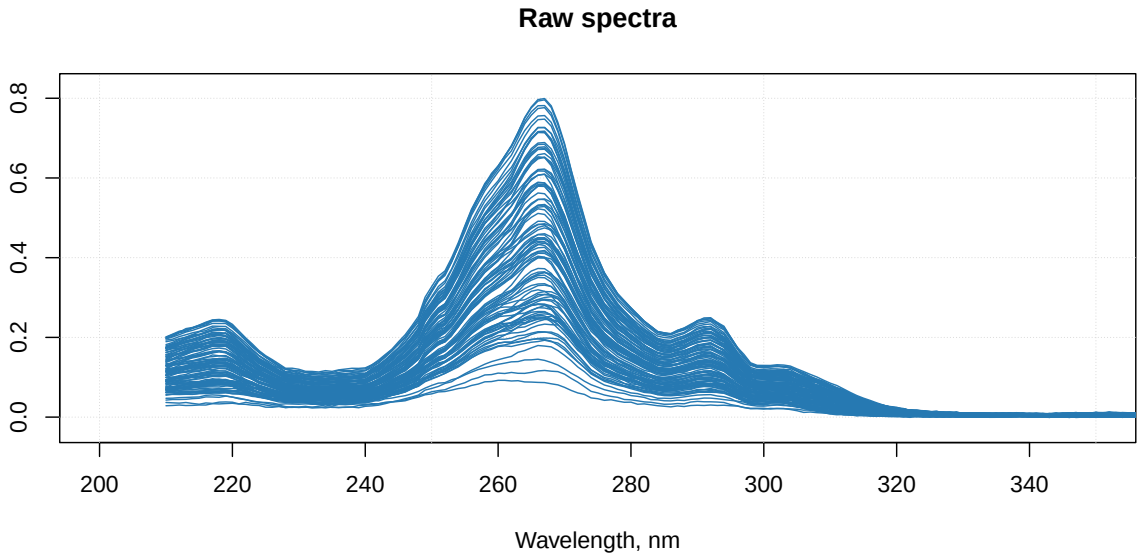
myprep <- list(
   prep("savgol", width = 5, porder = 2, dorder = 1),
   prep("norm", type = "snv"),
   prep("varsel", var.ind = w < 300)
)
```

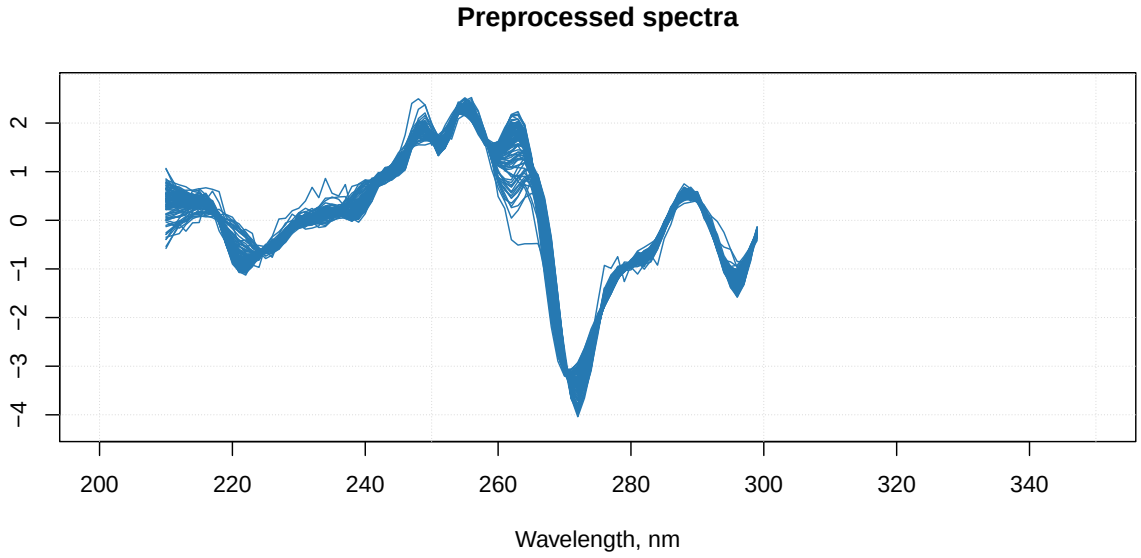
Now you can apply the whole sequence to any spectral data by using function `prep.apply()`:

```
Xc <- simdata$spectra.c
attr(Xc, "xaxis.values") <- w
attr(Xc, "xaxis.name") <- "Wavelength, nm"

Xcp <- prep.apply(myprep, Xc)

par(mfrow = c(2, 1))
mdaplot(Xc, type = "l", xlim = c(200, 350), main = "Raw spectra")
mdaplot(Xcp, type = "l", xlim = c(200, 350), main = "Preprocessed spectra")
```





Now let's consider another example, where we have a calibration set and a test set. We need to create preprocessing sequence for the calibration set and, let's say, we ended up with the following sequence:

- Smoothing with linear Savitzky-Golay filter (`width = 9`)
- MSC correction (EMSC with `degree = 0`)
- Normalization to unit area
- Removing the part with wavelength > 300 nm
- Mean centering

We have an issue here. Both centering and EMSC methods require additional parameters which are computed based on the training set. EMSC needs mean spectrum, which will be used as a reference. And centering needs values for centering. This issue can be solved by using method `prep.fit()`:

```
# define calibration and test sets
Xc <- simdata$spectra.c
Xt <- simdata$spectra.t
```

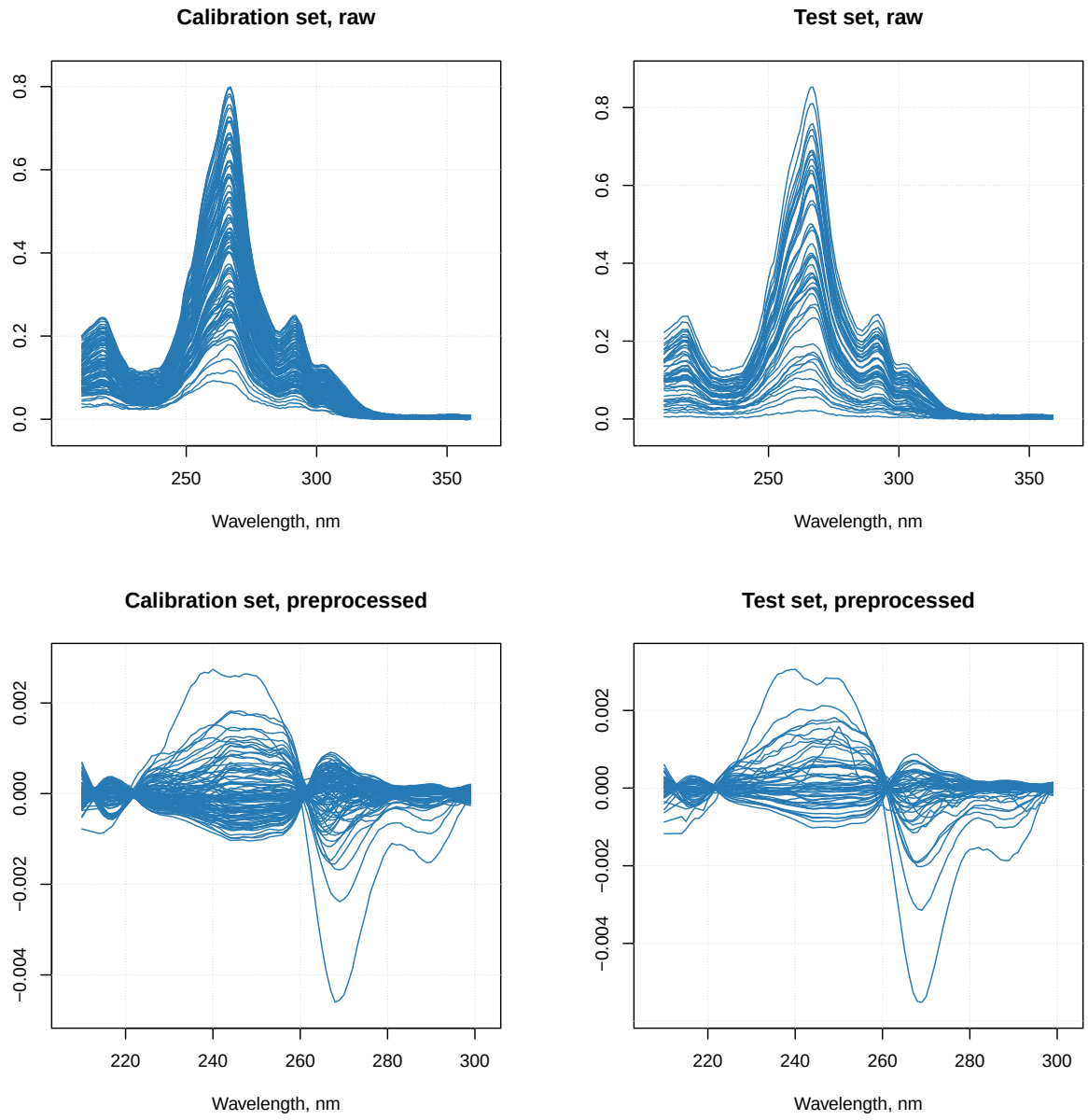
```r
w <- simdata$wavelength
attr(Xt, "xaxis.values") <- attr(Xc, "xaxis.values") <- w
attr(Xt, "xaxis.name") <- attr(Xc, "xaxis.name") <- "Wavelength, nm"

# create a sequence of preprocessing methods
myprep <- list(
   prep("savgol", width = 9, porder = 1),
   prep("emsc", degree = 0),
   prep("norm", type = "area"),
   prep("varsel", var.ind = w < 300),
   prep("center")
)

# fit the preprocessing model - will estimate all needed parameters based on Xc
myprep <- prep.fit(myprep, Xc)

# apply the fitted model to both sets
Xcp <- prep.apply(myprep, Xc)
Xtp <- prep.apply(myprep, Xt)

par(mfrow = c(2, 2))
mdaplot(Xc, type = "l", main = "Calibration set, raw")
mdaplot(Xt, type = "l", main = "Test set, raw")
mdaplot(Xcp, type = "l", main = "Calibration set, preprocessed")
mdaplot(Xtp, type = "l", main = "Test set, preprocessed")
```



The fitted preprocessing model can be saved to an RData file as any other R object.

Interoperability with web-applications

The *mdatools* package works nicely together with interactive web-applications available at https://mda.tools. There is a dedicated app, https://mda.tools/prep, which lets you create a preprocessing chain, train its parameters and apply it to new data. If you open the app and play with it you will notice that it contains all preprocessing methods mentioned above.

Any preprocessing model created using `prep.fit()` like in the example shown in the previous section can be saved into a JSON file and then be loaded into the *Apply* tab of the web-application. Here is an example:

```
# define calibration set
Xc <- simdata$spectra.c
w <- simdata$wavelength
```

```
attr(Xc, "xaxis.values") <- w
attr(Xc, "xaxis.name") <- "Wavelength, nm"

# create a sequence of preprocessing methods
myprep <- list(
   prep("savgol", width = 9, porder = 1),
   prep("emsc", degree = 0),
   prep("norm", type = "area"),
   prep("varsel", var.ind = w < 300),
   prep("center")
)

# train the preprocessing model
myprep <- prep.fit(myprep, Xc)

# save the model to JSON file
writeJSON(myprep, "prep-model.json")
```

If you run the code on your computer you will see a new `prep-model.json` file created in the working directory. Now open https://mda.tools/prep in your browser, select tab *Apply* and load the file with the preprocessing model. You will see something similar to the following:

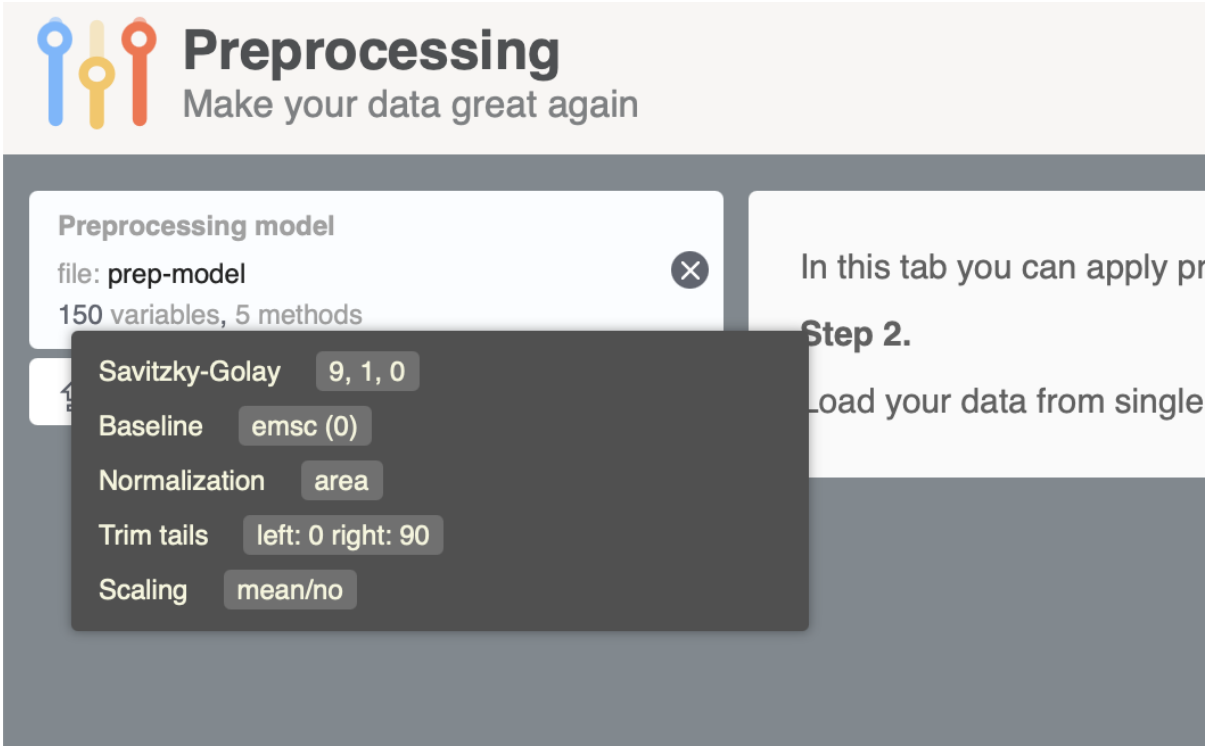


Figure 1: Screenshot with preprocessing model loaded to mda.tools/prep

After that you can upload dataset and preprocess it in the app.

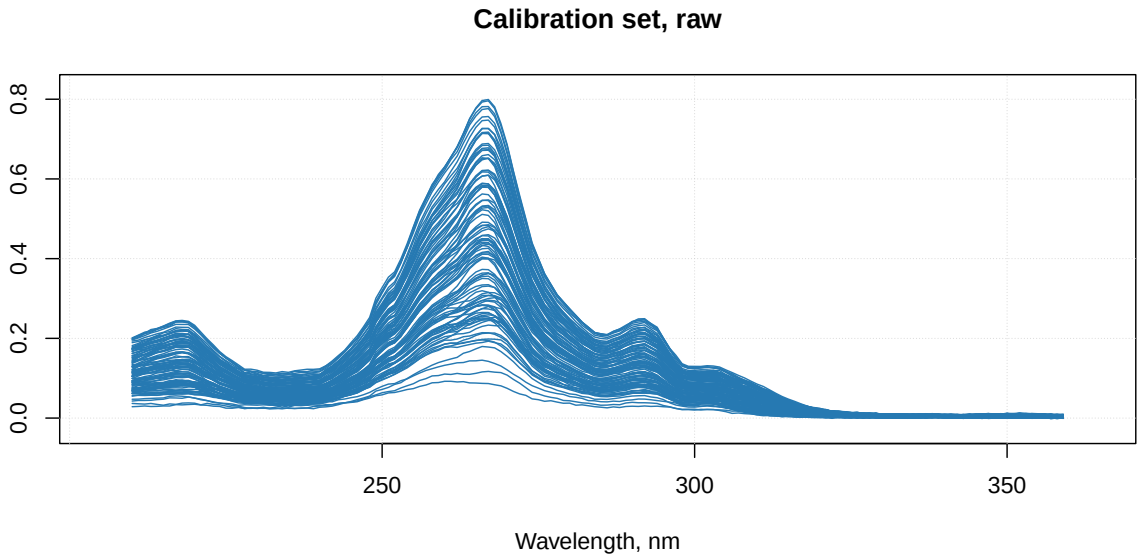
Likewise, any preprocessing model created in the application can be saved into a JSON file and loaded into R, so you can use it via `prep.apply()`. For example this JSON file I created using the *Train* tab of the app and *Simdata*. It does SG smoothing, SNV normalization, and mean centering. Download it to your working directory and run the following code (here I assume you already have `Xc` matrix from the previous block of

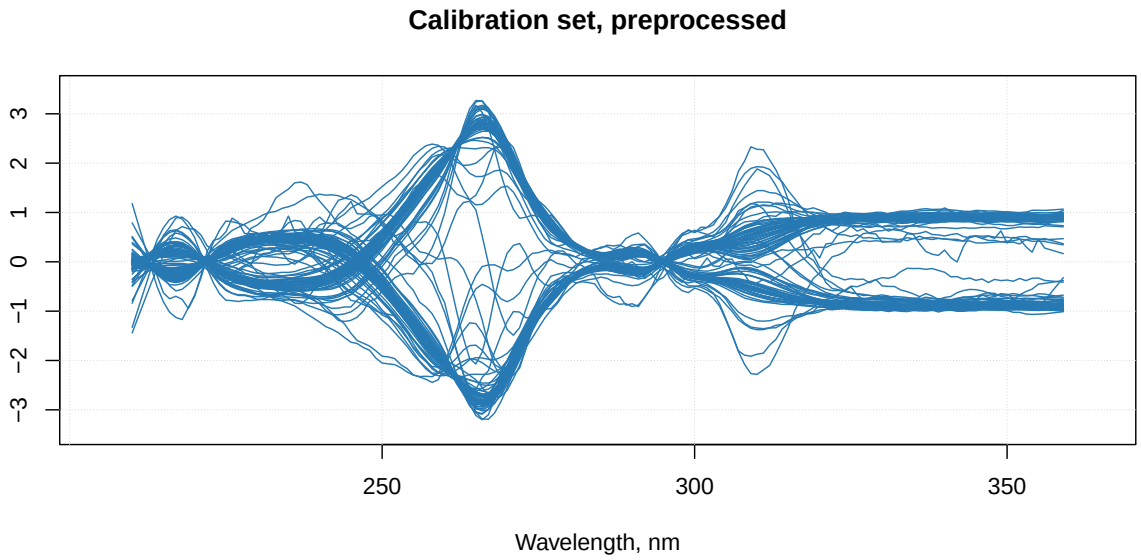
code):

```
# load preprocessing model from JSON file
myprep <- readJSON("prep-model-fromweb.json")

# apply it to Xc
Xcp <- prep.apply(myprep, Xc)

# show the result
par(mfrow = c(2, 1))
mdaplot(Xc, type = "l", main = "Calibration set, raw")
mdaplot(Xcp, type = "l", main = "Calibration set, preprocessed")
```





Additional methods and features

This section describes additional methods for data preprocessing. They work similar to any other method, the only difference is that they do not work with the `prep`, `prep.fit`, `prep.apply` methods. Just use them separately.

Element-wise transformations

Function `prep.transform()` allows you to apply an element-wise transformation — when the same transformation function is being applied to each element (each value) of the data matrix. This can be used, for example, in case of regression, when it is necessary to apply transformations which remove a non-linear relationship between predictors and responses.

Often such a transformation is either logarithmic or a power transformation. We can of course just apply a built-in R function e.g. `log()` or `sqrt()`, however in this case all additional attributes will be dropped in the preprocessed data. In order to tackle this and, also, to give the possibility of combining different preprocessing methods together, you can use a function `prep.transform()` for this purpose.

The syntax of the function is as follows: `prep.transform(data, fun, ...)`, where `data` is a matrix with the original data values, you want to preprocess (transform), `fun` is a reference to transformation function and `...` are optional additional arguments for the function. You can provide either one of the R functions, which are element-wise (meaning the function is being applied to each element of a matrix), such as `log`, `exp`, `sqrt`, etc. or define your own function.

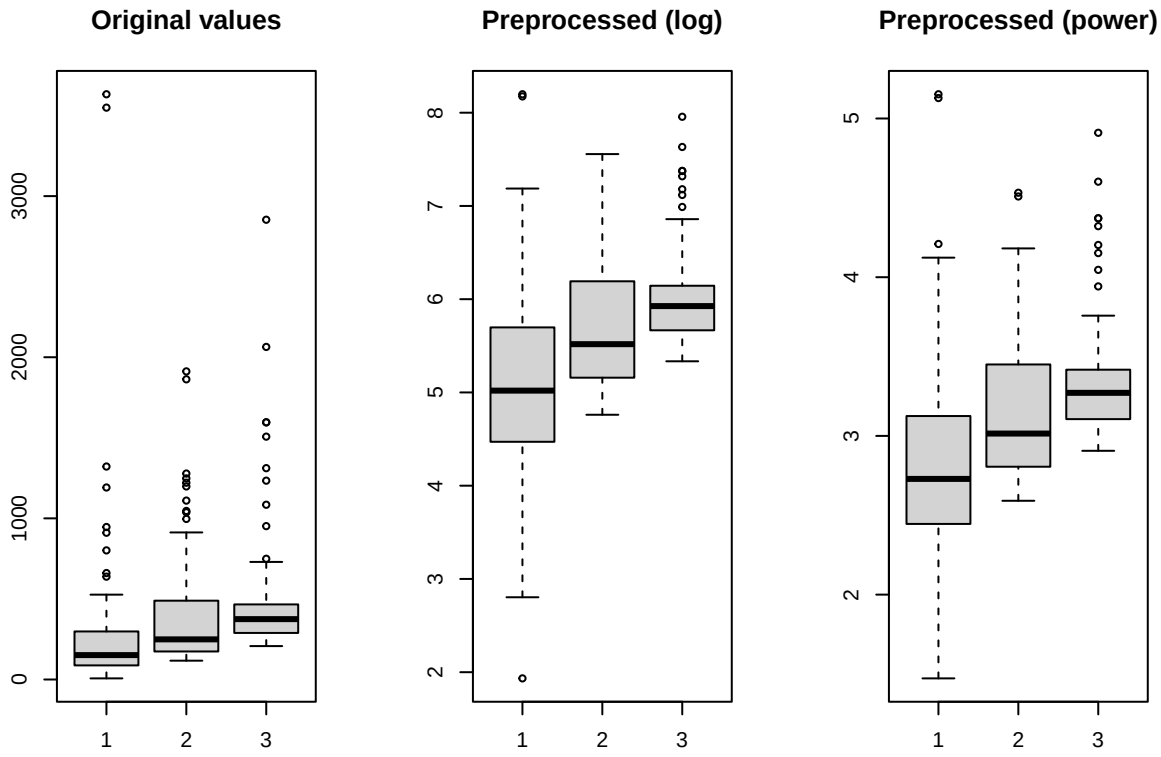
Here is an example:

```
# create a matrix with 3 variables (skewed random values)
X <- cbind(
   exp(rnorm(100, 5, 1)),
   exp(rnorm(100, 5, 1)) + 100 ,
   exp(rnorm(100, 5, 1)) + 200
)

# apply log transformation using built in "log" function
Y1 <- prep.transform(X, log)

# apply power transformation using manual function with additional argument
Y2 <- prep.transform(X, function(x, p) x^p, p = 0.2)

# show boxplots for the original and the transformed data
par(mfrow = c(1,3))
boxplot(X, main = "Original values")
boxplot(Y1, main = "Preprocessed (log)")
boxplot(Y2, main = "Preprocessed (power)")
```



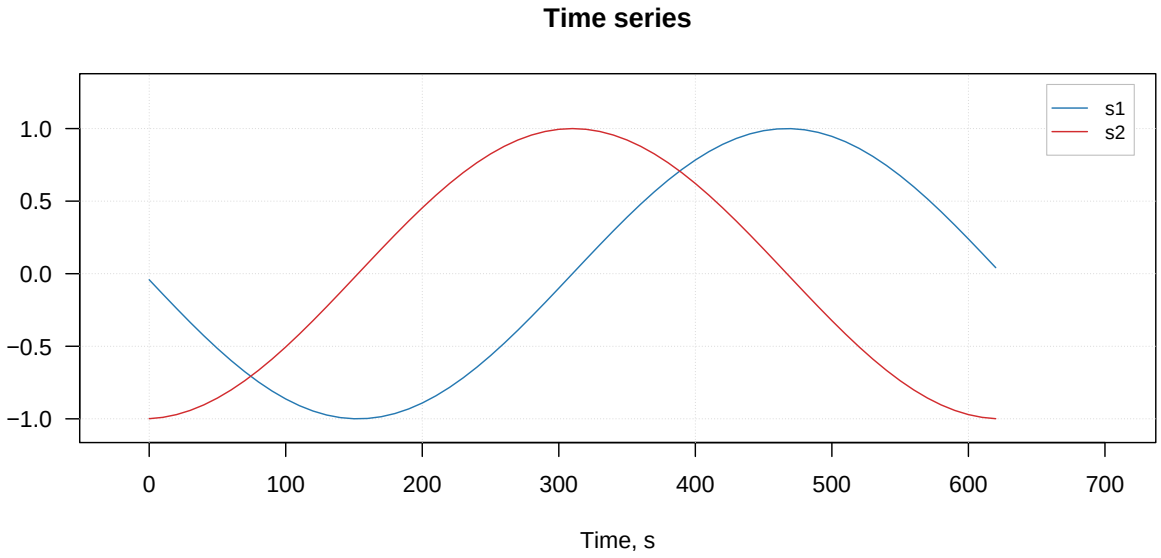
As already mentioned, the `prep.transform()` preserves all additional attributes, e.g. names and values for axes, excluded columns or rows, etc. Here is another example demonstrating this:

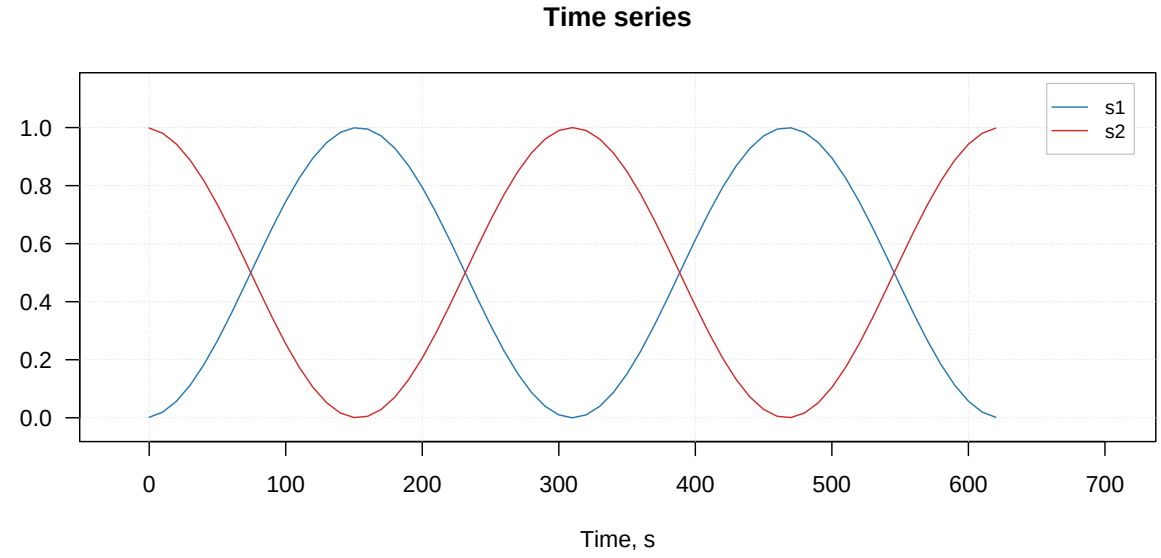
```
# generate two curves using sin() and cos() and add some attributes
t <- (-31:31)/10
X <- rbind(sin(t), cos(t))
rownames(X) <- c("s1", "s2")

# we make x-axis values as time, which span a range from 0 to 620 seconds
attr(X, "xaxis.name") <- "Time, s"
attr(X, "xaxis.values") <- (t * 10 + 31) * 10
attr(X, "name") <- "Time series"

# transform the dataset using squared transformation
Y <- prep.transform(X, function(x) x^2)

# show plots for the original and the transformed data
par(mfrow = c(2, 1))
mdaplotg(X, type = "l")
mdaplotg(Y, type = "l")
```





Notice that the x-axis values for the original and the transformed data (which we defined using corresponding attribute) are the same.

Kubelka-Munk transformation

Kubelka-Munk is a useful preprocessing method for diffuse reflection spectra (e.g. taken for powders or rough surface). It transforms the reflectance spectra R to K/M units as follows: $(1 - R)^2/2R$.

The method is implemented as `prep.ref2km(X)`, where `X` is a matrix with non-negative reflectance values.

Replacing missing values

This is not directly a preprocessing method, but can be considered as such. This method allows one to replace missing values in your dataset with the approximated ones.

The method uses PCA based approach described in this paper. The main idea is that we fit the dataset with a PCA model (e.g. PCA NIPALS algorithm can work even if the data contains missing values) and then approximate the missing values as if they were lying in the PC space.

The method has the same parameters as any PCA model. However, instead of specifying number of components you must specify another parameter, `expvarlim`, which specifies the portion of variance the model must explain. The default value is `0.95` which corresponds to 95% of the explained variance. You can also specify if data must be centered (default `TRUE`) and scaled/standardized (default `FALSE`). See more details by running `?pca.mvreplace`.

The example below shows a trivial case. First we generate a simple dataset. Then we replace some of the numbers with missing values (`NA`) and then apply the method to approximate them.

```
library(mdatools)

# generate a matrix with correlated variables
s = 1:6
odata = cbind(s, 2*s, 4*s)

# add some noise and labels for columns and rows
set.seed(42)
odata = odata + matrix(rnorm(length(odata), 0, 0.1), dim(odata))
colnames(odata) = paste0("X", 1:3)
rownames(odata) = paste0("O", 1:6)

# make a matrix with missing values
mdata = odata
mdata[5, 2] = mdata[2, 3] = NA

# replace missing values with approximated
rdata = pca.mvreplace(mdata, scale = TRUE)

# show all matrices together
show(round(cbind(odata, mdata, round(rdata, 2)), 3))
```

```
##       X1     X2     X3    X1     X2     X3   X1    X2    X3
## O1 1.137  2.151  3.861 1.137  2.151  3.861 1.14  2.15  3.86
## O2 1.944  3.991  7.972 1.944  3.991     NA 1.94  3.99  7.51
## O3 3.036  6.202 11.987 3.036  6.202 11.987 3.04  6.20 11.99
## O4 4.063  7.994 16.064 4.063  7.994 16.064 4.06  7.99 16.06
## O5 5.040 10.130 19.972 5.040     NA 19.972 5.04 10.21 19.97
## O6 5.989 12.229 23.734 5.989 12.229 23.734 5.99 12.23 23.73
```

```
# show the difference between original and approximated values
show(round(odata - rdata, 3))
```

```
##    X1     X2    X3
## O1  0  0.000 0.000
## O2  0  0.000 0.465
## O3  0  0.000 0.000
## O4  0  0.000 0.000
## O5  0 -0.076 0.000
## O6  0  0.000 0.000
```

As you can see, the method guessed that the two missing values must be 7.51 and 10.21, while the original values were 7.97 and 10.13.

The method works if the total number of missing values does not exceed 20% (10% if the dataset is small).

Deprecated methods

Here is a list of methods, which are deprecated, meaning they still work but it is recommended to replace them with the alternatives.

- `prep.autoscale()` — use `prep.center()` and `prep.scale()` instead.
- `prep.snv()` — use `prep.norm()` with parameter `type = "snv"` as an alternative.
- `prep.msc()` — use `prep.emsc()` with parameter `degree = 0`.
- `employ.prep()` — use `prep.apply()` instead.

Principal component analysis

In this chapter we will discuss how to use the PCA method implemented in *mdatools*. Besides that, we will use PCA examples to introduce some principles, which are common for most of the other methods (e.g. PLS, DD-SIMCA, PLS-DA, etc.) available in this package. This includes model and result objects, getting and visualization of performance statistics, validation, use of different kinds of plots, and so on.

Principal component analysis is one of the methods that decompose a data matrix $\mathbf{X}$ into a combination of three matrices: $\mathbf{X} = \mathbf{T}\mathbf{P}^{\mathrm{T}} + \mathbf{E}$. Here $\mathbf{P}$ is a matrix with unit vectors, defined in the original variable space. The unit vectors, also known as *loadings*, form a new basis — *principal components*. The components are mutually orthogonal and oriented in variable space to capture the direction of maximum variation of data points.

The data points are being projected to the principal components. Coordinates of these projections in principal component space, known as *scores*, form matrix $\mathbf{T} = \mathbf{X}\mathbf{P}$. The product of scores and loadings, $\mathbf{T}\mathbf{P}^{\mathrm{T}}$ gives coordinates of the projections in the original variable space. Matrix $\mathbf{E}$ contains residuals — difference between position of projected data points and their original locations. This difference is a part of data variation, which the PCA model does not capture, hence the name.

If the original data matrix has I rows (observations, objects) and J variables and PCA decomposition is made with A components, then matrix $\mathbf{P}$ will have dimension $J \times A$, matrix $\mathbf{T}$ — $I \times A$, and $\mathbf{T}\mathbf{P}^{\mathrm{T}}$ and $\mathbf{E}$ will have the same dimension as the original data matrix, $\mathbf{X}$.

The relationship between data objects and principal component space (PCA model) can be described using two distances (in some literature they are also called residual distances) — *orthogonal distance*, OD, and *score distance*, SD. The orthogonal distance is a squared Euclidean distance between the position of an object and its projection in original variable space. It can be computed by taking the sum of squared values from matrix $\mathbf{E} = \{e_{ij}\}$ along every row:

$$q_i = \sum_{j=1}^{J} e_{ij}^2$$

This distance usually is denoted as Q or q. It can be considered as a lack of fit measure for this particular object.

The score distance is a squared Mahalanobis distance between the projection of the objects and the origin. It is a measure of the extremeness of an object and usually denoted as h or T^2. The latter is used because the Hotelling T^2 distribution is often used to describe the distribution of the score distance values, so in many software packages the distance is called the *Hotelling T^2 distance*. The distance can be computed using standardized scores (so score values for every component have unit variance).

$$h_i = \sum_{a=1}^{A} \frac{t_{ia}^2}{\lambda_a}$$

Here λ are eigenvalues for the principal components, which correspond to the variance of the corresponding scores.

Both score and orthogonal distances are important statistics allowing one to assess how well objects are described by PCA model. They can be assessed visually, using the *Distance plot*, — scatter plot where orthogonal distance is plotted against the score distance for a particular number of components.

Both distances can be described using theoretical distributions. This helps to identify regular and extreme objects as well as outliers. All details will be shown later in this tutorial.

There are several methods to compute PCA loadings, including Singular Values Decomposition (SVD) and Non-linear Iterative Partial Least Squares (NIPALS). Both methods are implemented in this package and can be selected using `method` argument of main class `pca`. By default SVD is used. In addition, one can use a randomized version of the two methods, which can be efficient if the data matrix contains a large number of rows. This is explained in the last part of this chapter.

Models and results

In *mdatools*, any method for data analysis, such as PCA, PLS regression, DD-SIMCA classification and so on, can create two types of objects — a model and a result. Every time you build a model you get a *model object*. Every time you apply the model to a dataset you get a *result object*. Thus for PCA, the objects have classes `pca` and `pcares` correspondingly.

Each object includes a list with object's properties (e.g. loadings for model, scores and explained variance for result) and provides a number of methods for visualization and exploration.

Model calibration

Let's see how this works using a simple example — *People* data. We already used this data when we were playing with plots, it consists of 32 objects (persons from Scandinavian and Mediterranean countries, 16 male and 16 female) and 12 variables (height, weight, shoesize, annual income, beer and wine consumption and so on). More information about the data can be found using `?people`. We will first load the data matrix:

```
library(mdatools)
data(people)
```

Now let's calibrate the model:

```
m = pca(people, 7, scale = TRUE, info = "People PCA model")
m = selectCompNum(m, 5)
```

Here `pca` is a function that builds (calibrates, fits, trains) a PCA model and returns the model object. In terms of programming we can call function `pca()` a constructor of pca model object. The constructor has one mandatory argument — matrix or data frame with data values. Other arguments are optional as they all have default values. In this case we use three additional arguments: `7` is the maximum number of components, `scale = TRUE` tells that data values should be standardized (centering is a default option) and `info` adds a short text with information about the model which can be shown later.

Here `people` and `7` are *positional* arguments, meaning their role is defined by their position. Thus `pca()` function always expects that the first argument is data matrix or data frame and the second argument is the number of components. So if you swap them, you will get an error. The other arguments in the example above are *named* arguments, meaning their role is defined by name and therefore they can be provided in any order. The full list of arguments for `pca()` function and their default values can be seen by running `?pca`.

Function `selectCompNum()` sets an "optimal" number of components for the model. In our case we calibrate model with 7 principal components; however, e.g. after investigation of explained variance, we found that 5 components are optimal. In this case we have two choices. Either recalibrate the model using 5 components or use the model that is calibrated already but "tell" the model that 5 components is the optimal number.

In this case the model will keep all results calculated for 7 components but will use optimal number of components when necessary. For example, when showing distance plot for the model.

Function `print` prints information about the model object and its structure:

```
print(m)
```

```
##
## PCA model (class pca)
##
## Info:
## People PCA model
##
## Call:
## pca(x = people, ncomp = 7, scale = TRUE, info = "People PCA model")
##
## Major model fields:
##  $loadings - matrix with loadings
##  $eigenvals - eigenvalues for components
##  $ncomp - number of calculated components
##  $ncomp.selected - number of selected components
##  $center - values for centering data
##  $scale - values for scaling data
##  $alpha - significance level for critical limits
##  $gamma - significance level for outlier limits
##  $limParams - parameters for distribution of distances
##  $res - list with model results (calibration, test)
```

As you can see, there are no scores, explained variance values, distances and so on. Because they are not actually part of the PCA model, they are results of applying the model to a calibration set. But loadings, eigenvalues, number of calculated and selected principal components, vectors for centering and scaling the data, significance levels and corresponding critical limits for distances — all are parameters of the model.

Here is an example of how to get values from loading matrix having the model object:

```
m$loadings[1:4, 1:4]
```

```
##              Comp 1     Comp 2      Comp 3      Comp 4
## Height   -0.3752858 -0.1354585 -0.07237117  0.07431853
## Weight   -0.3811357 -0.1114472 -0.06810860 -0.03313762
## Hairleng  0.3377735  0.1501634  0.07901752  0.11431601
## Shoesize -0.3776970 -0.1508061 -0.00127806  0.06569227
```

One can also notice that the model object has a particular field — `res`, which is in fact a list of PCA *result objects* containing results of applying the model to the calibration set (`cal`) and (if available) validation/test set (`test`). In the case of regression and classification `res` can also contain an object with cross-validation results (`cv`). For example here is the description of object with calibration results:

```
print(m$res$cal)
```

```
##
## Results for PCA decomposition (class pcares)
##
## Major fields:
##  $scores - matrix with score values
##  $T2 - matrix with T2 distances
##  $Q - matrix with Q residuals
##  $ncomp.selected - selected number of components
```

```
##  $expvar - explained variance for each component
##  $cumexpvar - cumulative explained variance
```

And if we want to look at scores, here is the way:

```
m$res$cal$scores[1:4, 1:4]
```

```
##           Comp 1    Comp 2     Comp 3     Comp 4
## Lars   -5.332528 0.6765698  1.0673076  1.1008056
## Peter  -3.114319 0.2934016 -0.6707091 -1.3099103
## Rasmus -2.996848 0.3604689 -0.2118207 -1.1169333
## Lene    1.084240 1.8449249 -0.4091936 -0.1228956
```

Both model and result objects also have related functions (methods), first of all for visualizing various values (e.g. scores plot, loadings plot, etc.). Some of the functions will be discussed later in this chapter, a full list can be found in help for a proper constructor (e.g. `?pca`).

The *result object* is also created every time you apply a model to new data. Like in many built-in R methods, method `predict()` is used in this case. The first argument of the method is always a model object. Here is a PCA example (assuming we have already built the model):

```
# create data matrix with three new measurements
Xt = rbind(
   c(180, 80, -1, 44, 35, 35000, 200, 100, -1, 96, -1, 105),
   c(170, 67,  1, 40, 41, 29500, 100, 200,  1, 94,  1, 105),
   c(175, 70,  1, 41, 28, 21500, 110, 120,  1, 97,  1, 115)
)

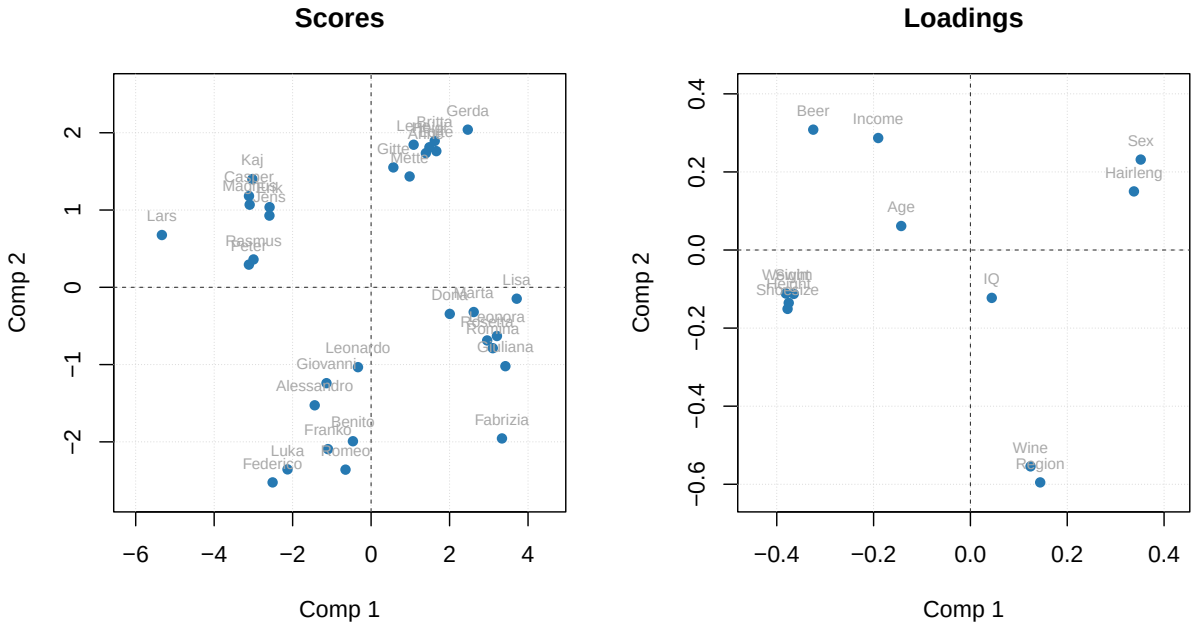
# project the new data to the previous PCA model and show the result object
res = predict(m, Xt)
print(res)
```

```
##
## Results for PCA decomposition (class pcares)
##
## Major fields:
##  $scores - matrix with score values
##  $T2 - matrix with T2 distances
##  $Q - matrix with Q residuals
##  $ncomp.selected - selected number of components
##  $expvar - explained variance for each component
##  $cumexpvar - cumulative explained variance
```

Plotting methods

First of all, you can use the methods `mdaplot()` and `mdaplotg()` (or any others, e.g. `ggplot2`) for easy visualisation of the results as they are all available as matrices with proper names, attributes, etc. In the example below I create scores and loadings plots for PC1 vs PC2. Here I assume that the model from previous section is already created and available in your Global Environment. As you can see, I take the matrix with scores from the objects with calibration results (`m$res$cal`).

```
par(mfrow = c(1, 2))
mdaplot(m$res$cal$scores, type = "p", show.labels = TRUE, show.lines = c(0, 0))
mdaplot(m$loadings, type = "p", show.labels = TRUE, show.lines = c(0, 0))
```



To simplify this routine, every model and result class also has a number of functions for visualization. Thus for PCA the function list includes scores and loadings plots, explained variance and cumulative explained variance plots, eigenvalues, distance plots, and many others.

A function that does the same for different models and results always has the same name. For example, `plotPredictions` will show predicted vs. measured plot for PLS model and PLS result, and so on, although the plot itself will be quite different. The first argument must always be either a model or a result object.

The major difference between plots for model and plots for result is as follows. A plot for result always shows one set of data objects — one set of points, lines or bars. For example, predicted vs. measured values for calibration set or scores values for test set and so on. For such plots method `mdaplot()` is used and you can provide any arguments, available for this method (e.g. color group scores for calibration results, add confidence ellipses for scores, etc.).

And a plot for a model in most cases shows several sets of data objects, e.g. predicted values for calibration and validation. In this case, a corresponding method uses `mdaplotg()` and, therefore, you can adjust the plot using arguments described for this method.

Here are some examples for results:

```
# create a factor for combination of Sex and Region values
g1 <- factor(people[, "Sex"], labels = c("M", "F"))
g2 <- factor(people[, "Region"], labels = c("S", "M"))
g <- interaction(g1, g2)

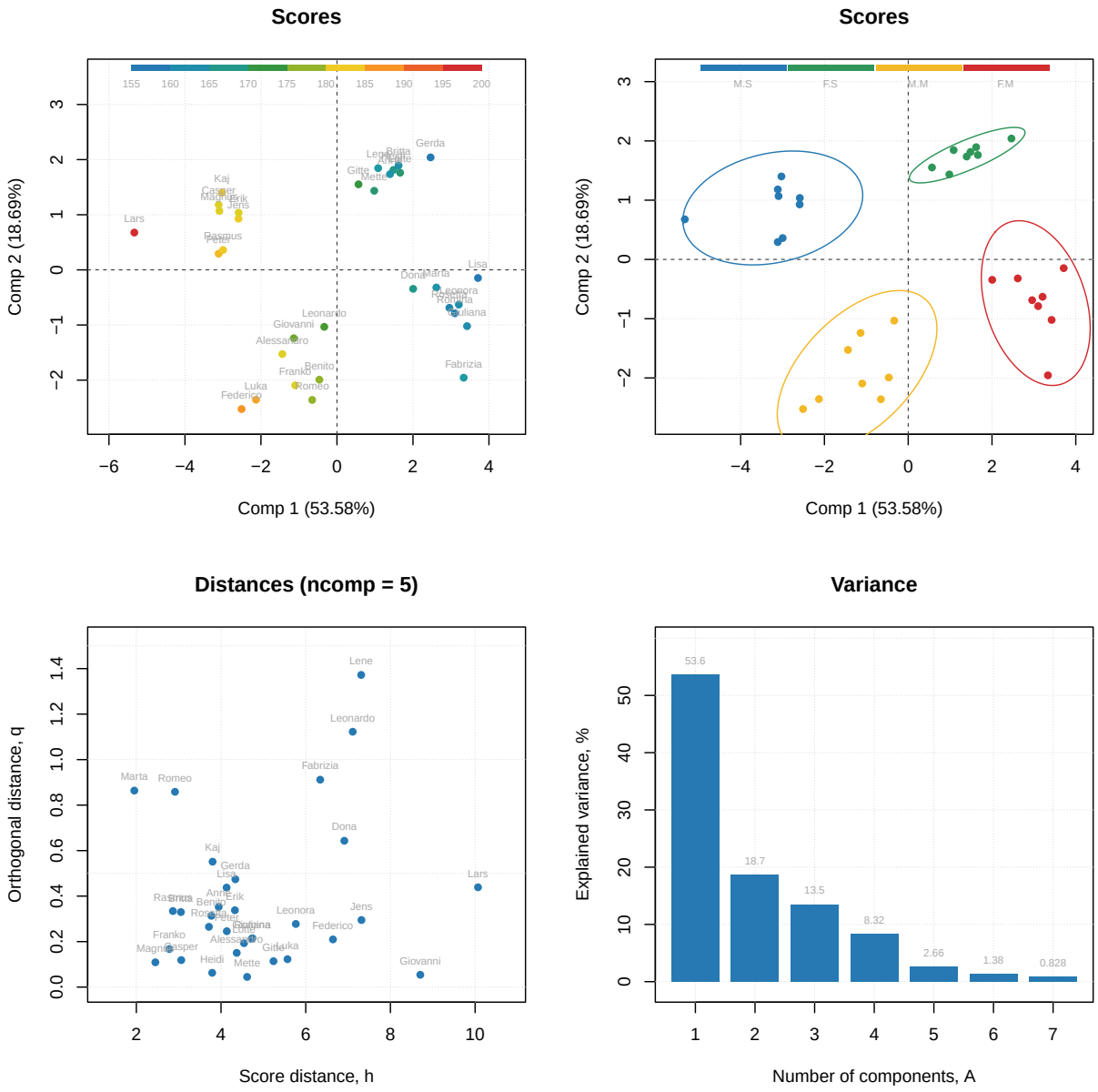
par(mfrow = c(2, 2))

# scores plot for calibration results colored by Height
plotScores(m$res$cal, show.labels = TRUE, cgroup = people[, "Height"])

# scores plot colored by the factor created above and confidence ellipses
p = plotScores(m$res$cal, c(1, 2), cgroup = g)
plotConfidenceEllipse(p)
```

```
# distance plot for calibration results with labels
plotDistances(m$res$cal, show.labels = TRUE)

# variance plot for calibration results with values as labels
plotVariance(m$res$cal, type = "h", show.labels = TRUE, labels = "values")
```



Notice that the distance plot in this case is shown using 5 PCs in the model. The model was calibrated using 7 components, but since we later selected 5 as optimal, it remembers this selection. You can also show this plot for any number of components between 1 and 7 by providing corresponding argument, e.g. `ncomp = 3`.

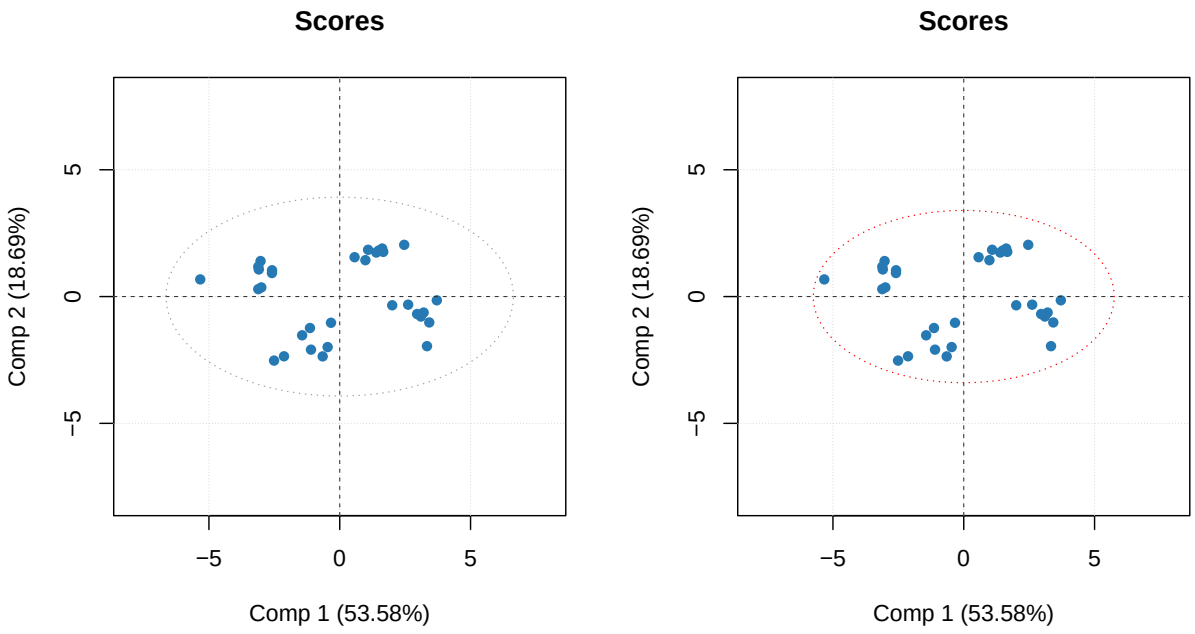
Scores plot can also be used together with `plotHotellingEllipse()` function. It works similar to `plotConfidenceEllipse()` or `plotConvexHull()` however does not require grouping of the values. You can add the ellipse both to score plot for results and score plot for model. In case of model, if you have e.g. both calibration and test set results you need to specify which one you want to use for creating the ellipse. Code

below shows several examples.

```
par(mfrow = c(1, 2))

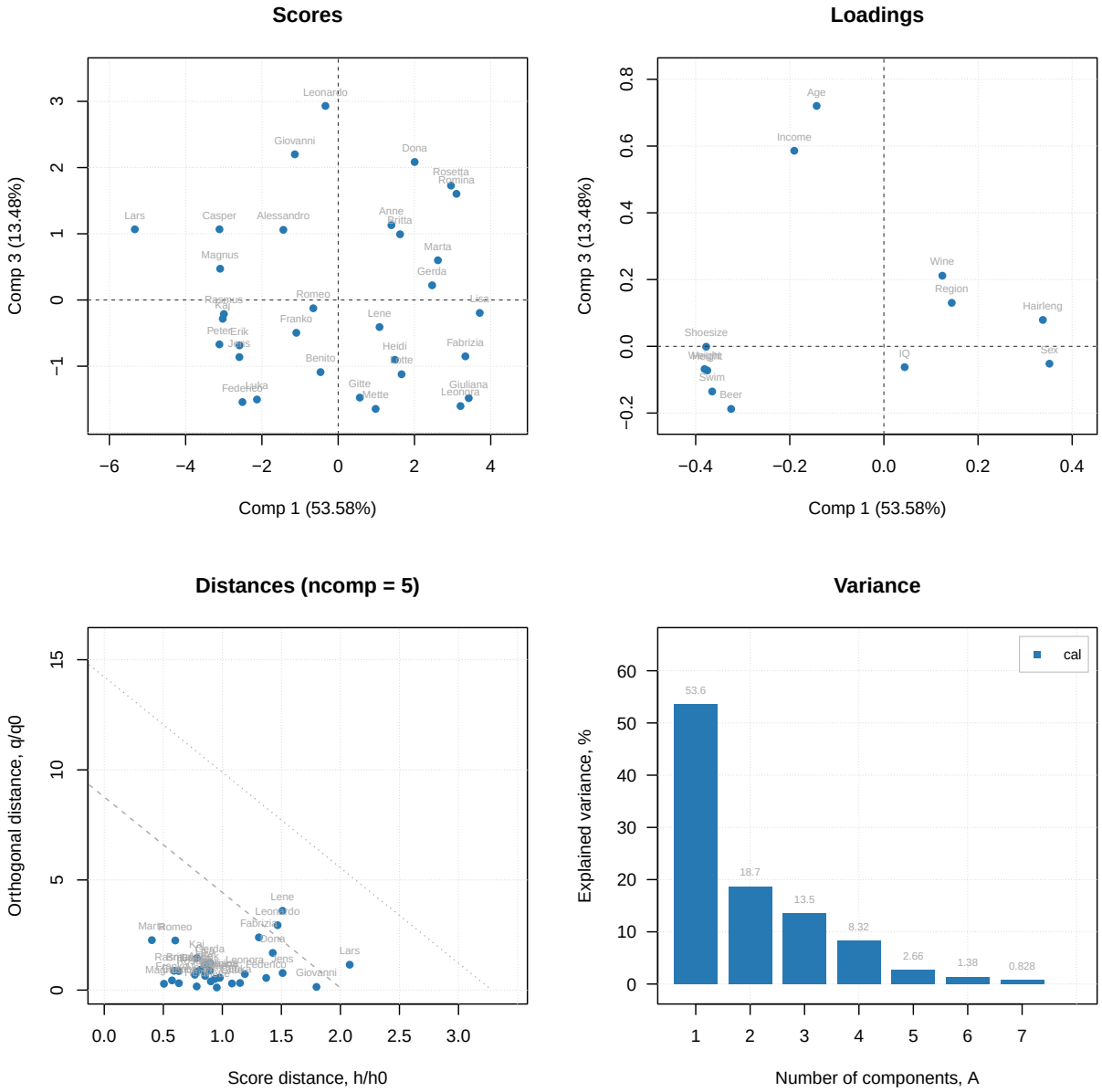
# default options for calibration results
p = plotScores(m$res$cal, xlim = c(-8, 8), ylim = c(-8, 8))
plotHotellingEllipse(p)

# also for calibration results but with specific significance limit and color
p = plotScores(m$res$cal, xlim = c(-8, 8), ylim = c(-8, 8))
plotHotellingEllipse(p, conf.lim = 0.9, col = "red")
```



Now let's look at similar plots (plus loadings) for a model.

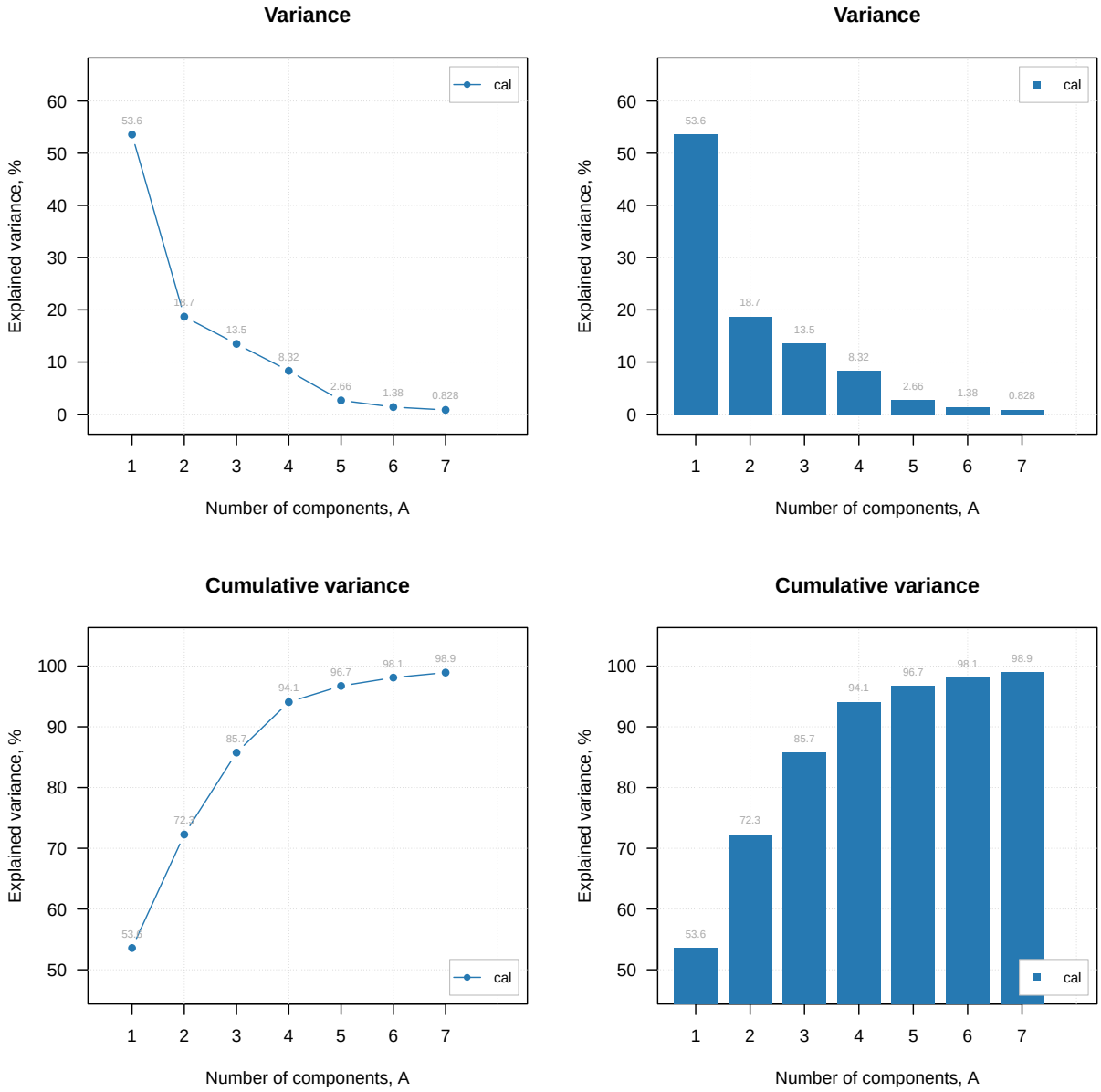
```
par(mfrow = c(2, 2))
plotScores(m, c(1, 3), show.labels = TRUE)
plotLoadings(m, c(1, 3), show.labels = TRUE)
plotDistances(m, show.labels = TRUE)
plotVariance(m, type = "h", show.labels = TRUE, labels = "values")
```



As you can see, in case of model, the distance plot also shows some lines. These are critical limits, all details about them as well as how to use the distance plot will be given in one of the next sections.

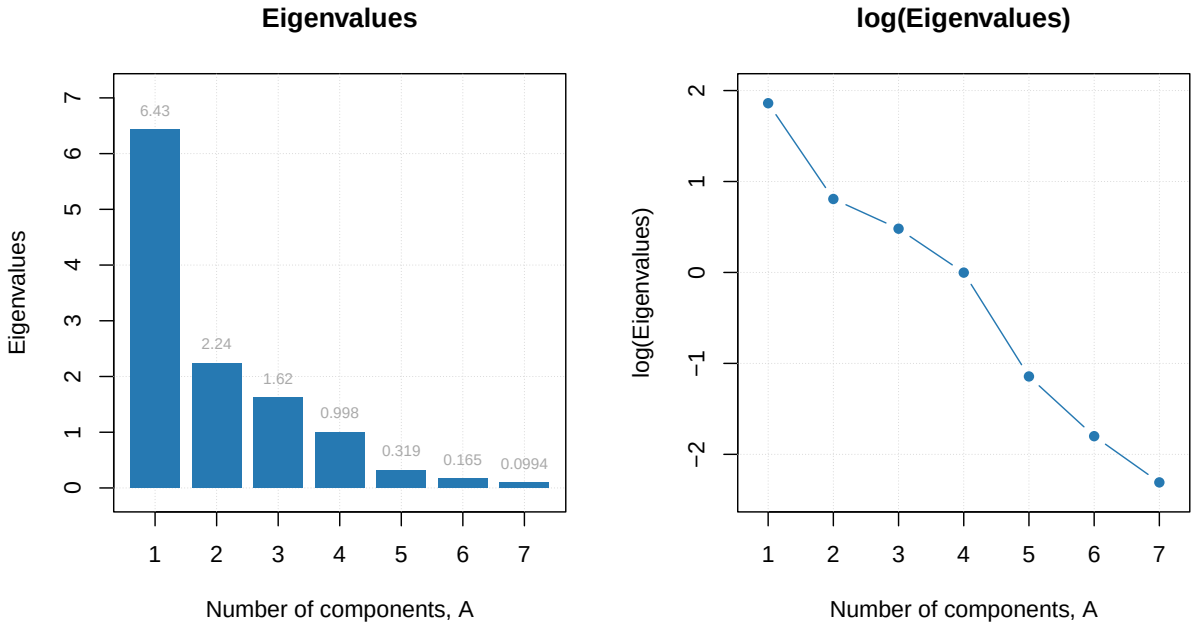
The variance can be plotted in two forms: individual (how much every component explains) or cumulative (how much is explained by this component and all previous together). Here is an example:

```
par(mfrow = c(2, 2))
plotVariance(m, type = "b", show.labels = TRUE, labels = "values")
plotVariance(m, type = "h", show.labels = TRUE, labels = "values")
plotCumVariance(m, type = "b", show.labels = TRUE, labels = "values")
plotCumVariance(m, type = "h", show.labels = TRUE, labels = "values")
```



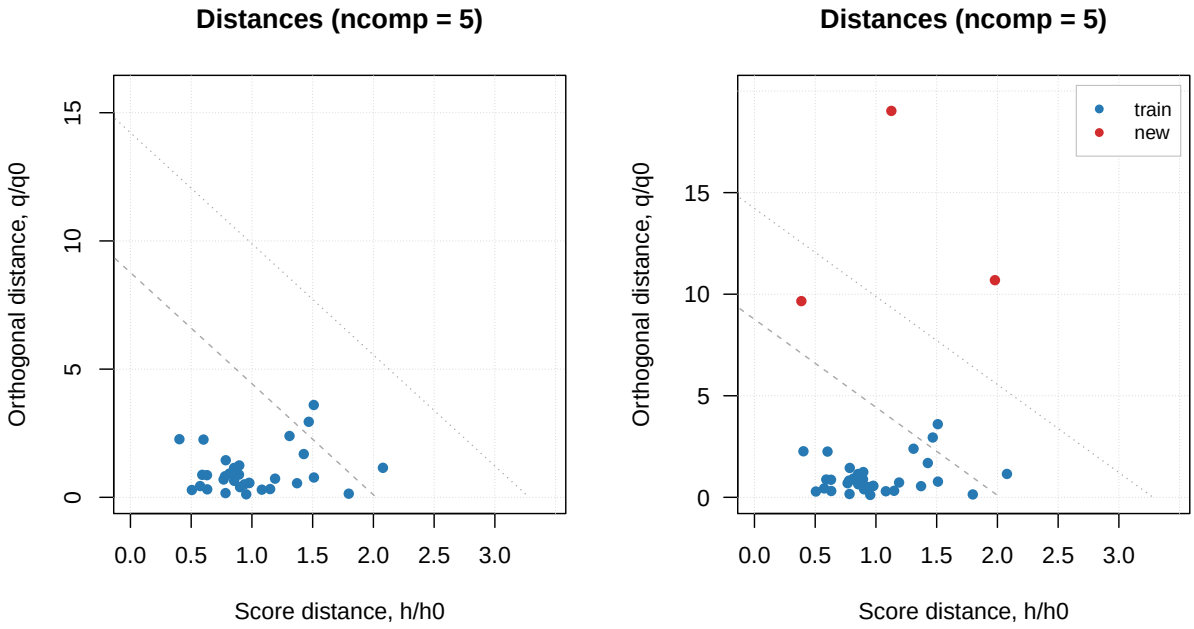
There is also a possibility to plot model's eigenvalues vs. number of components, either raw or transformed using log or sqrt transformation:

```
par(mfrow = c(1, 2))
plotEigenvalues(m, type = "h", show.labels = TRUE, labels = "values")
plotEigenvalues(m, type = "b", transform = "log")
```



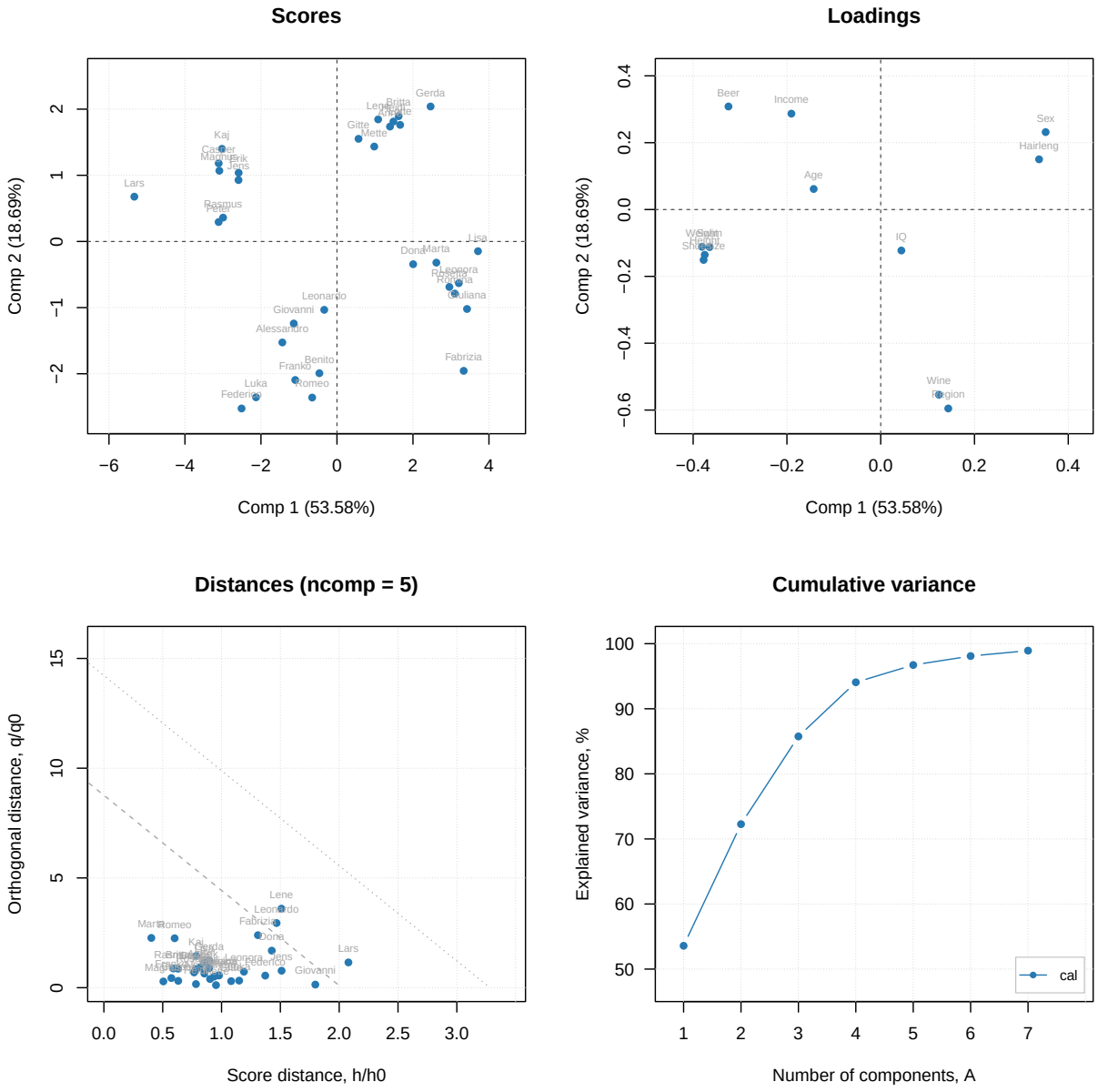
If model plot shows values from calibration and validation objects (e.g. scores, distances, etc), you can specify which result to show. In the example below left plot is made using default settings, for the right plot we specified a list with results explicitly (we show calibration results with label "train" and results for new measurements with label "new"). Note that the results should be specified as a named list (we also assume here that you ran code from the previous part of this section and have object `res` in your environment).

```
par(mfrow = c(1, 2))
plotDistances(m)
plotDistances(m, res = list("train" = m$res$cal, "new" = res))
```



Finally, method `plot()` shows the four main PCA plots as a model (or results) overview.

```
plot(m, show.labels = TRUE)
```



You do not have to care about labels, names, legend and so on, however if necessary you can always change almost anything. See full list of methods available for PCA by `?pca` and `?pcares`.

Manual x-values for loading line plot

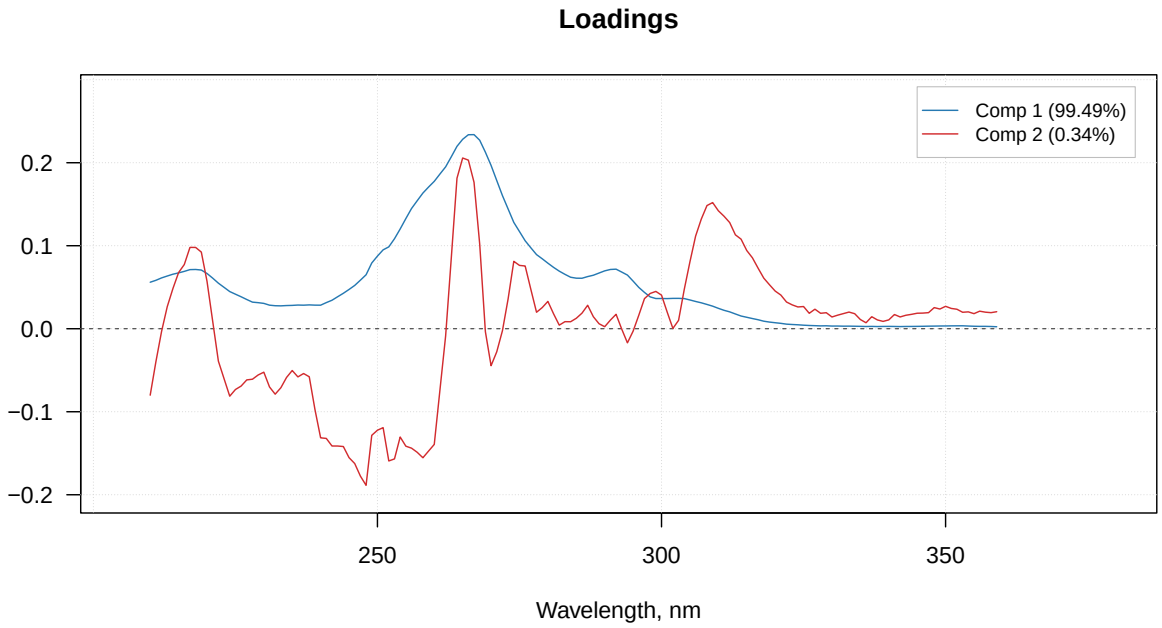
As discussed in the previous chapter, you can specify a special attribute, `"xaxis.values"` to a dataset, which will be used as manual x-values in bar and line plots. When we create any model and/or results the most important attributes, including this one, are inherited. For example, when you make a loadings line plot, it will be shown using the attribute values.

Here is an example that demonstrates this feature using PCA decomposition of the *Simdata* (UV/Vis spectra).

```
data(simdata)

# get the data matrix and add necessary attributes
X = simdata$spectra.c
attr(X, "xaxis.name") = "Wavelength, nm"
attr(X, "xaxis.values") = simdata$wavelength

# do PCA and show loadings plot for PC1 and PC2 as lines
m = pca(X, 3)
plotLoadings(m, 1:2, type = "l")
```



Excluding rows and columns

PCA as well as any other method in `mdatools` can exclude rows and columns from calculations when you train a model. For example, it can be useful if you have some candidates for outliers or do variable selection and do not want to remove rows and columns from the data matrix. In this case you can just specify two additional parameters, `exclcols` and `exclrows`, using either numbers or names of rows/columns to be excluded. You can also specify a vector with logical values (all `TRUE`s will be excluded).

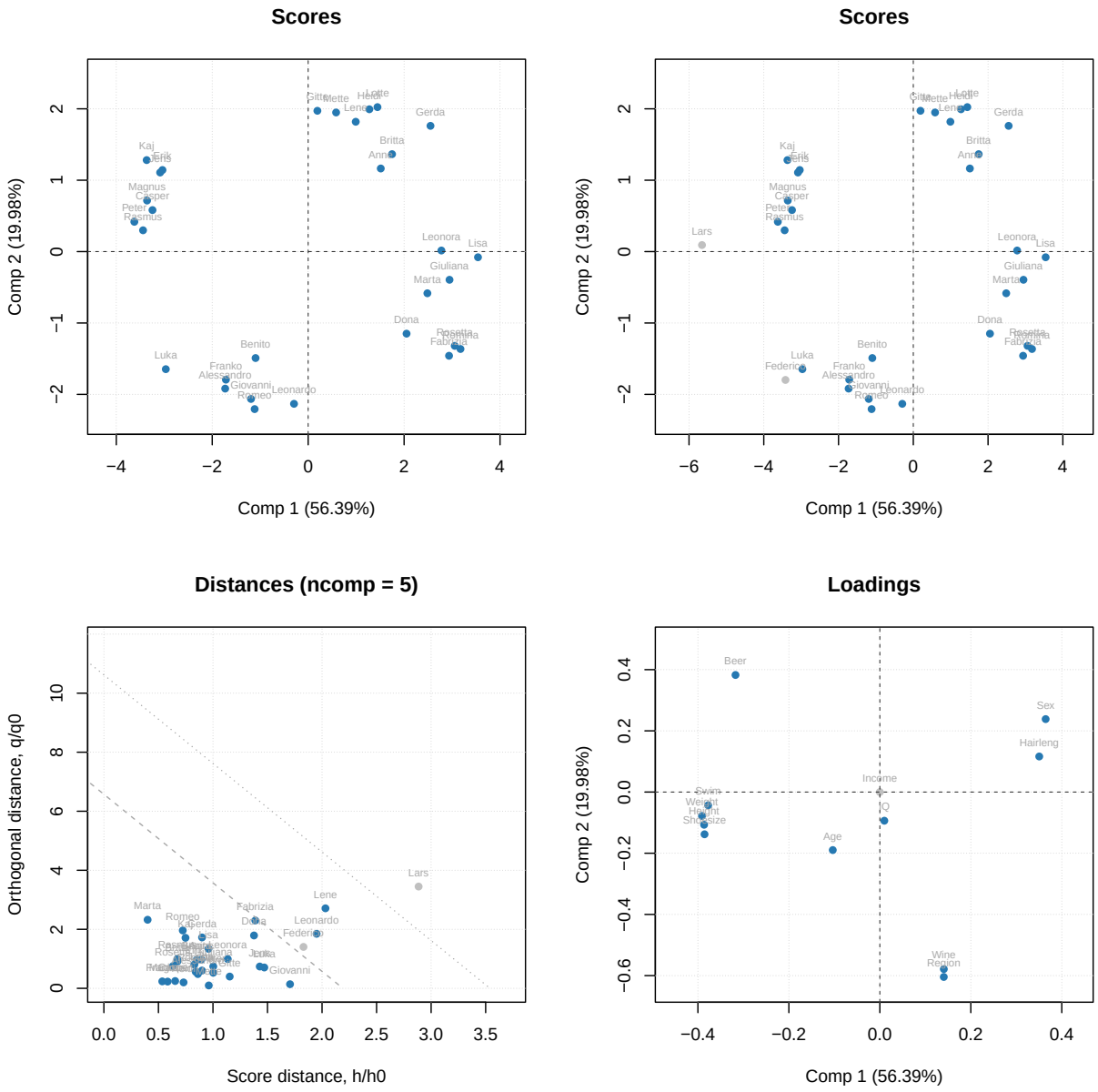
The excluded rows are not used for creating a model and calculation of model's and results' performance (e.g. explained variance). However, main results (for PCA — scores and distances) are calculated for these rows as well and are hidden, so you will not see them on plots. You can always show values for excluded objects by using parameter `show.excluded = TRUE`. It is implemented via attributes "known" for plotting methods from *mdatools* so if you use e.g. *ggplot2* you will see all points.

The excluded columns are not used for any calculations either, the corresponding results (e.g. loadings or regression coefficients) will have zero values for such columns and be also hidden on plots. Here is a simple example.

```
data(people)

m = pca(people, 5, scale = TRUE, exclrows = c("Lars", "Federico"), exclcols = "Income")
```

```
par(mfrow = c(2, 2))
plotScores(m, show.labels = TRUE)
plotScores(m, show.labels = TRUE, show.excluded = TRUE)
plotDistances(m, show.labels = TRUE, show.excluded = TRUE)
plotLoadings(m, show.labels = TRUE, show.excluded = TRUE)
```



As you can see, the excluded observations (or variables in case of loadings plot) are either hidden completely or shown as gray points if `show.excluded` parameter is set to `TRUE`.

Here is the matrix with loadings, note that variable `Income` has zeros for loadings and the matrix has attribute `exclrows` set to 6 (which is a position of the variable):

```
# show matrix with loadings (look at row Income and attribute "exclrows")
show(m$loadings)
```

```
##                Comp 1      Comp 2       Comp 3      Comp 4      Comp 5
## Height   -0.386393327 -0.10697019 -0.004829174 -0.12693029 -0.13128331
## Weight   -0.391013398 -0.07820097  0.051916032 -0.04049593 -0.14757465
## Hairleng  0.350435073  0.11623295 -0.103852349  0.04969503 -0.73669997
## Shoesize -0.385424793 -0.13805817 -0.069172117 -0.01049098 -0.17075488
## Age      -0.103466285 -0.18964288 -0.337243182  0.89254403 -0.02998028
## Income    0.000000000  0.00000000  0.000000000  0.00000000  0.00000000
## Beer     -0.317356319  0.38259695  0.044338872  0.03908064 -0.21419831
## Wine      0.140711271 -0.57861817 -0.059833970 -0.12347379 -0.41488773
## Sex       0.364537185  0.23838610  0.010818891 -0.04025631 -0.18263577
## Swim     -0.377470722 -0.04330411  0.008151288 -0.18149268 -0.30163601
## Region    0.140581701 -0.60435183  0.040969200 -0.15147464  0.17857614
## IQ        0.009849911 -0.09372132  0.927669306  0.32978247 -0.11815762
## attr(,"exclrows")
## [1] 6
## attr(,"name")
## [1] "Loadings"
## attr(,"xaxis.name")
## [1] "Components"
## attr(,"yaxis.name")
## [1] "Variables"
```

Such behavior will help to exclude and include rows and columns interactively, some examples will be shown later.

Support for images

As it was described before, images can be used as a source of data for any methods. In this case the results, related to objects/pixels will inherit all necessary attributes and can be shown as images as well. In the example below we make a PCA model for the image data from the package and show scores and distances.

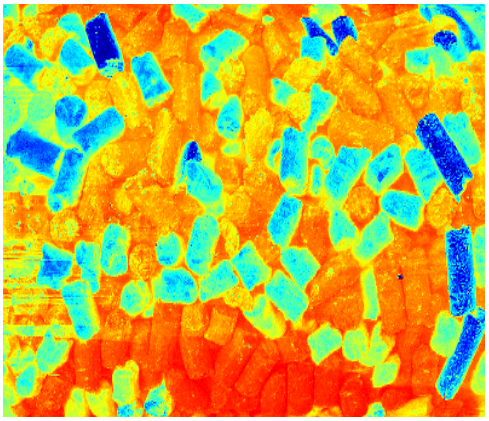
```
data(pellets)

# convert image to data matrix
X = mda.im2data(pellets)

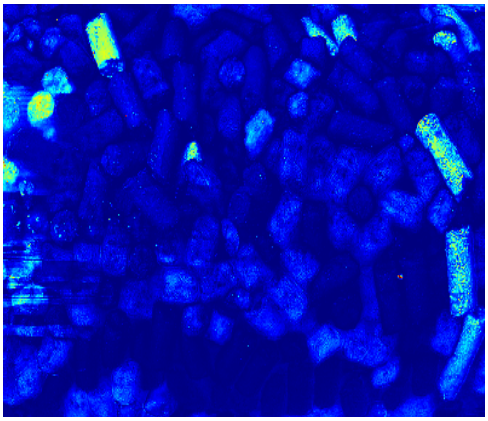
# apply PCA
m = pca(X)

# show PCA outcomes as images
par(mfrow = c(2, 2))
imshow(m$res$cal$scores, main = "Scores, PC1")
imshow(m$res$cal$Q, main = "Orthogonal distance, 1 PC")
imshow(m$res$cal$scores, 2, main = "Scores, PC2")
imshow(m$res$cal$Q, 3, main = "Orthogonal distance, 3 PCs")
```

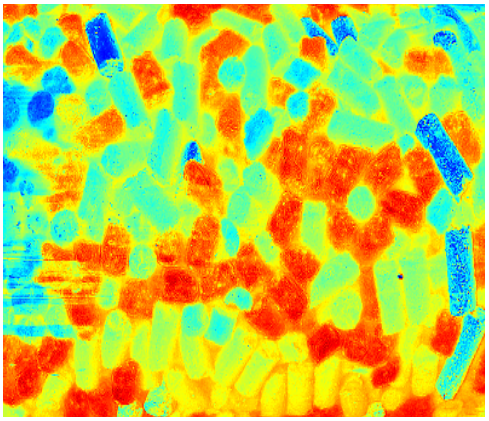
Scores, PC1



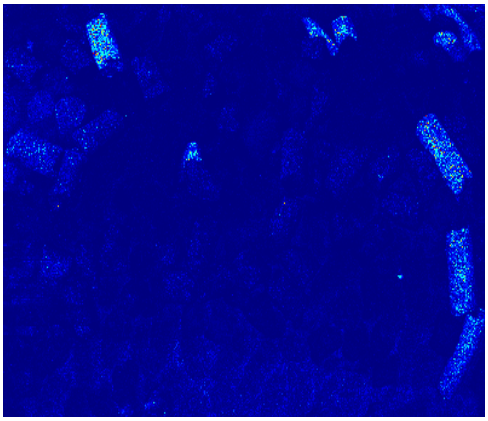
Orthogonal distance, 1 PC



Scores, PC2



Orthogonal distance, 3 PCs



Distances and critical limits

As it was written in the brief theoretical section about PCA, when data objects are being projected to a principal component space, two distances are calculated and stored as a part of PCA results (`pcares` object). The first is a squared orthogonal Euclidean distance from original position of an object to the PC space, also known as *orthogonal distance* and denoted as Q or q. The distance shows how well the object is fitted by the PCA model and allows one to detect objects that do not follow a common trend, captured by the PCs.

The second distance shows how far a projection of an object to the PC space is from the origin. This distance is also known as the Hotelling T^2 distance or the *score distance*, which is also denoted as h-distance. To compute the h-distance, scores should first be normalized by dividing them by the corresponding singular values (or eigenvalues if we deal with squared scores). The h-distance allows to see extreme objects — which are far from the origin.

> In previous versions of the *mdatools* method `plotDistances()` was known as `plotResiduals()`. It is still available for compatibility but if you write a new code, use `plotDistances()` instead.

In *mdatools* both distances are calculated for all objects in dataset and all possible components. So every distance is represented by $I \times A$ matrix (will remind here that I is the number of objects or rows in data matrix and A is number of components in PCA model), the first column contains distances for a model with only one component, second — for model with two components, and so on. The distances can be visualized using method `plotDistances()` which is available both for PCA results as well as for PCA model. In the case of model the plot shows distances both for calibration set and test set (if available).

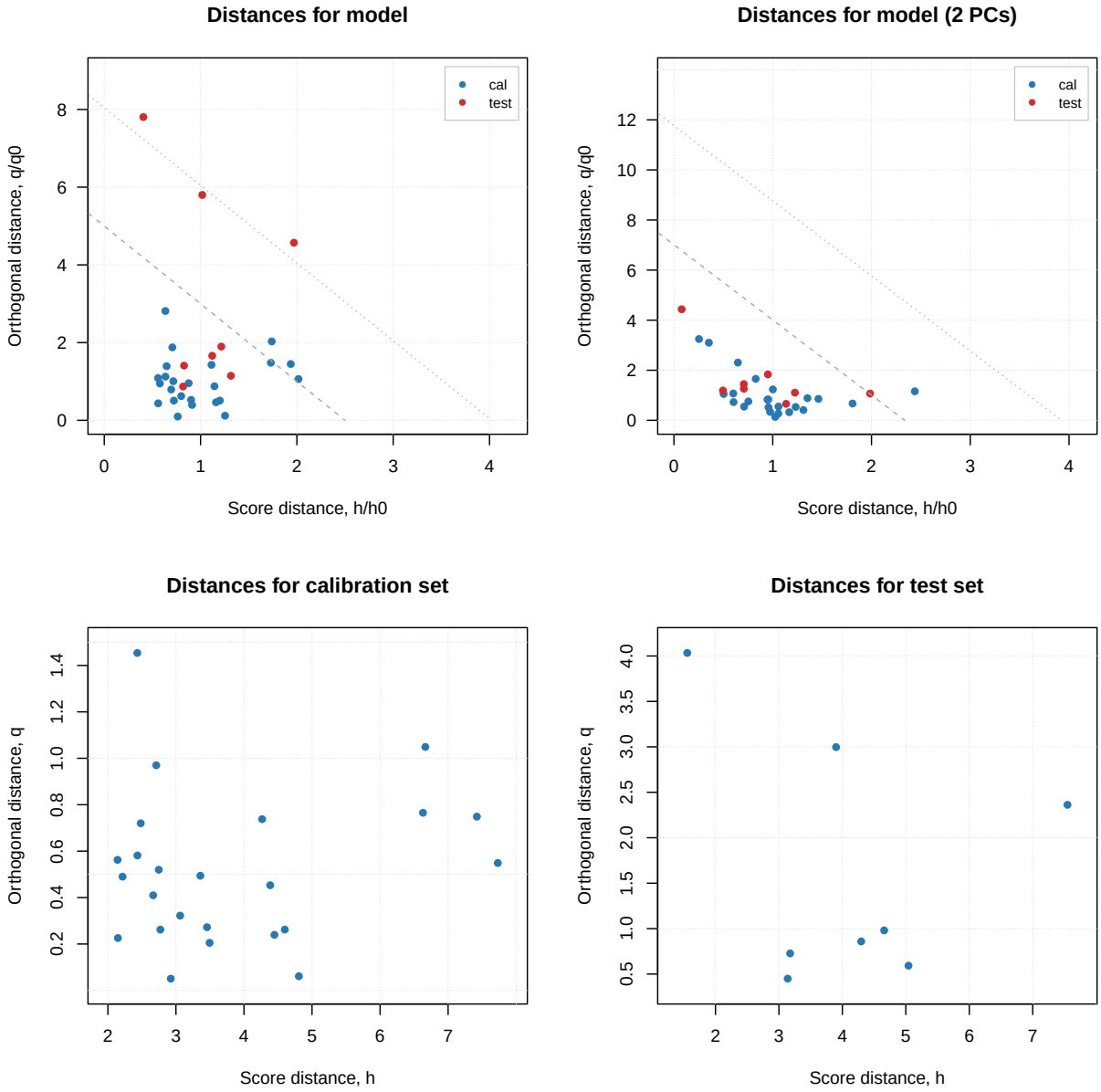
Here is an example, where I split People data to calibration and test set as in one of the examples in previous sections and show distance plot for model and result objects.

```
data(people)

# split People data by taking every 4th measurement to a test set
idx = seq(4, 32, 4)
Xc = people[-idx, ]
Xt = people[idx, ]

# train PCA model with test set validation
m = pca(Xc, 4, scale = TRUE, x.test = Xt)

# show the results
par(mfrow = c(2, 2))
plotDistances(m, main = "Distances for model")
plotDistances(m, ncomp = 2, main = "Distances for model (2 PCs)")
plotDistances(m$res$cal, main = "Distances for calibration set")
plotDistances(m$res$test, main = "Distances for test set")
```



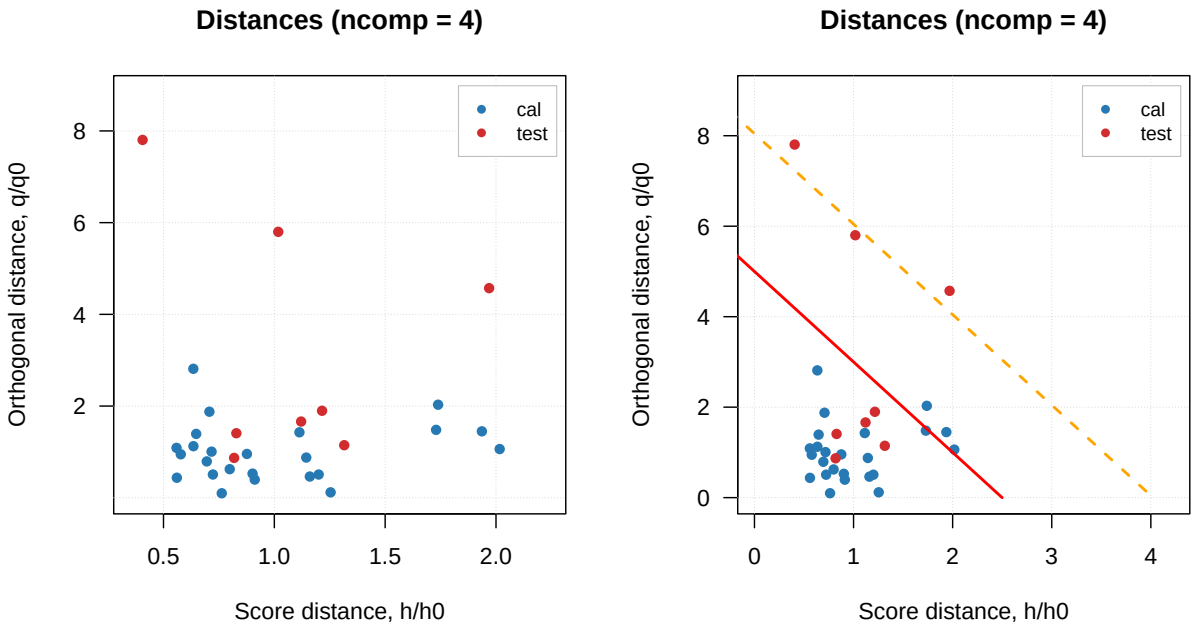
As it was briefly mentioned before, if residual/distance plot is made for a model, it also shows two critical limits: the dashed line is a limit for extreme objects and the dotted line is a limit for outliers. The position of the lines is defined by significance levels, `alpha` and `gamma`. For example, if `alpha = 0.05`, it is expected that 5% of the training samples will be beyond the critical limit for extreme objects (dashed line). Parameter `gamma` is a similar parameter for outlier detection, it defines the position of the dotted line (any object beyond this line is considered as outlier).

They are computed as critical limits using chi-square distribution, which approximates the distribution of the distance values. More details can be found in this paper.

If you want to hide them, just use additional parameter `show.limits = FALSE`. You can also change the color, line type and width for the lines by using options `lim.col`, `lim.lty` and `lim.lwd` as it is shown below.

```
par(mfrow = c(1, 2))
plotDistances(m, show.limits = FALSE)
```

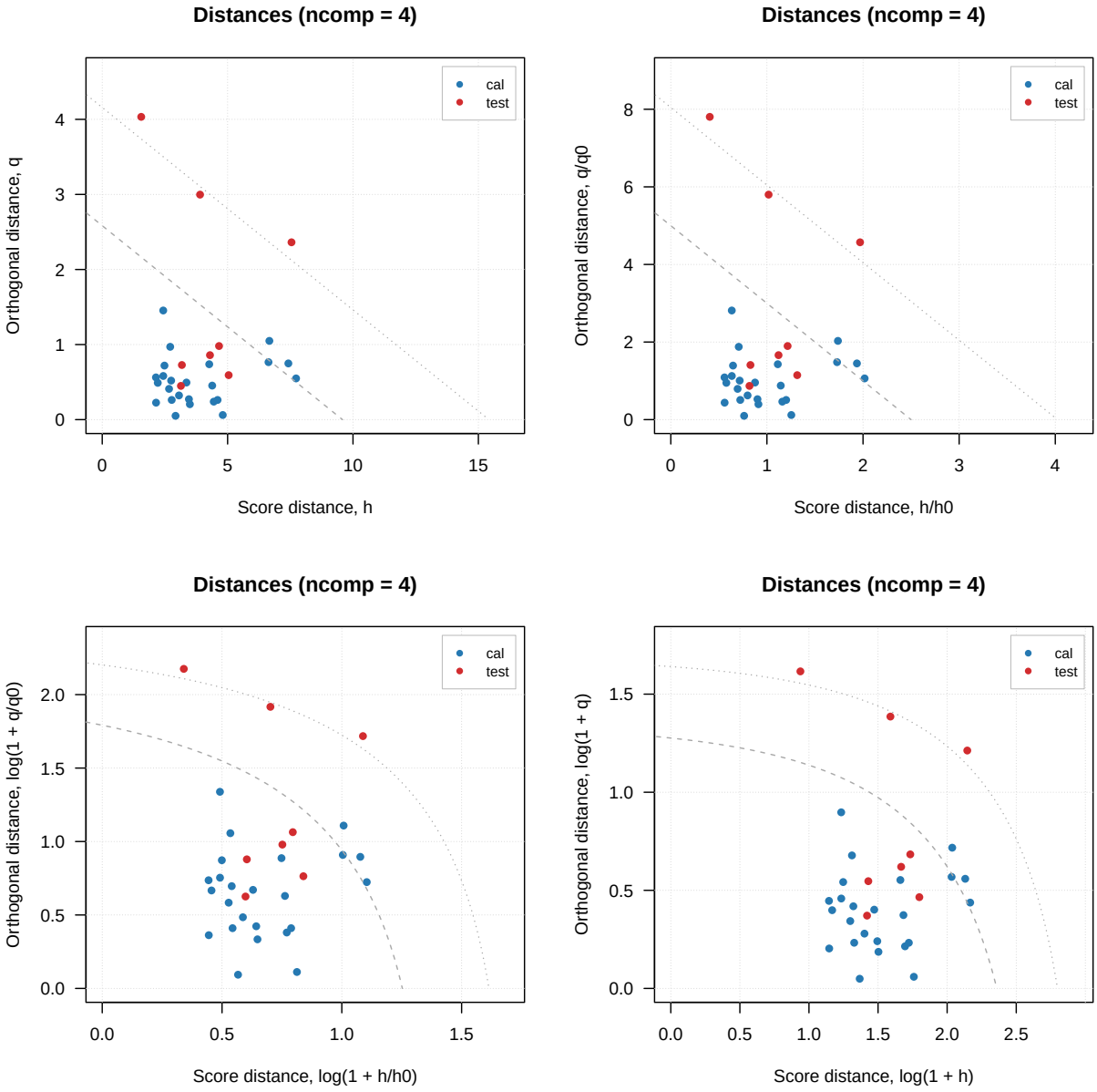
```
plotDistances(m, lim.col = c("red", "orange"), lim.lwd = c(2, 2), lim.lty = c(1, 2))
```



It is necessary to provide a vector with two values for each argument (first for extreme objects and second for outliers border).

Finally, you can also notice that for model plot the distance values are normalized (h/h_0 and q/q_0 instead of just h and q). This is needed to show the limits correctly and, again, will be explained in detail below. You can switch this option by using logical parameter `norm`. Sometimes, it also makes sense to use log transform for the values, which can be switched by using parameter `log`. Code below shows example for using these parameters.

```
par(mfrow = c(2, 2))
plotDistances(m, norm = FALSE)
plotDistances(m, norm = TRUE)
plotDistances(m, norm = TRUE, log = TRUE)
plotDistances(m, norm = FALSE, log = TRUE)
```



Model validation

PCA model can be validated using a separate validation set (in some literature *validation set* is also called *a test set*, although usually test set is used only for estimation of performance of the final, optimized model) — measurements for objects taken from the same population as the calibration set but which were not utilized for the model calibration. If you do not have a separate validation set you can generate one using Procrustes cross-validation. This section will show both approaches.

Using validation set

If a validation set is available it can be provided to `pca()` function as a value for named parameter `x.test`. In the example above we get UV/Vis spectra from `simdata` dataset. This dataset has spectra for both calibration and validation/test set. So we use the first set to calibrate a PCA model and then use the second

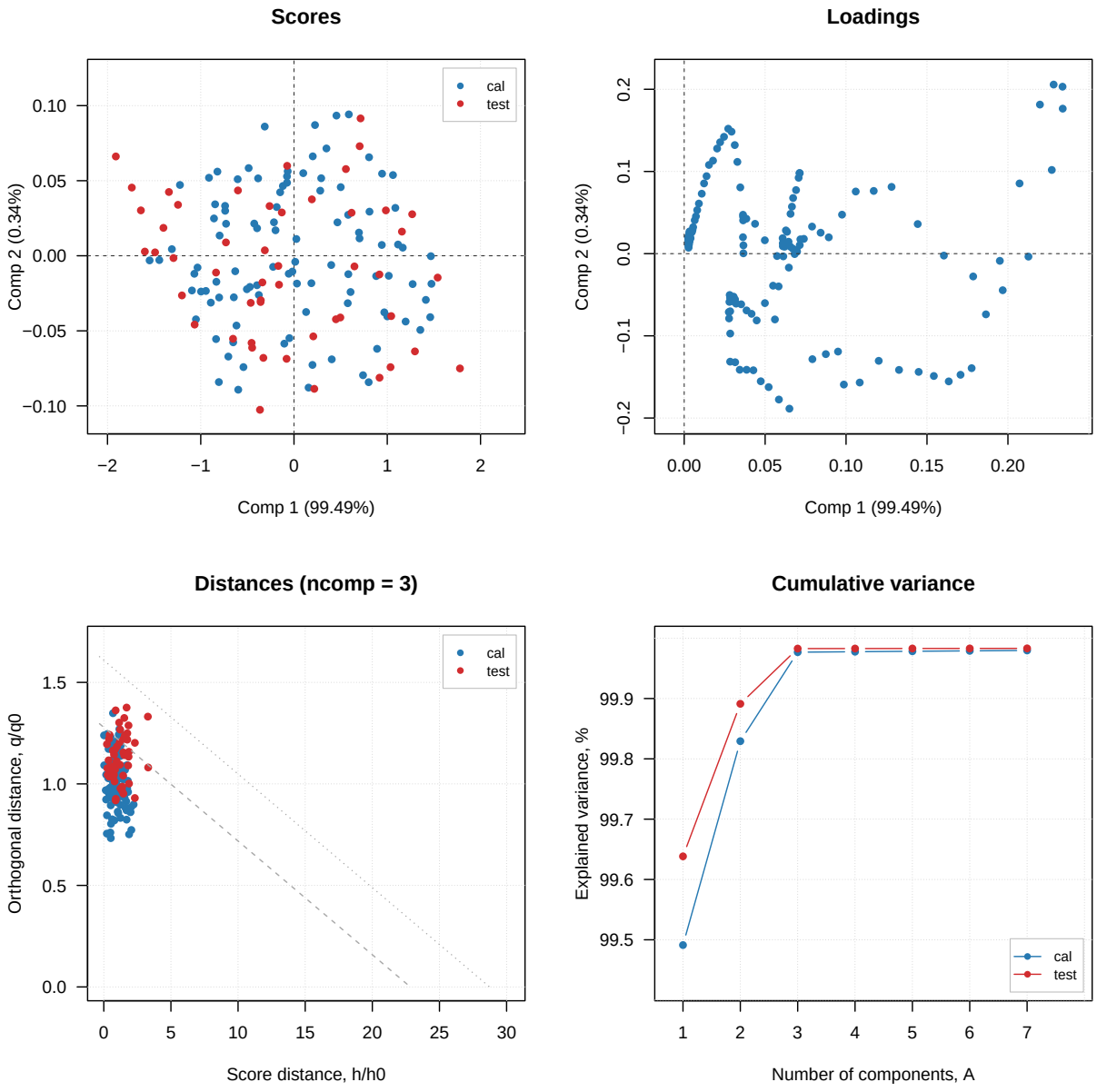
set as validation/test set. After that we created a plot overview for the model which has results from both sets.

```
data(simdata)

# define matrices with calibration and validation (test) set
Xc = simdata$spectra.c
Xt = simdata$spectra.t

# train PCA model and provide test set for validation
m = pca(Xc, 7, x.test = Xt)
```

```
# show the model overview plot
plot(m, ncomp = 3)
```



The validation results are, of course, represented by *result objects*, which are fields of a *model object* similar to `cal`, but with names `test`. And here is the info for both result objects:

```
print(m$res$cal)
```

```
##
## Results for PCA decomposition (class pcares)
##
## Major fields:
##  $scores - matrix with score values
##  $T2 - matrix with T2 distances
##  $Q - matrix with Q residuals
##  $ncomp.selected - selected number of components
##  $expvar - explained variance for each component
##  $cumexpvar - cumulative explained variance
```

```
print(m$res$test)
```

```
##
## Results for PCA decomposition (class pcares)
##
## Major fields:
##  $scores - matrix with score values
##  $T2 - matrix with T2 distances
##  $Q - matrix with Q residuals
##  $ncomp.selected - selected number of components
##  $expvar - explained variance for each component
##  $cumexpvar - cumulative explained variance
```

Let us compare, for example, the explained variance values for the results:

```
var = data.frame(
   cal = m$res$cal$expvar,
   test = m$res$test$expvar
)
show(round(var, 1))
```

```
##         cal test
## Comp 1 99.5 99.6
## Comp 2  0.3  0.3
## Comp 3  0.1  0.1
## Comp 4  0.0  0.0
## Comp 5  0.0  0.0
## Comp 6  0.0  0.0
## Comp 7  0.0  0.0
```

Every model and every result object has a method `summary()`, which shows some statistics for evaluation of a model performance. Here are some examples.

```
# summary for entire model
summary(m)
```

```
##
## Summary for PCA model (class pca)
## Type of limits: ddmoments
## Alpha: 0.05
## Gamma: 0.01
##
```

```
##        Eigenvals Expvar Cumexpvar  Nq Nh
## Comp 1     0.596  99.49     99.49   4  2
## Comp 2     0.002   0.34     99.83   3  5
## Comp 3     0.001   0.15     99.98 125  7
## Comp 4     0.000   0.00     99.98 116  8
## Comp 5     0.000   0.00     99.98 113 10
## Comp 6     0.000   0.00     99.98 107 12
## Comp 7     0.000   0.00     99.98 112 12
```

```
# summary for calibration results
summary(m$res$cal)
```

```
##
## Summary for PCA results
##
## Selected components: 7
##
##        Expvar Cumexpvar
## Comp 1  99.49     99.49
## Comp 2   0.34     99.83
## Comp 3   0.15     99.98
## Comp 4   0.00     99.98
## Comp 5   0.00     99.98
## Comp 6   0.00     99.98
## Comp 7   0.00     99.98
```

```
# summary for validation/test results
summary(m$res$test)
```

```
##
## Summary for PCA results
##
## Selected components: 7
##
##        Expvar Cumexpvar
## Comp 1  99.64     99.64
## Comp 2   0.25     99.89
## Comp 3   0.09     99.98
## Comp 4   0.00     99.98
## Comp 5   0.00     99.98
## Comp 6   0.00     99.98
## Comp 7   0.00     99.98
```

The same methodology is used for any other method, e.g. PLS or SIMCA. In the next section we will look at how to use plotting functions for models and results.

Procrustes cross-validation

If you do not have a dedicated validation/test set you can generate one using Procrustes cross-validation. The theory behind the method is available in this paper (it is Open Access), and all additional information can be found in GitHub repository of the project.

In order to use PCV you need to install a dedicated package, `pcv`, first. Here is a code which loads the `pcv` package (here I assume that you have already installed it) and creates a PV-set (analogue of validation set generated by PCV) using PCA model.

```
library(pcv)
Xpv = pcvpca(Xc, ncomp = 10, cv = list("ven", 3))
```

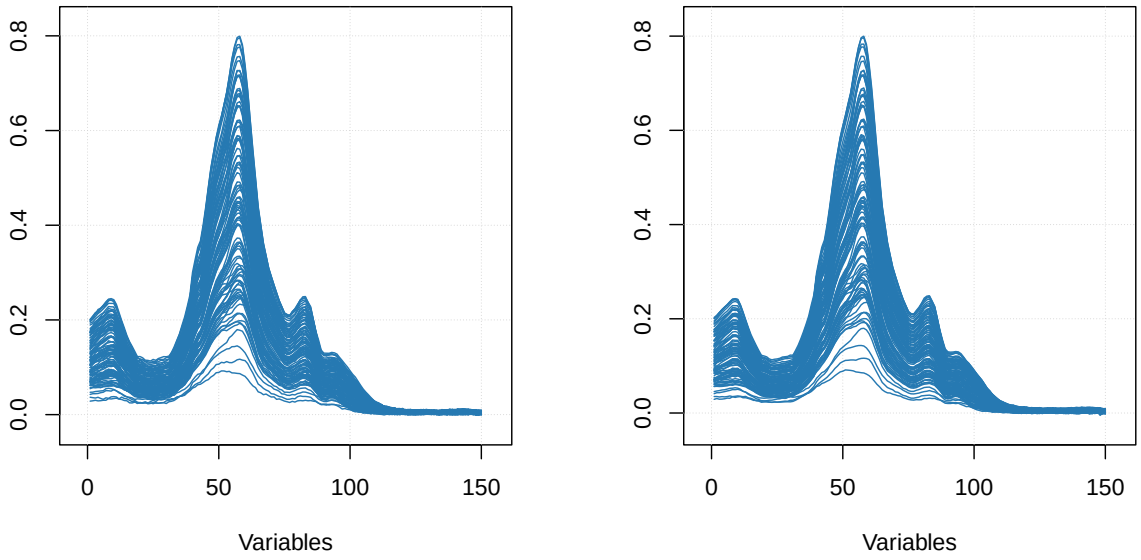
The idea behind PCV is as follows. PCV first splits rows of calibration set into several segments. Number of segments and the way to make the split (systematic or random) is defined by a value of parameter `cv`. As you can see in the code above, I used `cv = list("ven", 4)`. This means that I want to create 4 segments (subsets) using systematic split "Venetian blinds". So in the first segment I will have rows number 1, 11, 21, ... and in the second segment I will have rows: 2, 12, 22, etc.

The idea of splits is similar to cross-validation, and PCV is based on cross-validation resampling. You can also use random splits or leave-one-out splits. But in general Venetian blinds with relatively few segments (from 3 to 8) will work best. You also need to specify the number of components in the model, just use any number much larger than the expected optimal number. PCV is not sensitive to overfitting.

After the split is done it creates local PCA models for different subsets of your calibration set, measures uncertainties among the local models and then introduces this uncertainty to the calibration set hence creating a new set of measurements - *PV-set*.

The plot below compares spectra from the original calibration set (left) and the generated PV set (right). As you can see, although the shape of individual spectra looks similar the spectra are not identical.

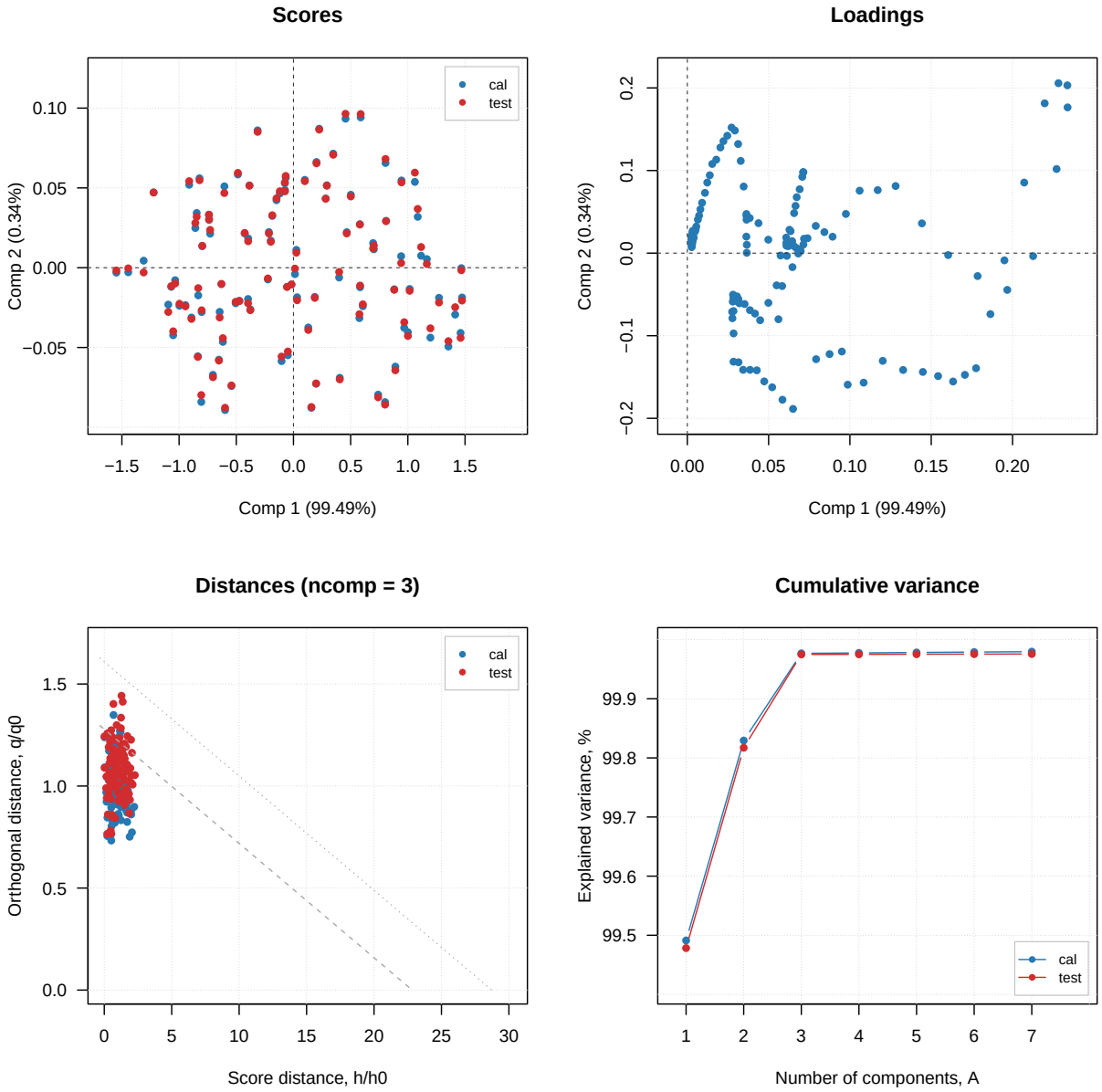
```
par(mfrow = c(1, 2))
mdaplot(Xc, type = "l")
mdaplot(Xpv, type = "l")
```



Now we can create a PCA model and validate it using the created PV-set similar to what we have done in the case of a dedicated validation set:

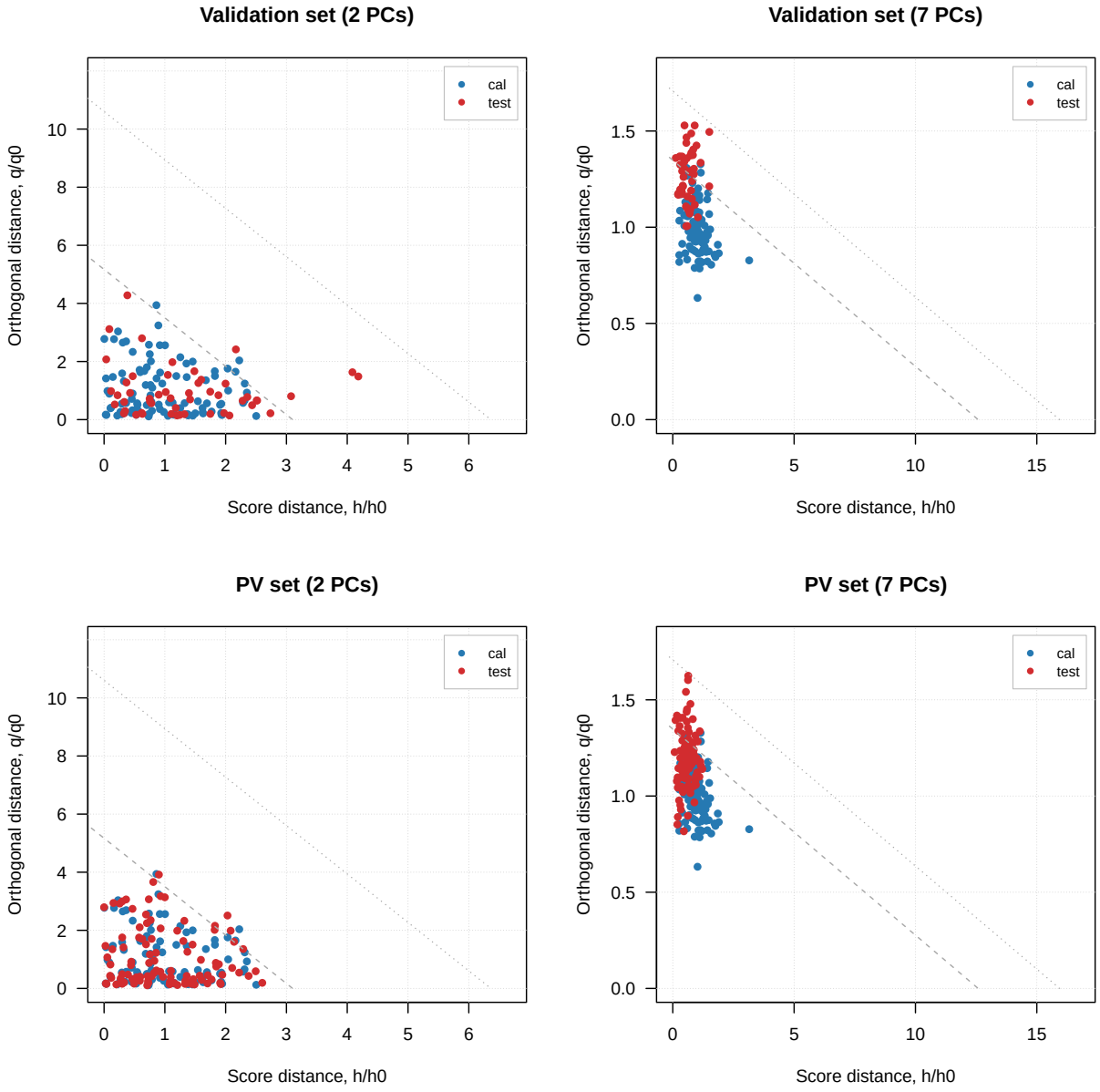
```
m2 = pca(Xc, 7, x.test = Xpv)
```

```
plot(m2, ncomp = 3)
```



The four plots below show distance plots for a model validated using a separate validation set (top) and the PV-set (bottom). The left plots are made for 2 principal components and the right plots are made for 7 components. As you can see both separate validation set and the one generated using PCV show similar patterns.

```
par(mfrow = c(2, 2))
plotDistances(m, ncomp = 2, main = "Validation set (2 PCs)")
plotDistances(m, ncomp = 7, main = "Validation set (7 PCs)")
plotDistances(m2, ncomp = 2, main = "PV set (2 PCs)")
plotDistances(m2, ncomp = 7, main = "PV set (7 PCs)")
```



Preprocessing as part of the model

From v. *0.15.0* preprocessing methods can be combined into a list and provided as additional argument to the models, including PCA. Once provided, PCA will take care of estimation of preprocessing parameters (e.g. reference spectrum for EMSC) and then will automatically apply all preprocessing methods to new data when the user calls function `predict()`.

Here is an example. We will create a PCA model for *Simdata* UV/Vis spectra but we would like to smooth the spectra first and apply SNV normalization. Here is how to do this:

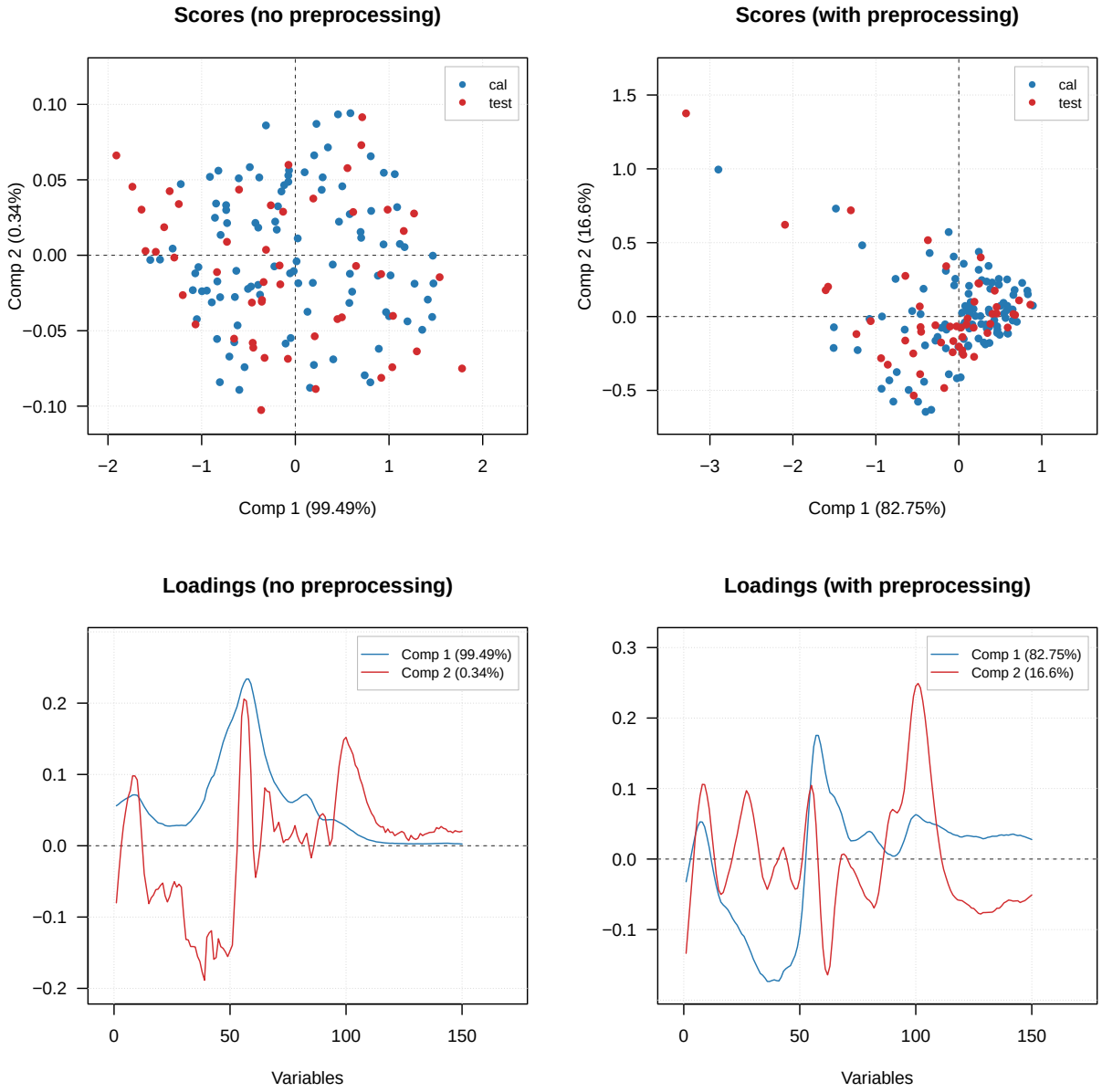
```
# load calibration and test set
Xc = simdata$spectra.c
Xt = simdata$spectra.t
```

```
# define chain of preprocessing methods
p = list(
   prep("savgol", width = 7, porder = 1, dorder = 0),
   prep("norm", type = "snv")
)

# create two PCA models - with and without preprocessing
m1 = pca(Xc, 6)
m2 = pca(Xc, 6, prep = p)

# apply the models to the test set
r1 = predict(m1, Xt)
r2 = predict(m2, Xt)

par(mfrow = c(2, 2))
plotScores(m1, res = list("cal" = m1$res$cal, "test" = r1), main = "Scores (no preprocessing)")
plotScores(m2, res = list("cal" = m2$res$cal, "test" = r2), main = "Scores (with preprocessing)")
plotLoadings(m1, comp = c(1, 2), type = "l", main = "Loadings (no preprocessing)")
plotLoadings(m2, comp = c(1, 2), type = "l", main = "Loadings (with preprocessing)")
```



As you can see there is no need to apply the preprocessing methods to the test set manually, method `predict()` takes care of everything.

Information about the preprocessing methods is also shown in the model summary:

```
summary(m2)
```

```
##
## Summary for PCA model (class pca)
## Type of limits: ddmoments
## Alpha: 0.05
## Gamma: 0.01
##
## Preprocessing methods:
##  - savgol: width = 7, porder = 1, dorder = 0
```

```
##  - norm: type = snv,
##
##        Eigenvals Expvar Cumexpvar Nq Nh
## Comp 1     0.380  82.75     82.75  1  1
## Comp 2     0.076  16.60     99.34  1  1
## Comp 3     0.001   0.20     99.55  2  1
## Comp 4     0.000   0.06     99.60  3  1
## Comp 5     0.000   0.04     99.65  3  1
## Comp 6     0.000   0.03     99.68  4  1
```

Model complexity

Complexity of PCA model is first of all associated with selection of proper (optimal) number of components. The optimal number should explain the systematic variation of the data points and keep the random variation uncaptured. Traditionally, number of components is selected by investigation of eigenvalues or looking at plot with residual or explained variance. However, this is quite a complicated issue and result of selection depends very much on quality of data and the purpose the PCA model is built for. More details can be found in this paper.

In *mdatools* there are several additional instruments both to select proper number of components as well as to see if, for example, new data (test set or new set of measurements) are well fitted by the model.

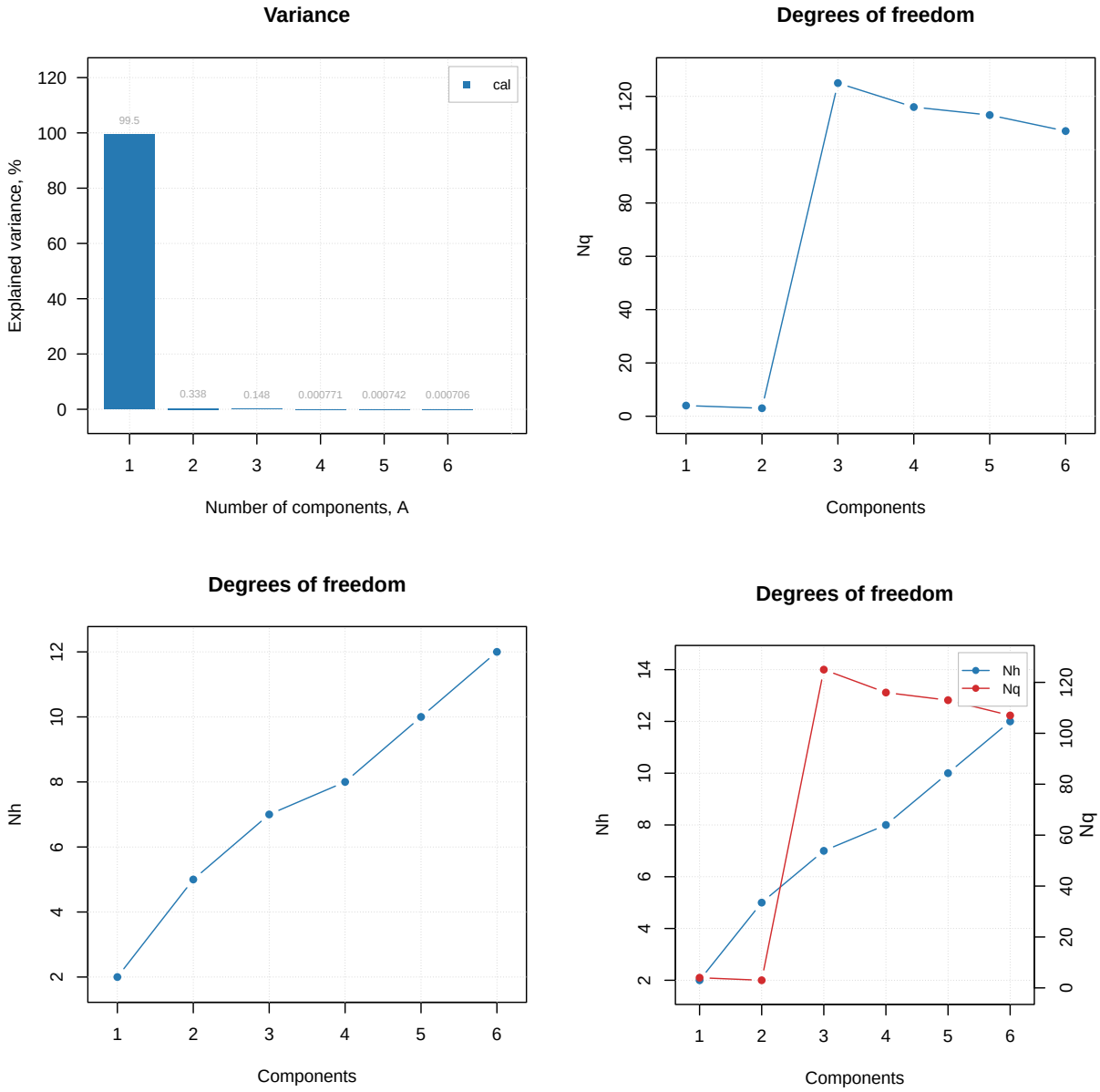
The first tool is the use of degrees of freedom for orthogonal and score distances. As we briefly mentioned before, the distances follow scaled chi-square distribution, which has two parameters, the scaling factor (h_0 for score distance, q_0 for orthogonal distance) and the number of degrees of freedom, DoF (N_h and N_q correspondingly).

A simple plot, where the number of degrees of freedom is plotted against the number of components in PCA model can often reveal the presence of overfitting — the DoF value for the orthogonal distance, N_q jumps up significantly.

The code and its outcome below show how to make such plot separately for each distance and for both. The first plot in the figure is a conventional plot with variance explained by each component. The model is made for *Simdata*, where it is known that the optimal number of components is two.

```
data(simdata)
Xc = simdata$spectra.c
Xt = simdata$spectra.t
m = pca(Xc, 6)

par(mfrow = c(2, 2))
plotVariance(m, type = "h", show.labels = TRUE)
plotQDoF(m)
plotT2DoF(m)
plotDistDoF(m)
```



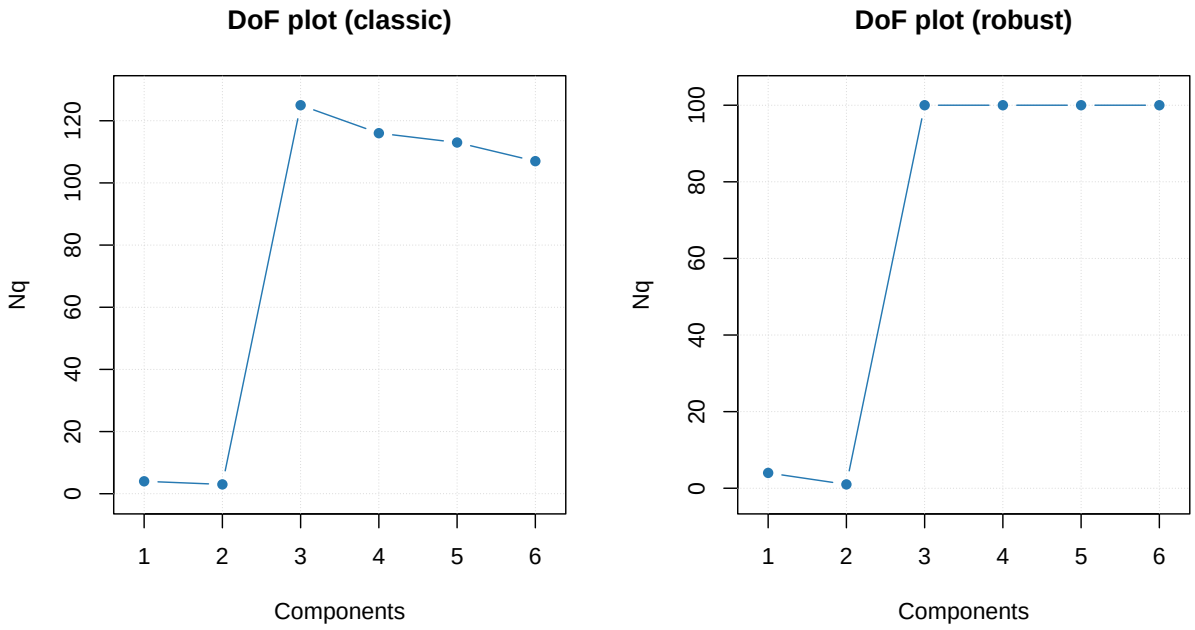
As you can see from the plots, the second component explains less than 1% of variance and can be considered insignificant. However, plot for N_q shows a clear break at $A = 3$, indicating that both first and second PCs are important. The N_h values in this case do not provide any useful information.

If outliers are present it can be useful to investigate the plot for N_q using classical and robust methods for estimation as shown in example below.

```
m = pca(Xc, 6)

par(mfrow = c(1, 2))
plotQDoF(m, main = "DoF plot (classic)")

m = setDistanceLimits(m, lim.type = "ddrobust")
plotQDoF(m, main = "DoF plot (robust)")
```



In this case both plots demonstrate quite similar behavior, however if they look different it can be an indicator of the presence of outliers. The DoF plots work only if a data-driven method is used for computing of critical limits.

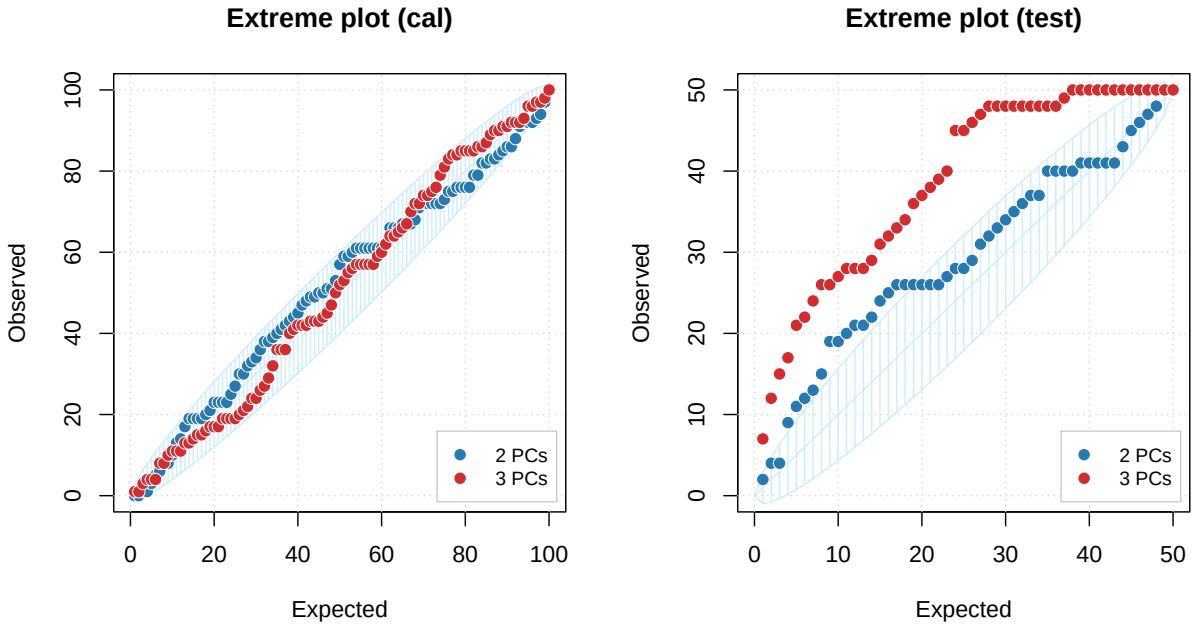
Another way to see how many components are optimal in the model, in case you have a test set or just a new set of measurements you want to use the model with, is to employ the Extreme plot. The plot shows a number of extreme values for different α. Imagine that you make several distance plots with different α values and count how many objects model found as extremes. On the other hand you know that the expected value is αI. If you plot the number of extreme values vs. the expected number — this is the Extreme plot.

If the model captures systematic variation both for calibration and test set the points on this plot will lie within a confidence ellipse shown using light blue color. However, if model is overfitted, the points are getting outside it.

Below you can see two Extreme plots made for the same data as the previous example. The left plot shows results for calibration set for number of components in the PCA model equal to 2 and 3. The right plot shows the results for the test set.

```
Xc = simdata$spectra.c
Xt = simdata$spectra.t
m = pca(Xc, 6, x.test = Xt)

par(mfrow = c(1, 2))
plotExtreme(m, comp = 2:3, main = "Extreme plot (cal)")
plotExtreme(m, comp = 2:3, res = m$res$test, main = "Extreme plot (test)")
```



As you can see, in case of calibration set, the points are lying within the confidence ellipse for both $A = 2$ and $A = 3$. However for the test set, the picture is quite different. In case of $A = 2$ most of the points are inside the interval, but for $A = 3$ all of them are clearly outside.

We hope these new tools will make the use of PCA more efficient.

Randomized PCA algorithms

Both SVD and NIPALS are not very efficient when the number of rows in a dataset is very large (e.g. hundreds of thousands values or even more). Such datasets can be easily obtained in case of for example hyperspectral images. Direct use of the traditional algorithms with such datasets often leads to a lack of memory and long computational time.

One of the solutions here is to use probabilistic algorithms, which allow to reduce the number of values needed for estimation of principal components. Starting from *0.9.0* a probabilistic approach is also implemented in *mdatools*. The original idea can be found in this paper and some examples on using the approach for PCA analysis of hyperspectral images are described here.

The approach is based on calculation of matrix B, which has a much smaller number of rows compared to the original data, but captures the action of the data. To produce proper values for B, several parameters are used. First of all it is a rank of original data, k, if it is known. Second, it is oversampling parameter, p, which is important if rank is underestimated. The idea is to use p to overestimate the rank thus making solution more stable. The third parameter, q, is the number of iterations needed to make B more robust. Usually using $p = 5$ and $q = 1$ will work well on most of the datasets and, at the same time, will take less time for finding a solution compared with conventional methods. By default, k is set to number of components, used for PCA decomposition.

The example below uses two methods (classic SVD and randomized SVD) for PCA decomposition of 100 000 x 300 dataset and compares time and main outcomes (loadings and explained variance).

```
# create a dataset as a linear combination of three sin curves with random "concentrations"
n = 100000
X = seq(0, 29.9, by = 0.1)
```

```
S = cbind(sin(X), sin(10 * X), sin(5 * X))
C = cbind(runif(n, 0, 1), runif(n, 0, 2), runif(n, 0, 3))
D = C %*% t(S)
D = D + matrix(runif(300 * n, 0, 0.5), ncol = 300)
show(dim(D))
```

```
## [1] 100000    300
```

```
# conventional SVD
t1 = system.time({m1 = pca(D, ncomp = 2)})
show(t1)
```

```
##    user  system elapsed
##  10.359   1.036   1.850
```

```
# randomized SVD with p = 5 and q = 1
t2 = system.time({m2 = pca(D, ncomp = 2, rand = c(5, 1))})
show(t2)
```

```
##    user  system elapsed
##   2.409   0.179   0.893
```

```
# compare variances
summary(m1)
```

```
##
## Summary for PCA model (class pca)
## Type of limits: ddmoments
## Alpha: 0.05
## Gamma: 0.01
##
##        Eigenvals Expvar Cumexpvar Nq Nh
## Comp 1   112.557  62.08     62.08  4  3
## Comp 2    49.971  27.56     89.63  6  5
```

```
summary(m2)
```

```
##
## Summary for PCA model (class pca)
##
## Parameters for randomized algorithm: p = 5, q = 1
## Type of limits: ddmoments
## Alpha: 0.05
## Gamma: 0.01
##
##        Eigenvals Expvar Cumexpvar Nq Nh
## Comp 1   112.557  62.08     62.08  4  3
## Comp 2    49.971  27.56     89.63  6  5
```

```
# compare loadings
show(m1$loadings[1:10, ])
```

```
##              Comp 1        Comp 2
##  [1,]  7.153341e-05 -1.674804e-05
##  [2,]  3.866555e-02 -6.926634e-02
##  [3,]  6.827530e-02 -7.513861e-02
##  [4,]  8.137169e-02 -1.238775e-02
##  [5,]  7.458268e-02  6.125458e-02
```

```
##  [6,]  4.930915e-02  7.793674e-02
##  [7,]  1.166580e-02  2.290402e-02
##  [8,] -2.893568e-02 -5.313758e-02
##  [9,] -6.217729e-02 -7.988570e-02
## [10,] -7.994828e-02 -3.243878e-02
```

```
show(m2$loadings[1:10, ])
```

```
##              Comp 1        Comp 2
##  [1,]  7.156171e-05  1.684982e-05
##  [2,]  3.866553e-02  6.926619e-02
##  [3,]  6.827530e-02  7.513861e-02
##  [4,]  8.137169e-02  1.238777e-02
##  [5,]  7.458266e-02 -6.125460e-02
##  [6,]  4.930914e-02 -7.793681e-02
##  [7,]  1.166579e-02 -2.290410e-02
##  [8,] -2.893569e-02  5.313758e-02
##  [9,] -6.217728e-02  7.988575e-02
## [10,] -7.994829e-02  3.243875e-02
```

As you can see the explained variance values, eigenvalues and loadings are almost identical in the two models and the second method is several times faster.

It is possible to make PCA decomposition even faster if only loadings and scores are needed. In this case you can use method `pca.run()` and skip other steps, like calculation of distances, variances, critical limits and so on. But in this case data matrix must be centered (and scaled if necessary) manually prior to the decomposition. Here is an example using the data generated in previous code.

```
D = scale(D, center = TRUE, scale = FALSE)

# conventional SVD
t1 = system.time({m1 = pca.run(D, method = "svd", ncomp = 2)})
show(t1)
```

```
##    user  system elapsed
##   8.338   0.351   0.821
```

```
# randomized SVD with p = 5 and q = 1
t2 = system.time({m2 = pca.run(D, method = "svd", ncomp = 2, rand = c(5, 1))})
show(t2)
```

```
##    user  system elapsed
##   0.585   0.151   0.104
```

```
# compare loadings
show(m1$loadings[1:10, ])
```

```
##               [,1]          [,2]
##  [1,]  7.153341e-05 -1.674804e-05
##  [2,]  3.866555e-02 -6.926634e-02
##  [3,]  6.827530e-02 -7.513861e-02
##  [4,]  8.137169e-02 -1.238775e-02
##  [5,]  7.458268e-02  6.125458e-02
##  [6,]  4.930915e-02  7.793674e-02
##  [7,]  1.166580e-02  2.290402e-02
##  [8,] -2.893568e-02 -5.313758e-02
##  [9,] -6.217729e-02 -7.988570e-02
## [10,] -7.994828e-02 -3.243878e-02
```

```
show(m2$loadings[1:10, ])
```

```
##                [,1]          [,2]
##  [1,]  7.153313e-05  1.685131e-05
##  [2,]  3.866554e-02  6.926639e-02
##  [3,]  6.827530e-02  7.513868e-02
##  [4,]  8.137170e-02  1.238765e-02
##  [5,]  7.458270e-02 -6.125456e-02
##  [6,]  4.930916e-02 -7.793674e-02
##  [7,]  1.166579e-02 -2.290399e-02
##  [8,] -2.893567e-02  5.313742e-02
##  [9,] -6.217731e-02  7.988569e-02
## [10,] -7.994828e-02  3.243883e-02
```

As you can see the loadings are still the same but the probabilistic algorithm is about 10 times faster. The exact difference depends on the computer, of course, so if you run these examples on your computer you will get a bit different results, however the randomized algorithm will always be a clear winner.

Interoperability with web-applications

Similar to preprocessing there is an interactive web-application, https://mda.tools/pca, which lets you create PCA models and apply them to a new dataset in any modern browser. Any PCA model created using `pca()` can be saved into a JSON file and then be loaded into the *Project* tab of the web-application. Including preprocessing methods!

Here is an example based on *Simdata*:

```
# define calibration set
Xc <- simdata$spectra.c

# create a sequence of preprocessing methods
p <- list(
   prep("savgol", width = 9, porder = 1, dorder = 0),
   prep("norm", type = "snv")
)

# create a PCA model with preprocessing as a part of the model
m <- pca(Xc, 6, prep = p)

# save the model to JSON file
writeJSON(m, "pca-model.json")
```

If you run the code on your computer you will see a new `pca-model.json` file created in the working directory. Now open https://mda.tools/pca in your browser, select tab *Project* and load the file with the model. You will see something similar to the following:

After that you can upload dataset and see the result of projecting it to the PCA model directly in the app.

Likewise, any PCA model created in the application can be saved into a JSON file and loaded into R, so you can use it via `predict()`. For example this JSON file I created using the *Train* tab of the app and *Simdata*. It contains a PCA model on data with a preprocessing chain that includes SG smoothing, unit area normalization, and mean centering. Download it to your working directory and run the following code (here I assume you already have `Xc` matrix from the previous blocks of code):

```
# load PCA model from JSON file
m <- readJSON("pca-model-fromweb.json")
```

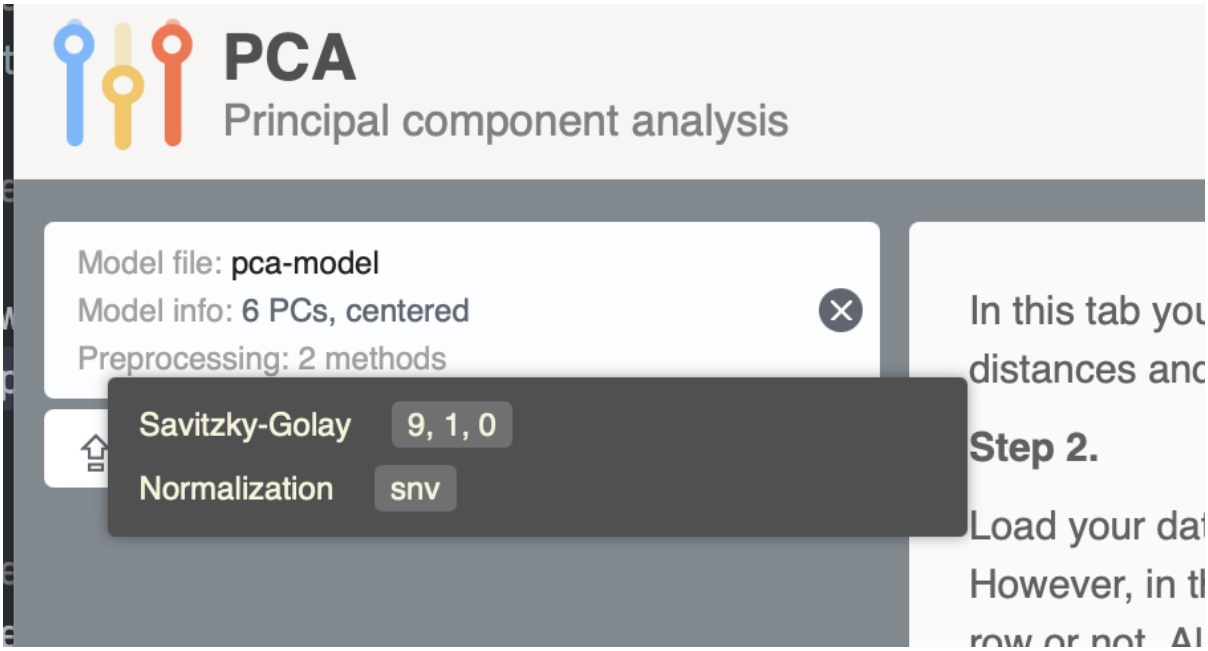
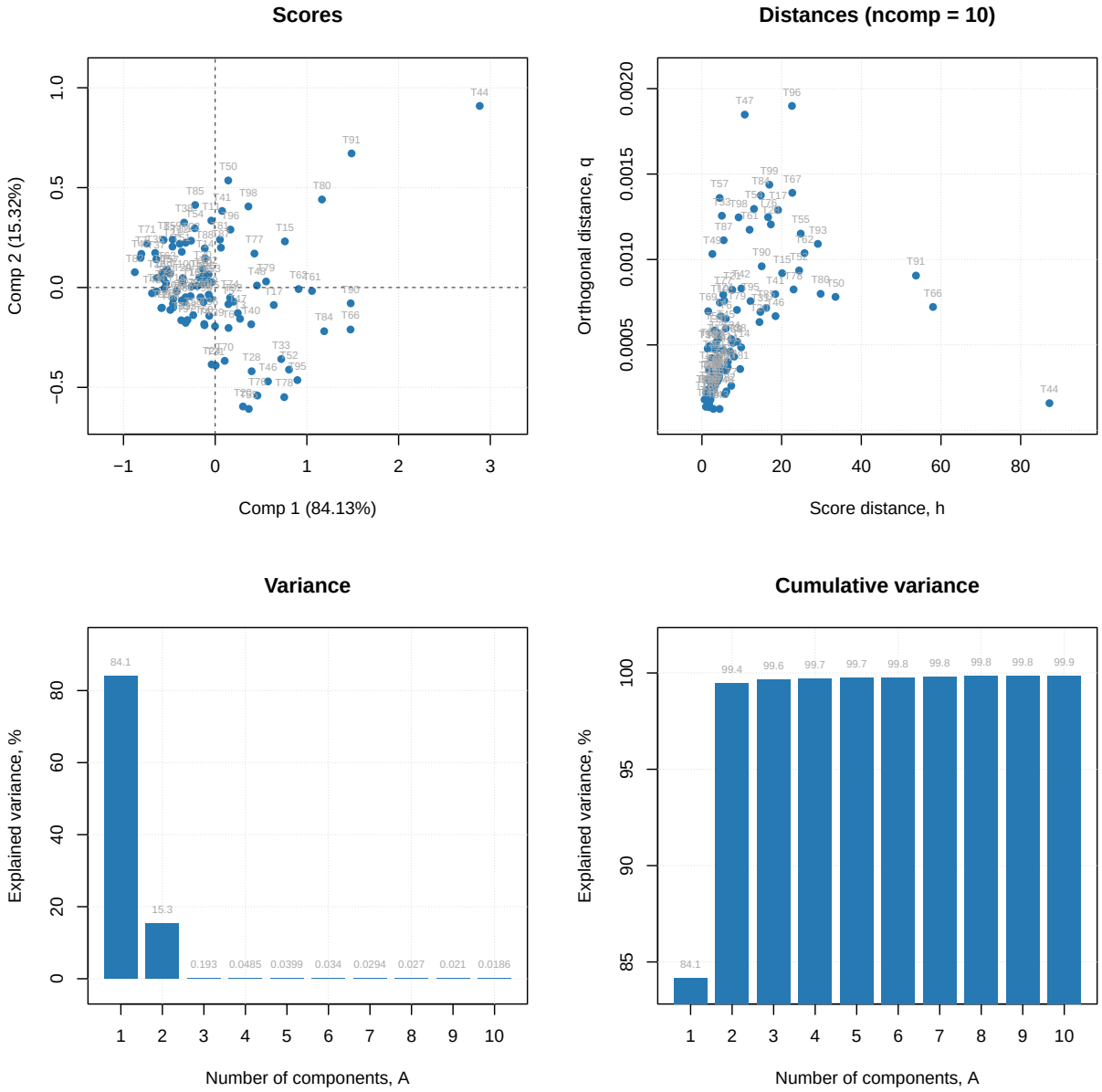


Figure 2: Screenshot with PCA model loaded to mda.tools/pca

```
# apply it to Xc
r <- predict(m, Xc)
summary(r)
```

```
##
## Summary for PCA results
##
## Selected components: 10
##
##          Expvar Cumexpvar
## Comp 1    84.13     84.13
## Comp 2    15.32     99.45
## Comp 3     0.19     99.64
## Comp 4     0.05     99.69
## Comp 5     0.04     99.73
## Comp 6     0.03     99.76
## Comp 7     0.03     99.79
## Comp 8     0.03     99.82
## Comp 9     0.02     99.84
## Comp 10    0.02     99.86
```

```
# show plots
plot(r)
```



There is also a limitation though, model loaded from JSON file created by the web-application does not contain calibration results, it can only be used for predictions/projections:

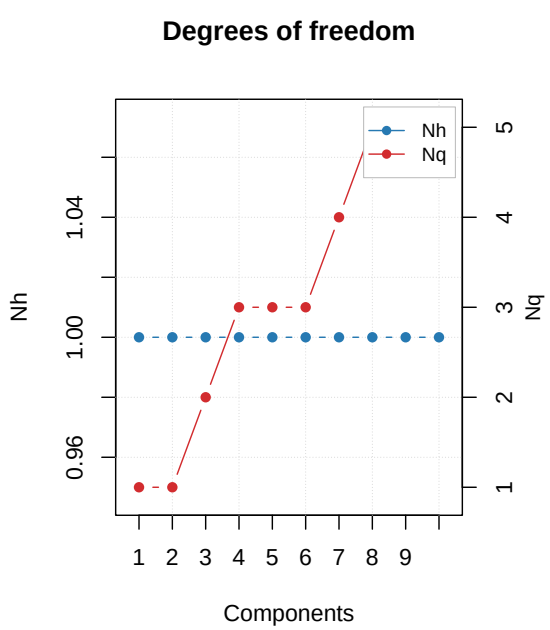
```
summary(m)
```

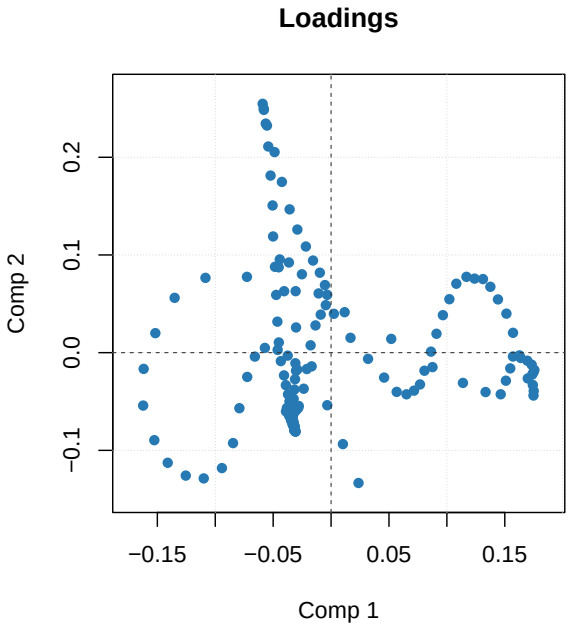
```
##
## Summary for PCA model (class pca)
## Type of limits: ddmoments
## Alpha: 0.05
## Gamma: 0.01
##
## Preprocessing methods:
##  - savgol: width = 9, porder = 1, dorder = 0
##  - norm: type = snv,
```

```
## Calibration results not found (most probably this model is loaded from web-application).
```

Most of the plots, which rely on calibration set results will not work with such model object and give you the same message as above. However plots, which are only based on model outcomes, such as loadings, eigenvalues and degrees of freedom are available:

```
par(mfrow = c(1, 2))
plotDistDoF(m)
plotLoadings(m)
```





DD-SIMCA classification

Data Driven SIMCA (Soft Independent Modelling of Class Analogy) is a simple but efficient one-class classification method mainly based on PCA. The general idea is to create a PCA model using only samples/objects belonging to a target class and classify new objects based on how well the model can fit them. The decision is made using the two distances we discussed in the PCA chapter — orthogonal and score distances and corresponding critical limits.

Critical limits computed for both distances (or their combination) are used to cut-off the strangers (extreme objects) and accept class members with a pre-defined expected ratio of false negatives (α). In DD-SIMCA the critical limits and other outcomes are computed by fitting the distance values with scaled chi-square distribution.

All theoretical background as well as some practical aspects of DD-SIMCA can be found in this paper. It has open access, freely available for everyone. Moreover, it is advised to read the paper first, as all examples below are based on similar examples and the same *Oregano* dataset, as discussed in the paper.

It should also be noted that any DD-SIMCA model is also a PCA model object and any DD-SIMCA result is also a PCA result object, therefore all plots, methods, statistics, available for PCA, can be used for SIMCA model and result objects as well.

Calibration and validation

The model calibration is similar to PCA, but there are several additional arguments, which are important for classification. First of all, it is a class name, which is a second mandatory argument. The class name is a string, which can be used later e.g. for identifying class members for testing.

You can also provide significance level for extreme objects, `alpha`, and significance level for detection of outliers, `gamma`. By default 0.05 (5%) and 0.01 (1%) will be used. These two parameters can be also adjusted later without rebuilding the whole model.

By default DD-SIMCA model computes critical limits and does all classification using two types of estimator: `"classic"` (sometimes also referred by `"moments"`) and `"robust"`. The robust estimator should be used for detection of outliers, while the final model should use the classic one. You do not need to preselect the type of the limits, any plot and any outcomes can be shown for both types of limits by providing argument `limType` as it will be shown below.

To run all examples you need to download dataset *Oregano* from web-application: here is a direct link: https://mda.tools/ddsimca/Oregano.zip. Download and unzip it to the same folder where you plan to keep your R code. In my case all data files are collected inside `Oregano` folder.

Let's load the training set first and prepare it for the model:

```
library(mdatools)

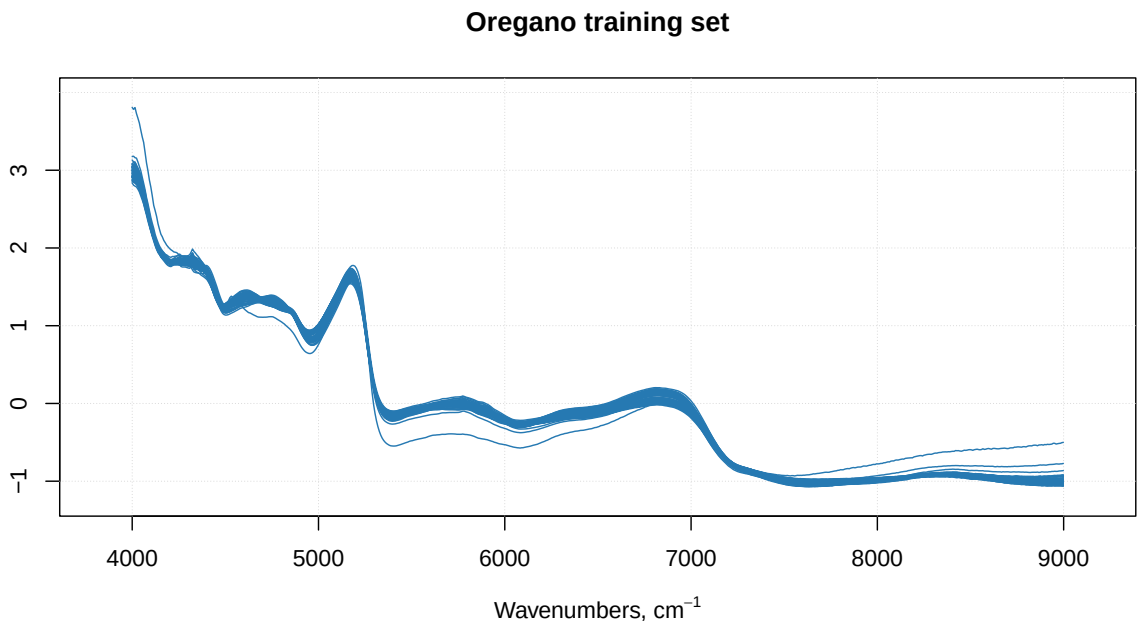
dc <- read.csv("./Oregano/Target_Train.csv", row.names = 1, check.names = FALSE)
show(dc[1:5, 1:5])
```

```
##         Class  8998.367  8990.654  8982.939  8975.226
## Bio01 Oregano -1.036600 -1.036965 -1.034487 -1.032063
## Bio02 Oregano -1.040406 -1.035694 -1.035552 -1.035854
## Bio03 Oregano -1.042949 -1.041892 -1.039863 -1.041063
## Bio04 Oregano -1.052614 -1.050210 -1.050617 -1.052789
## Bio05 Oregano -1.034859 -1.032732 -1.033896 -1.032058
```

As you can see, the first column, `Class`, contains the class labels, which we do not need for training (we need to know the label though). The other columns contain values of NIR spectra of Oregano samples, already preprocessed. The column names are wavenumbers in reciprocal centimeters, let's take them and assign as xaxis-values:

```
xc = as.matrix(dc[, -1])
w = as.numeric(colnames(xc))
attr(xc, "xaxis.values") = w
attr(xc, "xaxis.name") = expression("Wavenumbers, cm"^{-1})

mdaplot(xc, type = "l", main = "Oregano training set")
```



We can notice one or two spectra which behave differently, we will discuss them later.

Now we can train the model:

```
m = ddsimca(xc, "Oregano", ncomp = 10, pcv = list("ven", 4))
summary(m)
```

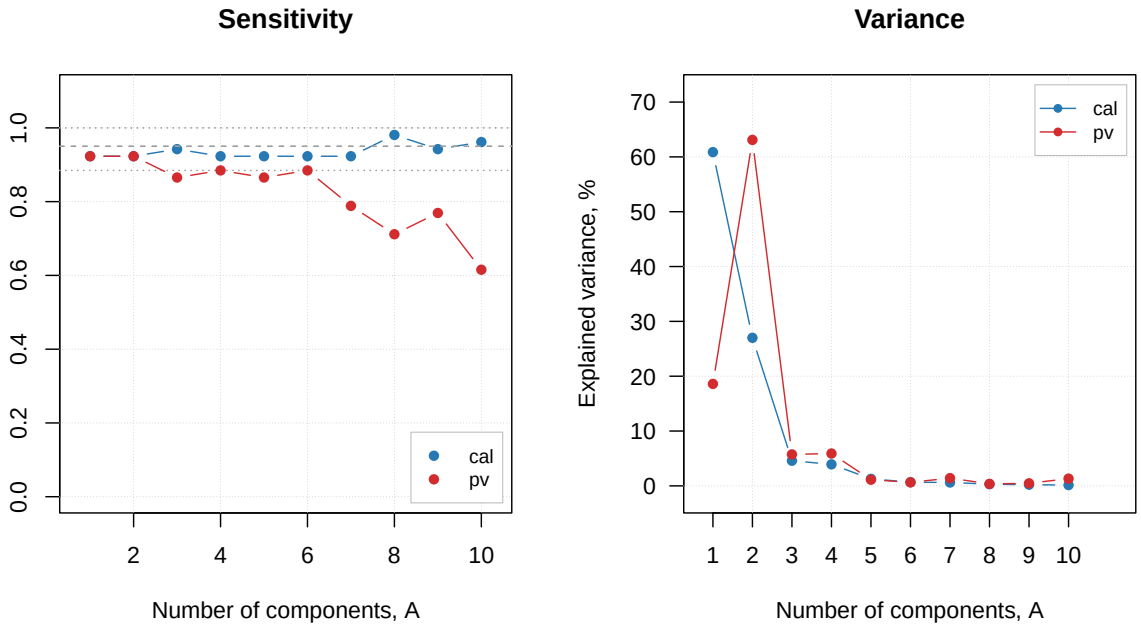
```
##
## DD-SIMCA model for class 'Oregano'
##
## Number of components: 10
## Number of selected components: 10
## Type of limits: classic
## Alpha: 0.05
## Gamma: 0.01
```

```
##
##     Expvar Cumexpvar In Out TP FN   Sens
## Cal 0.1458     99.61 50   2 50  2 0.9615
## Pv  1.3179     98.64 32  20 32 20 0.6154
```

As you may notice, we provided value for argument `pcv`. These are parameters for Procrustes cross-validation, which was introduced in the PCA chapter. Here, instead of creating the PV-set manually, DD-SIMCA does everything itself. As one can see, the model with $A = 10$ components performs badly on PV-set with sensitivity of 0.6154.

Let's start with looking at the *Sensitivity plot* and the PC variance:

```
par(mfrow = c(1, 2))
plotSensitivity(m, show.ci = TRUE)
plotVariance(m)
```

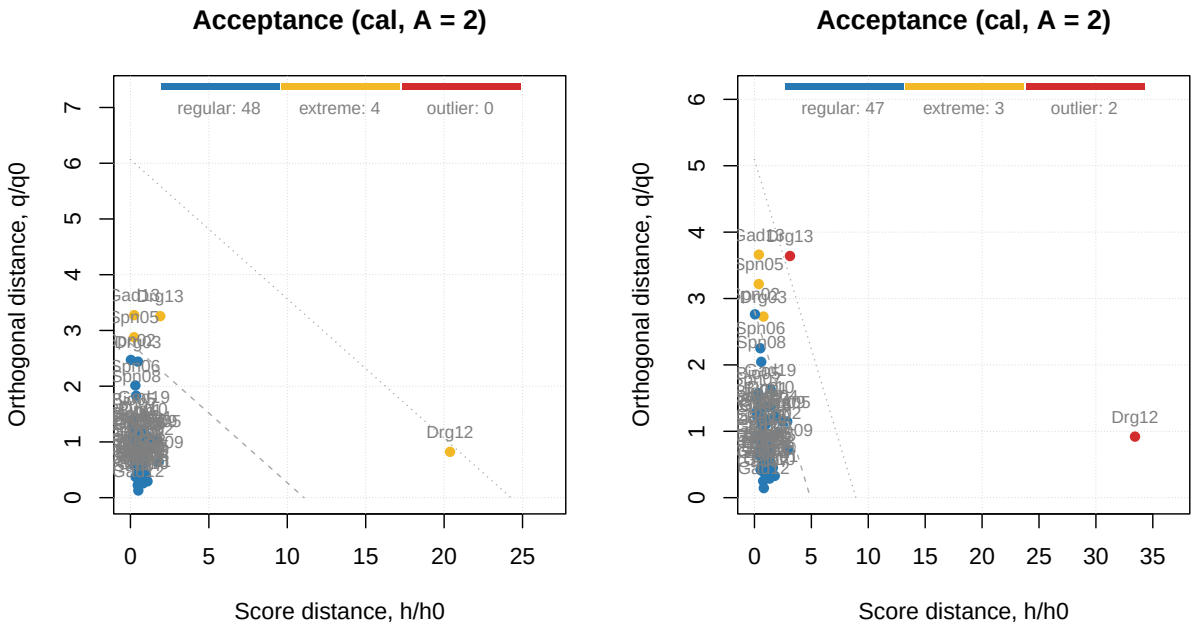


The parameter `show.ci` for sensitivity plot adds 95% confidence interval for the expected sensitivity which is computed based on the significance level and the size of the dataset. The dashed point in the middle shows the expected value and the two dotted lines — the boundaries of the interval.

As one can see, the sensitivity for PV-set (red color) is within the interval for the first two PCs, and then falling apart. After $A = 6$ the overfitting becomes obvious. The variance plot shows a large discrepancy between the individual variance values for training and PV-sets which possibly indicates the presence of outliers.

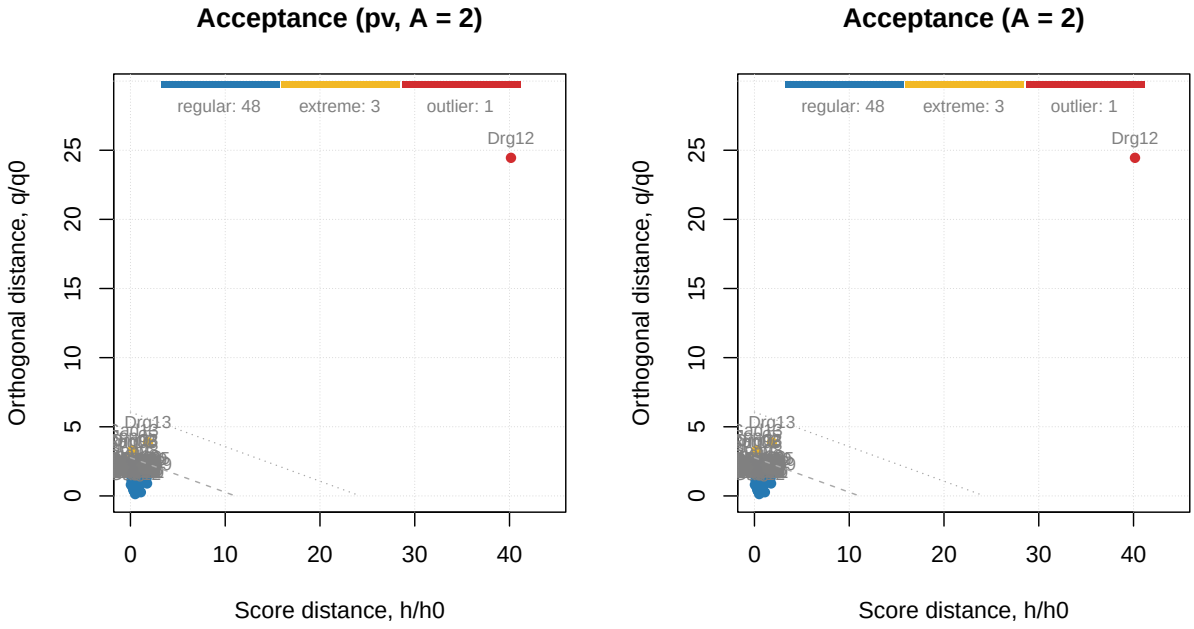
The next code shows *Acceptance plot* for $A = 2$. This plot is similar to PCA's *Distance plot*; however, in this case the objects are also color coded according to their role (regular, extreme or outlier). The plot can be shown using robust and classic estimators. The code in fact reproduces two top plots from the Figure 2 of the paper.

```
par(mfrow = c(1, 2))
plotAcceptance(m, ncomp = 2, show.labels = TRUE)
plotAcceptance(m, ncomp = 2, limType = "robust", show.labels = TRUE)
```



As you can see, the plot is made for calibration set. If you want to make it for PV-set, either specify parameter `res` or simply make it for the result object:

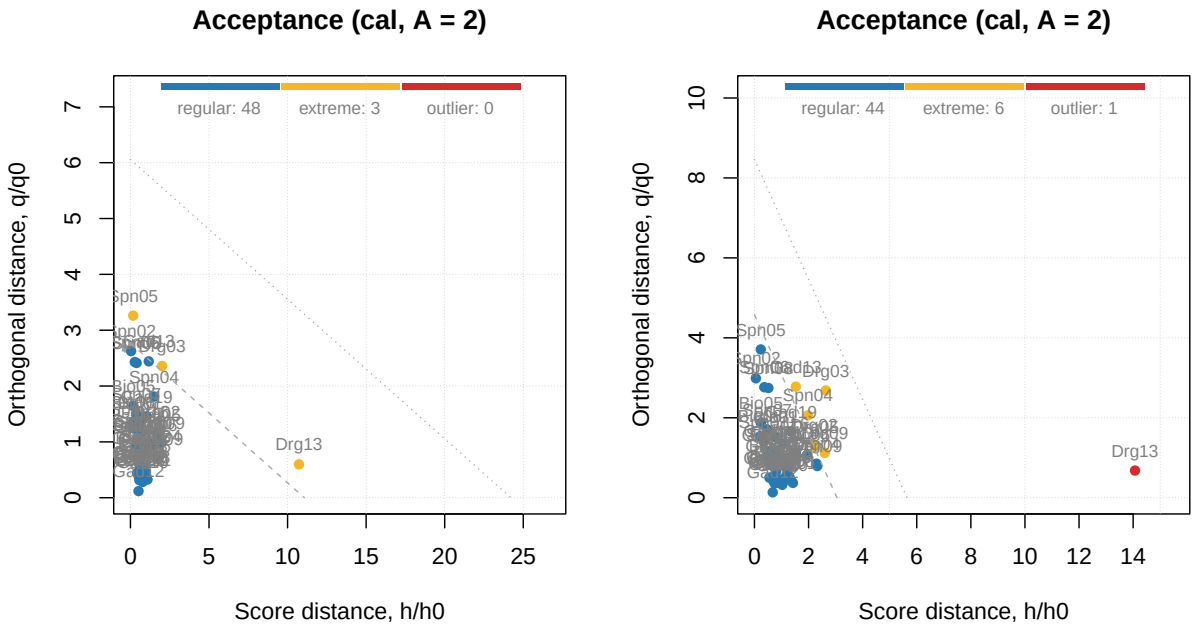
```
par(mfrow = c(1, 2))
plotAcceptance(m, res = "pv", ncomp = 2, show.labels = TRUE)
plotAcceptance(m$res$pv, ncomp = 2, show.labels = TRUE)
```



Anyway, on both sets of plots, `Drg12` is clearly an outlier. Let's remove it and re-create the model. There are two possibilities, either you can remove it from the original dataset or just provide the name of the sample to the model using `exclrows` parameter. Let's try the second option:

```
m = ddsimca(xc, "Oregano", ncomp = 10, pcv = list("ven", 4), exclrows = c("Drg12"))

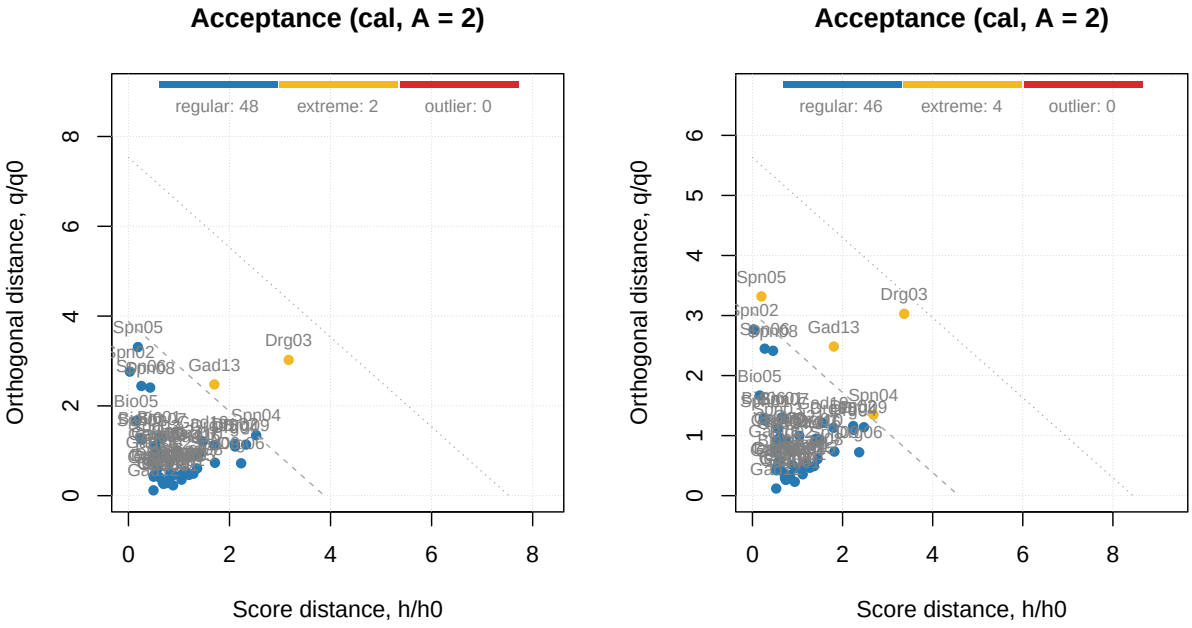
par(mfrow = c(1, 2))
plotAcceptance(m, ncomp = 2, show.labels = TRUE)
plotAcceptance(m, ncomp = 2, limType = "robust", show.labels = TRUE)
```



Apparently the object `Drg13` is an outlier as well, let's exclude it:

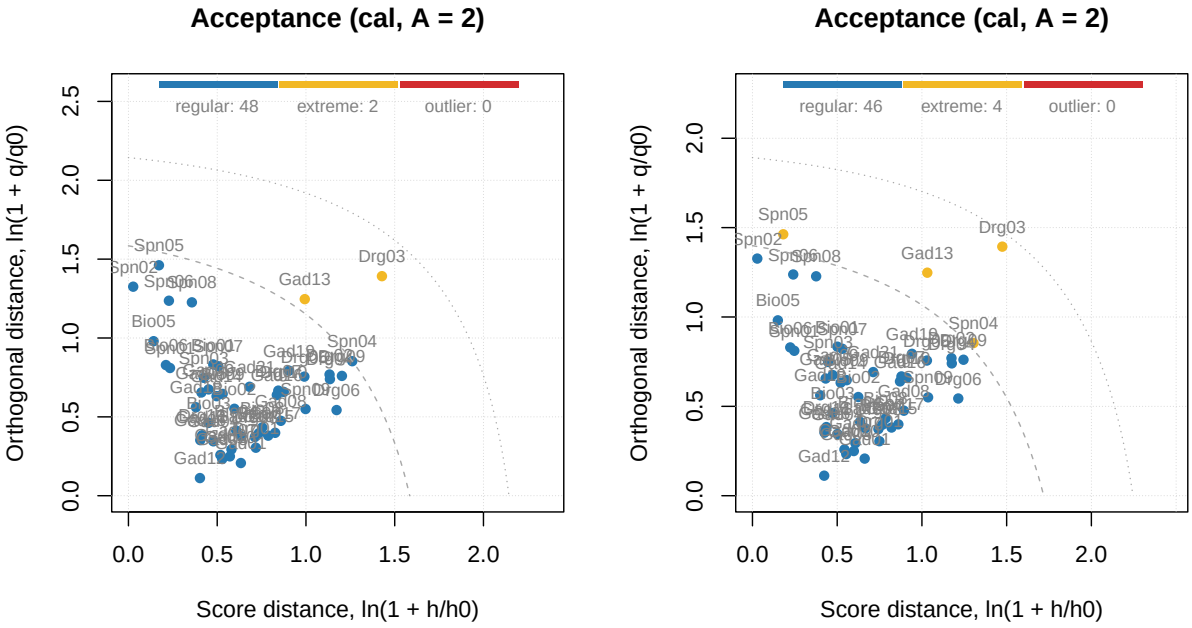
```
m = ddsimca(xc, "Oregano", ncomp = 10, pcv = list("ven", 4), exclrows = c("Drg12", "Drg13"))

par(mfrow = c(1, 2))
plotAcceptance(m, ncomp = 2, show.labels = TRUE)
plotAcceptance(m, ncomp = 2, limType = "robust", show.labels = TRUE)
```



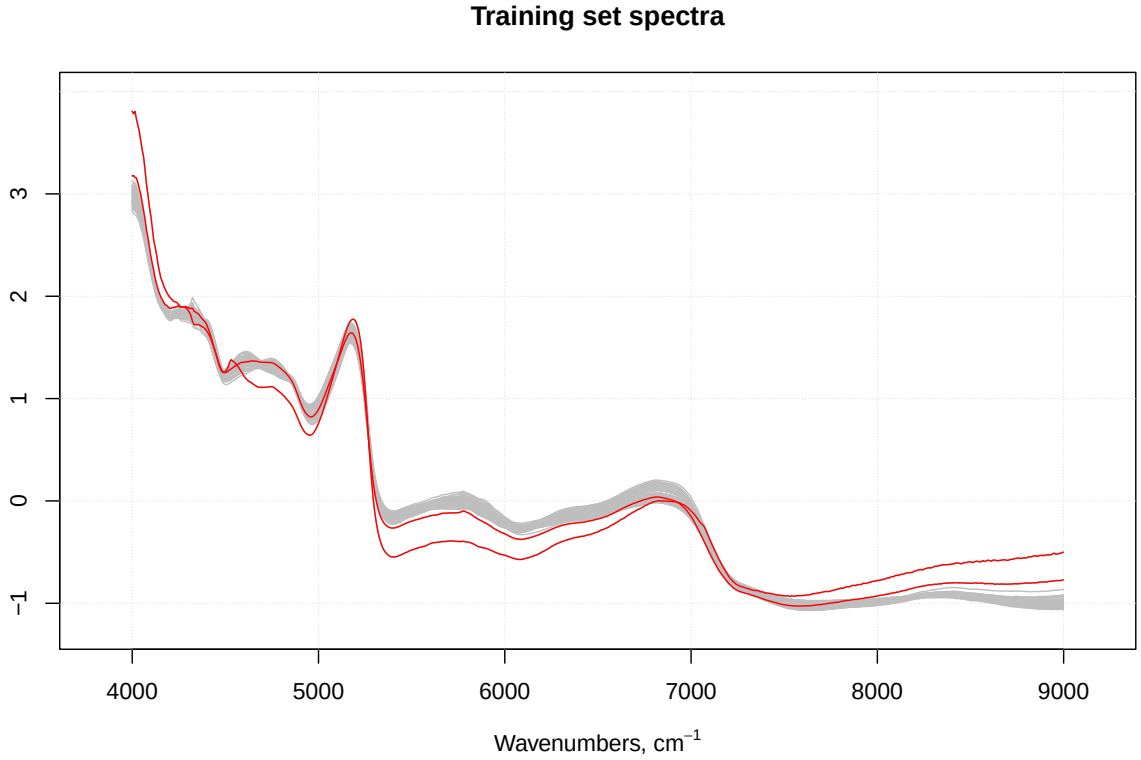
Now both classic and robust estimators show a decent picture without outliers. You can also show the plots using log-transformed distances:

```
par(mfrow = c(1, 2))
plotAcceptance(m, ncomp = 2, log = TRUE, show.labels = TRUE)
plotAcceptance(m, ncomp = 2, log = TRUE, limType = "robust", show.labels = TRUE)
```



Before we proceed with figure of merits and optimization of number of components, let's look at the spectra of the removed objects to confirm their different behaviour:

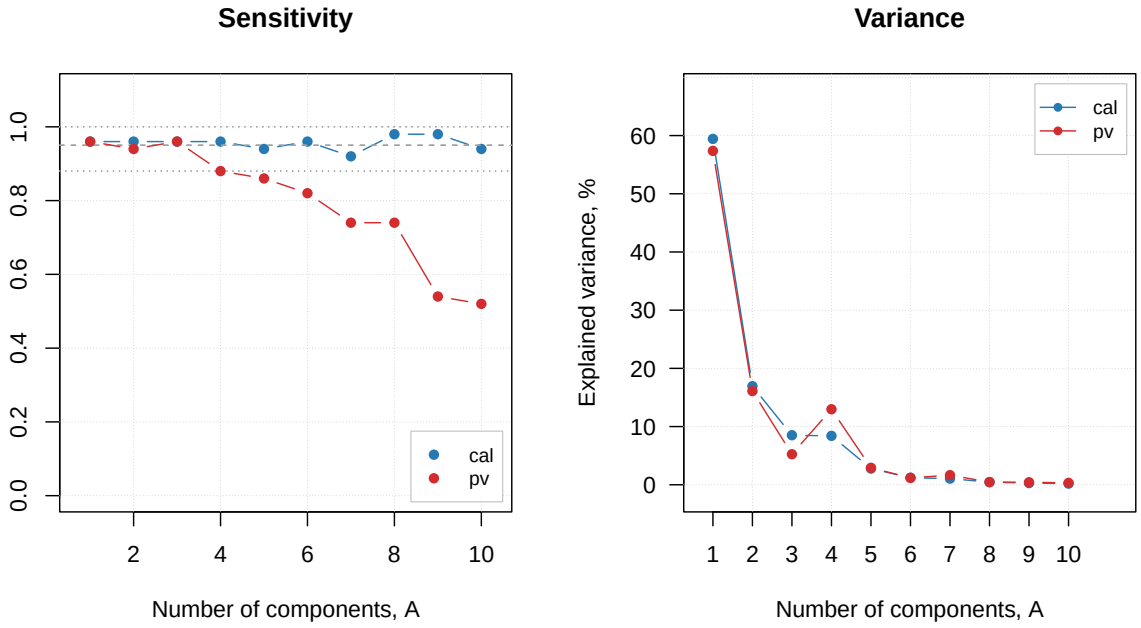
```
par(mfrow = c(1, 1))
mdaplot(xc, type = "l", col = "gray", main = "Training set spectra")
mdaplot(mda.subset(xc, c("Drg12", "Drg13")), type = "l", col = "red", show.axes = FALSE)
```



Apparently they indeed look like outliers.

Now let's check at the sensitivity and variance again:

```
par(mfrow = c(1, 2))
plotSensitivity(m, show.ci = TRUE)
plotVariance(m)
```



Now the variance of training and PV-set looks similar and the sensitivity plot points at $A = 3$ as the optimal number of components. Let's set this parameter for the model and check the summary:

```
m = selectCompNum(m, 3)
summary(m)
```

```
##
## DD-SIMCA model for class 'Oregano'
##
## Excluded rows: 2
## Number of components: 10
## Number of selected components: 3
## Type of limits: classic
## Alpha: 0.05
## Gamma: 0.01
##
##     Expvar Cumexpvar In Out TP FN Sens
## Cal  8.512     84.88 48   4 48  2 0.96
## Pv   5.248     78.72 48   2 48  2 0.96
```

If you for any reason want to change significance level for the model, you can do it as follows:

```
m2 = setParams(m, alpha = 0.01, gamma = 0.001)
summary(m2)
```

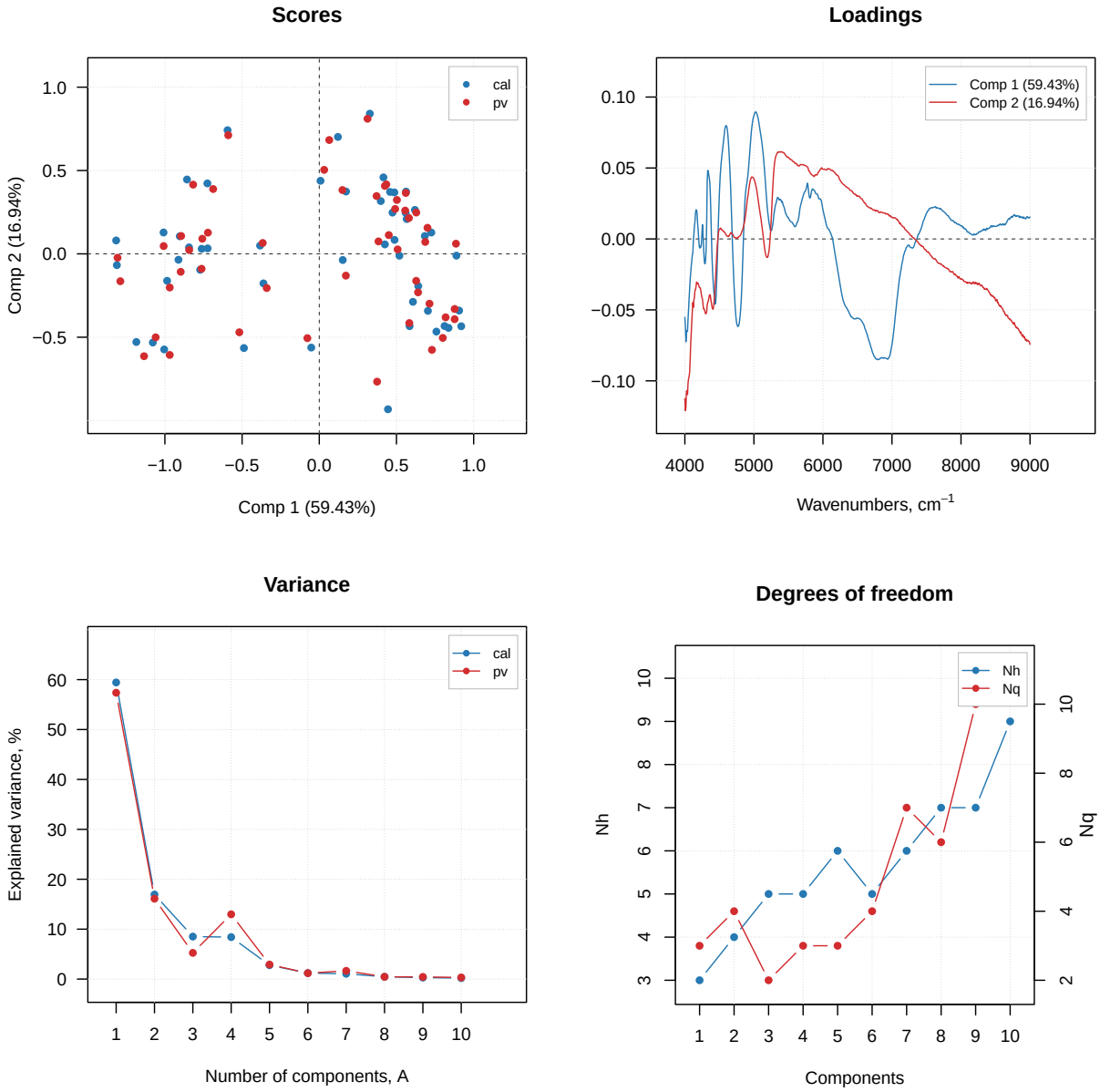
```
##
## DD-SIMCA model for class 'Oregano'
##
## Excluded rows: 2
## Number of components: 10
## Number of selected components: 3
## Type of limits: classic
## Alpha: 0.01
```

```
## Gamma: 0.001
##
##     Expvar Cumexpvar In Out TP FN Sens
## Cal  8.512     84.88 49   3 49  1 0.98
## Pv   5.248     78.72 49   1 49  1 0.98
```

As you can see, the results and the figures of merit for training set and PV-set are recomputed automatically. In the code above, we saved the modified model to a separate variable, so we can continue with default parameters.

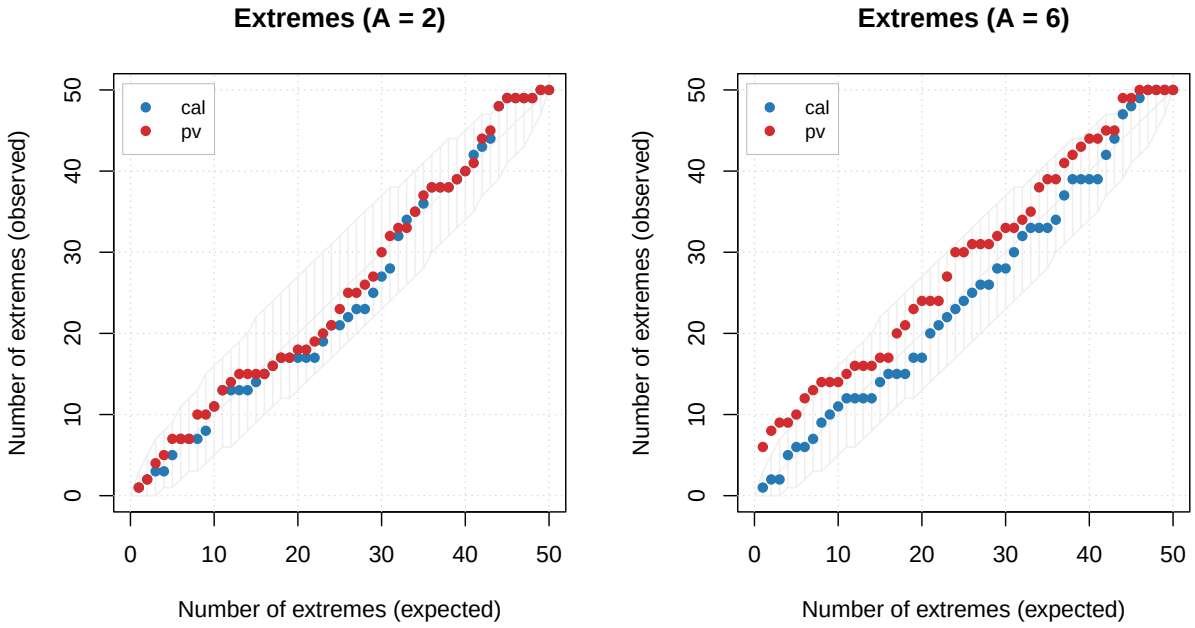
Since DD-SIMCA model is also a PCA model, you can show all plots relevant for PCA:

```
par(mfrow = c(2, 2))
plotScores(m)
plotLoadings(m, type = "l")
plotVariance(m)
plotDistDoF(m)
```



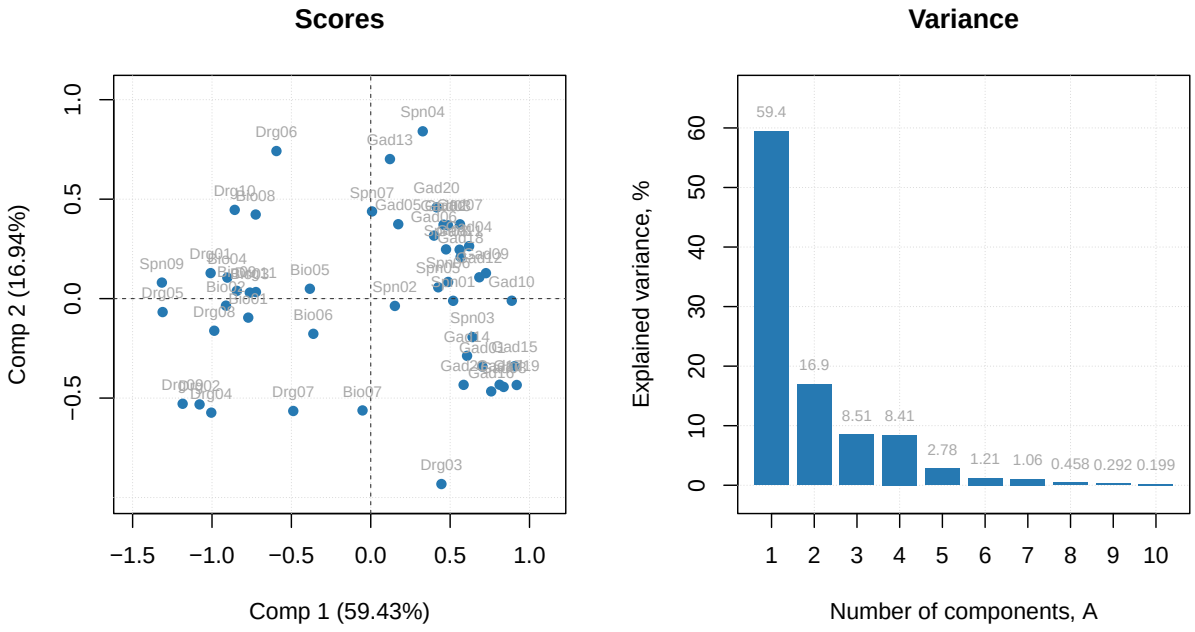
And of course you can look at the *Extremes plot* to check for possible overfitting:

```
par(mfrow = c(1, 2))
plotExtremes(m, ncomp = 2)
plotExtremes(m, ncomp = 6)
```



The same applies to the results object.

```
par(mfrow = c(1, 2))
plotScores(m$res$cal, show.labels = TRUE)
plotVariance(m$res$cal, type = "h", show.labels = TRUE)
```



Making predictions

Dataset with target class only

Now let's check the model performance using independent test set consisting only of the target class members. First of all, we need to load it and split to matrix with predictors and the vector with class labels:

```
dtt <- read.csv("./Oregano/Target_Test.csv", row.names = 1, check.names = FALSE)

ctt = dtt[, 1]
xtt = as.matrix(dtt[, -1])
```

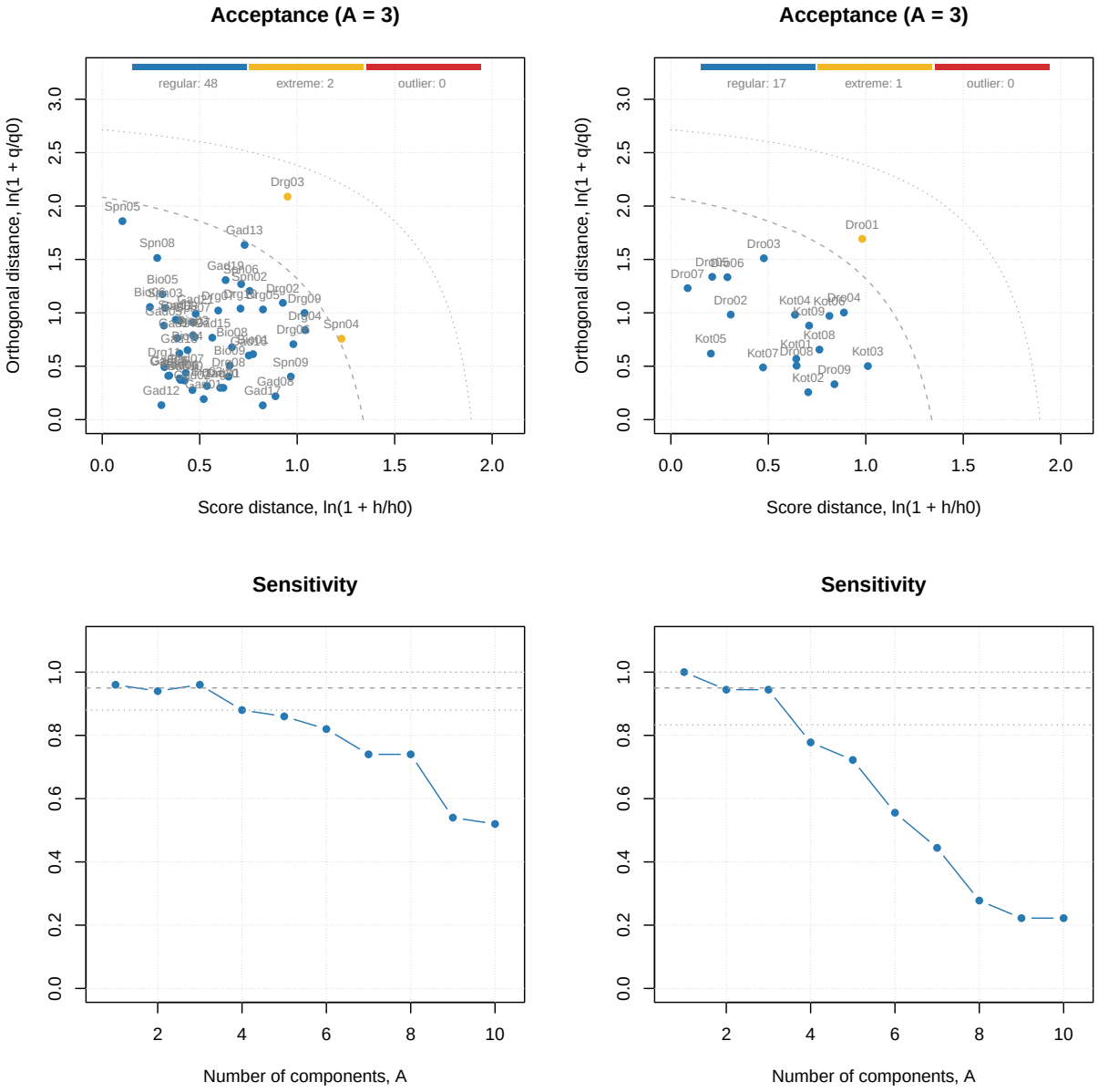
And apply the model:

```
rtt <- predict(m, xtt, ctt)
summary(rtt)
```

```
##
## DD-SIMCA results
##
## Target class: Oregano
## Number of components: 10
## Number of selected components: 3
## Type of limits: classic
## Alpha: 0.050
##
## Number of objects: 18
##  - target class members: 18
##  - from non-target classes: 0
##  - from unknown classes: 0
##
##          Expvar Cumexpvar In Out TP FN   Sens
## Comp 1  38.1465     38.15 18   0 18  0 1.0000
## Comp 2  20.4210     58.57 17   1 17  1 0.9444
## Comp 3* 11.5118     70.08 17   1 17  1 0.9444
## Comp 4   9.9178     80.00 14   4 14  4 0.7778
## Comp 5   5.4689     85.47 13   5 13  5 0.7222
## Comp 6   4.6626     90.13 10   8 10  8 0.5556
## Comp 7   5.3661     95.49  8  10  8 10 0.4444
## Comp 8   0.8855     96.38  5  13  5 13 0.2778
## Comp 9   0.3749     96.76  4  14  4 14 0.2222
## Comp 10  1.2238     97.98  4  14  4 14 0.2222
```

Summary text for result object shows values for all components, but the selected one is marked by * in the table. As one can see, the sensitivity is very similar to the one obtained for PV-set. Let's compare these two results also with plots:

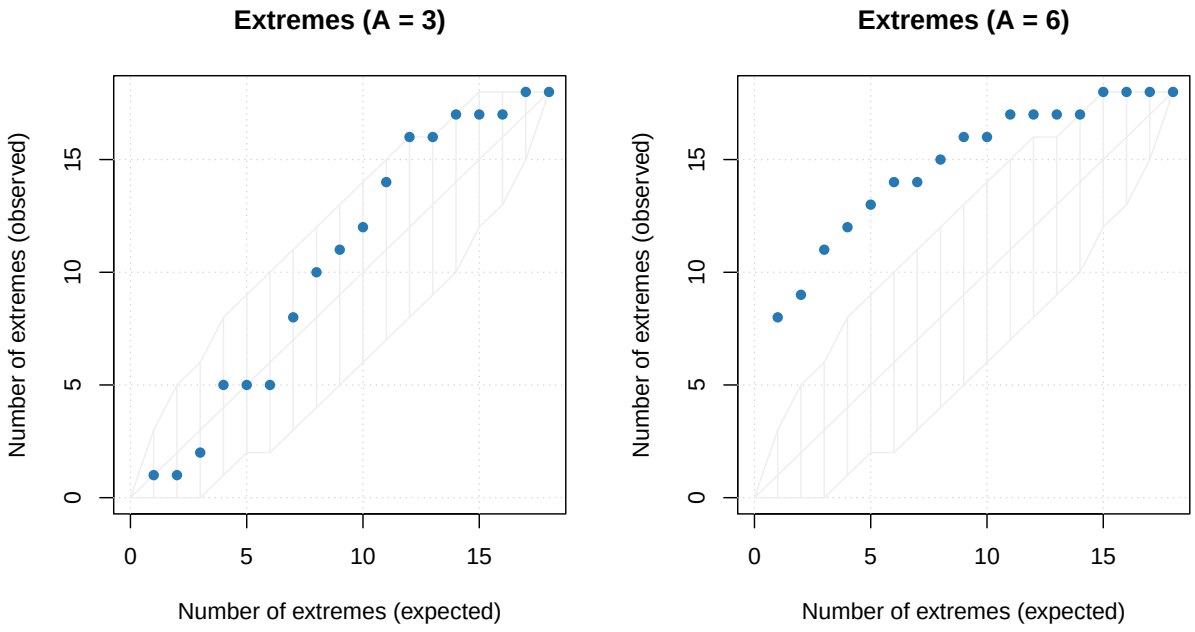
```
par(mfrow = c(2, 2))
plotAcceptance(m$res$pv, log = TRUE, show.labels = TRUE)
plotAcceptance(rtt, log = TRUE, show.labels = TRUE)
plotSensitivity(m$res$pv, show.ci = TRUE)
plotSensitivity(rtt, show.ci = TRUE)
```



Apparently, both PV-set and test set point to $A = 3$ as optimal selection.

As in case of PCA, one can make *Extremes plot* for the DD-SIMCA results, which can be helpful in identifying overfitting:

```
par(mfrow = c(1, 2))
plotExtremes(rtt, ncomp = 3)
plotExtremes(rtt, ncomp = 6)
```



You can get all classification details by converting the DD-SIMCA result object into a data frame:

```
df.tt <- as.data.frame(rtt)
head(df.tt)
```

```
##         class    role decision      h/h0     q/q0         f
## Dro01 Oregano extreme      out 1.6683669 4.433422 17.208679
## Dro02 Oregano regular       in 0.3595882 1.673141  5.144223
## Dro03 Oregano regular       in 0.6114075 3.530630 10.118297
## Dro04 Oregano regular       in 1.4298180 1.726167 10.601424
## Dro05 Oregano regular       in 0.2364948 2.807929  6.798333
## Dro06 Oregano regular       in 0.3366462 2.794651  7.272534
```

By default the data frame is created using current model settings. If necessary, you can change this as follows:

```
df.tt <- as.data.frame(rtt, ncomp = 2, limType = "robust")
head(df.tt)
```

```
##         class    role decision      h/h0     q/q0         f
## Dro01 Oregano extreme      out 0.9142328 4.027964 27.824718
## Dro02 Oregano regular       in 0.5176074 1.110331  8.732413
## Dro03 Oregano regular       in 0.9710622 2.265733 17.478646
## Dro04 Oregano regular       in 0.5654677 2.270967 15.887671
## Dro05 Oregano regular       in 0.3515385 1.817433 12.310749
## Dro06 Oregano regular       in 0.2007634 2.019881 12.922342
```

This is possible because any DD-SIMCA result object contains outcomes computed for all number of components and both robust and classic estimators for the distance parameters and critical limits. However, if you want to change significance level, you need to change the model first, using method `setParams()`, and then re-apply the new model to the dataset.

Datasets with objects from alternative classes

Now let's load data containing non-target class members (spectra of herbs, which are not Oregano). This dataset is organized a bit differently, the row names are located in a dedicated column, plus some of the row names are not unique, therefore we use a bit modified procedure:

```
dtn = read.csv("./Oregano/NonTarget_Non_Or.csv", check.names = FALSE)

# get rownames and make them unique
objnames = dtn[, 1]
objnames = paste0(objnames, ".", seq_along(objnames))

ctn = dtn[, 2]
xtn = as.matrix(dtn[, -c(1, 2)])
rownames(xtn) = objnames
```

And apply the model:

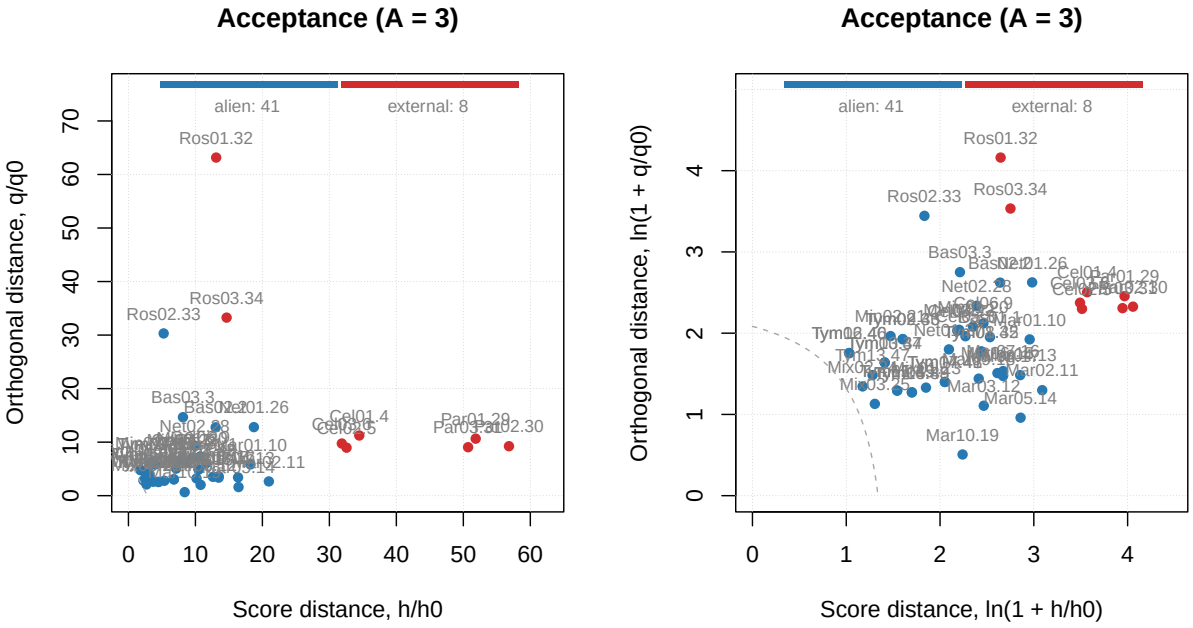
```
rtn <- predict(m, xtn, ctn)
summary(rtn)
```

```
##
## DD-SIMCA results
##
## Target class: Oregano
## Number of components: 10
## Number of selected components: 3
## Type of limits: classic
## Alpha: 0.050
##
## Number of objects: 49
##  - target class members: 0
##  - from non-target classes: 49
##  - from unknown classes: 0
##
##          Expvar Cumexpvar In Out TN FP  Spec    Sel
## Comp 1  17.0005     17.00  5  44 44  5 0.898 0.8997
## Comp 2  35.5382     52.54  0  49 49  0 1.000 0.9692
## Comp 3* 29.5319     82.07  0  49 49  0 1.000 0.9682
## Comp 4   3.6730     85.74  0  49 49  0 1.000 0.9835
## Comp 5   2.1846     87.93  0  49 49  0 1.000 0.9894
## Comp 6   6.4556     94.38  0  49 49  0 1.000 0.9840
## Comp 7   1.3279     95.71  0  49 49  0 1.000 0.9948
## Comp 8   1.6068     97.32  0  49 49  0 1.000 0.9943
## Comp 9   0.2181     97.54  0  49 49  0 1.000 0.9988
## Comp 10  0.5038     98.04  0  49 49  0 1.000 0.9997
```

Now you can spot the difference. First of all, because this dataset does not contain any target class members, the table does not have columns for true positives (TP), false negatives (FN) and sensitivity. In contrast, it has columns for true negatives (TN), false positives (FP), as well as selectivity and specificity.

Therefore the *Acceptance plot* also looks different:

```
par(mfrow = c(1, 2))
plotAcceptance(rtn, show.labels = TRUE)
plotAcceptance(rtn, log = TRUE, show.labels = TRUE)
```



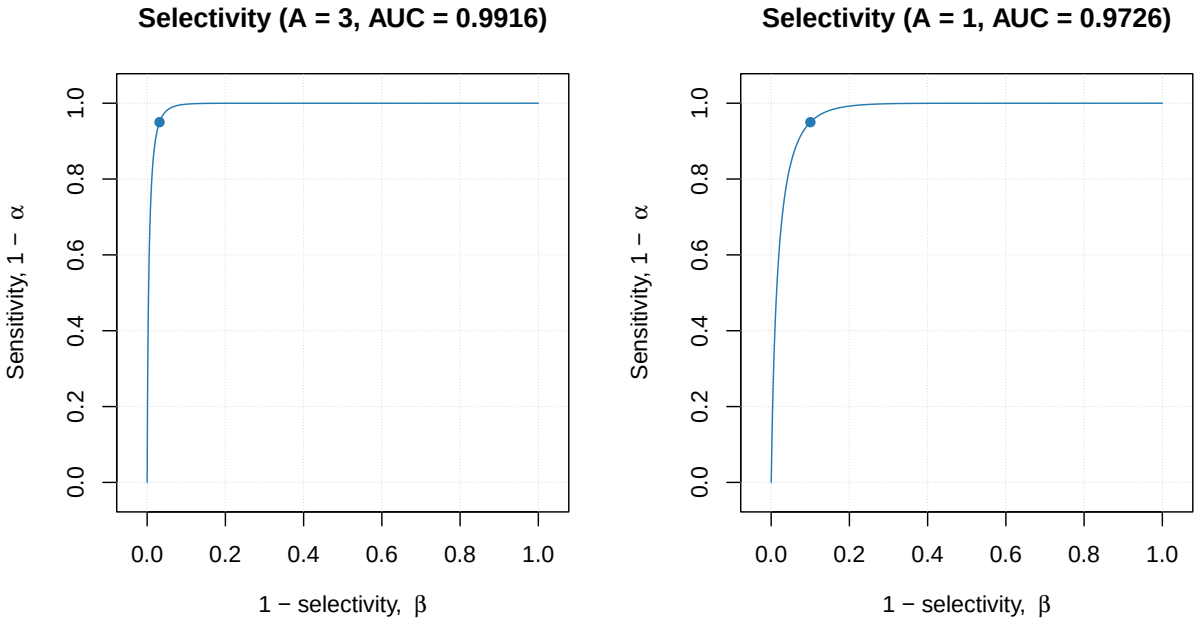
In this case we know in advance that all objects are from non-target classes, so they are supposed to be extreme. But they are extreme in a different way. The ones marked with role "alien" are objects from possibly the same population, closest to the target class. The objects marked as "external" are possibly from other groups/populations which are the most distinct from the target class. The plot also does not contain the limits for outliers, as "outlier" is a role that can be assigned only to a target class member.

The specificity is computed based on all non-target class objects. While selectivity is computed only by taking the "aliens" into account. Moreover, in this case we fit aliens with non-central chi-square distribution and find the overlap between the distribution of the target class objects and the distribution of the aliens. The selectivity is related to this overlap.

This approach is also known as *power of test* in statistics. It gives more realistic estimate on how the model will behave for rejecting the aliens. The value complementary to selectivity is a chance of making Type II error, β. For example for $A = 3$ the beta is $1 - 0.9682 = 0.0318$ or approximately 3%.

The selectivity can be visualized as ROC curve, showing changes in selectivity values depending on significance level used in the model:

```
par(mfrow = c(1, 2))
plotSelectivityArea(rtn)
plotSelectivityArea(rtn, ncomp = 1)
```



The plot shows the dependence as a curve and the current model as a point on the curve. It also computes the area under the curve (AUC), which can be used as auxiliary figure of merit.

Datasets with objects from both target and alternative classes

One can also apply DD-SIMCA model to a dataset containing both target and non-target class members. In this case one can use a *compliance* approach for finding optimal number of components. Let's load the data first:

```
dta = read.csv("./Oregano/All_Test.csv", check.names = FALSE)

objnames = dta[, 1]
cta = dta[, 2]
xta = as.matrix(dta[, -c(1, 2)])
rownames(xta) = objnames
```

And apply the model

```
rta <- predict(m, xta, cta)
summary(rta)
```

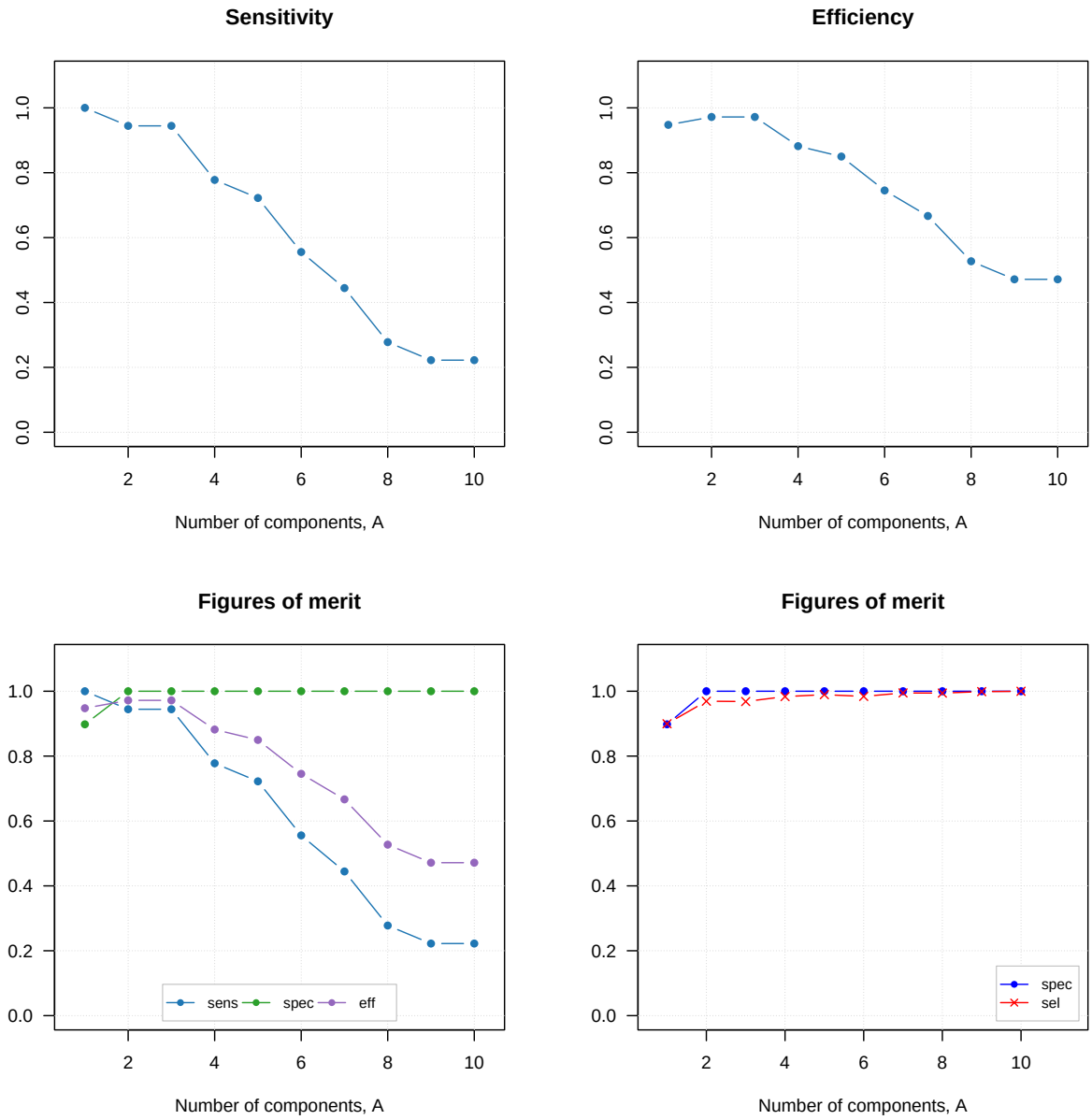
```
##
## DD-SIMCA results
##
## Target class: Oregano
## Number of components: 10
## Number of selected components: 3
## Type of limits: classic
## Alpha: 0.050
##
## Number of objects: 67
##  - target class members: 18
##  - from non-target classes: 49
```

```
##  - from unknown classes: 0
##
##          Expvar Cumexpvar In Out TP FN   Sens TN FP  Spec    Sel    Acc    Eff
## Comp 1  17.9265     17.93 23  44 18  0 1.0000 44  5 0.898 0.8997 0.9254 0.9476
## Comp 2  34.8762     52.80 17  50 17  1 0.9444 49  0 1.000 0.9692 0.9851 0.9718
## Comp 3* 28.7428     81.55 17  50 17  1 0.9444 49  0 1.000 0.9682 0.9851 0.9718
## Comp 4   3.9465     85.49 14  53 14  4 0.7778 49  0 1.000 0.9835 0.9403 0.8819
## Comp 5   2.3285     87.82 13  54 13  5 0.7222 49  0 1.000 0.9894 0.9254 0.8498
## Comp 6   6.3771     94.20 10  57 10  8 0.5556 49  0 1.000 0.9840 0.8806 0.7454
## Comp 7   1.5047     95.70  8  59  8 10 0.4444 49  0 1.000 0.9948 0.8507 0.6667
## Comp 8   1.5752     97.28  5  62  5 13 0.2778 49  0 1.000 0.9943 0.8060 0.5270
## Comp 9   0.2250     97.50  4  63  4 14 0.2222 49  0 1.000 0.9988 0.7910 0.4714
## Comp 10  0.5353     98.04  4  63  4 14 0.2222 49  0 1.000 0.9997 0.7910 0.4714
```

As you can see the summary table in this case is much larger containing all outcomes and figures of merit. In addition to sensitivity, specificity and selectivity, it also has columns for efficiency (the geometric mean of the two) and accuracy.

All these figures of merit can be visualized individually or all together. Here are some examples:

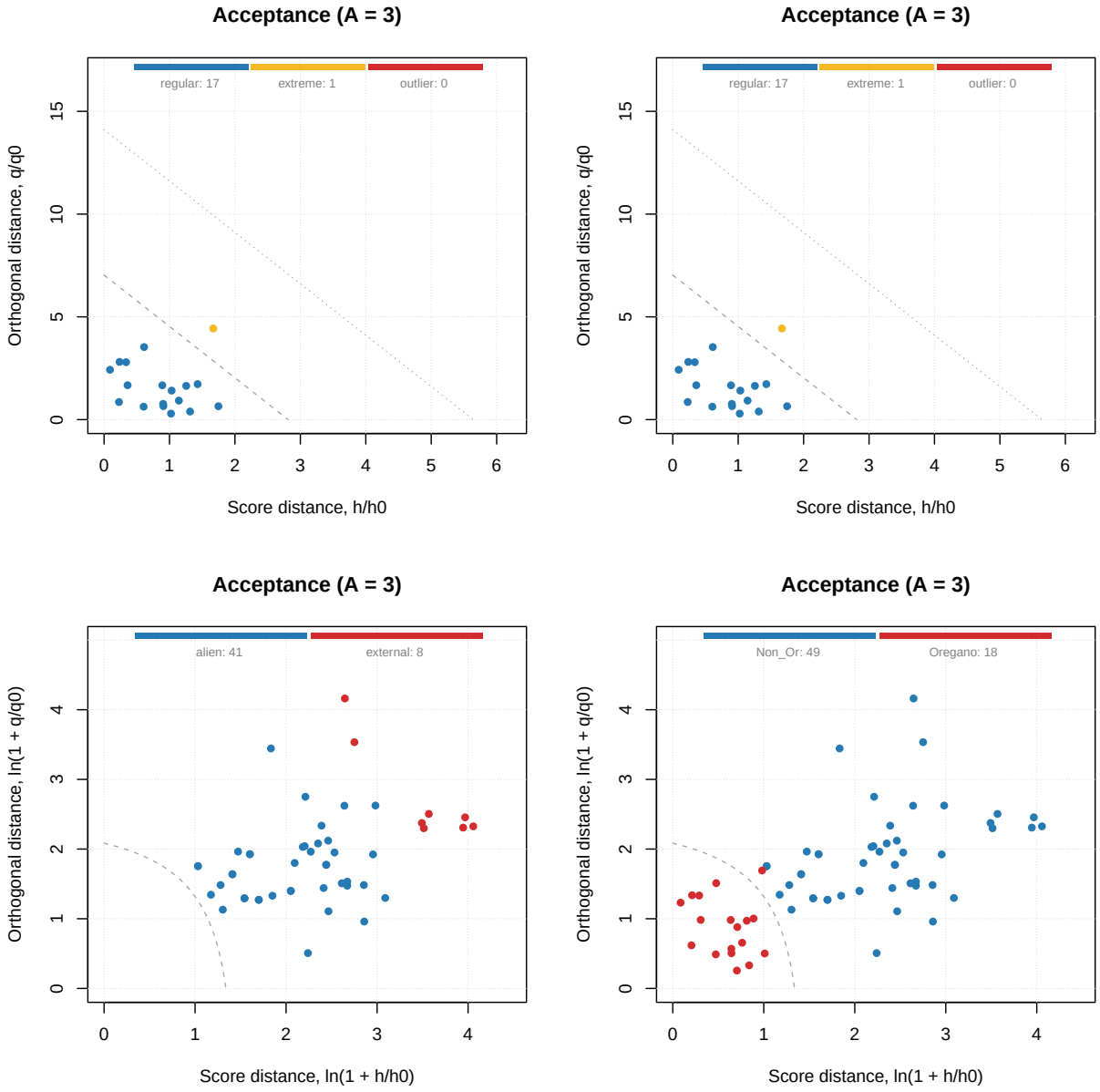
```
par(mfrow = c(2, 2))
plotFoM(rta, fom = "sens")
plotFoM(rta, fom = "eff")
plotFoMs(rta, fom = c("sens", "spec", "eff"))
plotFoMs(rta, fom = c("spec", "sel"), legend.position = "bottomright",
   pch = c(16, 4), col = c("blue", "red"))
```



And of course similar to other plots one can define own colors, markers, etc.

When you make Acceptance plot, you need to specify, whether you want to show only members (this is a default option), only strangers or both. In the latter case the objects will be color grouped by the reference classes:

```
par(mfrow = c(2, 2))
plotAcceptance(rta)
plotAcceptance(rta, show = "members")
plotAcceptance(rta, show = "strangers", log = TRUE)
plotAcceptance(rta, show = "all", log = TRUE)
```



Datasets without reference classes

If no reference class labels are provided, the objects are considered as unknown (new) and no FoMs are computed. In this case only acceptance and distance plots are available (as well as all PCA plots of course). Let's use the non-target class data again (I assume you have it loaded), but we will not provide the vector with reference classes this time:

```
rn <- predict(m, xtn)
summary(rn)
```

```
##
## DD-SIMCA results
##
## Target class: Oregano
```

```
## Number of components: 10
## Number of selected components: 3
## Type of limits: classic
## Alpha: 0.050
##
## Number of objects: 49
##  - target class members: 0
##  - from non-target classes: 0
##  - from unknown classes: 49
##
##          Expvar Cumexpvar In Out
## Comp 1  17.0005     17.00  5  44
## Comp 2  35.5382     52.54  0  49
## Comp 3* 29.5319     82.07  0  49
## Comp 4   3.6730     85.74  0  49
## Comp 5   2.1846     87.93  0  49
## Comp 6   6.4556     94.38  0  49
## Comp 7   1.3279     95.71  0  49
## Comp 8   1.6068     97.32  0  49
## Comp 9   0.2181     97.54  0  49
## Comp 10  0.5038     98.04  0  49
```

And here is how the data frame with outcomes looks like:

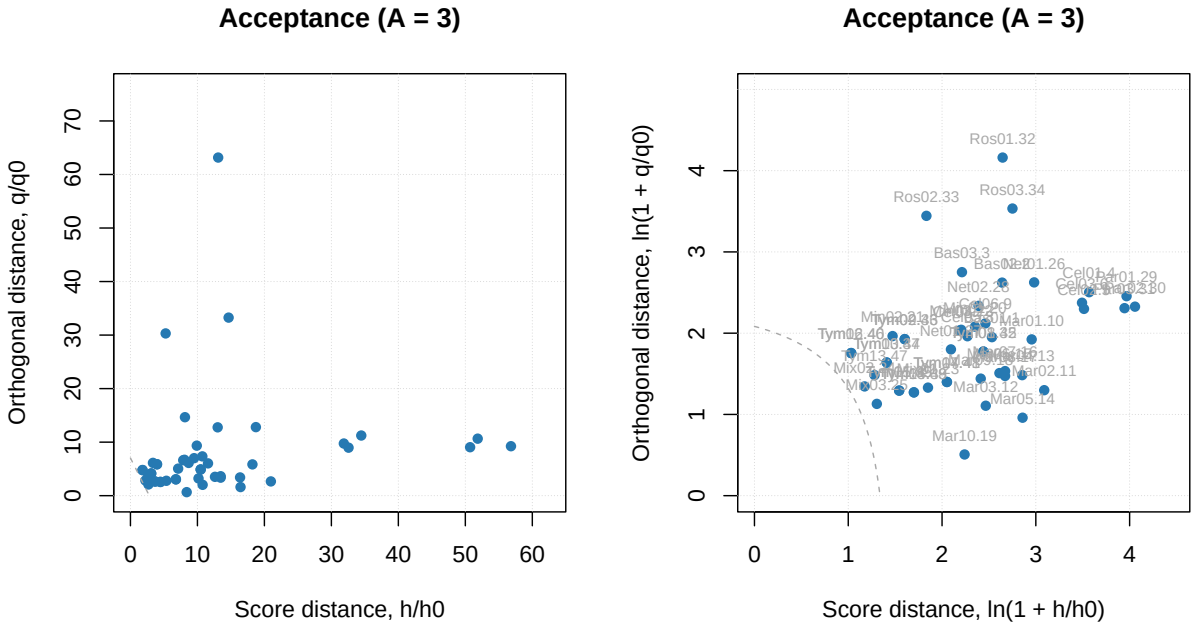
```
dfn <- as.data.frame(rn)
head(dfn)
```

```
##         decision      h/h0      q/q0         f
## Bas01.1      out 11.580328  6.034440  69.97052
## Bas02.2      out 13.022050 12.767634  90.64552
## Bas03.3      out  8.137095 14.658761  70.00300
## Cel01.4      out 34.464145 11.237176 194.79508
## Cel02.5      out 32.563306  8.966258 180.74904
## Cel03.6      out 31.860073  9.742277 178.78492
```

It does not contain column "roles" anymore, only the decision ("in" or "out").

If you try to make a plot with figures of merit, you will get an error, as none of them can be found. But *Acceptance plot* still works of course:

```
par(mfrow = c(1, 2))
plotAcceptance(rn)
plotAcceptance(rn, log = TRUE, show.labels = TRUE)
```



Using preprocessing

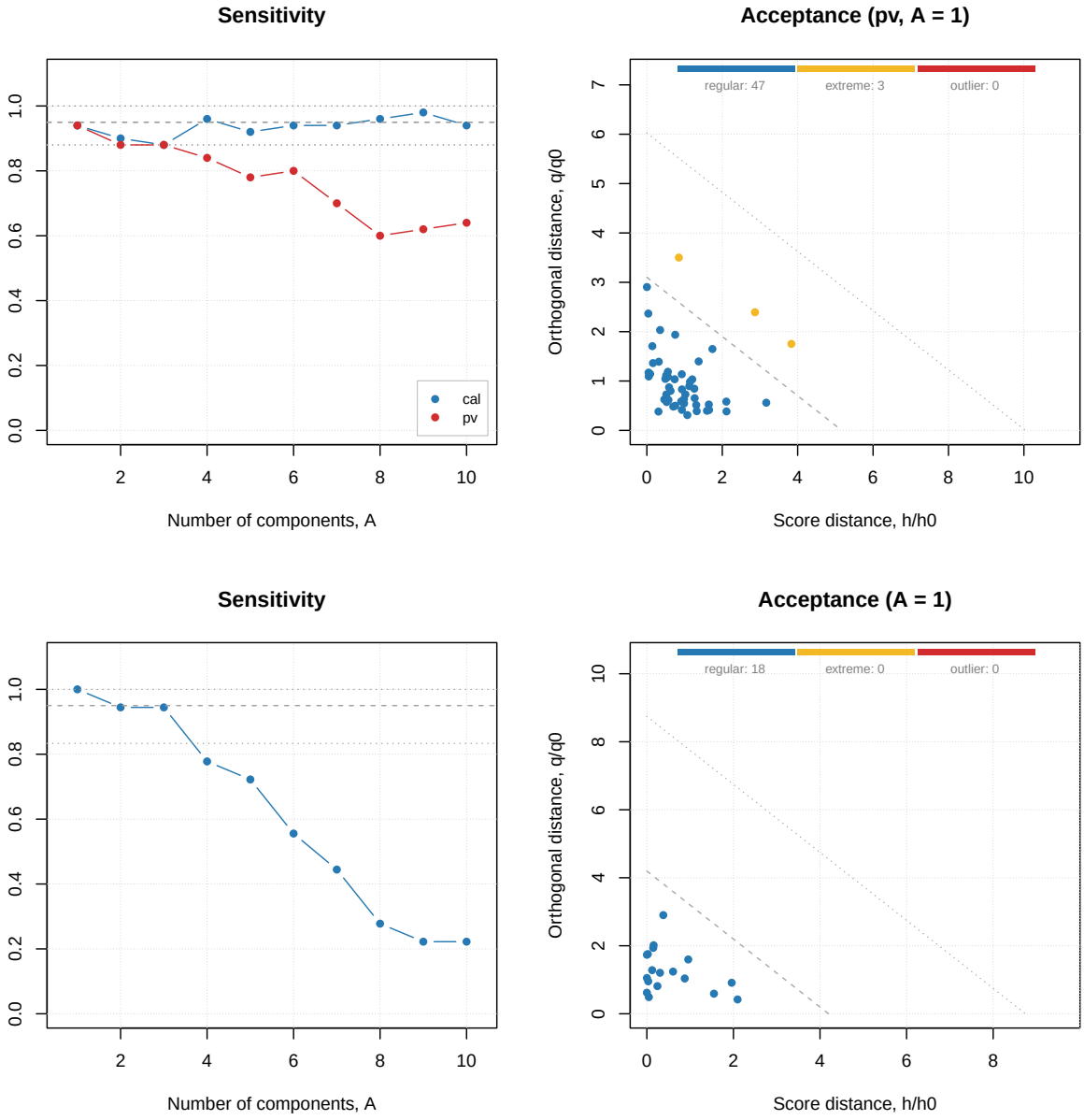
Like for PCA, you can pre-define a sequence of preprocessing methods and provide it as argument `prep` to the DD-SIMCA model. In this case it will train the preprocessing model and apply the methods automatically every time you use the model for predictions:

```
p = list(
   prep("savgol", width = 9, porder = 1, dorder = 1),
   prep("norm", type = "snv")
)

m = ddsimca(xc, "Oregano", ncomp = 10, pcv = list("ven", 4), prep = p, exclrows = c("Drg12", "Drg13"))

r = predict(m, xtt, ctt)

par(mfrow = c(2, 2))
plotSensitivity(m, show.ci = TRUE)
plotAcceptance(m, res = "pv", ncomp = 1)
plotSensitivity(rtt, show.ci = TRUE)
plotAcceptance(rtt, ncomp = 1)
```



Interoperability with web-application

Similar to PCA and preprocessing there is an interactive web-application, https://mda.tools/ddsimca, which lets you create DD-SIMCA models and apply them to a new dataset in any modern browser. Any DD-SIMCA model created using `ddsimca()` can be saved into a JSON file and then be loaded into the *Project* tab of the web-application. Including preprocessing methods.

Here is an example based on the model created in the previous section (let's repeat this code again here, just in case):

```
p = list(
   prep("savgol", width = 9, porder = 1, dorder = 1),
   prep("norm", type = "snv")
)
```

```
m = ddsimca(xc, "Oregano", ncomp = 10, pcv = list("ven", 4), prep = p, exclrows = c("Drg12", "Drg13"))
m = selectCompNum(m, 3)

# save the model to JSON file
writeJSON(m, "ddsimca-model.json")
```

If you run the code on your computer you will see a new `ddsimca-model.json` file created in the working directory. Now open https://mda.tools/ddsimca in your browser, select tab *Test* and load the file with the model. You will see something similar to the following:

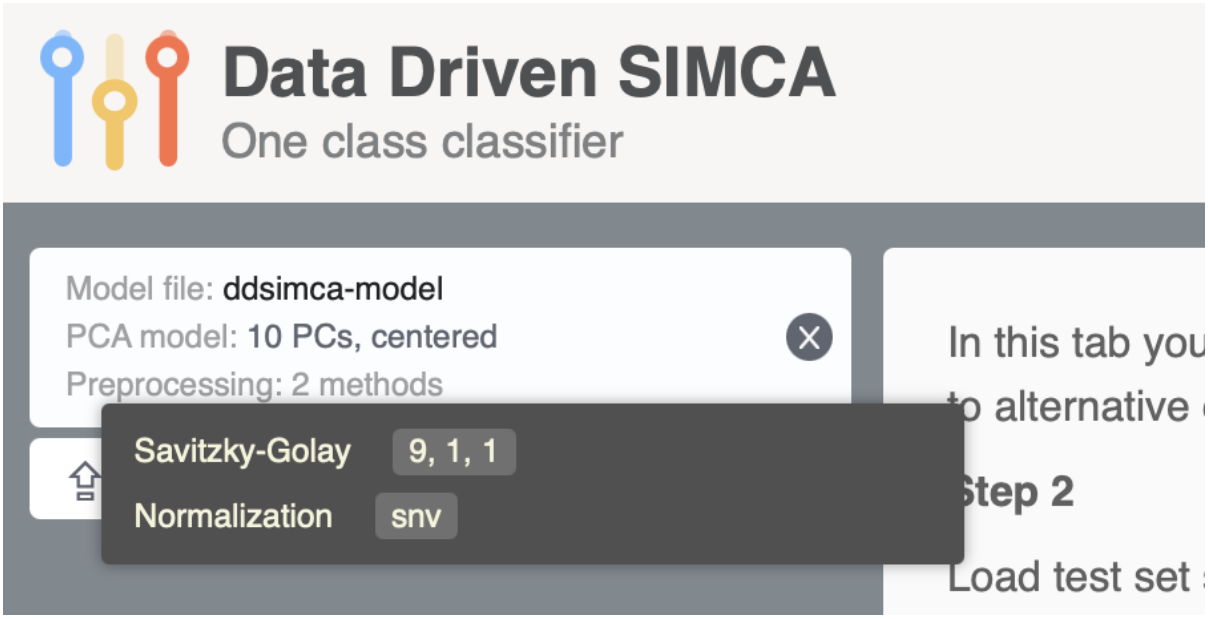


Figure 3: Screenshot with DD-SIMCA model loaded to mda.tools/ddsimca

After that you can upload any CSV file we used in this chapter and see the result of classification directly in the app.

Likewise, any DD-SIMCA model created in the application can be saved into a JSON file and loaded into R, so you can use it via `predict()`. For example this JSON file I created using the same Oregano training set but it has no preprocessing, $A = 2$ as the optimal number of components, and significance levels $\alpha = 0.01$, $\gamma = 0.001$.

Download it to your working directory and run the following code (here I assume you already have `xta` matrix and corresponding `cta` vector from the previous blocks of code):

```
# load PCA model from JSON file
m <- readJSON("ddsimca-model-fromweb.json")

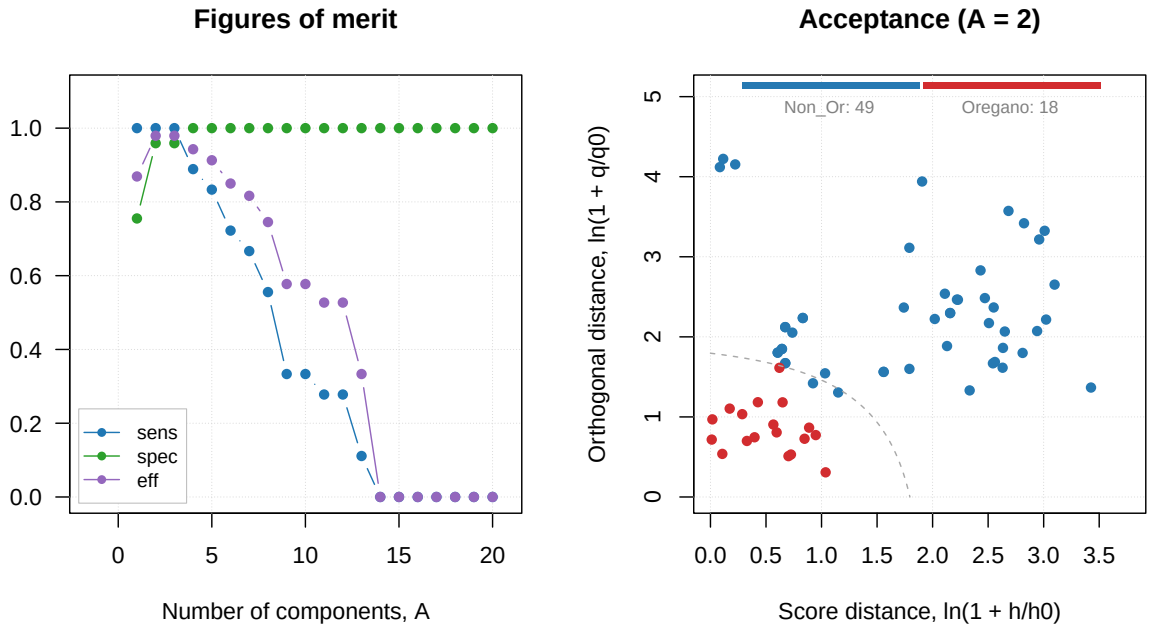
# apply it to test set with all objects
r <- predict(m, xta, cta)
summary(r)
```

```
##
## DD-SIMCA results
##
## Target class: Oregano
## Number of components: 20
```

```
## Number of selected components: 2
## Type of limits: classic
## Alpha: 0.010
##
## Number of objects: 67
##  - target class members: 18
##  - from non-target classes: 49
##  - from unknown classes: 0
##
##           Expvar Cumexpvar In Out TP FN   Sens TN FP   Spec    Sel    Acc
## Comp 1  17.92650     17.93 30  37 18  0 1.0000 37 12 0.7551 0.8155 0.8209
## Comp 2* 34.87615     52.80 20  47 18  0 1.0000 47  2 0.9592 0.9347 0.9701
## Comp 3  28.74276     81.55 20  47 18  0 1.0000 47  2 0.9592 0.9350 0.9701
## Comp 4   3.94647     85.49 16  51 16  2 0.8889 49  0 1.0000 0.9621 0.9701
## Comp 5   2.32847     87.82 15  52 15  3 0.8333 49  0 1.0000 0.9755 0.9552
## Comp 6   6.37712     94.20 13  54 13  5 0.7222 49  0 1.0000 0.9638 0.9254
## Comp 7   1.50471     95.70 12  55 12  6 0.6667 49  0 1.0000 0.9858 0.9104
## Comp 8   1.57521     97.28 10  57 10  8 0.5556 49  0 1.0000 0.9845 0.8806
## Comp 9   0.22496     97.50  6  61  6 12 0.3333 49  0 1.0000 0.9960 0.8209
## Comp 10  0.53530     98.04  6  61  6 12 0.3333 49  0 1.0000 0.9990 0.8209
## Comp 11  0.12531     98.16  5  62  5 13 0.2778 49  0 1.0000 0.9997 0.8060
## Comp 12  0.06786     98.23  5  62  5 13 0.2778 49  0 1.0000 0.9999 0.8060
## Comp 13  0.09338     98.32  2  65  2 16 0.1111 49  0 1.0000 1.0000 0.7612
## Comp 14  0.07814     98.40  0  67  0 18 0.0000 49  0 1.0000 1.0000 0.0000
## Comp 15  0.07341     98.48  0  67  0 18 0.0000 49  0 1.0000 1.0000 0.0000
## Comp 16  0.12804     98.60  0  67  0 18 0.0000 49  0 1.0000 1.0000 0.0000
## Comp 17  0.03662     98.64  0  67  0 18 0.0000 49  0 1.0000 1.0000 0.0000
## Comp 18  0.09743     98.74  0  67  0 18 0.0000 49  0 1.0000 1.0000 0.0000
## Comp 19  0.07831     98.82  0  67  0 18 0.0000 49  0 1.0000 1.0000 0.0000
## Comp 20  0.13056     98.95  0  67  0 18 0.0000 49  0 1.0000 1.0000 0.0000
##            Eff
## Comp 1  0.8690
## Comp 2* 0.9794
## Comp 3  0.9794
## Comp 4  0.9428
## Comp 5  0.9129
## Comp 6  0.8498
## Comp 7  0.8165
## Comp 8  0.7454
## Comp 9  0.5774
## Comp 10 0.5774
## Comp 11 0.5270
## Comp 12 0.5270
## Comp 13 0.3333
## Comp 14 0.0000
## Comp 15 0.0000
## Comp 16 0.0000
## Comp 17 0.0000
## Comp 18 0.0000
## Comp 19 0.0000
## Comp 20 0.0000
```

```
par(mfrow = c(1, 2))
plotFoMs(r, fom = c("sens", "spec", "eff"), legend.position = "bottomleft")
```

```
plotAcceptance(r, show = "all", log = TRUE)
```



There is also a limitation though, model loaded from JSON file created by the web-application does not contain calibration results, it can only be used for predictions/projections:

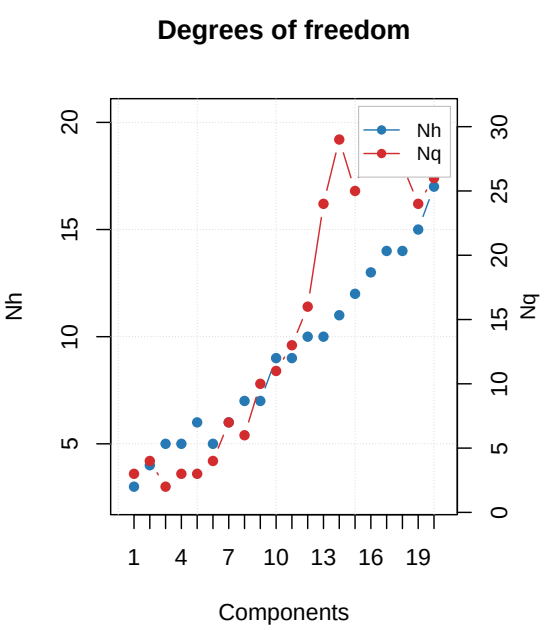
```
summary(m)
```

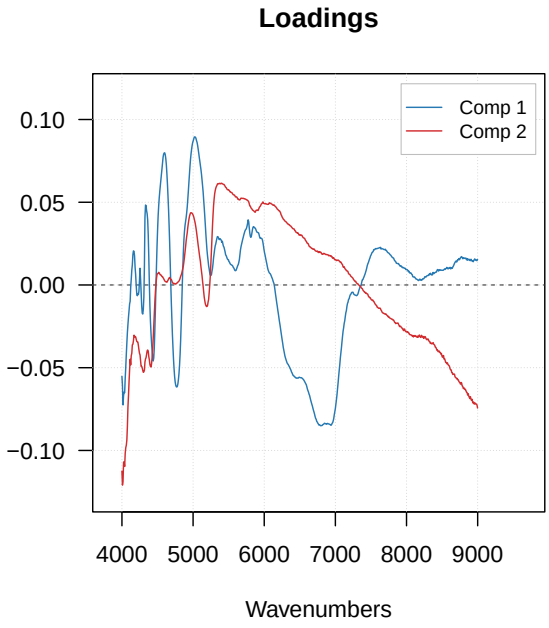
```
##
## DD-SIMCA model for class 'Oregano'
##
## Number of components: 20
## Number of selected components: 2
## Type of limits: robust
## Alpha: 0.01
## Gamma: 0.001
```

```
## Calibration results not found (most probably this model is loaded from web-application).
```

Most of the plots, which rely on calibration set results will not work with such model object and give you the same message as above. However plots, which are only based on model outcomes, such as loadings, eigenvalues and degrees of freedom are available:

```
par(mfrow = c(1, 2))
plotDistDoF(m)
plotLoadings(m, type = "l")
```


Degrees of freedom
Nh
Nq
Components
1
4
7
10
13
16
19
5
10
15
20
0
5
10
15
20
25
30


Loadings
Comp 1
Comp 2
Wavenumbers
4000
5000
6000
7000
8000
9000
0.10
0.05
0.00
−0.05
−0.10

DD-SIMCA for 3-way data

Data Driven SIMCA can also be used for developing one-class classification models based on 3-way data, e.g. excitation-emission profiles, GC-MS, and similar. In this case, a 3-way decomposition (PARAFAC or Tucker) is used to fit the training set data and compute distances, parameters, etc.

This section describes how to do this using a simple simulated dataset. It assumes that you already know how conventional 2-way implementation of DD-SIMCA (`ddsimca`) works (as many plots and methods are similar) and understand the basics of PARAFAC and Tucker decompositions. We will describe the PARAFAC version (`ddsimca.parafac()`) in detail, and then show what is different in the Tucker version (`ddsimca.tucker()`).

Prepare datasets

Three-way datasets are stored as tensors (three-dimensional arrays). However, both methods support two ways of providing the data - natural (as a 3-way array) as well as unfolded into a 2-way matrix (rows are samples and columns are column-wise unfolded sample matrices).

This tutorial is based on simulated 3-way data, the simulation was done similarly to what is described in 10.1021/acs.analchem.3c05096. Target samples contain two components in random concentrations. Alternative class samples contain the same two components + a third component in random concentrations. The component instrumental profiles are partially overlapped Gaussian functions, defined in 40 and 50 channels in each data mode (hence the data matrices have 50×40 dimension). Gaussian noise was then added to all signals at a level of 10% with respect to the mean signal. The author is grateful to Alejandro Olivieri for sharing a MATLAB script implementing simulations and helping with the tutorial.

You can load the data by running:

```
library(mdatools)

data(data3w)
print(str(data3w))
```

```
## List of 3
##  $ X3w : num [1:80, 1:50, 1:40] 0.012 0.03 0.0228 0.0437 0.0353 ...
##  $ X3wt: num [1:20, 1:50, 1:40] 0.02798 0.00561 0.03457 0.03881 0.02374 ...
##  $ X3wa: num [1:60, 1:50, 1:40] 0.0249 0.01478 0.03083 0.01011 0.00696 ...
## NULL
```

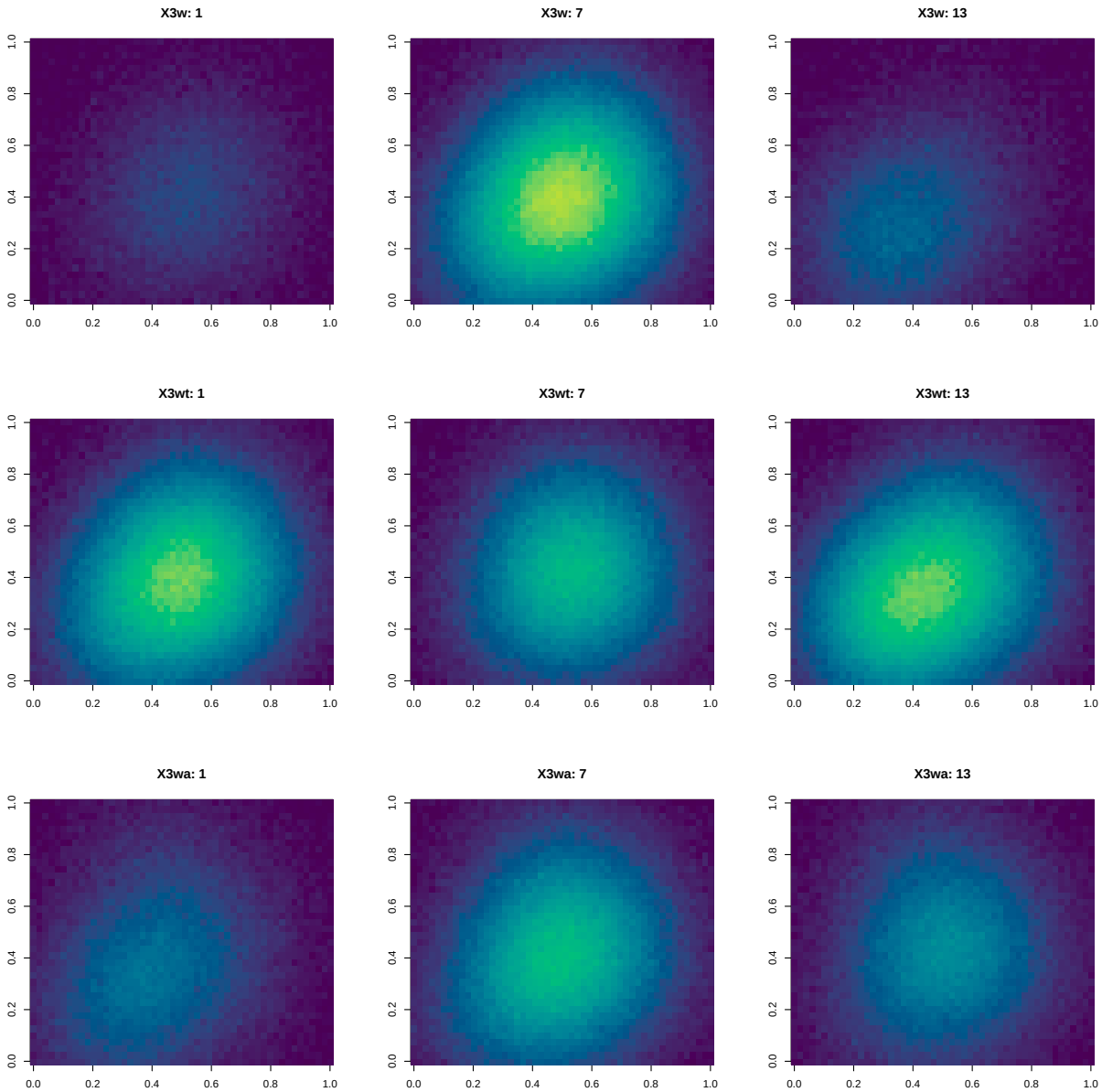
As we can see, all datasets are represented in the form of 3-way arrays, the training set, `X3w`, contains 80 samples, the test target set`X3wt`, contains 20 samples and the test alternative set, `X3wa`, contains 60 samples. The dimensions of each sample matrix are 50 x 40.

Let's visualize some of them in the form of heatmaps.

```
col <- hcl.colors(256, "Viridis")

par(mfrow = c(3, 3))
```

```
for (n in names(data3w)) {
   X = data3w[[n]]
   X[X < 0] = 0
   for (i in c(1, 7, 13)) {
      image(X[i, , ], main = sprintf("%s: %d", n, i),
         zlim = c(0, 0.35), useRaster = TRUE, col = col)
   }
}
```



As mentioned above, it is not necessary to reshape three-way data. However, we will create a reshaped array for the training set just to demonstrate that the methods work on both representations. In addition to that we will also generate vectors with reference class labels.

```
data(data3w)

# attach the list, so we can use "X3w" instead of "data3w$X3w"
attach(data3w)
```

```
## The following object is masked _by_ .GlobalEnv:
##
##     X3w
```

```
## The following objects are masked from data3w (pos = 4):
##
##     X3w, X3wa, X3wt
```

```
# prepare unfolded 2-way data for training set plus generate the class labels
X2w <- X3w
dim(X2w) <- c(dim(X3w)[1], prod(dim(X3w)[2:3]))
c.train <- rep("target", nrow(X2w))

# generate class labels for target and alternative class samples in test set
c.test.target <- rep("target", dim(X3wt)[1])
c.test.alt <- rep("alt", dim(X3wa)[1])

# combine the target and alternative class samples into a single test set
nt <- dim(X3wt)[1]
na <- dim(X3wa)[1]
n <- nt + na
c.test <- c(c.test.target, c.test.alt)
X3wb <- array(0, dim = c(n, dim(X3wt)[2], dim(X3wt)[3]))
X3wb[1:nt, , ] <- X3wt
X3wb[(nt + 1):n, , ] <- X3wa
```

PARAFAC DD-SIMCA

Training a PARAFAC DD-SIMCA model is similar to training a conventional DD-SIMCA model, with one additional argument — you either need to provide 3-way data or provide 2-way data and the dimensions of the sample matrix (in our case 50×40):

Let's start with 3-way:

```
m <- ddsimca.parafac(X3w, "target", ncomp = 4)
summary(m)
```

```
##
## DD-SIMCA-PARAFAC model for class 'target' (50 x 40)
##
## Number of components: 4
## Number of selected components: 4
## Alpha: 0.05
## Gamma: 0.01
##
##       Expvar Cumexpvar In Out TP FN   Sens
## Cal 0.001243     99.51 79   1 79  1 0.9875
```

Now the same but with 2-way data and additional argument:

```
m <- ddsimca.parafac(X2w, "target", ncomp = 4, dim = c(50, 40))
summary(m)
```

```
##
## DD-SIMCA-PARAFAC model for class 'target' (50 x 40)
##
## Number of components: 4
## Number of selected components: 4
## Alpha: 0.05
## Gamma: 0.01
##
##       Expvar Cumexpvar In Out TP FN   Sens
## Cal 0.001781     99.51 79   1 79  1 0.9875
```

As you can see the result is the same. A small discrepancy for explained variance of the last component is due to random initialization of the PARAFAC algorithm and also because our model is overfitted (there are 2 pure components in the data).

There is an important difference from conventional PCA-based DD-SIMCA. In the case of conventional DD-SIMCA training, providing `ncomp = 4` will result in a model based on a single PCA decomposition (a single PCA model) with 4 components. In the case of PARAFAC, it will train four different models: the first PARAFAC model will use 1 component (1 factor), the second model — 2 components (factors), and so on.

Therefore, when we run the code above, R makes four models, and the models with 3 and 4 components are overfitted. You can see the effect of overfitting if you add argument `verbose = TRUE` when training. In this case the PARAFAC algorithm for models with 3 and 4 components cannot converge and you will see a corresponding message:

```
m <- ddsimca.parafac(X3w, "target", ncomp = 4, verbose = TRUE)
```

```
## ncomp = 3: parafac.als reached max iterations (500) before converging

## ncomp = 4: parafac.als reached max iterations (500) before converging
```

Here are the statistics for calibration set:

```
summary(m$res$cal)
```

```
##
## DD-SIMCA results
##
## Target class: target
## Number of components: 4
## Number of selected components: 4
## Type of limits: classic
## Alpha: 0.050
##
## Number of objects: 80
##  - target class members: 80
##  - from non-target classes: 0
##  - from unknown classes: 0
##
##            Expvar Cumexpvar In Out TP FN   Sens
## Comp 1  95.648972     95.65 76   4 76  4 0.9500
## Comp 2   3.862370     99.51 79   1 79  1 0.9875
## Comp 3   0.001884     99.51 79   1 79  1 0.9875
## Comp 4*  0.001266     99.51 79   1 79  1 0.9875
```

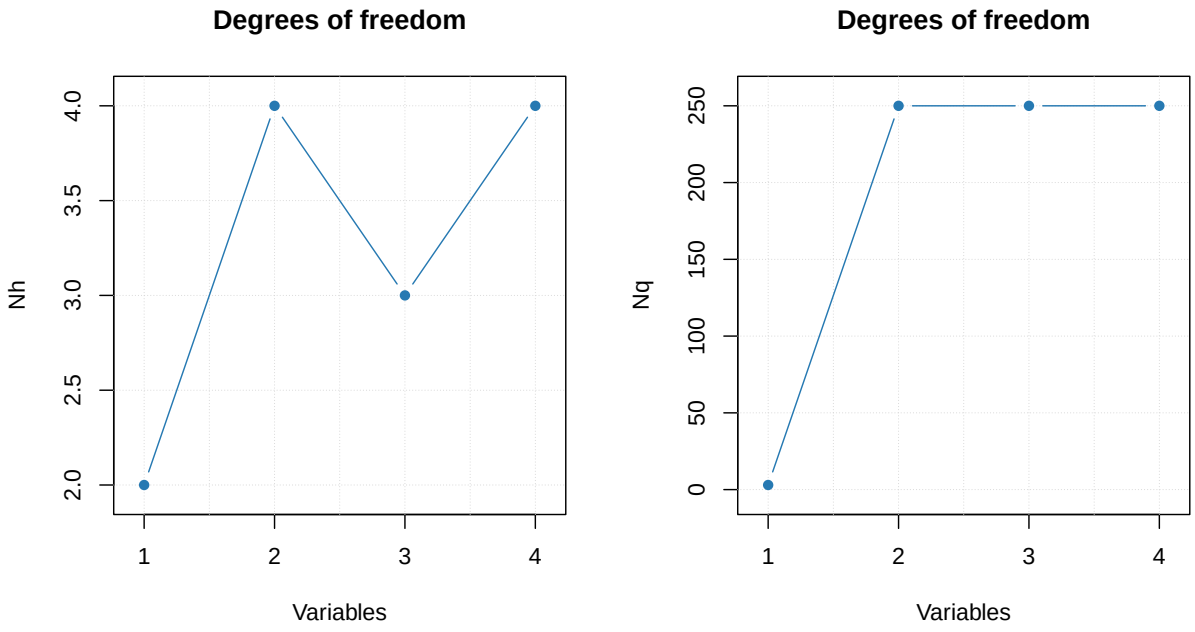
As you can see, there is further evidence that the model with 2 components is enough. By default, the last model (with 4 components in our case) is considered optimal. We can change this by using method `selectCompNum()`:

```
m <- selectCompNum(m, 2)
summary(m)
```

```
##
## DD-SIMCA-PARAFAC model for class 'target' (50 x 40)
##
## Number of components: 4
## Number of selected components: 2
## Alpha: 0.05
## Gamma: 0.01
##
##     Expvar Cumexpvar In Out TP FN   Sens
## Cal  3.862     99.51 79   1 79  1 0.9875
```

Like in conventional DD-SIMCA you can look at the number of degrees of freedom.

```
par(mfrow = c(1, 2))
plotT2DoF(m)
plotQDoF(m)
```



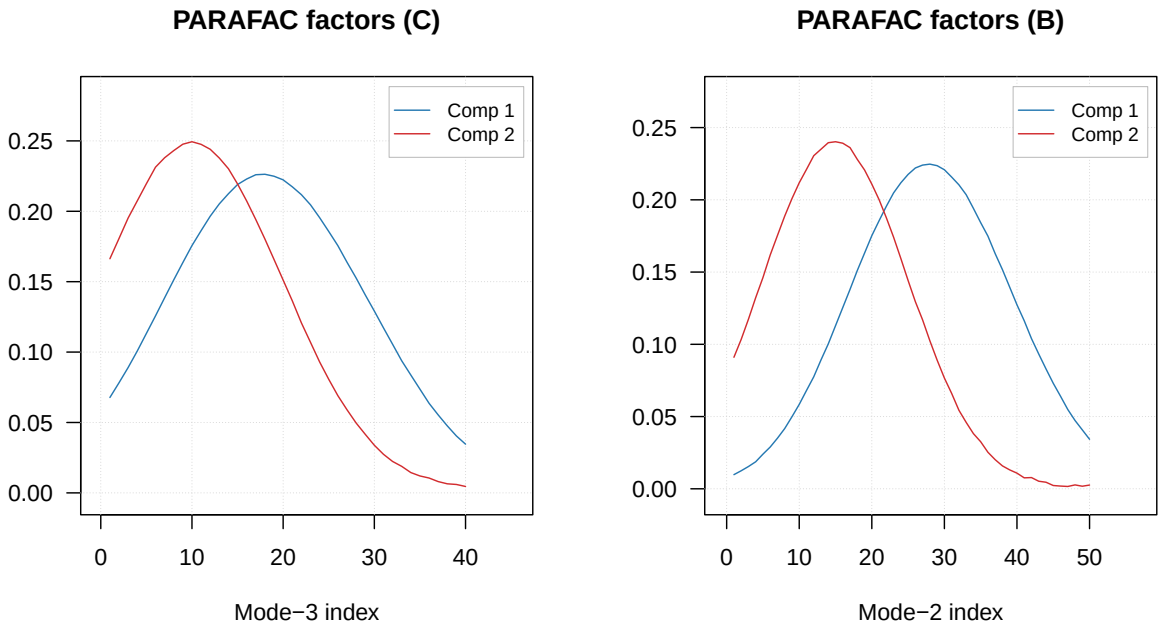
Again, it is important to underline that both `Nh` and `Nq` values are related to different PARAFAC models. Thus one can see that for the PARAFAC model with 2 components/factors, `Nq` is 250. It is the largest possible value because 2 components explain all systematic variation in the training data, so there is nothing left for the residual distance. Any model with number of factors > 2 will demonstrate similar behaviour.

The degrees of freedom for the full distance, N_f, are computed differently here. There are two options - either use the same approach as in conventional DD-SIMCA ($N_f = N_h + N_q$). Or compute N_f based on statistics of the full distance distribution, the same way as N_q and N_h are computed. By default the last method is used, but you can change it by setting the argument `nf.as.sum = TRUE`.

Another difference is that there is no `plotLoadings()` method in this case, but you can show the B and C

factors instead:

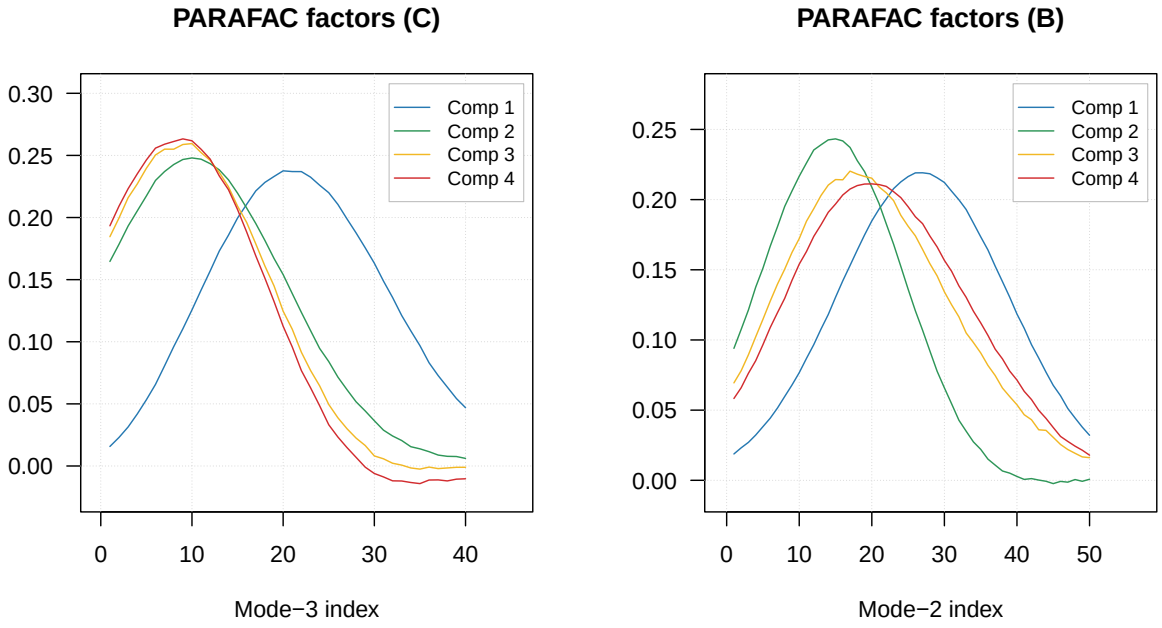
```
par(mfrow = c(1, 2))
plotFactors(m, comp = 1:2, "C")
plotFactors(m, comp = 1:2, "B")
```



We see two partially overlapped Gaussian curves used for simulation of the data.

This method shows factors only for the optimal PARAFAC model (selected using `selectCompNum()` method). If you want to see factors for the PARAFAC model with 4 components, you need to select it first:

```
m <- selectCompNum(m, 4)
par(mfrow = c(1, 2))
plotFactors(m, comp = 1:4, "C")
plotFactors(m, comp = 1:4, "B")
```



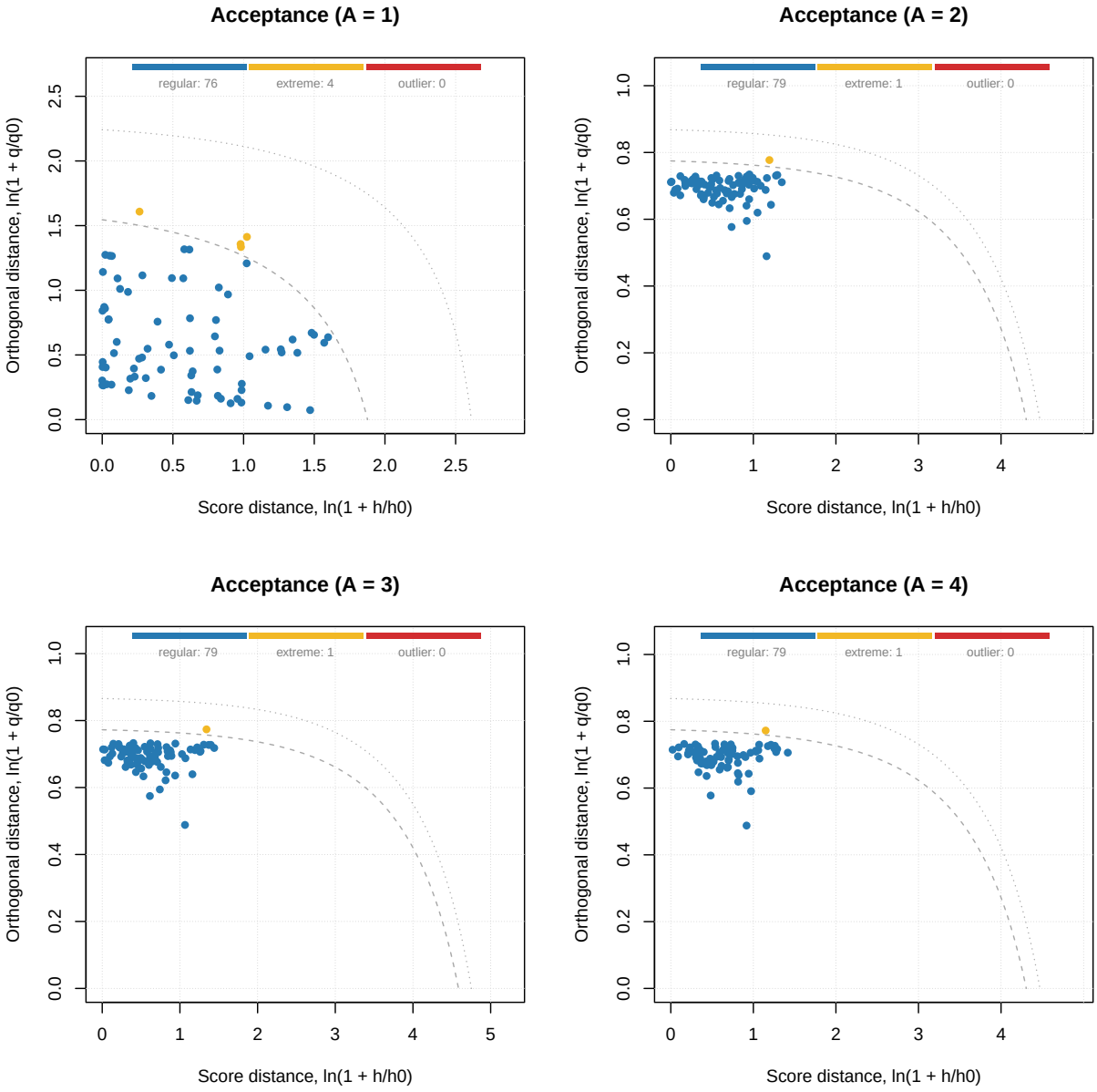
Let's get back to the optimal model.

```
m <- selectCompNum(m, 2)
```

Like in conventional DD-SIMCA, to get the results, you need to apply this model to a dataset. However, the results for the training set are already part of the model, as with any other method in the `mdatools` package. Procrustes cross-validation is not available for 3-way methods yet though.

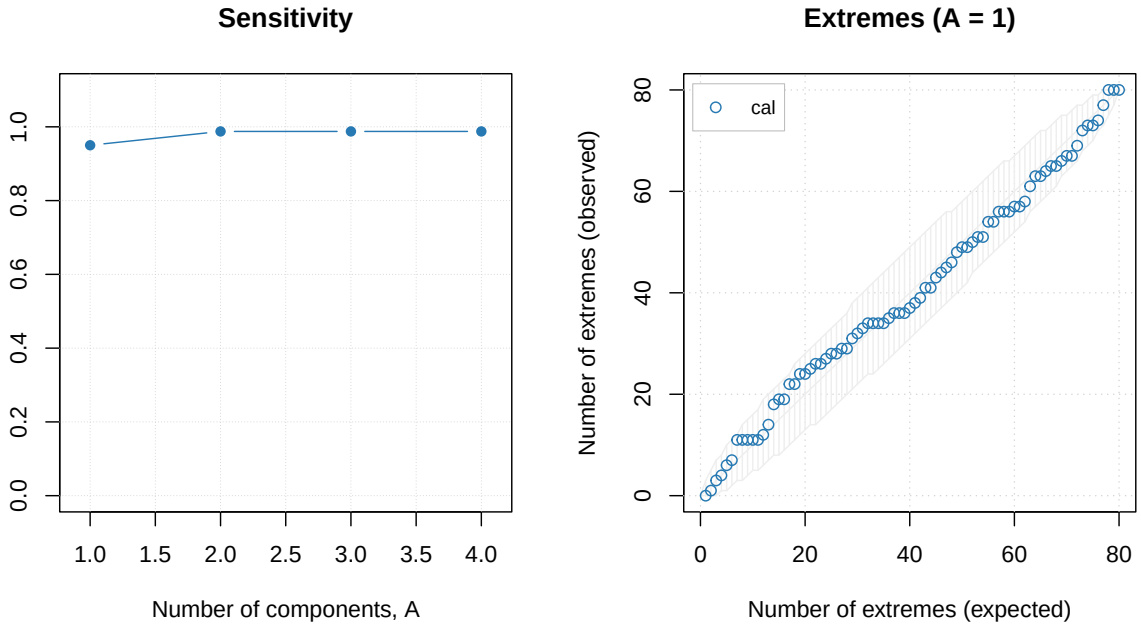
Let's check the acceptance plot for the training set using different models. Although the model now knows that the decomposition with 2 components is optimal and should be used as default, the distance values and the acceptance limits are still computed for all models, so we can show the acceptance plot for models with different numbers of components/factors.

```
par(mfrow = c(2, 2))
for (A in 1:4)
   plotAcceptance(m$res$cal, ncomp = A, log = TRUE)
```



And, like in conventional DD-SIMCA, we can of course show sensitivity and extreme plots.

```
par(mfrow = c(1, 2))
plotSensitivity(m$res$cal)
plotExtreme(m, ncomp = 1, pch = 1)
```



Like in most of the `mdatools` methods, to get results for a new dataset, you need to use method `predict()`. Here is how to do it for the combined test set:

```
r.test <- predict(m, X3wb, c.test)
summary(r.test)
```

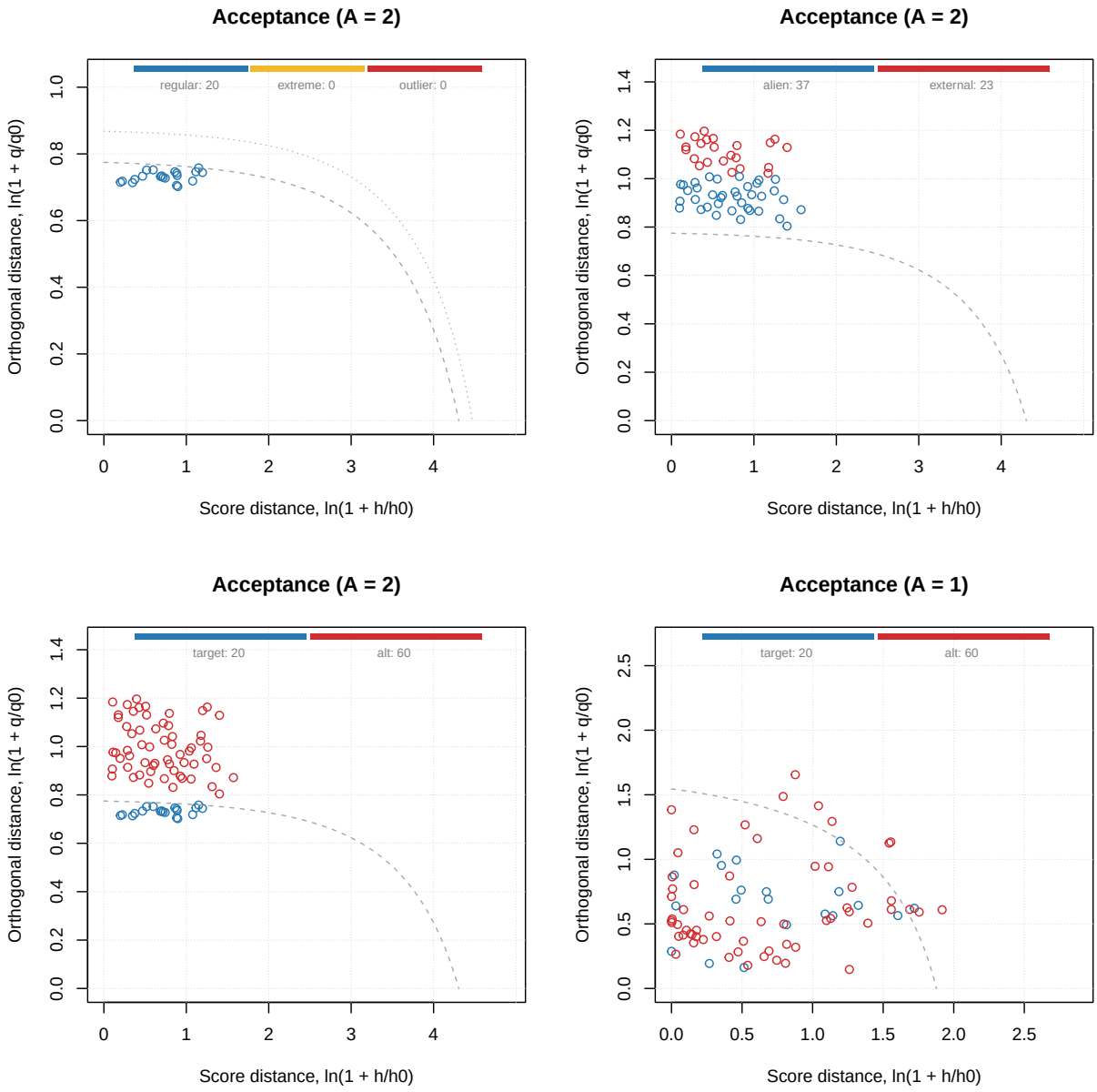
```
##
## DD-SIMCA results
##
## Target class: target
## Number of components: 4
## Number of selected components: 2
## Type of limits: classic
## Alpha: 0.050
##
## Number of objects: 80
##  - target class members: 20
##  - from non-target classes: 60
##  - from unknown classes: 0
##
##           Expvar Cumexpvar In Out TP FN Sens TN FP Spec     Sel     Acc     Eff
## Comp 1  95.45842     95.46 70  10 19  1 0.95  9 51 0.15 0.02427 0.3500 0.3775
## Comp 2*  3.77964     99.24 20  60 20  0 1.00 60  0 1.00 0.99846 1.0000 1.0000
## Comp 3   0.13615     99.37 20  60 20  0 1.00 60  0 1.00 0.99930 1.0000 1.0000
## Comp 4  -0.01752     99.36 19  61 19  1 0.95 60  0 1.00 0.99958 0.9875 0.9747
```

Most of the settings from conventional DD-SIMCA, such as selection of `alpha` and `gamma` values, will work here as well.

Here are different acceptance plots for the test set predictions.

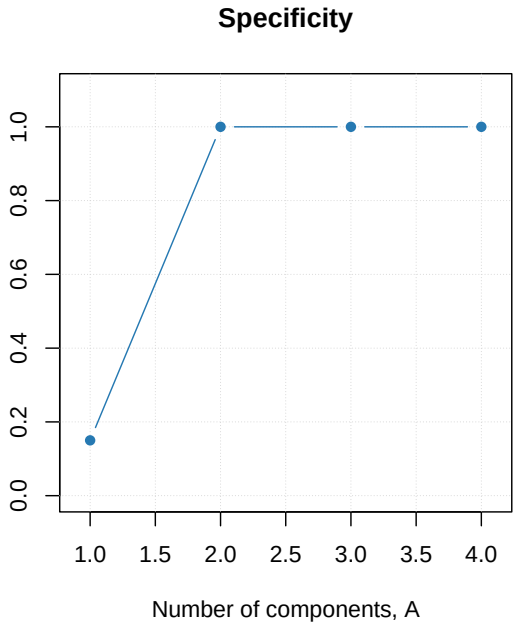
```
par(mfrow = c(2, 2))
plotAcceptance(r.test, pch = 1, log = TRUE)
plotAcceptance(r.test, pch = 1, log = TRUE, show = "strangers")
```

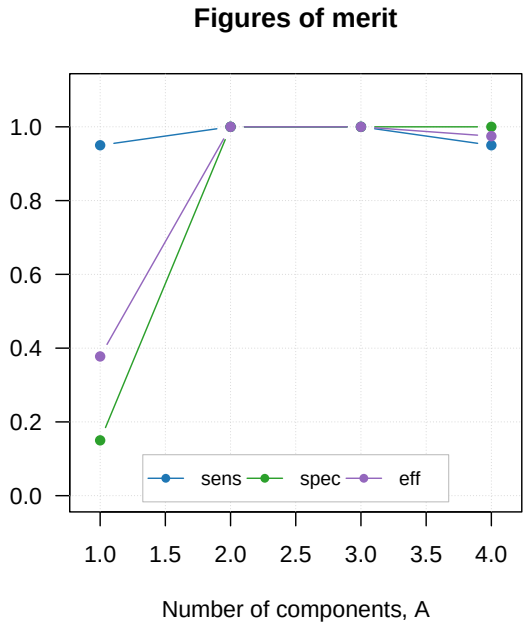
```
plotAcceptance(r.test, pch = 1, log = TRUE, show = "all")
plotAcceptance(r.test, pch = 1, ncomp = 1, log = TRUE, show = "all")
```



And the figures of merit.

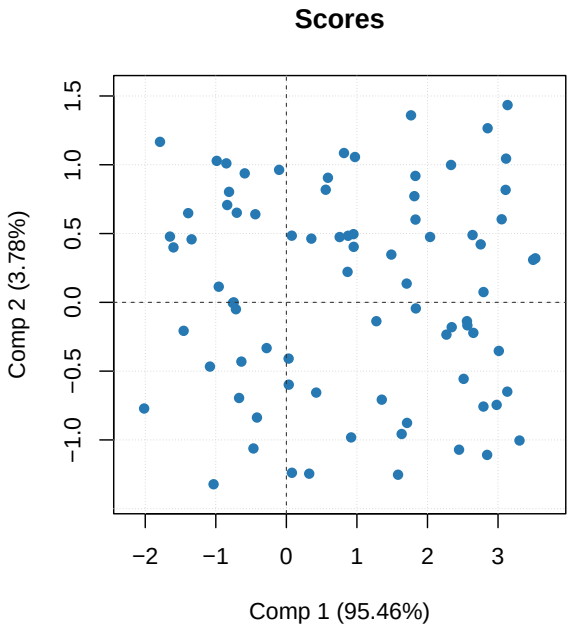
```
par(mfrow = c(1, 2))
plotFoM(r.test, fom = "spec")
plotFoMs(r.test)
```

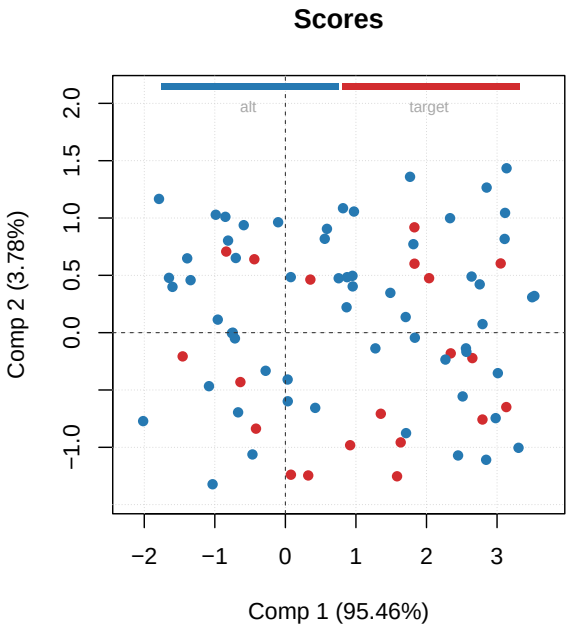




One can also show a plot with PARAFAC scores, but they are shown only for the optimal model.

```
par(mfrow = c(1, 2))
plotScores(r.test, c(1, 2))
plotScores(r.test, cgroup = as.factor(c.test))
```





And of course convert the DD-SIMCA results to a data frame:

```
df.test <- as.data.frame(r.test)
head(df.test)
```

```
##    class    role decision      h/h0     q/q0        f
## 1 target regular       in 1.1043853 1.069744 271.8536
## 2 target regular       in 2.1575837 1.134071 292.1480
## 3 target regular       in 0.6802476 1.120346 282.8076
## 4 target regular       in 0.2506432 1.050417 263.6068
## 5 target regular       in 0.4541723 1.061343 267.1525
## 6 target regular       in 0.5986852 1.081811 272.8476
```

Tucker DD-SIMCA

Because the Tucker-based DD-SIMCA is very similar to the conventional and PARAFAC DD-SIMCA, we will show only what is different here and omit the rest of the functionality.

The main difference between PARAFAC and Tucker is that for Tucker, matrices A, B, and C may have different numbers of components/factors, e.g. N_A, N_B, N_C. So when you train the model you need to provide all three numbers, not just one.

The first number, N_A, is treated as the maximum number of components. `ddsimca.tucker()` will create N_A decompositions, starting from $[1, N_B, N_C]$ and ending with $[N_A, N_B, N_C]$. So when you select the optimal number of components, or when you show a plot with figures of merit vs. the number of components, it is always the sample-mode component that varies — the other two are fixed.

Therefore the main difference is that the value of `ncomp` for `ddsimca.tucker` is a tuple of three numbers.

Here is how to train the model:

```
m <- ddsimca.tucker(X3w, "target", ncomp = c(3, 2, 3))
summary(m)
```

```
##
## DD-SIMCA-TUCKER model for class 'target' (50 x 40)
##
## Tucker ranks: (1x2x3-3x2x3) total, (3x2x3) optimal
## Number of components: 3
## Number of selected components: 3
## Alpha: 0.05
## Gamma: 0.01
##
##       Expvar Cumexpvar In Out TP FN   Sens
## Cal 0.003073     99.51 79   1 79  1 0.9875
```

As you can see, the line with the total number of components shows three items: `3` — the maximum number of components for the sample mode; `1x2x3` — the smallest number of components in the model; and `3x2x3` — the largest. As mentioned above, the last two are fixed while the first varies from 1 to the maximum number.

Here is what happens if you change the optimal number:

```
m <- selectCompNum(m, 2)
summary(m)
```

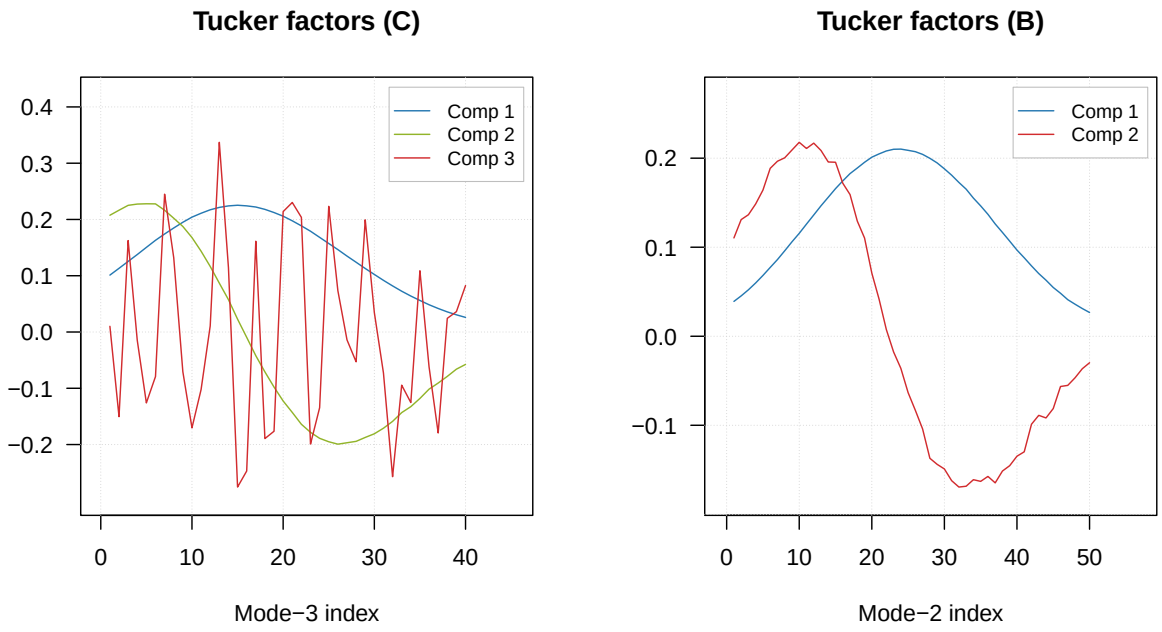
```
##
## DD-SIMCA-TUCKER model for class 'target' (50 x 40)
##
## Tucker ranks: (1x2x3-3x2x3) total, (2x2x3) optimal
## Number of components: 3
## Number of selected components: 2
## Alpha: 0.05
## Gamma: 0.01
```

```
##
##     Expvar Cumexpvar In Out TP FN   Sens
## Cal   2.96      99.5 79   1 79  1 0.9875
```

The number of components is also related to the sample mode, so 1 here means $1 \times 2 \times 3$, and so on.

Like in PARAFAC, you can show the factors as plots — just remember that the number of factors for B and C can be different (as in our model, we have 2 factors for B and 3 factors for C):

```
par(mfrow = c(1, 2))
plotFactors(m, comp = 1:3, "C")
plotFactors(m, comp = 1:2, "B")
```



The third C factor is clearly just noise, though.

All other functionality is identical to PARAFAC DD-SIMCA.

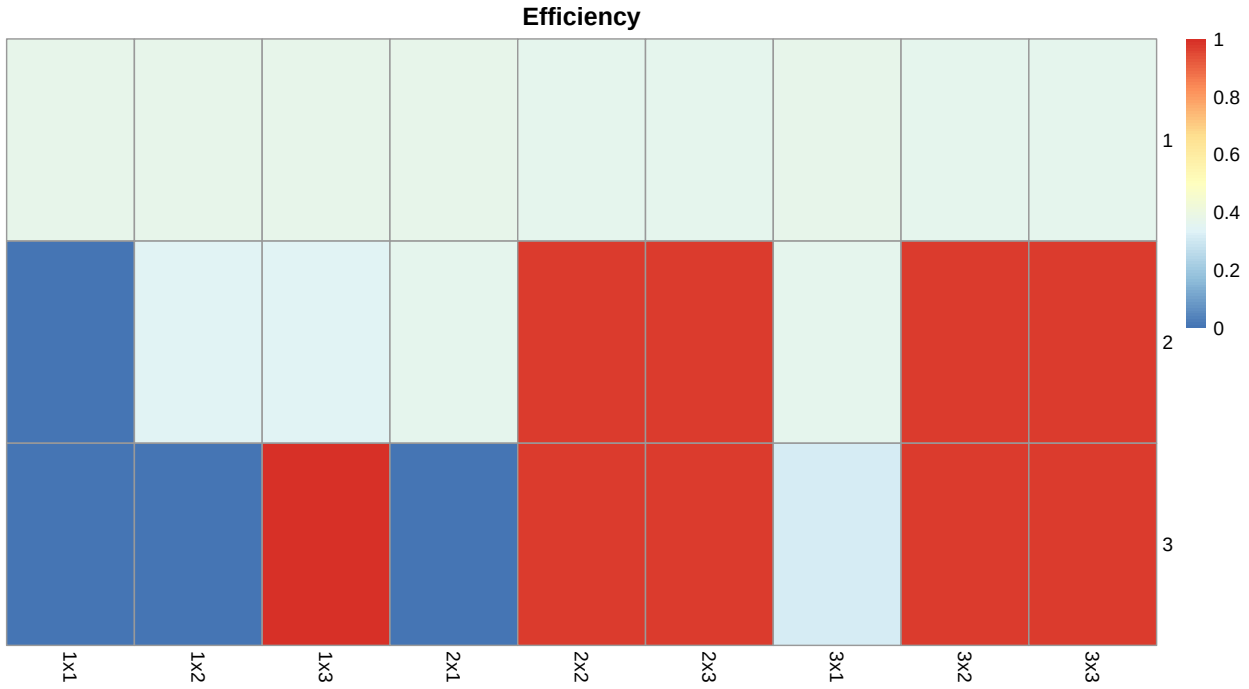
If you want to find optimal efficiency across all 3 modes, just use loops. Here is an example where we will search for the best combination of N_A, N_B, N_C in a range $[1, 3]$:

```
library(pheatmap)


eff <- matrix(0, 9, 3)
labels <- rep("", 9)
n <- 1
for (b in 1:3) {
   for (c in 1:3) {
      a <- min(c(3, b * c))
      m <- ddsimca.tucker(X3w, "target", ncomp = c(a, b, c))
      r <- predict(m, X3wb, c.test)
      eff[n, 1:a] <- r$simca$outcomes$moments$eff
      labels[n] <- sprintf("%dx%d", b, c)
      n <- n + 1
   }
```

```
}

colnames(eff) <- 1:3
rownames(eff) <- labels
pheatmap(t(eff),
   main = "Efficiency",
   cluster_rows = FALSE,
   cluster_cols = FALSE
)
```

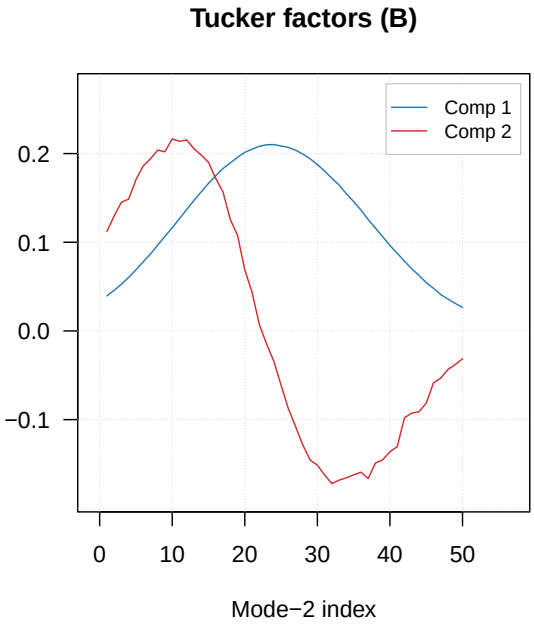


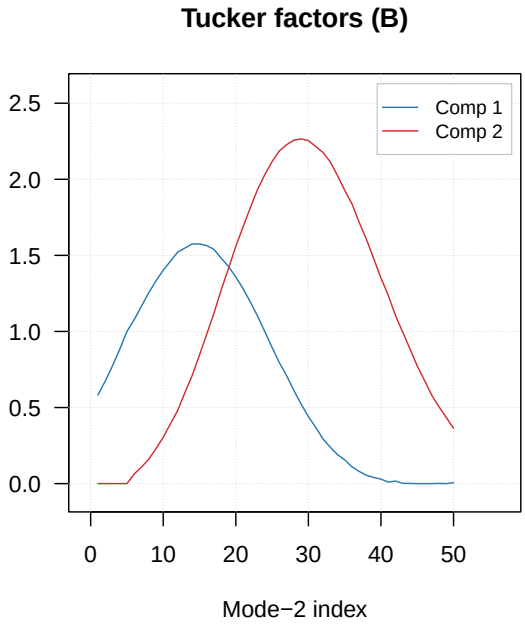
Imposing non-negativity

Both `ddsimca.parafac` and `ddsimca.tucker` can use a non-negativity constraint for the corresponding decomposition. Simply specify `non.negative = TRUE` when training a model. Here is an example for Tucker:

```
X3w[X3w < 0] <- 0
m1 <- ddsimca.tucker(X3w, "target", ncomp = c(3, 2, 3))
m2 <- ddsimca.tucker(X3w, "target", ncomp = c(3, 2, 3), non.negative = TRUE)

par(mfrow = c(1, 2))
plotFactors(m1, "B", comp = 1:2)
plotFactors(m2, "B", comp = 1:2)
```






```
detach(data3w)
```

Partial least squares regression

Partial least squares regression (PLS) is a linear regression method, which uses principles similar to PCA: data is decomposed using latent variables. Because in this case we have two datasets, a matrix with predictors ($\mathbf{X}$) and a matrix with responses ($\mathbf{Y}$) we do decomposition for both, computing scores, loadings and residuals: $\mathbf{X} = \mathbf{T}\mathbf{P}^{\mathrm{T}} + \mathbf{E}_x$, $\mathbf{Y} = \mathbf{U}\mathbf{Q}^{\mathrm{T}} + \mathbf{E}_y$. In addition to that, orientation of latent variables in PLS is selected to maximize the covariance between the X-scores, $\mathbf{T}$, and Y-scores $\mathbf{U}$. This approach makes it possible to work with datasets where more traditional Multiple Linear Regression fails — when the number of variables exceeds the number of observations and when X-variables are mutually correlated. But, at the end, PLS-model is a linear model, where response value is just a linear combination of predictors, so the main outcome is a vector with regression coefficients.

There are two main algorithms for PLS, *NIPALS* and *SIMPLS*, in the *mdatools* only the last one is implemented. PLS model and PLS results objects have a lot of properties and performance statistics, which can be visualized via plots. Besides that, there is also a possibility to compute selectivity ratio (SR) and VIP scores, which can be used for selection of the most important variables. Another additional option is a randomization test which helps to select the optimal number of components. We will discuss most of the methods in this chapter and you can get the full list using `?pls`.

Models and results

Like we discussed for PCA, *mdatools* creates two types of objects — a model and a result. Every time you build a PLS model you get a *model object*. Every time you apply the model to a dataset you get a *result object*. For PLS, the objects have classes `pls` and `plsres` correspondingly.

Model calibration

Let's use the same *People* data and create a PLS-model for prediction of *Shoesize* (column number four) using other 11 variables as predictors. As usual, we start with preparing datasets (we will also split the data into calibration and test subsets):

```
library(mdatools)
data(people)

idx = seq(4, 32, 4)
Xc = people[-idx, -4]
yc = people[-idx, 4, drop = FALSE]
Xt = people[idx, -4]
yt = people[idx, 4, drop = FALSE]
```

So `Xc` and `yc` are predictors and response values for calibration subset. Now let's calibrate the model and show an information about the model object:

```
m = pls(Xc, yc, 7, scale = TRUE, info = "Shoesize prediction model")
```

```
## Warning in selectCompNum.pls(model, selcrit = ncomp.selcrit): No validation
## results were found.
```

You can notice that the calibration succeeded but there is also a warning about lack of validation results. For supervised models, which have complexity parameter (in this case — number of components), doing proper validation is important as it helps to find the optimal complexity. When you calibrate a PLS model the calibration also tries to find the optimal number (details will be discussed later in this chapter) and this needs some validation. How to do proper validation of PLS models is discussed in the next section.

Here is an info for the model object:

```
print(m)
```

```
##
## PLS model (class pls)
##
## Call:
## selectCompNum.pls(obj = model, selcrit = ncomp.selcrit)
##
## Major fields:
##  $ncomp - number of calculated components
##  $ncomp.selected - number of selected components
##  $coeffs - object (regcoeffs) with regression coefficients
##  $xloadings - matrix with x loadings
##  $yloadings - matrix with y loadings
##  $weights - matrix with weights
##  $res - list with results (calibration, cv, etc)
##
## Try summary(model) and plot(model) to see the model performance.
```

As expected, we see loadings for predictors and responses, matrix with weights, and a special object (`regcoeffs`) for regression coefficients.

Result object

Similar to PCA, model object contains a list with result objects (`res`), obtained using calibration set (`cal`), cross-validation (`cv`) and test set validation (`test`). All three have class `plsres`, here is how `res$cal` looks like:

```
print(m$res$cal)
```

```
##
## PLS results (class plsres)
##
## Call:
## plsres(y.pred = yp, y.ref = y.ref, ncomp.selected = object$ncomp.selected,
##     xdecomp = xdecomp, ydecomp = ydecomp)
##
## Major fields:
##  $ncomp.selected - number of selected components
##  $y.pred - array with predicted y values
##  $y.ref - matrix with reference y values
##  $rmse - root mean squared error
##  $r2 - coefficient of determination
##  $slope - slope for predicted vs. measured values
##  $bias - bias for prediction vs. measured values
##  $ydecomp - decomposition of y values (ldecomp object)
```

```
##  $xdecomp - decomposition of x values (ldecomp object)
```

The `xdecomp` and `ydecomp` are objects similar to `pcares`, they contain scores, residuals and variances for decomposition of X and Y correspondingly.

```
print(m$res$cal$xdecomp)
```

```
##
## Results of data decomposition (class ldecomp).
##
## Major fields:
##  $scores - matrix with score values
##  $T2 - matrix with T2 distances
##  $Q - matrix with Q residuals
##  $ncomp.selected - selected number of components
##  $expvar - explained variance for each component
##  $cumexpvar - cumulative explained variance
```

Other fields are mostly various performance statistics, including slope, coefficient of determination (R^2), bias, and root mean squared error (RMSE). Besides that, the results also include reference y-values and an array with predicted y-values. The array has dimension *nObjects x nComponents x nResponses*.

PLS predictions for a new set can be obtained using method `predict`:

```
res = predict(m, Xt, yt)
print(res)
```

```
##
## PLS results (class plsres)
##
## Call:
## plsres(y.pred = yp, y.ref = y.ref, ncomp.selected = object$ncomp.selected,
##     xdecomp = xdecomp, ydecomp = ydecomp)
##
## Major fields:
##  $ncomp.selected - number of selected components
##  $y.pred - array with predicted y values
##  $y.ref - matrix with reference y values
##  $rmse - root mean squared error
##  $r2 - coefficient of determination
##  $slope - slope for predicted vs. measured values
##  $bias - bias for prediction vs. measured values
##  $ydecomp - decomposition of y values (ldecomp object)
##  $xdecomp - decomposition of x values (ldecomp object)
```

If reference y-values are not provided to `predict()` function, then all predictions are computed anyway, but performance statistics (and corresponding plot) will not be available.

Validation

Validation of PLS (and PLS-DA) models can be done by one of the following methods:

1. Using separate validation/test set
2. Using Procrustes cross-validation
3. Using conventional cross-validation

Below we will show each of the method in detail. For all examples we will use *Simdata* with UV/VIS spectra of mixtures of three carbohydrates, this dataset has been already used for examples in Preprocessing and

PCA chapters. It contains spectra and reference concentrations for two sets of measurements.

Using validation/test set

If you have a dedicated validation/test set, you need to provide it as two separate matrices — one for predictors (parameter `x.test`) and one for response (`y.test`) values. The rest will be done automatically and you will see the results for test set in all plots and summary tables.

The example below shows how to create such model for *Simdata* (here we will make PLS1 model for prediction of concentration of the first mixture component — first column of the concentration matrix):

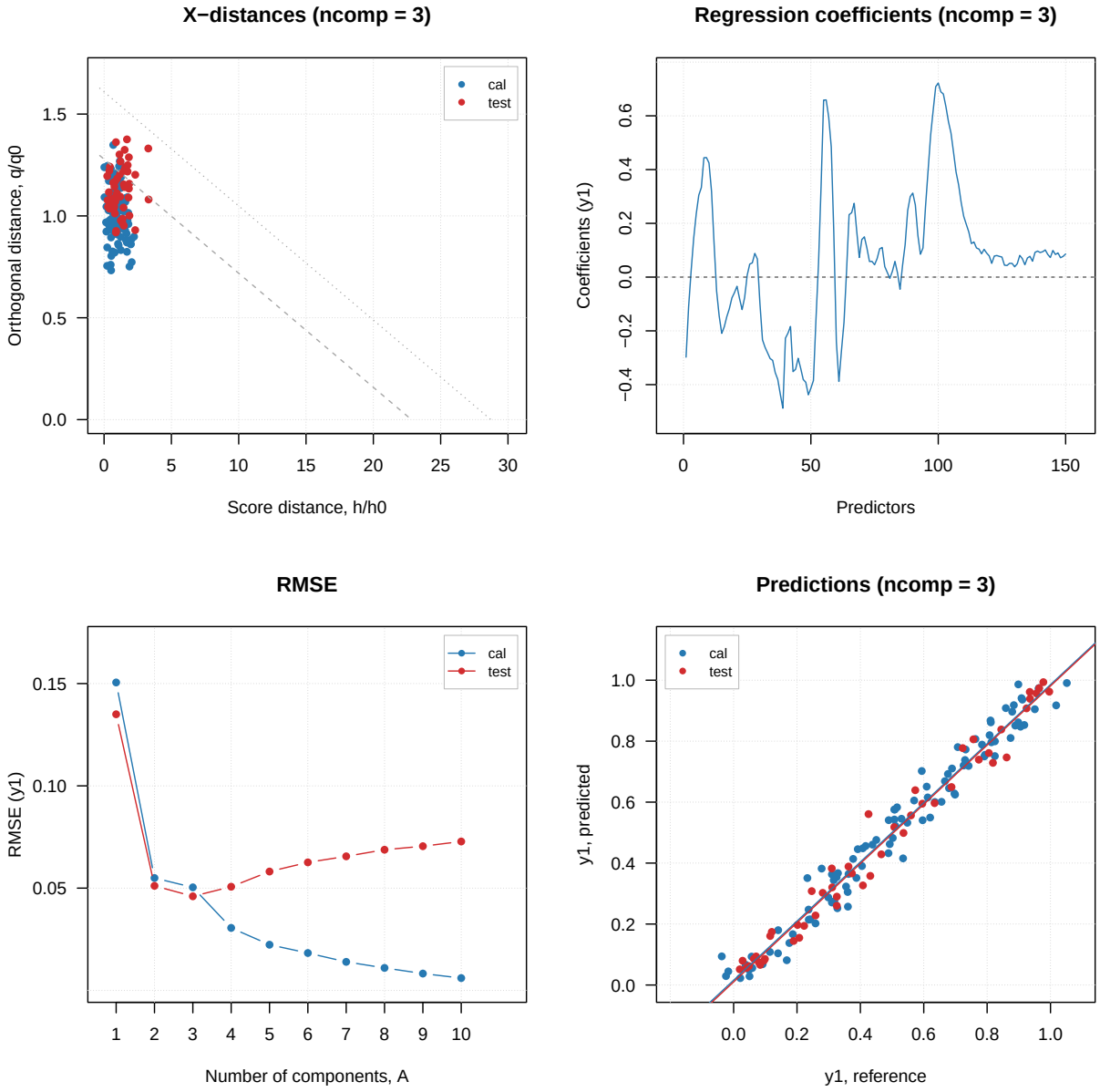
```
data(simdata)
Xc = simdata$spectra.c
yc = simdata$conc.c[, 1]
Xt = simdata$spectra.t
yt = simdata$conc.t[, 1]

m = pls(Xc, yc, 10, x.test = Xt, y.test = yt)
summary(m)
```

```
##
## PLS model (class pls) summary
## -------------------------------
## Info:
## Number of selected components: 3
## Cross-validation: none
##
##      X cumexpvar Y cumexpvar    R2  RMSE Slope  Bias  RPD
## Cal       99.977      96.960 0.970 0.050 0.970 0.000 5.76
## Test      99.983      97.951 0.979 0.046 0.971 0.003 6.93
```

As you can see, in this case we do not have a warning about absence of validation results as we have seen in the example above. Summary information is available for optimal number of components for both calibration and test set. Model was able to detect that 3 components are optimal despite the provided number of 10 components. How this selection is done is described later in this chapter, meanwhile let's look at the plot overview:

```
plot(m)
```



Distance plot, RMSE and plot with predicted and measured y-values have points for both calibration and test set. The RMSE plot shows clearly that after third component the model becomes overfitted as error for the test set is increasing.

As mentioned, method `summary()` for validated model shows performance statistics calculated using optimal number of components for each of the results (calibration and test set validation in our case).

If you want more details run `summary()` for one of the result objects.

```
summary(m$res$test)
```

```
##
## PLS regression results (class plsres) summary
## Info: test set validation results
## Number of selected components: 3
##         X expvar X cumexpvar Y expvar Y cumexpvar    R2  RMSE Slope    Bias
```

```
## Comp 1    99.638      99.638   82.362      82.362 0.816 0.135 0.868 -0.0234
## Comp 2     0.246      99.884   15.109      97.471 0.974 0.051 0.962  0.0039
## Comp 3     0.099      99.983    0.480      97.951 0.979 0.046 0.971  0.0030
## Comp 4     0.000      99.983   -0.438      97.513 0.974 0.051 0.975  0.0037
## Comp 5     0.000      99.983   -0.781      96.732 0.966 0.058 0.982  0.0024
## Comp 6     0.000      99.983   -0.524      96.208 0.961 0.063 0.989  0.0035
## Comp 7     0.000      99.983   -0.366      95.842 0.957 0.066 0.996  0.0064
## Comp 8     0.000      99.983   -0.419      95.422 0.952 0.069 0.995  0.0083
## Comp 9     0.000      99.983   -0.232      95.190 0.950 0.070 0.996  0.0099
## Comp 10    0.000      99.983   -0.319      94.871 0.947 0.073 0.997  0.0106
##          RPD
## Comp 1  2.39
## Comp 2  6.24
## Comp 3  6.93
## Comp 4  6.29
## Comp 5  5.48
## Comp 6  5.09
## Comp 7  4.88
## Comp 8  4.66
## Comp 9  4.56
## Comp 10 4.42
```

In this case, the statistics are shown for all available components and explained variance for individual components is added.

Using Procrustes cross-validation

As in case of PCA, you can also generate the validation set using Procrustes cross-validation. Here I assume that you already have package `pcv` installed and read about the method (or at least tried it with PCA examples).

Let's create the PV-set first:

```
library(pcv)
Xpv = pcvpls(Xc, yc, 20, cv = list("ven", 4))
```

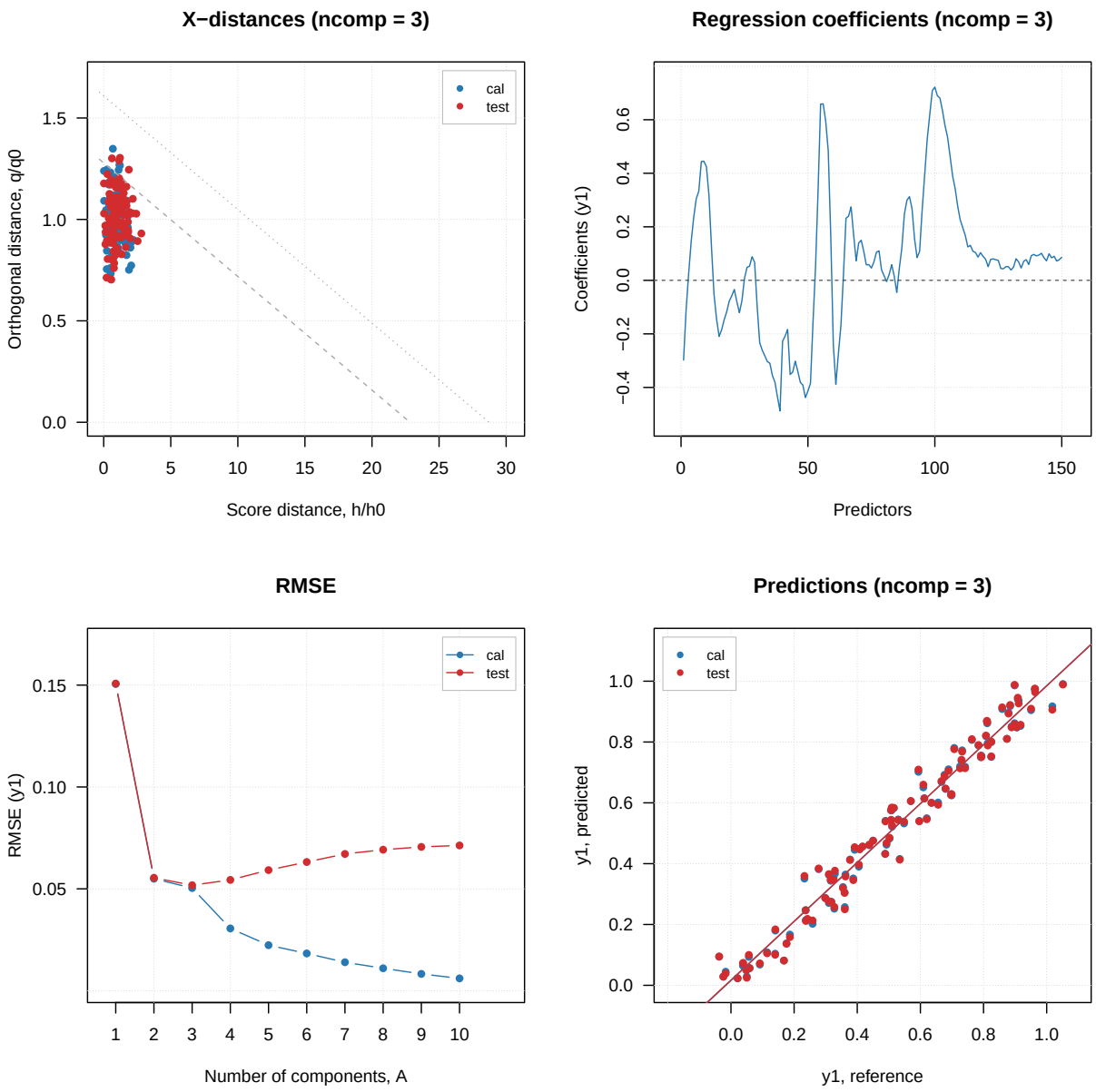
In this case we generate only predictors, the response values will be the same as for the calibration set. Now we can create a model similar to what we did for dedicated validation set (note that we provide `yc` as values for the `y.test` argument in this case):

```
m2 = pls(Xc, yc, 10, x.test = Xpv, y.test = yc)
summary(m2)
```

```
##
## PLS model (class pls) summary
## -------------------------------
## Info:
## Number of selected components: 3
## Cross-validation: none
##
##      X cumexpvar Y cumexpvar    R2  RMSE Slope   Bias  RPD
## Cal       99.977      96.960 0.970 0.050 0.970  0e+00 5.76
## Test      99.977      96.789 0.968 0.052 0.967 -4e-04 5.61
```

As you can see in this case PLS also selected 3 components as optimal. Here is the overview of the model.

```
plot(m2)
```



The behaviour of RMSE values is quite similar to the one we got for the test set.

Using conventional cross-validation

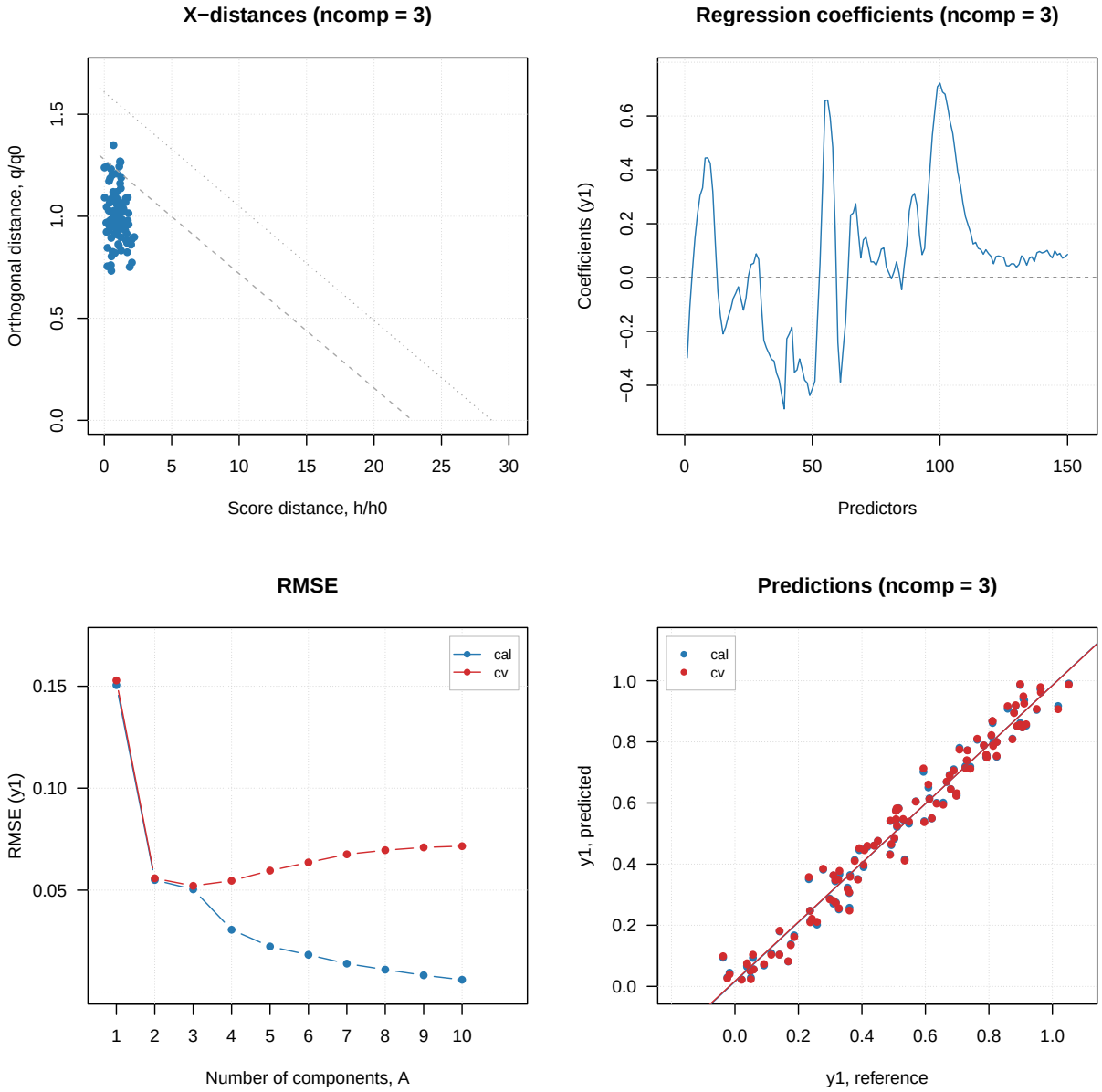
Alternatively you can use conventional cross-validation as well. But in this case only figures of merit (e.g. RMSE and R2 values) will be computed, as computing variances and distances cannot be done correctly. This is enough though to optimize the parameters of your model (number of components, preprocessing, variable selection) and then apply the final optimized model to the test set.

In this case you do not need to provide any extra data, but just set cross-validation parameter `cv` similar to what you do when generate PV-set:

```
m3 = pls(Xc, yc, 10, cv = list("ven", 4))
summary(m3)
```

```
##
## PLS model (class pls) summary
## -------------------------------
## Info:
## Number of selected components: 3
## Cross-validation: venetian blinds with 4 segments
##
##     X cumexpvar Y cumexpvar    R2  RMSE Slope   Bias  RPD
## Cal    99.97681    96.96024 0.970 0.050 0.970  0e+00 5.76
## Cv           NA          NA 0.968 0.052 0.967 -5e-04 5.58
```

```
plot(m3)
```



As you can see, all results are very similar to what you got with PV-set, however some of the outcomes (e.g. distances, explained variance, etc) are not available.

In the examples above we set parameter `cv` to `list("ven", 4)`. But in fact it can be a number, a list or a vector with segment values manually assigned for each measurement.

If it is a number, it will be used as number of segments for random cross-validation, e.g. if `cv = 2` cross-validation with two segments will be carried out with measurements being randomly split into the segments. For full cross-validation use `cv = 1` like in the example above. This option is a bit confusing, logically we have to set `cv` equal to the number of measurements in the calibration set. But this option was used many years ago when the package was created and it is kept for backward compatibility.

For more advanced selection you can provide a list with name of cross-validation method, number of segments and number of iterations, e.g. `cv = list("rand", 4, 4)` for running random cross-validation with four segments and four repetitions or `cv = list("ven", 8)` for systematic split into eight segments (*venetian blinds*). In case of venetian splits, the measurements will be automatically ordered from smallest to largest

response (y) values.

Finally, you can also provide a vector with manual slits. For example, if you create a model with the following value for `cv` parameter: `cv = rep(1:4, length.out = nrow(Xc))` the vector of values you provide will look as `1 2 3 4 1 2 3 4 1 2 ...`, which corresponds to systematic splits but without ordering y-values, so it will use the current order of measurements (rows) in the data. Use this option if you have specific requirements for cross-validation and the implemented methods do not meet them.

Local and global CV scope

From *0.14.1* it is possible to set how cross-validation subsets are centred and scaled inside the cross-validation loop. This is regulated by a new parameter `cv.scope` in methods `pls()`, `plsda()` and `ipls()`.

Cross-validation splits the original training set into local calibration and validation sets inside the loop. For example, if you have dataset with 100 objects/observations and use cross-validation with four segments (with random or systematic splits), in each iteration 25 objects will be taken as local validation set and remaining 75 objects will form a local calibration set.

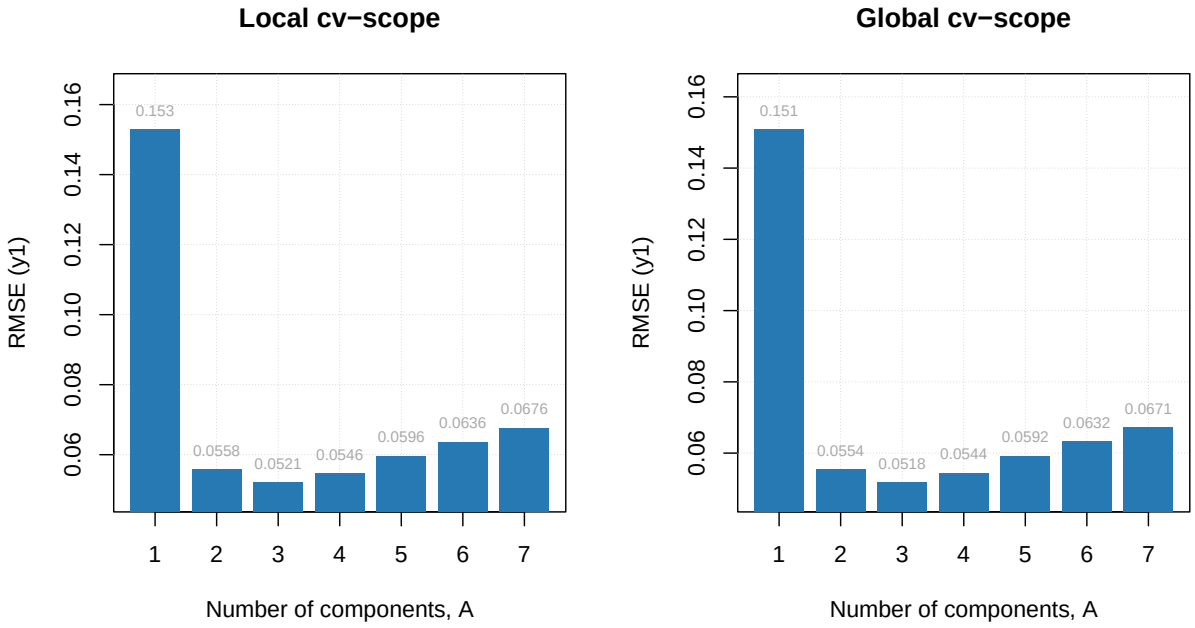
If `cv.scope` parameter is set to `"local"`, then mean values for centering and standard deviation values for scaling (if this option is selected) will be computed based on the local set with 75 objects in the example below. So, every local model will have its own center in the variable space.

If `cv.scope` parameter is set to `"global"`, then the mean values and standard deviation values will be computed for the original (global) training set hence all local models will have the same center as the global model.

The default value is `"local"` and this was also the way `mdatools` worked before this option was introduced in *0.14.1*. The small example below shows how both options work (it is assumed that you already have `Xc` and `yc` variables from the previous examples):

```
m1 = pls(Xc, yc, 7, cv = list("ven", 4), cv.scope = "local")
m2 = pls(Xc, yc, 7, cv = list("ven", 4), cv.scope = "global")

par(mfrow = c(1, 2))
plotRMSE(m1$res$cv, type = "h", show.labels = TRUE, main = "Local cv-scope")
plotRMSE(m2$res$cv, type = "h", show.labels = TRUE, main = "Global cv-scope")
```



As you can see, the cross-validation results are very similar, however RMSE values for locally autoscaled models tend to be a bit higher as expected.

Selection of optimal number of components

When you create a PLS model, it tries to select optimal number of components automatically (which, of course, you can always change later). To do that, the method uses RMSE values, calculated for different number of components and validation predictions. So, if we do not use any validation, it warns us about this as it was mentioned in the beginning of this chapter.

If validation (any of the three methods described above) was used, there are two different ways/criteria for automatic selection of the components based on validation results. One is using first local minimum on the RMSE plot and second is so called Wold criterion, based on a ratio between PRESS values for current and next component.

You can select which criterion to use by specifying parameter `ncomp.selcrit` (either `'min'` or `'wold'`) as it is shown below (here we use cross-validation).

```
m1 = pls(Xc, yc, 7, cv = list("ven", 4), ncomp.selcrit = "min")
show(m1$ncomp.selected)
```

```
## [1] 3
```

```
m2 = pls(Xc, yc, 7, cv = list("ven", 4), ncomp.selcrit = "wold")
show(m2$ncomp.selected)
```

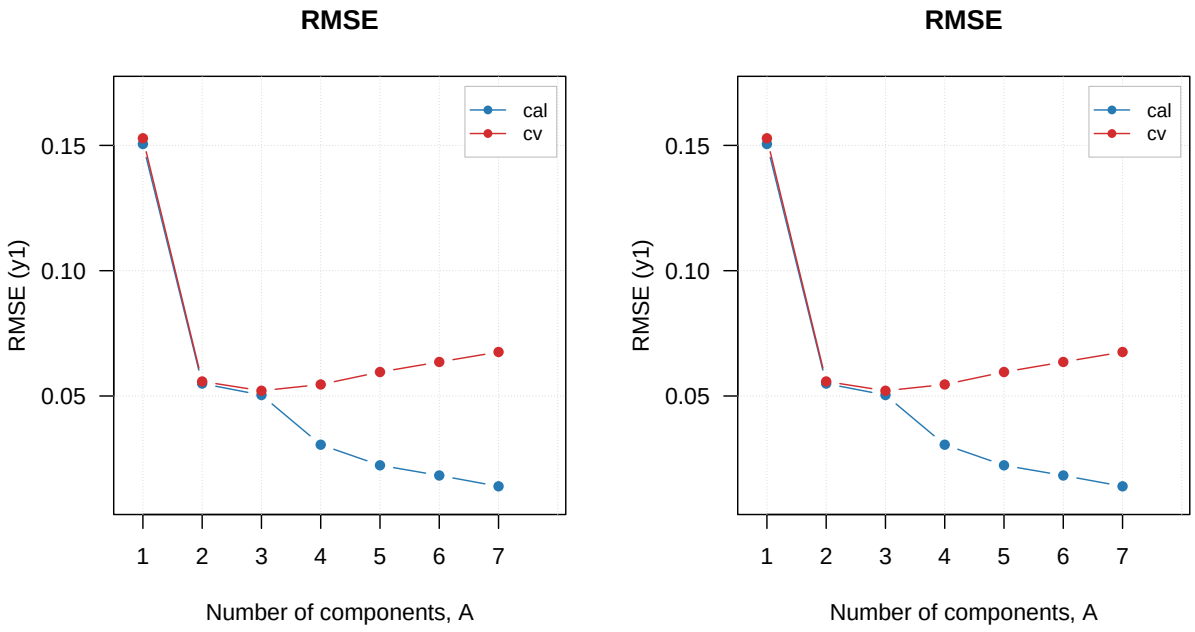
```
## [1] 3
```

Well, in this case both pointed to the same number, 3, but sometimes they give different solutions.

And here are the RMSE plots (they are identical of course), where you can see how error depends on number of components for both calibration set and cross-validated predictions. Apparently the minimum for cross-validation error (RMSECV) is indeed at 3:

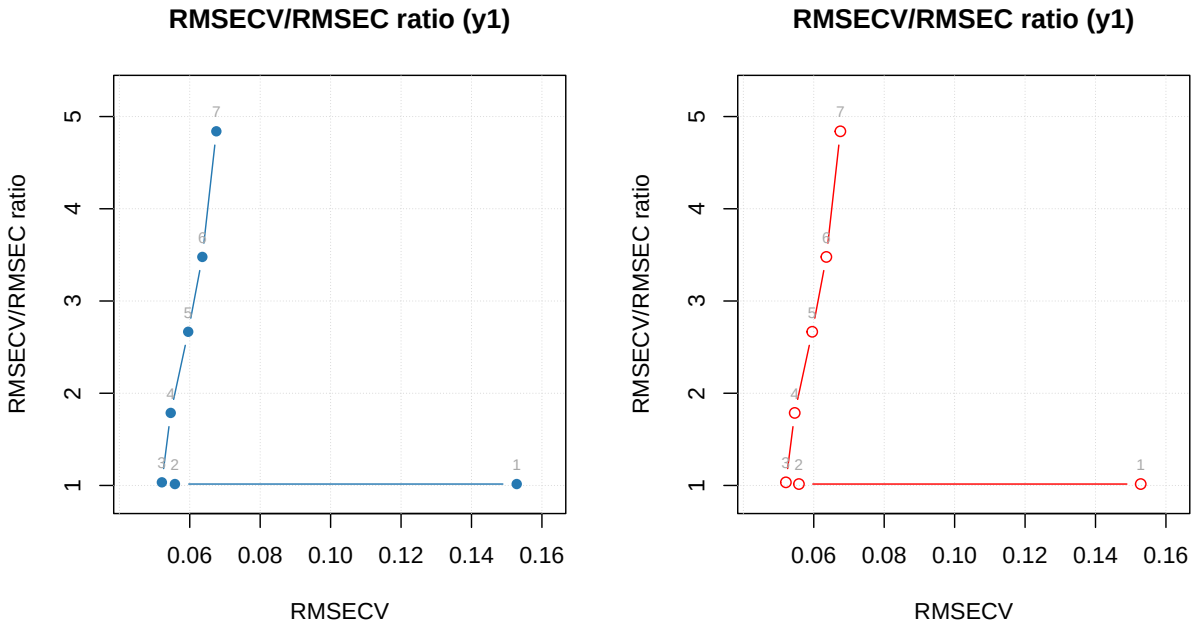
```
par(mfrow = c(1, 2))
plotRMSE(m1)
```

```
plotRMSE(m2)
```



Another useful plot in this case is a plot which shows ratio between cross-validated RMSE values, RMSECV, and the calibrated ones, RMSEC. You can see an example in the figure below and read more about this plot in blog post by Barry M. Wise.

```
par(mfrow = c(1, 2))
plotRMSERatio(m1)
plotRMSERatio(m2, pch = 1, col = "red")
```



Regression coefficients

First of all let us re-create the PLS model for prediction of Shoesize from People data we used at the beginning of this chapter:

```
library(mdatools)
data(people)

idx = seq(4, 32, 4)
Xc = people[-idx, -4]
yc = people[-idx, 4, drop = FALSE]
Xt = people[idx, -4]
yt = people[idx, 4, drop = FALSE]
```

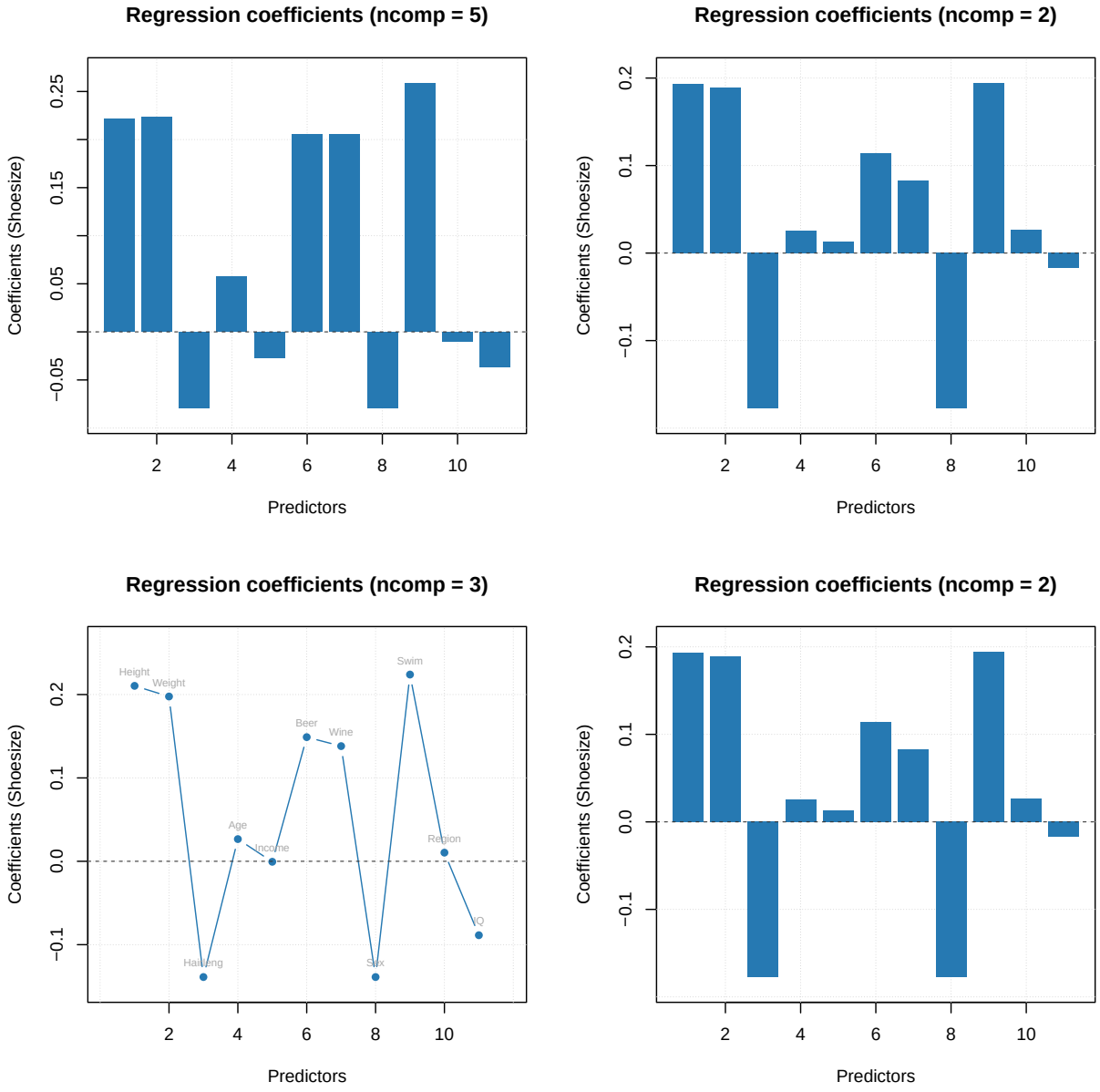
But this time I will create the model using full cross-validation:

```
m = pls(Xc, yc, 7, scale = TRUE, cv = 1, info = "Shoesize prediction model")
```

The values for regression coefficients are available in `m$coeffs$values`, it is an array with dimension *nVariables x nComponents x nPredictors*. The reason to use the object instead of just an array is mainly for being able to get and plot regression coefficients for different methods. Besides that, it is possible to calculate confidence intervals and other statistics for the coefficients using Jack-Knife method (will be shown later), which produces extra entities.

The regression coefficients can be shown as plot using either function `plotRegcoeffs()` for the PLS model object or function `plot()` for the object with regression coefficients. You need to specify for which predictor (if you have more than one y-variable) and which number of components you want to see the coefficients for. By default it shows values for the optimal number of components and first y-variable as it is shown on example below.

```
par(mfrow = c(2, 2))
plotRegcoeffs(m)
plotRegcoeffs(m, ncomp = 2)
plot(m$coeffs, ncomp = 3, type = "b", show.labels = TRUE)
plot(m$coeffs, ncomp = 2)
```



The model keeps regression coefficients, calculated for centered and standardized data, without intercept, etc. Here are the values for three PLS components.

```
show(m$coeffs$values[, 3, 1])
```

```
##       Height       Weight     Hairleng          Age       Income         Beer
##  0.210411676  0.197646483 -0.138824482  0.026613035 -0.000590693  0.148917913
##         Wine          Sex         Swim       Region           IQ
##  0.138138095 -0.138824482  0.223962000  0.010392542 -0.088658626
```

You can see a summary for the regression coefficients object by calling function `summary()` for the object `m$coeffs` like it is shown below. By default it shows only estimated regression coefficients for the selected y-variable and number of components.

However, if you use cross-validation, Jack-Knifing method will be used to compute some statistics, including standard error, p-value (for test if the coefficient is equal to zero in population) and confidence interval. All

statistics in this case will be shown automatically with `summary()` as you can see below.

```
summary(m$coeffs)
```

```
##
## Regression coefficients for Shoesize (ncomp = 1)
## -----------------------------------------------
##                Coeffs  Std. err. t-value p-value        2.5%       97.5%
## Height    0.176077659 0.01594024   11.03   0.000  0.14310275  0.20905257
## Weight    0.175803980 0.01598815   10.98   0.000  0.14272997  0.20887799
## Hairleng -0.164627444 0.01638528  -10.04   0.000 -0.19852297 -0.13073192
## Age       0.046606027 0.03718827    1.25   0.225 -0.03032377  0.12353583
## Income    0.059998121 0.04047132    1.47   0.155 -0.02372318  0.14371942
## Beer      0.133136867 0.01116749   11.89   0.000  0.11003515  0.15623859
## Wine      0.002751573 0.03542518    0.08   0.936 -0.07053100  0.07603415
## Sex      -0.164627444 0.01638528  -10.04   0.000 -0.19852297 -0.13073192
## Swim      0.173739533 0.01516461   11.44   0.000  0.14236915  0.20510992
## Region   -0.031357608 0.03590576   -0.87   0.395 -0.10563433  0.04291911
## IQ       -0.003353428 0.03841171   -0.08   0.934 -0.08281410  0.07610725
##
## Degrees of freedom (Jack-Knifing): 23
```

You can also get the corrected coefficients, which can be applied directly to the raw data (without centering and standardization), by using method `getRegcoeffs()`:

```
show(getRegcoeffs(m, ncomp = 3))
```

```
##               Estimated
## Intercept  1.251537e+01
## Height     8.105287e-02
## Weight     5.110732e-02
## Hairleng  -5.375404e-01
## Age        1.147785e-02
## Income    -2.580586e-07
## Beer       6.521476e-03
## Wine       1.253340e-02
## Sex       -5.375404e-01
## Swim       1.164947e-01
## Region     4.024083e-02
## IQ        -2.742712e-02
## attr(,"name")
## [1] "Regression coefficients for  Shoesize"
```

Plotting methods

Plotting methods, again, work similarly to PCA, so in this section we will look in more detail at the available methods instead of on how to customize them. PLS has a lot of different results and much more possible plots. Here is a list of methods, which will work both for a model and for a particular results.

Plotting methods for summary statistics.

Method	Description
`plotRMSE(obj, ny, ...)`	RMSE values vs. number of components in a model
`plotRMSERatio(obj, ny, ...)`	RMSECV/RMSEC ratio values vs. RMSECV in a model
`plotXVariance(obj, ...)`	explained variance for X decomposition for each component

Method	Description
`plotXCumVariance(obj, ...)`	same as above but for cumulative variance
`plotYVariance(obj, ...)`	explained variance for Y decomposition for each component
`plotYCumVariance(obj, ...)`	same as above but for cumulative variance

Plotting methods for objects.

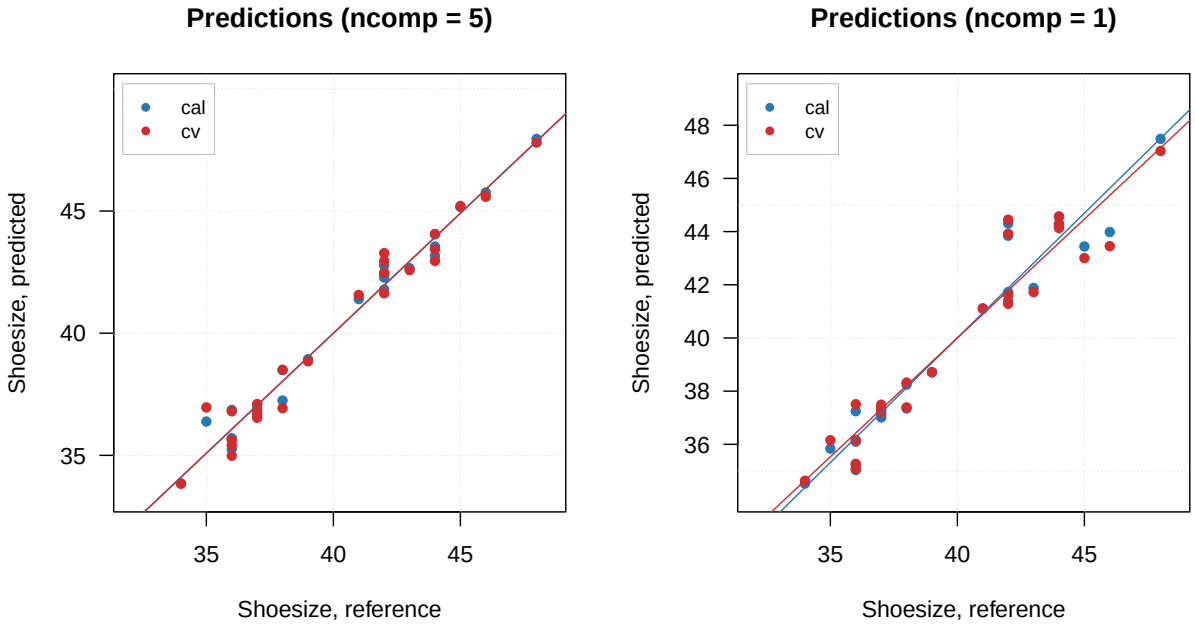
Method	Description
`plotPredictions(obj, ny, ncomp, ...)`	plot with predicted vs. measured (reference) y-values
`plotXScores(obj, comp, ...)`	scores for decomposition of X (similar to PCA plot)
`plotYScores(obj, comp, ...)`	scores for decomposition of Y (similar to PCA plot)
`plotXResiduals(obj, ncomp, ...)`	distance plot for decomposition of X (similar to PCA)
`plotYResiduals(obj, ncomp, ...)`	distance plot for decomposition of Y
`plotXYResiduals(obj, ncomp, ...)`	distance plot for both X and Y decomposition
`plotXYScores(obj, ncomp, ...)`	Y-scores vs. X-scores for a particular PLS component.

Parameter `obj` is either a model or a result object and it is the only mandatory argument for the plots. All other parameters have reasonable default values. Parameter `ny` is used to specify which y-variable you want to see a plot for (if **Y** is multivariate). You can also provide any parameter from `mdaplot()` or `mdaplotg()` thus change limits or labels for axis, main title, colors, line and marker style etc.

Parameter `comp` allows to provide a number of selected components (one or several) to show the plot for, while parameter `ncomp` assumes that only one number is expected (number of components in a model or particular individual component). So if e.g. you create model for five components and selected three as optimal, you can also see, for example, prediction plot for having only one or four components in your model.

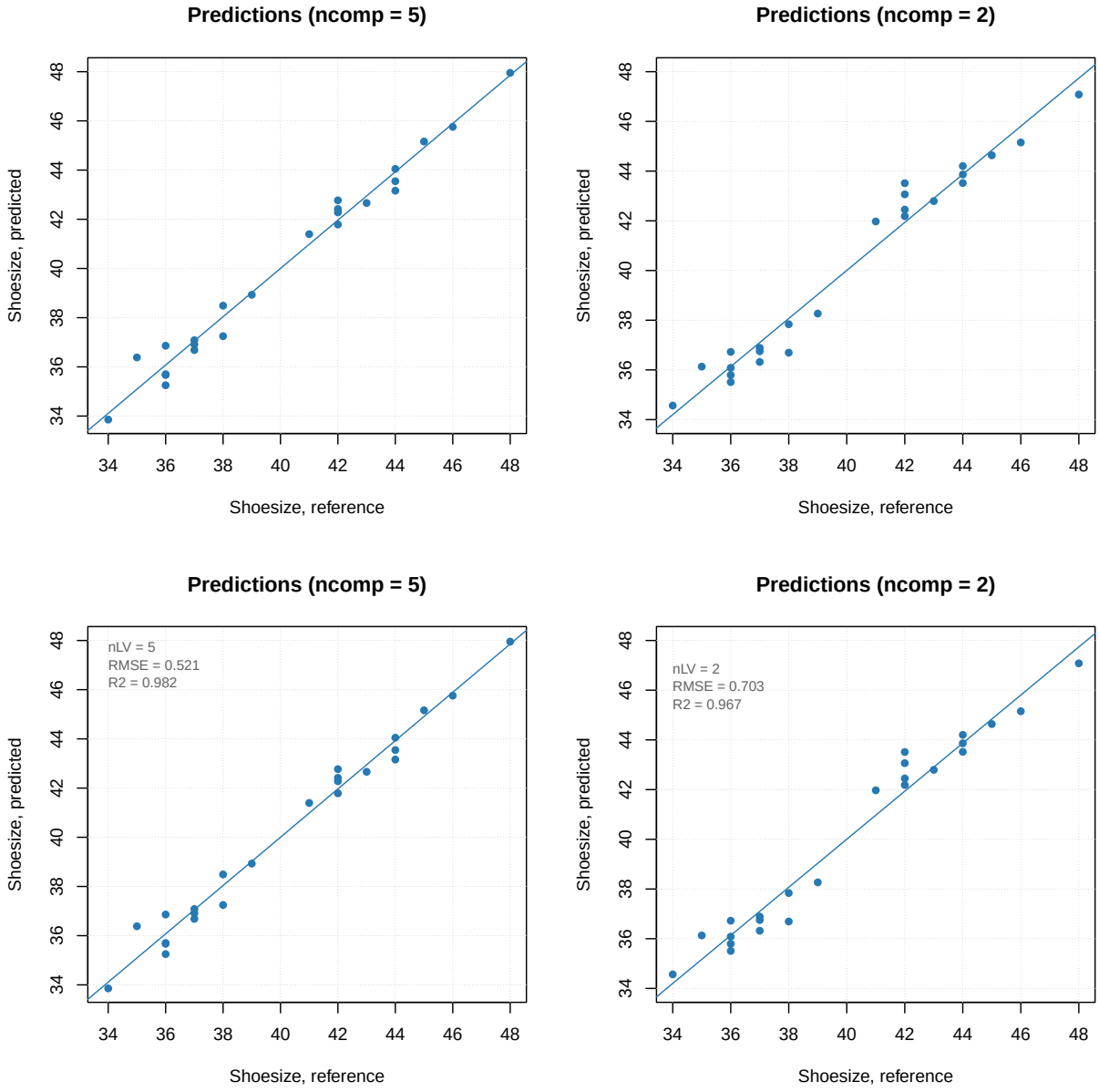
Here is an example for creating prediction plot for `m` model (left for automatically selected number of components, right for user specified value):

```
par(mfrow = c(1, 2))
plotPredictions(m)
plotPredictions(m, ncomp = 1)
```



By the way, when `plotPredictions()` is made for results object, you can show performance statistics on the plot:

```
par(mfrow = c(2, 2))
plotPredictions(m$res$cal)
plotPredictions(m$res$cal, ncomp = 2)
plotPredictions(m$res$cal, show.stat = TRUE)
plotPredictions(m$res$cal, ncomp = 2, show.stat = TRUE)
```

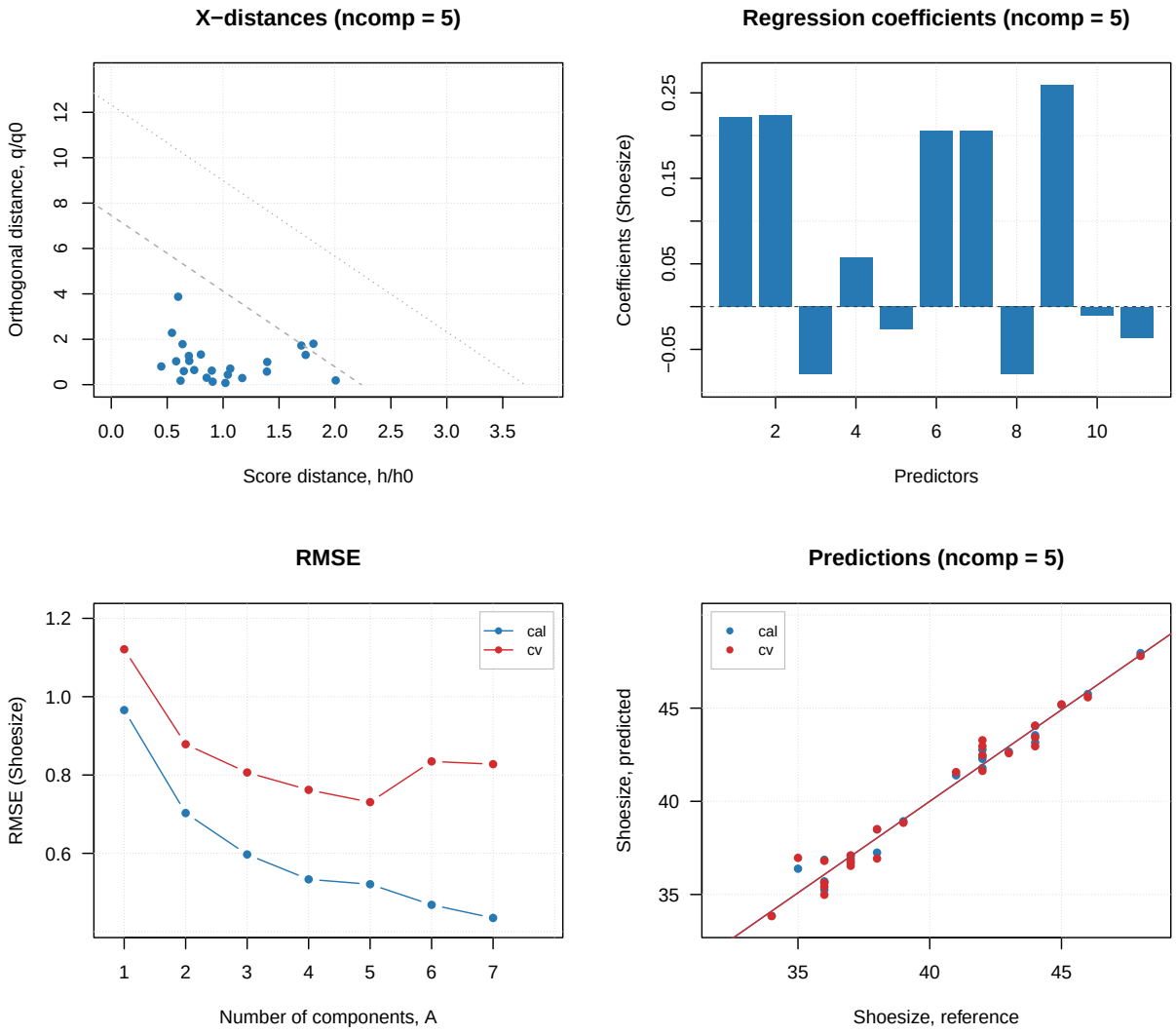


The plots for variables are available only for a model object and include:

Methods	Description
`plotXLoadings(obj, comp)`	loadings plot for decomposition of X
`plotXYLoadings(obj, comp)`	loadings plot for both X and Y decomposition
`plotWeights(obj, comp)`	plot with weights (W) for PLS decomposition
`plotRegcoeffs(obj, ny, ncomp)`	plot with regression coefficients
`plotVIPScores(obj, ny)`	VIP scores plot
`plotSelectivityRatio(obj, ny, ncomp)`	Selectivity ratio plot

And, of course, both model and result objects have method `plot()` for giving an overview.

```
plot(m)
```



Excluding rows and columns

PLS, like PCA, also can exclude rows and columns from calculations. The implementation works similar to what was described for PCA. For example, it can be useful if you have some candidates for outliers or do variable selection and do not want to remove rows and columns physically from the data matrix. In this case you can just specify two additional parameters, `exclcols` and `exclrows`, using either numbers or names of rows/columns to be excluded. You can also specify a vector with logical values (all `TRUE`s will be excluded).

The excluded rows are not used for creating a model and calculation of model's and results' performance (e.g. explained variance). However main results (for PLS — scores, predictions, distances) are calculated for these rows as well and are hidden, so you will not see them on plots. You can always e.g. show scores for excluded objects by using `show.excluded = TRUE`. It is implemented via attributes "known" for plotting methods from *mdatools* so if you use e.g. *ggplot2* you will see all points.

The excluded columns are not used for any calculations either, the corresponding results (e.g. loadings, weights or regression coefficients) will have zero values for such columns and be also hidden on plots.

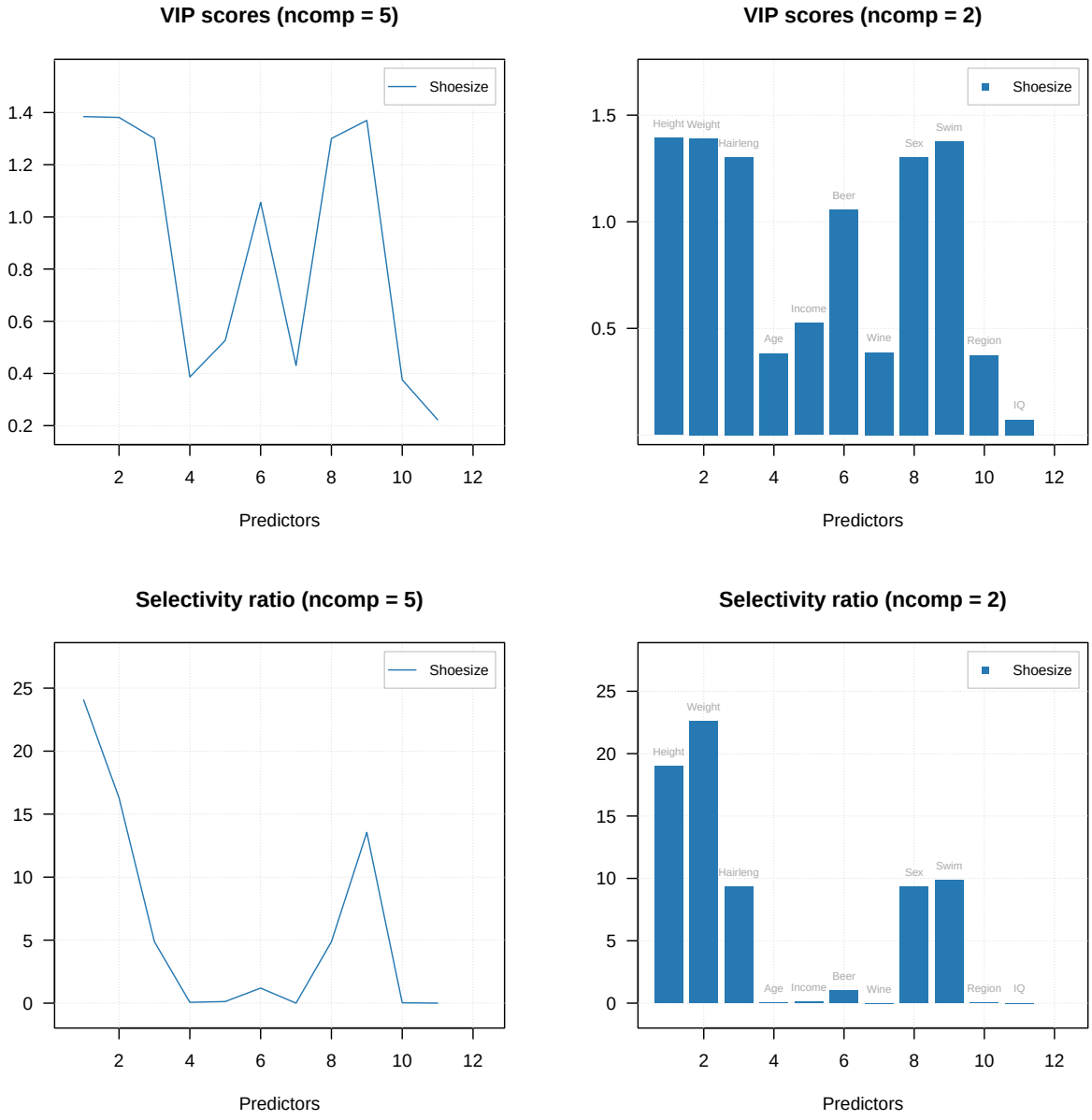
Variable selection

PLS allows one to calculate several statistics, which can be used to select most important (or remove least important) variables in order to improve performance and make model simpler. The first two are VIP-scores (variables important for projection) and Selectivity ratio. All details and theory can be found e.g. here.

Both parameters can be shown as plots and as vector of values for a selected y-variable. Take into account that when you make a plot for VIP scores or Selectivity ratio, the corresponding values should be computed first, which can take some time for large datasets.

Here is an example of corresponding plots.

```
par(mfrow = c(2, 2))
plotVIPScores(m)
plotVIPScores(m, ncomp = 2, type = "h", show.labels = TRUE)
plotSelectivityRatio(m)
plotSelectivityRatio(m, ncomp = 2, type = "h", show.labels = TRUE)
```



To compute the values without plotting use `vipscores()` and `selratio()` functions. In the example below, I create two other PLS models by excluding variables with VIP score or selectivity ratio below a threshold (I use 0.5 and 2 correspondingly) and show the performance for both.

```
vip = vipscores(m, ncomp = 2)
m2 = pls(Xc, yc, 4, scale = T, cv = 1, exclcols = (vip < 0.5))
summary(m2)
```

```
##
## PLS model (class pls) summary
## -------------------------------
## Info:
## Number of selected components: 2
## Cross-validation: full (leave one out)
## Excluded columns: 4
```

```
##
## Response variable: Shoesize
##     X cumexpvar Y cumexpvar    R2  RMSE Slope   Bias  RPD
## Cal    89.80722    95.27241 0.953 0.842 0.953 0.0000 4.70
## Cv           NA          NA 0.933 1.005 0.935 0.0254 3.94
```

```
sr = selratio(m, ncomp = 2)
m2 = pls(Xc, yc, 4, scale = T, cv = 1, exclcols = (sr < 2))
summary(m2)
```

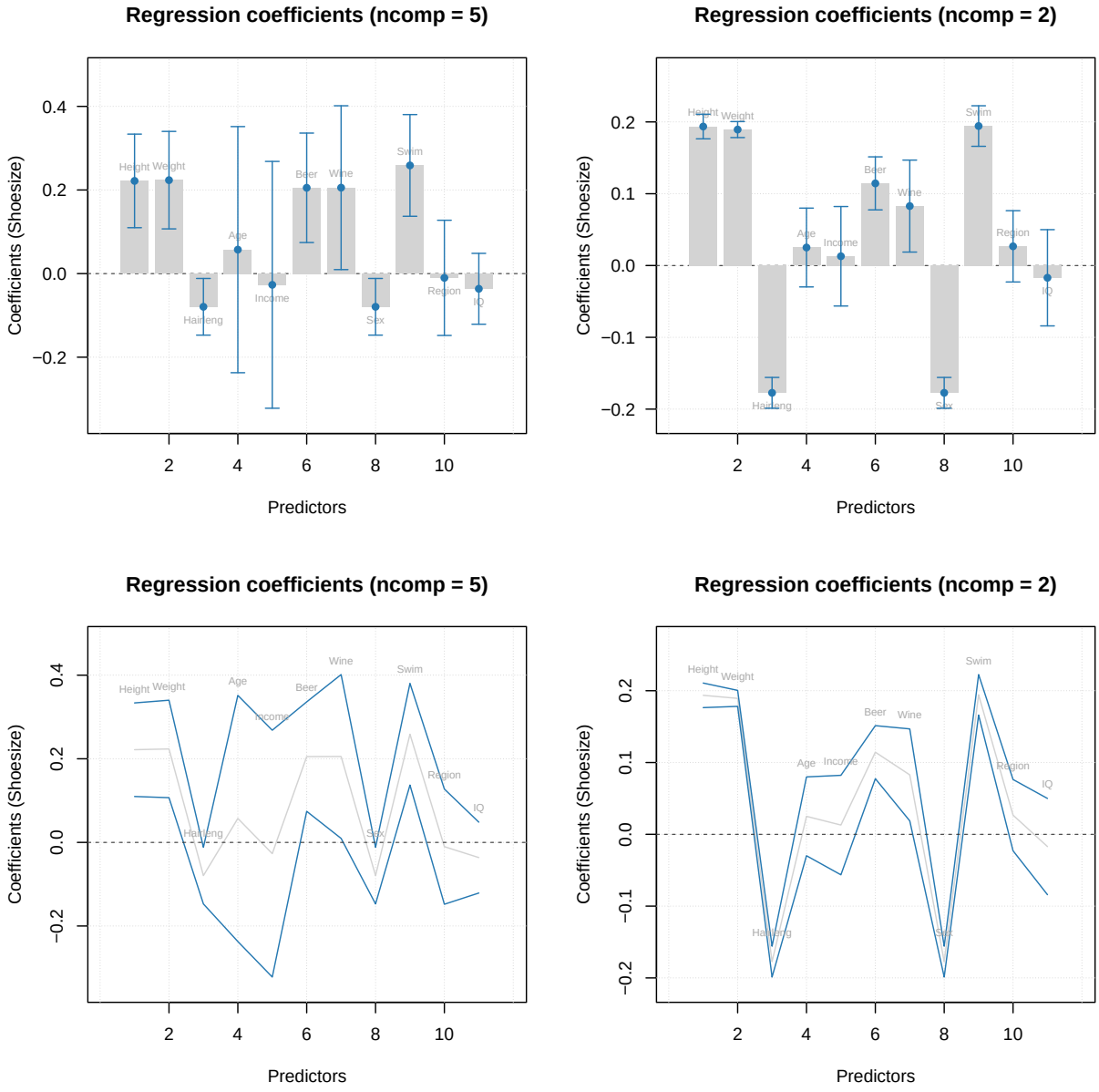
```
##
## PLS model (class pls) summary
## ------------------------------
## Info:
## Number of selected components: 2
## Cross-validation: full (leave one out)
## Excluded columns: 6
##
## Response variable: Shoesize
##     X cumexpvar Y cumexpvar    R2  RMSE Slope  Bias  RPD
## Cal    98.77528    95.79889 0.958 0.794 0.958 0.000 4.98
## Cv           NA          NA 0.946 0.903 0.944 0.014 4.38
```

Another way is to make an inference about regression coefficients and calculate confidence intervals and p-values for each variable. This can be done using Jack-Knife approach, when model is cross-validated using a sufficient number of segments (at least ten) and statistics are calculated using the distribution of regression coefficient values obtained for each step. The statistics are automatically computed when you use full cross-validation.

```
mjk = pls(Xc, yc, 7, scale = TRUE, cv = 1)
```

The statistics are calculated for each y-variable and each available number of components. When you show a plot for regression coefficients, you can show the confidence intervals by using parameter `show.ci = TRUE` as shown in examples below.

```
par(mfrow = c(2, 2))
plotRegcoeffs(mjk, type = "h", show.ci = TRUE, show.labels = TRUE)
plotRegcoeffs(mjk, ncomp = 2, type = "h", show.ci = TRUE, show.labels = TRUE)
plotRegcoeffs(mjk, type = "l", show.ci = TRUE, show.labels = TRUE)
plotRegcoeffs(mjk, ncomp = 2, type = "l", show.ci = TRUE, show.labels = TRUE)
```



Calling function `summary()` for regression coefficients allows to get all calculated statistics.

```
summary(mjk$coeffs, ncomp = 2)
```

```
##
## Regression coefficients for Shoesize (ncomp = 2)
## -------------------------------------------------
##               Coeffs   Std. err. t-value p-value        2.5%       97.5%
## Height     0.19357436 0.008270469   23.39   0.000  0.17646559  0.21068313
## Weight     0.18935489 0.005386414   35.12   0.000  0.17821224  0.20049754
## Hairleng -0.17730737 0.010368912  -17.09   0.000 -0.19875710 -0.15585764
## Age        0.02508113 0.026525148    0.95   0.351 -0.02979032  0.07995258
## Income     0.01291497 0.033491494    0.39   0.702 -0.05636746  0.08219740
## Beer       0.11441573 0.017851922    6.40   0.000  0.07748621  0.15134524
## Wine       0.08278735 0.030960265    2.68   0.013  0.01874117  0.14683354
```

```
## Sex      -0.17730737 0.010368912  -17.09   0.000 -0.19875710 -0.15585764
## Swim      0.19426871 0.013654151   14.21   0.000  0.16602295  0.22251448
## Region    0.02673161 0.023989628    1.12   0.275 -0.02289471  0.07635794
## IQ       -0.01705852 0.032399035   -0.53   0.602 -0.08408103  0.04996399
##
## Degrees of freedom (Jack-Knifing): 23
```

```
summary(mjk$coeffs, ncomp = 2, alpha = 0.01)
```

```
##
## Regression coefficients for Shoesize (ncomp = 2)
## -------------------------------------------------
##               Coeffs   Std. err. t-value p-value         0.5%       99.5%
## Height    0.19357436 0.008270469   23.39   0.000  0.170356376  0.21679234
## Weight    0.18935489 0.005386414   35.12   0.000  0.174233417  0.20447636
## Hairleng -0.17730737 0.010368912  -17.09   0.000 -0.206416384 -0.14819835
## Age       0.02508113 0.026525148    0.95   0.351 -0.049383866  0.09954612
## Income    0.01291497 0.033491494    0.39   0.702 -0.081106896  0.10693683
## Beer      0.11441573 0.017851922    6.40   0.000  0.064299387  0.16453206
## Wine      0.08278735 0.030960265    2.68   0.013 -0.004128502  0.16970321
## Sex      -0.17730737 0.010368912  -17.09   0.000 -0.206416384 -0.14819835
## Swim      0.19426871 0.013654151   14.21   0.000  0.155936928  0.23260050
## Region    0.02673161 0.023989628    1.12   0.275 -0.040615325  0.09407855
## IQ       -0.01705852 0.032399035   -0.53   0.602 -0.108013488  0.07389645
##
## Degrees of freedom (Jack-Knifing): 23
```

Function `getRegcoeffs()` in this case may also return corresponding t-value, standard error, p-value, and confidence interval for each of the coefficient (except intercept) if the user specifies a parameter `full`. The standard error and confidence intervals are also computed for raw, non-standardized, variables (similar to coefficients).

```
show(getRegcoeffs(mjk, ncomp = 2, full = TRUE))
```

```
##               Estimated    Std. err.     t-value      p-value          2.5%
## Intercept  1.342626e+01           NA          NA           NA            NA
## Height     7.456695e-02 3.185875e-03  23.3854388 0.000000e+00  0.0679764667
## Weight     4.896328e-02 1.392816e-03  35.1242970 0.000000e+00  0.0460820210
## Hairleng  -6.865495e-01 4.014933e-02 -17.0926429 1.443290e-14 -0.7696047104
## Age        1.081716e-02 1.143995e-02   0.9524214 3.507861e-01 -0.0128481769
## Income     5.642220e-06 1.463158e-05   0.3875343 7.019238e-01 -0.0000246255
## Beer       5.010542e-03 7.817789e-04   6.3971862 1.579903e-06  0.0033933089
## Wine       7.511377e-03 2.809055e-03   2.6802915 1.336266e-02  0.0017004042
## Sex       -6.865495e-01 4.014933e-02 -17.0926429 1.443290e-14 -0.7696047104
## Swim       1.010496e-01 7.102257e-03  14.2147650 7.018830e-13  0.0863574493
## Region     1.035071e-01 9.288992e-02   1.1174095 2.753561e-01 -0.0886503166
## IQ        -5.277164e-03 1.002285e-02  -0.5285419 6.021868e-01 -0.0260110098
##                    97.5%
## Intercept             NA
## Height     8.115743e-02
## Weight     5.184454e-02
## Hairleng  -6.034943e-01
## Age        3.448250e-02
## Income     3.590994e-05
## Beer       6.627775e-03
## Wine       1.332235e-02
```

```
## Sex       -6.034943e-01
## Swim       1.157417e-01
## Region     2.956646e-01
## IQ         1.545668e-02
## attr(,"name")
## [1] "Regression coefficients for  Shoesize"
```

It is also possible to change significance level for confidence intervals.

```
show(getRegcoeffs(mjk, ncomp = 2, full = TRUE, alpha = 0.01))
```

```
##               Estimated    Std. err.     t-value      p-value          0.5%
## Intercept  1.342626e+01           NA          NA           NA            NA
## Height     7.456695e-02 3.185875e-03  23.3854388 0.000000e+00  6.562313e-02
## Weight     4.896328e-02 1.392816e-03  35.1242970 0.000000e+00  4.505318e-02
## Hairleng  -6.865495e-01 4.014933e-02 -17.0926429 1.443290e-14 -7.992621e-01
## Age        1.081716e-02 1.143995e-02   0.9524214 3.507861e-01 -2.129862e-02
## Income     5.642220e-06 1.463158e-05   0.3875343 7.019238e-01 -3.543353e-05
## Beer       5.010542e-03 7.817789e-04   6.3971862 1.579903e-06  2.815826e-03
## Wine       7.511377e-03 2.809055e-03   2.6802915 1.336266e-02 -3.745830e-04
## Sex       -6.865495e-01 4.014933e-02 -17.0926429 1.443290e-14 -7.992621e-01
## Swim       1.010496e-01 7.102257e-03  14.2147650 7.018830e-13  8.111117e-02
## Region     1.035071e-01 9.288992e-02   1.1174095 2.753561e-01 -1.572661e-01
## IQ        -5.277164e-03 1.002285e-02  -0.5285419 6.021868e-01 -3.341467e-02
##                    99.5%
## Intercept             NA
## Height     8.351077e-02
## Weight     5.287338e-02
## Hairleng  -5.738369e-01
## Age        4.293295e-02
## Income     4.671797e-05
## Beer       7.205258e-03
## Wine       1.539734e-02
## Sex       -5.738369e-01
## Swim       1.209880e-01
## Region     3.642803e-01
## IQ         2.286034e-02
## attr(,"name")
## [1] "Regression coefficients for  Shoesize"
```

The p-values, t-values and standard errors are stored each as a 3-way array similar to regression coefficients. The selection can be made by comparing e.g. p-values with a threshold similar to what we have done with VIP-scores and selectivity ratio.

```
exclcols = mjk$coeffs$p.values[, 2, 1] > 0.05
show(exclcols)
```

```
##   Height   Weight Hairleng      Age   Income     Beer     Wine      Sex
##    FALSE    FALSE    FALSE     TRUE     TRUE    FALSE    FALSE    FALSE
##     Swim   Region       IQ
##    FALSE     TRUE     TRUE
```

Here `p.values[, 2, 1]` means values for all predictors, model with two components, first y-variable.

```
newm = pls(Xc, yc, 3, scale = TRUE, cv = 1, exclcols = exclcols)
summary(newm)
```

```
##
```

```
## PLS model (class pls) summary
## ------------------------------
## Info:
## Number of selected components: 3
## Cross-validation: full (leave one out)
## Excluded columns: 4
##
## Response variable: Shoesize
##     X cumexpvar Y cumexpvar    R2  RMSE Slope    Bias  RPD
## Cal    97.85476    97.79127 0.978 0.575 0.978  0.0000 6.87
## Cv           NA          NA 0.965 0.727 0.972 -0.0161 5.44
```

```
show(getRegcoeffs(newm))
```

```
##              Estimated
## Intercept  4.952691046
## Height     0.091079963
## Weight     0.052708955
## Hairleng  -0.315927643
## Age        0.000000000
## Income     0.000000000
## Beer       0.009320427
## Wine       0.018524717
## Sex       -0.315927643
## Swim       0.134354170
## Region     0.000000000
## IQ         0.000000000
## attr(,"exclrows")
##    Age Income Region     IQ
##      4      5     10     11
## attr(,"name")
## [1] "Regression coefficients for  Shoesize"
```

As you can see, the variables *Age*, *Income*, *Region* and *IQ* have been excluded as they are not related to the *Shoesize*, which seems to be correct.

Variable selection as well as all described above can be also carried out for PLS discriminant analysis (PLS-DA), which will be explained later in one of the next chapters.

Distances and outlier detection

Distance plot for X-decomposition

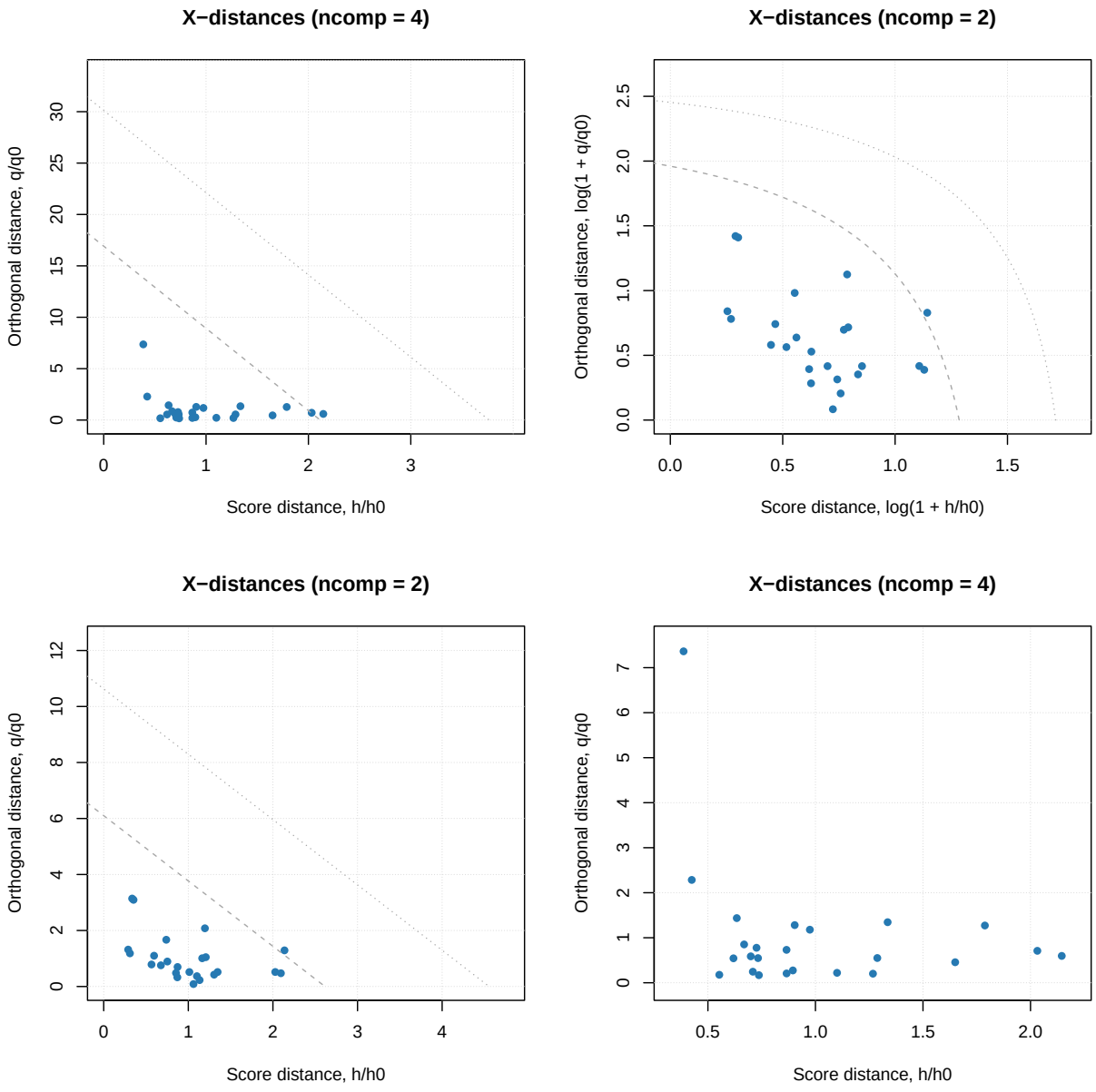
For decomposition of X-values the orthogonal and score distances are computed and treated in the same way as in PCA. The `pls()` constructor takes the three arguments for computing the critical limits, similar to PCA (`lim.type`, `alpha`, `gamma`). The default value for `lim.type` is the same as for PCA (`"ddmoments"`). Distance plot can be made using `plotXResiduals()`, it works identical to `plotResiduals()` for PCA.

Below is example on how to use the plot.

```
m = pls(Xc, yc, 4, scale = TRUE, cv = 1)

par(mfrow = c(2, 2))
plotXResiduals(m)
plotXResiduals(m, ncomp = 2, log = TRUE)
plotXResiduals(m, ncomp = 2, res = list("cal" = m$res$cal))
```

```
plotXResiduals(m$res$cal)
```

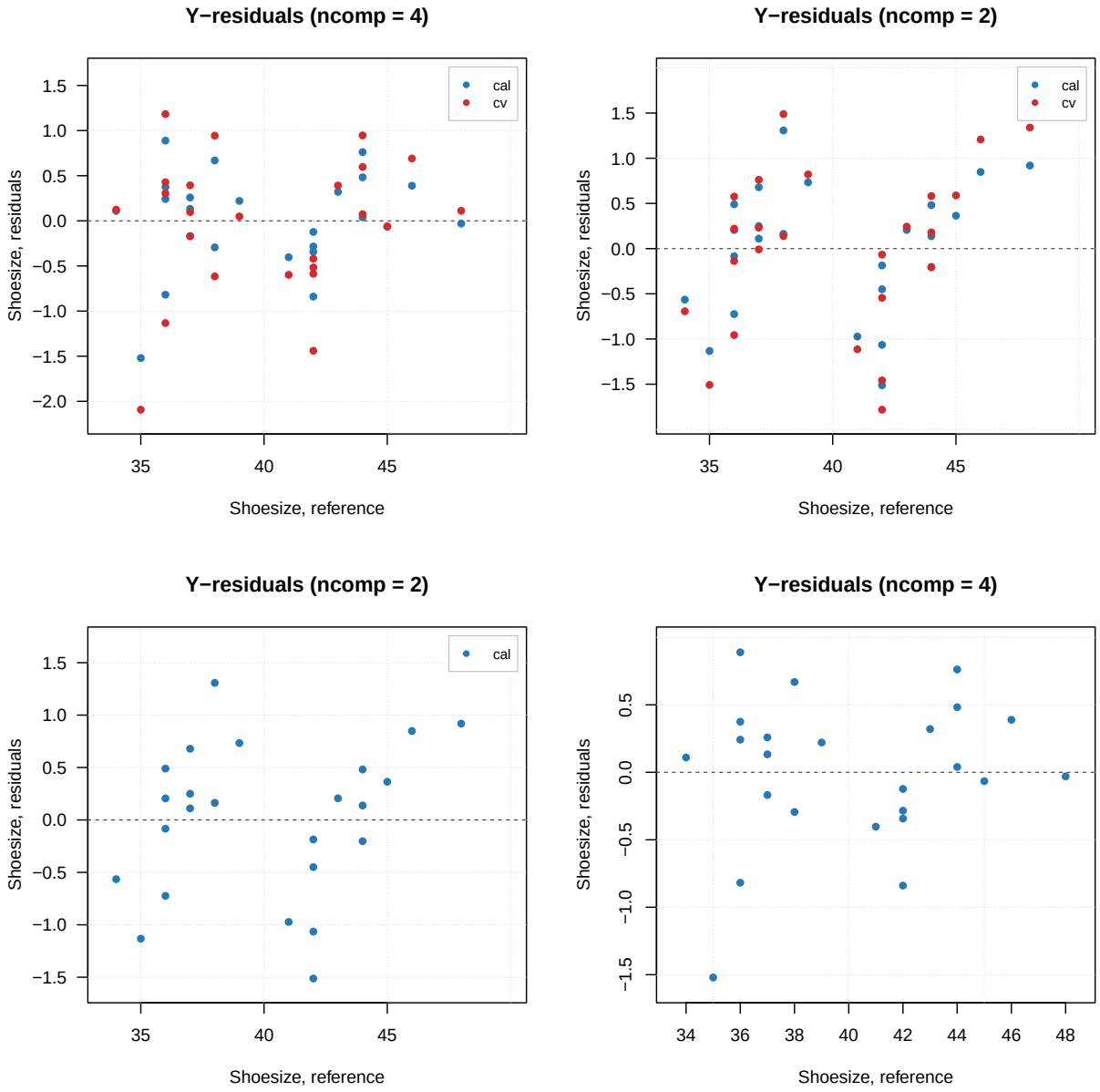


PLS also has function `categorize()` which allows to identify extreme objects and outliers. However, it works in a different way and takes into account not only total distances for X-decomposition but also similar measure for decomposition of Y. This is explained in following sections.

Distance plot for Y-decomposition and total distance

The distance for Y-decomposition is calculated in a different way — as a difference between predicted and reference y-values. The difference for selected y-variable is expected to be random and normally distributed. It can be checked visually by using function `plotYResiduals()` as shown in example below. In this plot the difference is plotted against the reference value.

```
par(mfrow = c(2, 2))
plotYResiduals(m)
plotYResiduals(m, ncomp = 2)
plotYResiduals(m, ncomp = 2, res = list("cal" = m$res$cal))
plotYResiduals(m$res$cal)
```



Since predictions are also computed for cross-validation, the plot shows the distance values for cross-validated results.

The distance between reference and predicted values can be used to compute orthogonal squared distance, similar to how it is done for X-decomposition, as it is shown below:

$$z_i = \sum_{k=1}^{K} (y_{ik} - \hat{y}_{ik})^2$$

Here y_{ik} is a reference value for sample i and y-variable k and $\hat{y}_{ik}$ is the corresponding predicted value, K is the number of y-variables. Apparently, z values also follow chi-square distribution similar to how [it is done in PCA][Critical limits]:

$$N_z \frac{z}{z_0} \propto \chi^2(N_z)$$

In case of PLS1 (when there is only one y-variable), $N_z = 1$. Otherwise, both z_0 and N_z are computed using [data driven approach][Data driven approach], similar to q_0, N_q, h_0, N_h, using either classical (based on moments) or robust approach.

Full distance for X-decomposition, f and the Y-distance, z can be combined into XY total distance, g:

$$g = N_f \frac{f}{f_0} + N_z \frac{z}{z_0} \propto \chi^2(N_g)$$

Here $N_g = N_f + N_z$. This approach was proposed by Rodionova and Pomerantsev and described in this paper. The distances and the critical limits can be visualized using function `plotXYResiduals()` as illustrated in the next example.

The example is based on People data (prediction of shoesize), however we will first introduce two outliers. We will change response value for row #9 to 25, which is apparently too small value for shoesize of an adult person. The predictor values will be the same. Then we will change height of first person (row #1) to 125 cm, which is again quite a small value. And keep the response value unchanged. After that, we create a PLS model with classic data driven approach for computing critical limits and show the XY-distance plot. Finally, that we change the method for limits to robust and show the plot again:

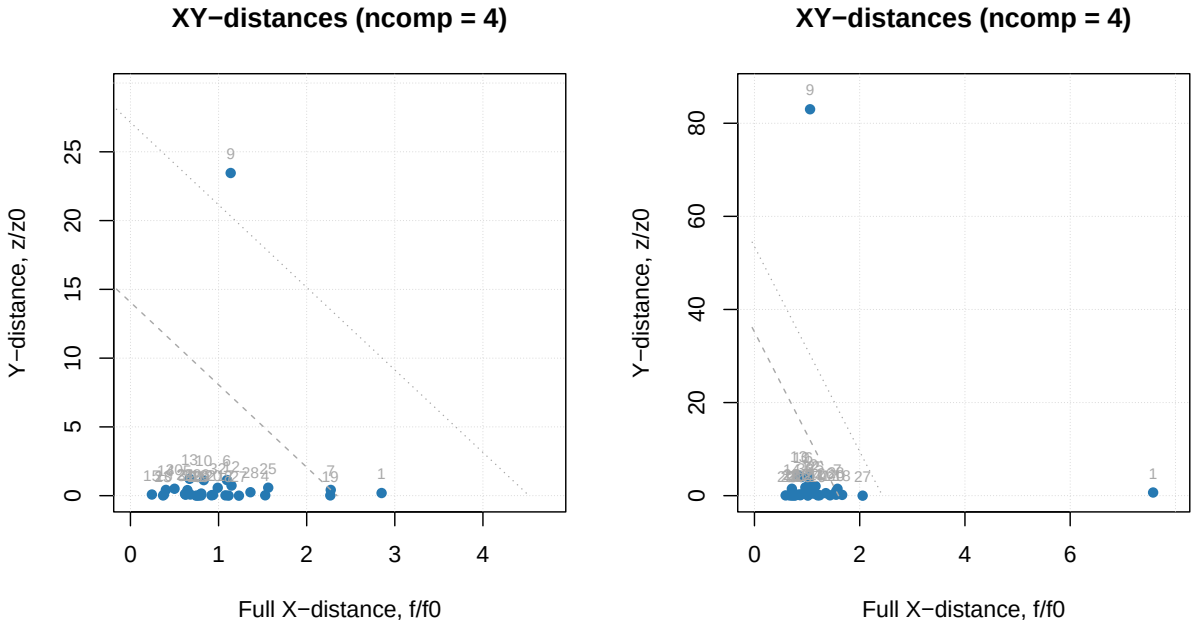
```
# prepare data
data(people)
X = people[, -4]
y = people[, 4, drop = FALSE]

# add outliers
y[9] = 25
X[1, 1] = 125

# create model and show plots
m = pls(X, y, 4, scale = TRUE, cv = 1, lim.type = "ddmoments")

par(mfrow = c(1, 2))
plotXYResiduals(m, show.labels = TRUE, labels = "indices")

m = setDistanceLimits(m, lim.type = "ddrobust")
plotXYResiduals(m, show.labels = TRUE, labels = "indices")
```



As one can see, both classic and robust approach detect sample number 9 (with wrong y-value) as a clear outlier. Using robust approach helps to identify the second outlier as well. However, you can see that the location of the two samples is different — sample #9 has large Y-distance (z/z_0), while sample #1 has large full X-distance (f/f_0). Which is in agreement with the way we created the outliers.

The limits shown on the plot are made for the total distance, g. For example, to detect outliers we need to compare the total distance, g, with critical limit computed for predefined significance level, γ (shown as dotted line on the plots):

$$g > \chi^{-2}\left((1-\gamma)^{1/I}, N_g\right)$$

The proper procedure for detection and removing outliers can be found in the paper and will be very briefly touched upon here. The detection can be done using function `categorize()` which works similar to the same function for PCA decomposition — returns vector with categories for each measurement, as shown in an example below:

```
c = categorize(m, m$res$cal)
print(c)
```

```
##  [1] outlier regular regular regular regular regular extreme regular outlier
## [10] regular regular regular regular regular regular regular regular extreme
## [19] regular regular regular regular regular regular regular regular extreme
## [28] regular regular regular regular regular
## Levels: regular extreme outlier
```

A simplified description of the recommended procedure proposed in the mentioned paper is as follows:

1. Make a PLS model with robust estimator of critical limits
2. Detect outliers if any, remove them and keep the removed objects separately
3. Repeat steps 1-2 until no outliers are detected
4. Apply model to the collected outliers (use `predict`)
5. If any of them appear as regular objects, return them back to the dataset and go to step 1
6. Repeat all steps until step 5 does not reveal any regular objects

7. Make a final PLS model using classic estimator for the critical limits

If test set validation is used, the outlier detection should be done for both sets. And it is of course important to use optimal number of components, which can be identified using RMSE plot and plot for explained Y-variance.

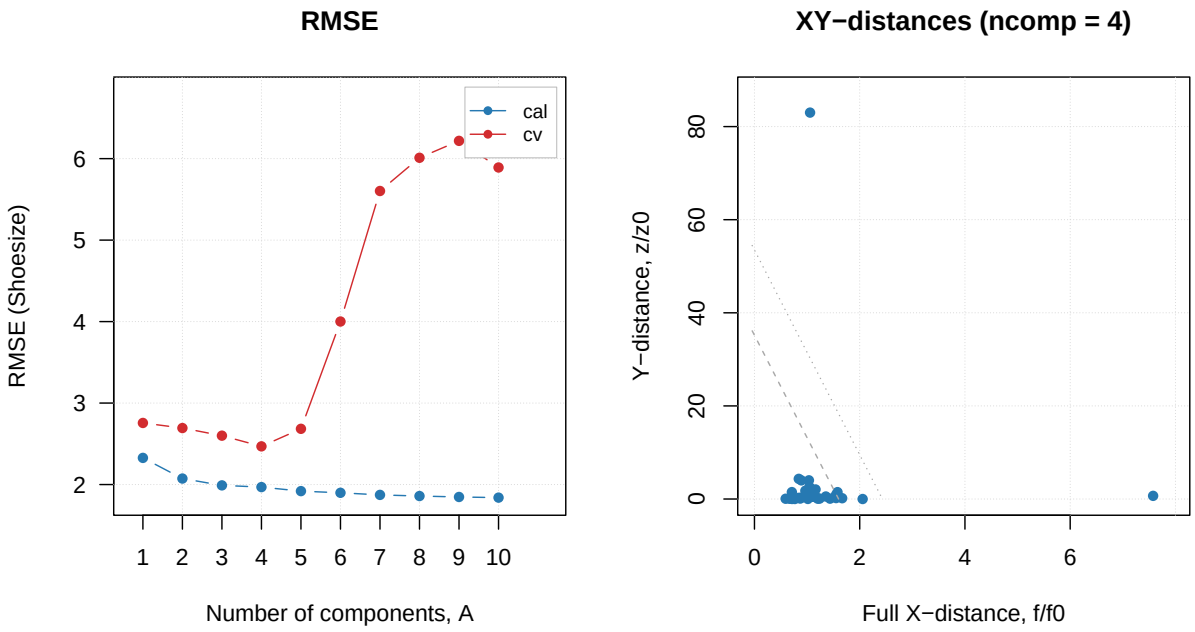
The code chunks below show the steps for detection of outliers in the People data used in an example earlier. We start with creating PLS model for the whole data.

```
# prepare data
data(people)
X = people[, -4]
y = people[, 4, drop = FALSE]

# add outliers
y[9] = 25
X[1, 1] = 125

# compute initial PLS model with all data
m = pls(X, y, 10, scale = TRUE, cv = 1, lim.type = "ddrobust")

# look at RMSE and XY-residuals plot
par(mfrow = c(1, 2))
plotRMSE(m)
plotXYResiduals(m)
```



Apparently four components selected automatically is indeed a proper value. The XY-distance plot shows that we have two outliers, let's find and remove them:

```
# get row indices for outliers in calibration set
outliers = which(categorize(m, m$res$cal) == "outlier")

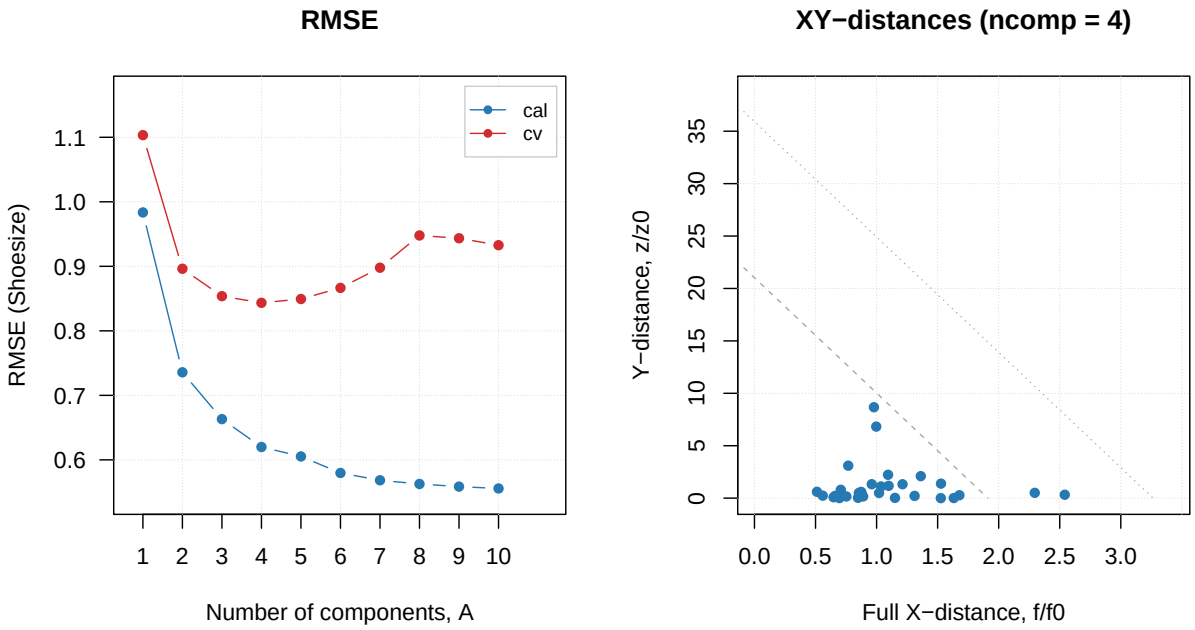
# keep data for outliers in separate matrices
Xo = X[outliers, , drop = FALSE]
```

```
yo = y[outliers, , drop = FALSE]

# remove the rows with outliers from the data
X = X[-outliers, , drop = FALSE]
y = y[-outliers, , drop = FALSE]

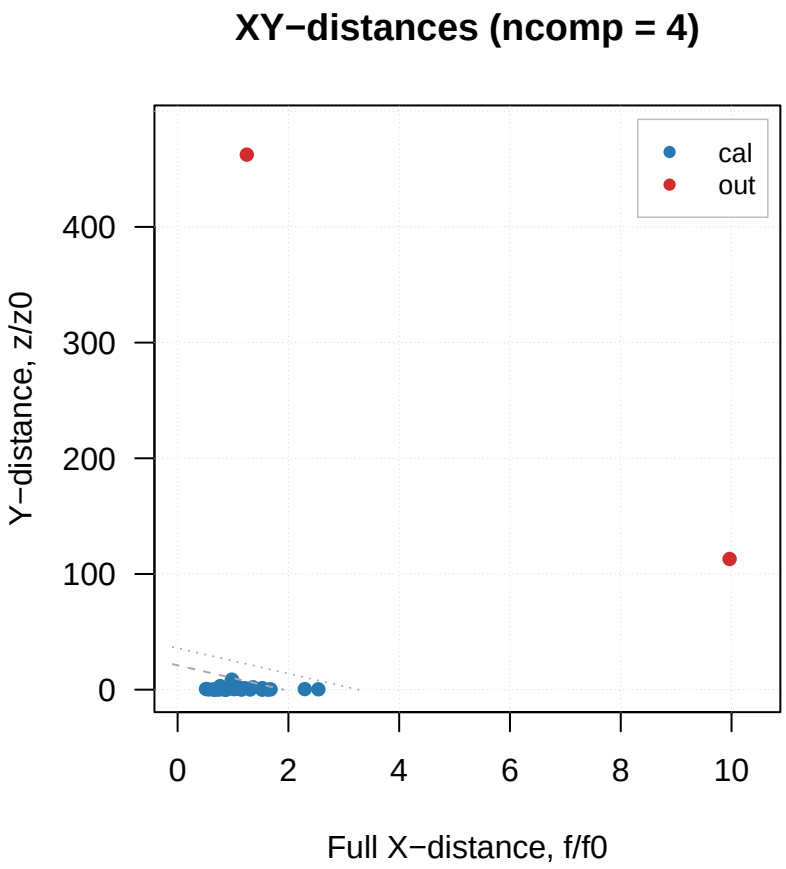
# make a new model for outlier-free data
m = pls(X, y, 10, scale = TRUE, cv = 1, lim.type = "ddrobust")

# look at RMSE and XY-distance plot
par(mfrow = c(1, 2))
plotRMSE(m)
plotXYResiduals(m)
```



Again, four or three components seem to be optimal and this time no extra outliers are detected. Let's now apply the model to the two outliers found previously and see their status.

```
res = predict(m, Xo, yo)
plotXYResiduals(m, res = list("cal" = m$res$cal, "out" = res))
```

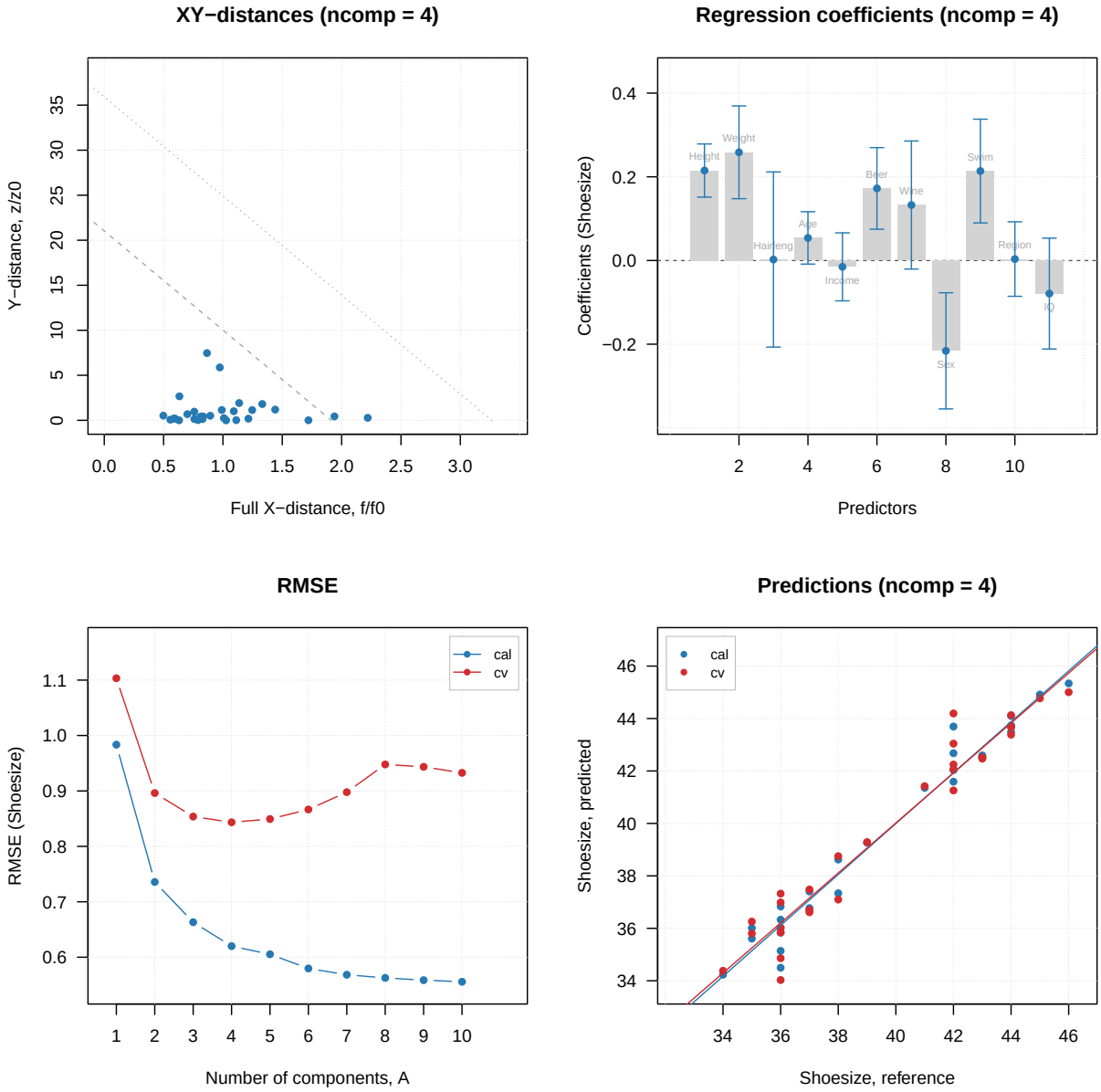


The two objects seem now even more extreme for the model built using the outlier-free data, no need to have them back. So we just need to make a final model and look at it:

```
# make a new model for outlier free data and explore it
m = pls(X, y, 10, scale = TRUE, cv = 1, lim.type = "ddmoments")
summary(m)
```

```
##
## PLS model (class pls) summary
## ------------------------------
## Info:
## Number of selected components: 4
## Cross-validation: full (leave one out)
##
## Response variable: Shoesize
##     X cumexpvar Y cumexpvar    R2  RMSE Slope    Bias  RPD
## Cal     85.9168    97.03856 0.970 0.620 0.970  0.0000 5.91
## Cv           NA          NA 0.945 0.843 0.953 -0.0261 4.35
```

```
par(mfrow = c(2, 2))
plotXYResiduals(m)
plotRegcoeffs(m, type = "h", show.labels = TRUE, show.ci = TRUE)
plotRMSE(m)
plotPredictions(m)
```



Randomization test

Another additional option for PLS regression implemented in *mdatools* is randomization test for estimation of optimal number of components. The description of the method can be found in this paper. The basic idea is that for each component from 1 to `ncomp` we compute a statistic T, which is a covariance between X-scores and the reference Y values. After that, this procedure is repeated for randomly permuted Y-values and distribution of the statistic is obtained. A parameter `alpha` is computed to show how often the statistic T, calculated for permuted Y-values, is the same or higher than the same statistic, calculated for original response values without permutations.

If a component is important, then the covariance for non-permuted data should be larger than the covariance for permuted data and therefore the value for `alpha` will be quite small (there is still a small chance to get similar covariance). This makes `alpha` very similar to p-value in a statistical test.

The function `randtest()` calculates alpha for each component, the values can be observed using `summary()` or `plot()` functions. There are also several functions, allowing e.g. to show distribution of statistics and the critical value for each component.

In the code example below most of the functions are shown.

```
data(people)

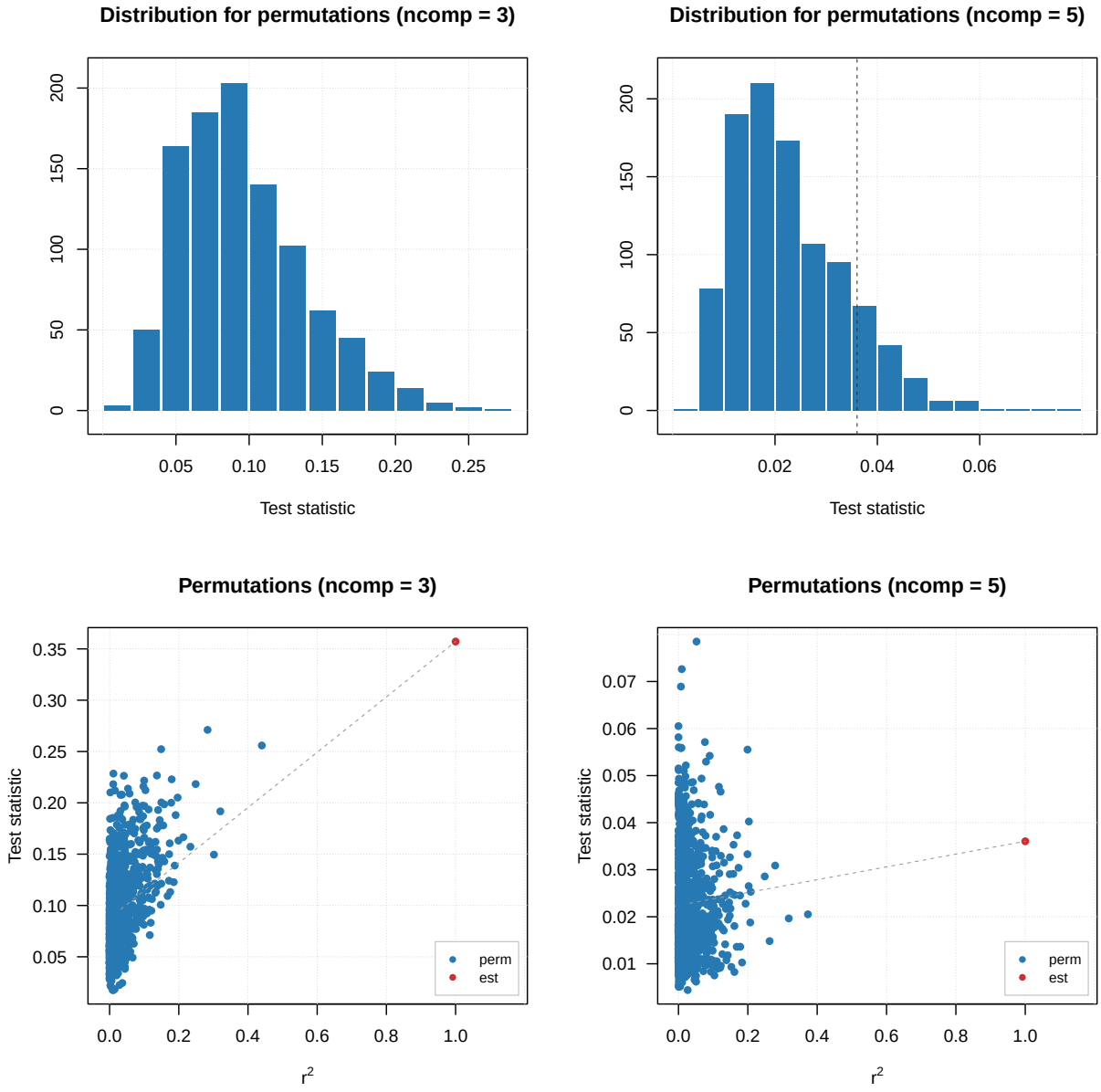
y = people[, 4, drop = FALSE]
X = people[, -4]

r = randtest(X, y, ncomp = 5, nperm = 1000, silent = TRUE)
summary(r)
```

```
##
## Summary for permutation test results
## Number of permutations: 1000
## Suggested number of components: 4
##
## Statistics and alpha values:
##              Comp 1    Comp 2    Comp 3    Comp 4     Comp 5
## Alpha     0.0550000 0.0000000 0.0000000 0.0000000 0.13000000
## Statistic 0.2403837 0.4349094 0.3571243 0.2678767 0.03603819
```

As you can see, `alpha` is very small for components 2–4 and then jumps up.

```
par( mfrow = c(2, 2))
plotHist(r, ncomp = 3)
plotHist(r, ncomp = 5)
plotCorr(r, ncomp = 3)
plotCorr(r, ncomp = 5)
```



Preprocessing as part of the model

From v. *0.15.0* preprocessing methods can be combined into a list and provided as additional argument to the models, including PLS. Once provided, PLS will take care of estimation of preprocessing parameters (e.g. reference spectrum for EMSC) and then will automatically apply all preprocessing methods to new data when the user calls function `predict()`.

Here is an example. We will create a PLS model for *Simdata* UV/Vis spectra but we would like to smooth the spectra first and apply SNV normalization. Here is how to do this:

```
# load calibration and test set
Xc = simdata$spectra.c
yc = simdata$conc.c[, 2, drop = FALSE]
Xt = simdata$spectra.t
```

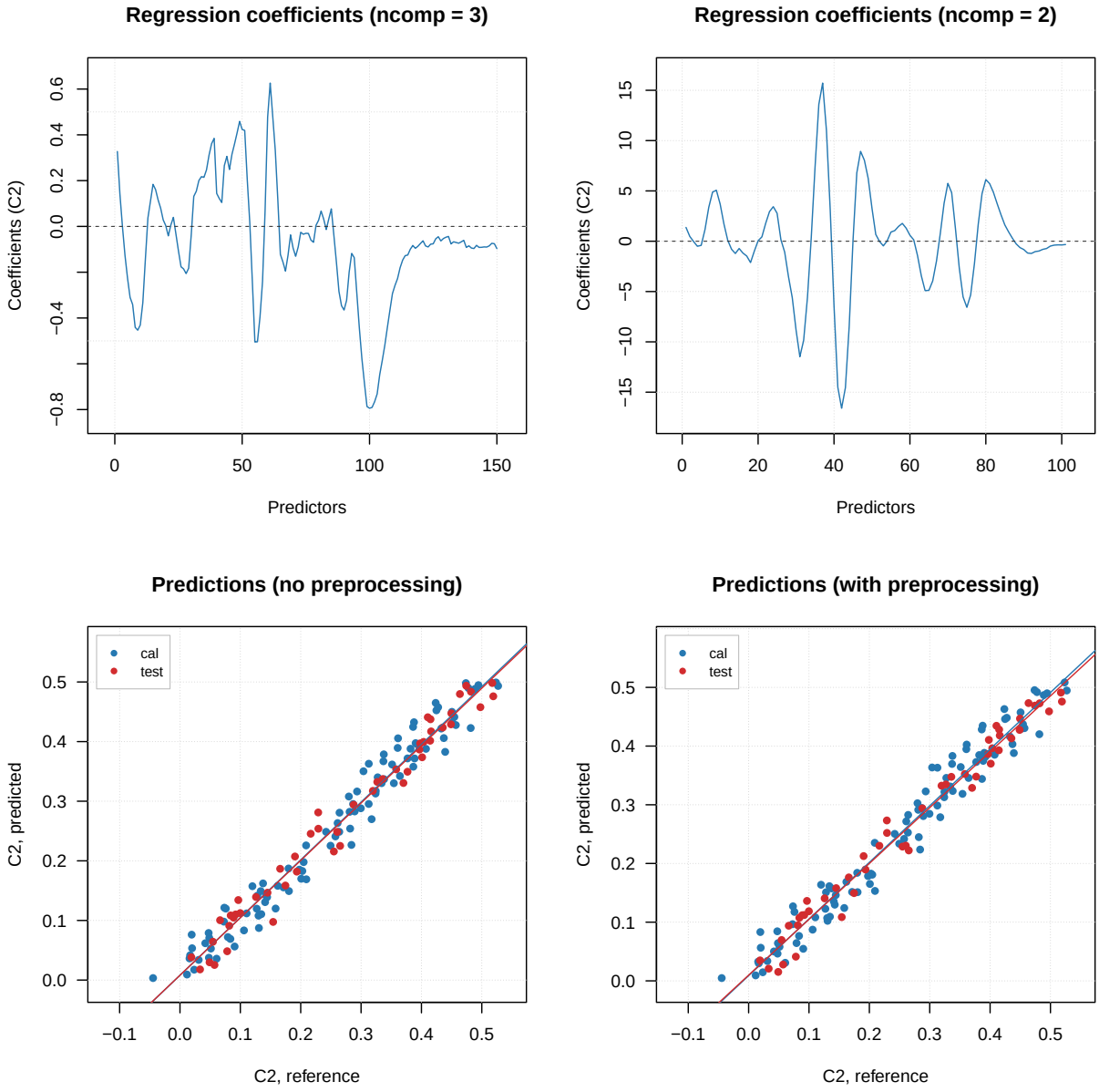
```
yt = simdata$conc.t[, 2, drop = FALSE]

# define chain of preprocessing methods
p = list(
   prep("savgol", width = 11, porder = 2, dorder = 2),
   prep("varsel", var.ind = 20:120)
)

# create two PLS models - with and without preprocessing
m1 = pls(Xc, yc, 6, cv = list("ven", 10))
m2 = pls(Xc, yc, 6, cv = list("ven", 10), prep = p)

# apply the models to the test set
r1 = predict(m1, Xt, yt)
r2 = predict(m2, Xt, yt)

par(mfrow = c(2, 2))
plotRegcoeffs(m1)
plotRegcoeffs(m2)
plotPredictions(m1, res = list("cal" = m1$res$cal, "test" = r1), main = "Predictions (no preprocessing)
plotPredictions(m2, res = list("cal" = m2$res$cal, "test" = r2), main = "Predictions (with preprocessin
```



As you can see there is no need to apply the preprocessing methods to the test set manually, method `predict()` takes care of everything.

Information about the preprocessing methods is also shown in the model summary:

```
summary(m2)
```

```
##
## PLS model (class pls) summary
## ------------------------------
## Info:
## Number of selected components: 2
## Cross-validation: venetian blinds with 10 segments
##
## Preprocessing methods:
```

```
##  - savgol: width = 11, porder = 2, dorder = 2
##  - varsel: var.ind = 20:120
##
## Response variable: C2
##     X cumexpvar Y cumexpvar    R2  RMSE Slope  Bias  RPD
## Cal    99.04458    96.48095 0.965 0.028 0.965 0e+00 5.36
## Cv           NA          NA 0.962 0.029 0.963 2e-04 5.18
```

Interoperability with web-applications

Similar to preprocessing there is an interactive web-application, https://mda.tools/pls, which lets you create PLS models and apply them to a new dataset in any modern browser. Any PLS model created using `pls()` can be saved into a JSON file and then be loaded into the *Project* tab of the web-application. Including preprocessing methods!

Here is an example based on *Simdata*:

```
# define calibration set
Xc = simdata$spectra.c
yc = simdata$conc.c[, 2, drop = FALSE]

# create a sequence of preprocessing methods
p <- list(
   prep("savgol", width = 11, porder = 2, dorder = 2),
   prep("varsel", var.ind = 20:120)
)

# create a PCA model with preprocessing as a part of the model
m <- pls(Xc, yc, 6, prep = p)
```

```
## Warning in selectCompNum.pls(model, selcrit = ncomp.selcrit): No validation
## results were found.
```

```
# save the model to JSON file
writeJSON(m, "pls-model.json")
```

If you run the code on your computer you will see a new `pls-model.json` file created in the working directory. Now open https://mda.tools/pls in your browser, select tab *Project* and load the file with the model. You will see something similar to the following:

After that you can upload dataset and see the result of projecting it to the PLS model directly in the app.

Likewise, any PLS model created in the application can be saved into a JSON file and loaded into R, so you can use it via `predict()`. For example this JSON file I created using the *Train* tab of the app and *Simdata*. It contains a PLS model on data with a preprocessing chain that includes SG smoothing with first derivative, trimming tails, and mean centering. Download it to your working directory and run the following code (here I assume you already have `Xc` and `yc` matrices from the previous blocks of code):

```
# load preprocessing model from JSON file
m <- readJSON("pls-model-fromweb.json")

# apply it to Xc
r <- predict(m, Xc, yc)
summary(r)
```

```
##
## PLS regression results (class plsres) summary
```

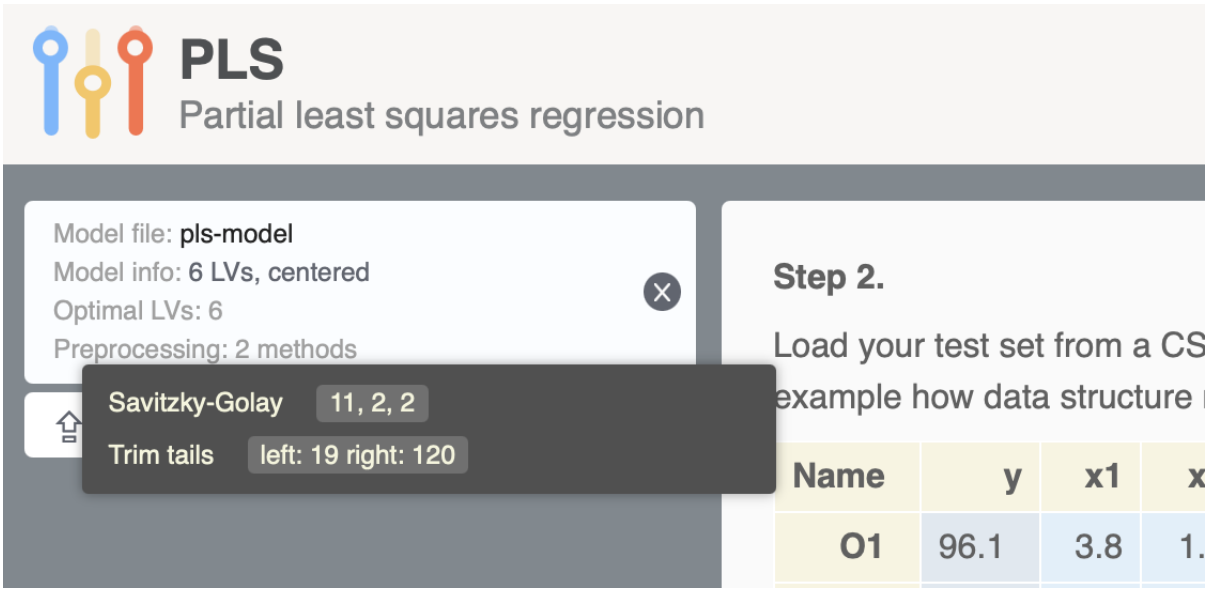
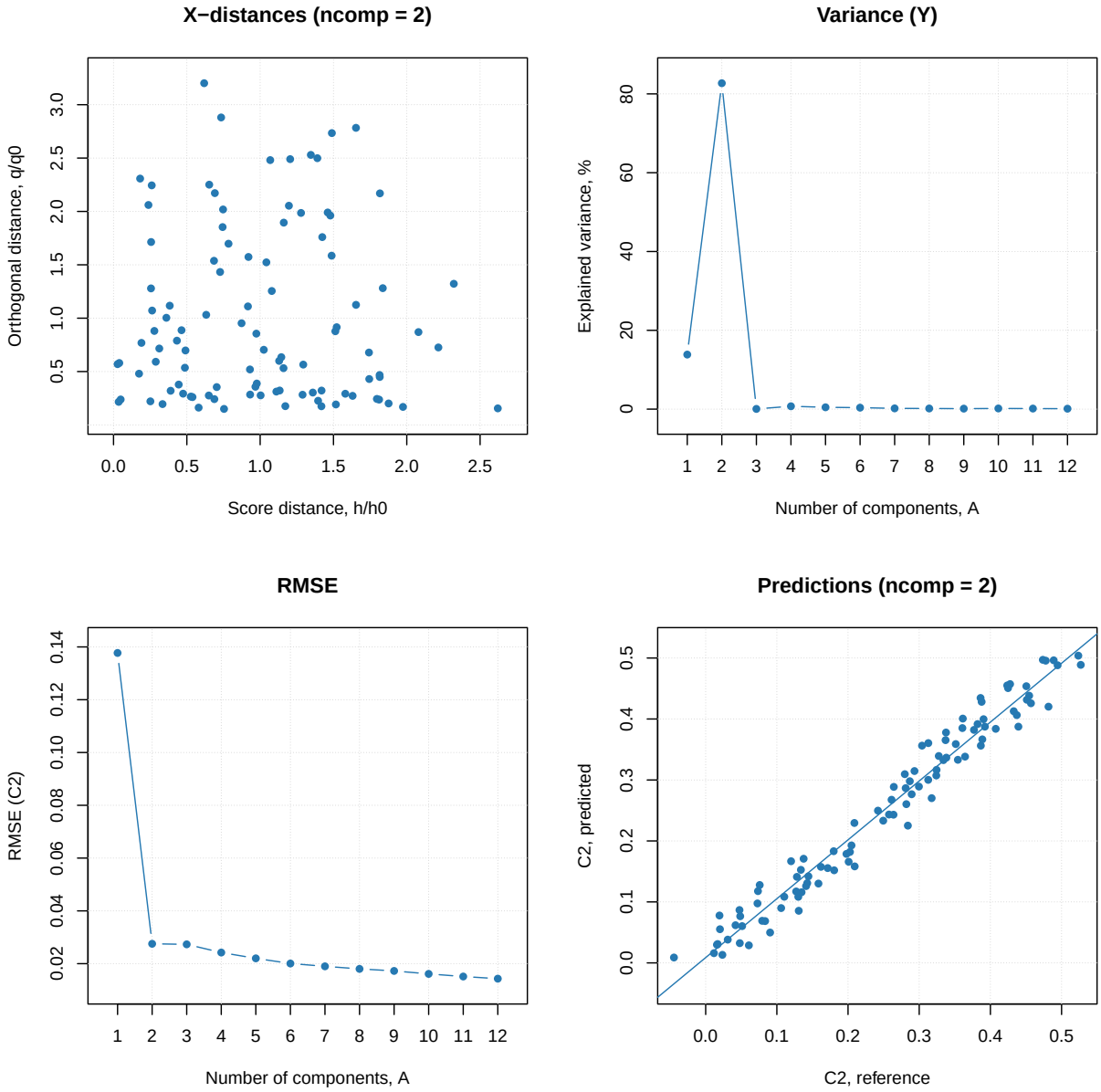


Figure 4: Screenshot with PLS model loaded to mda.tools/pls

```
## Info:
## Number of selected components: 2
##
## Response variable C2:
##         X expvar X cumexpvar Y expvar Y cumexpvar    R2  RMSE Slope Bias   RPD
## Comp 1    97.380      97.380   13.844      13.844 0.138 0.138 0.138    0  1.08
## Comp 2     2.217      99.597   82.714      96.557 0.966 0.028 0.966    0  5.42
## Comp 3     0.326      99.923    0.052      96.609 0.966 0.027 0.966    0  5.46
## Comp 4     0.005      99.928    0.727      97.336 0.973 0.024 0.973    0  6.16
## Comp 5     0.003      99.931    0.464      97.799 0.978 0.022 0.978    0  6.78
## Comp 6     0.003      99.934    0.372      98.172 0.982 0.020 0.982    0  7.43
## Comp 7     0.003      99.937    0.190      98.362 0.984 0.019 0.984    0  7.85
## Comp 8     0.003      99.940    0.163      98.525 0.985 0.018 0.985    0  8.27
## Comp 9     0.003      99.943    0.124      98.648 0.986 0.017 0.986    0  8.64
## Comp 10    0.002      99.945    0.171      98.819 0.988 0.016 0.988    0  9.25
## Comp 11    0.002      99.947    0.141      98.960 0.990 0.015 0.990    0  9.86
## Comp 12    0.003      99.950    0.109      99.069 0.991 0.014 0.991    0 10.42
```

```
# show plots
plot(r)
```



There is also a limitation though, model loaded from JSON file created by the web-application does not contain calibration results, it can only be used for predictions:

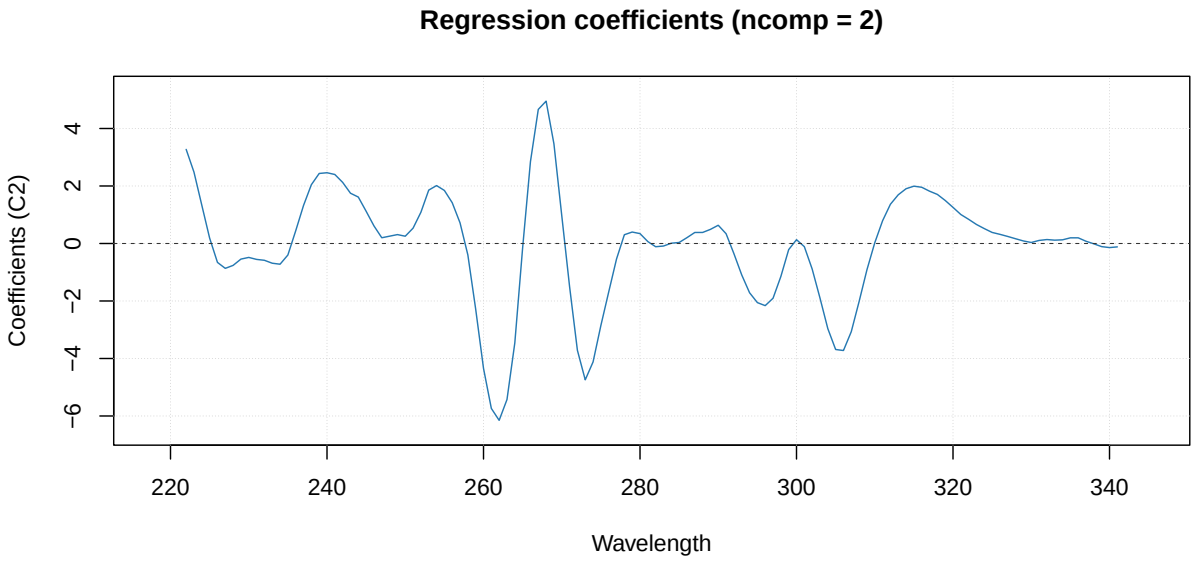
```
summary(m)
```

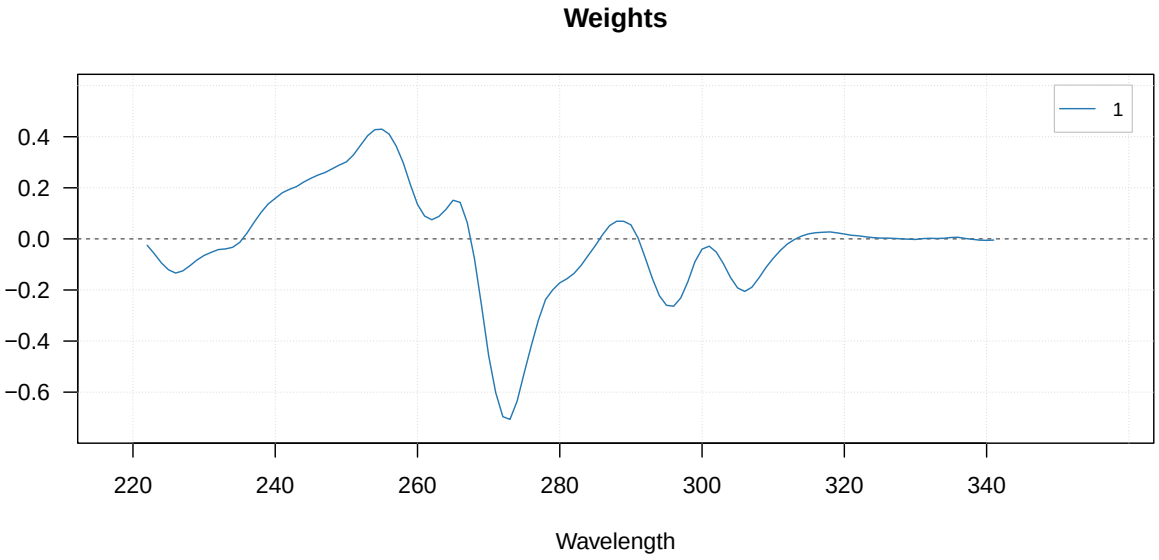
```
##
## PLS model (class pls) summary
## ------------------------------
## Info:
## Number of selected components: 2
## Cross-validation: none
##
## Preprocessing methods:
##  - savgol: width = 9, porder = 1, dorder = 1
##  - varsel: var.ind = 13:132
```

```
## Objects with results not found (most probably this model is loaded from web-application).
```

Most of the plots, which rely on calibration set results will not work with such model object and give you the same message as above. However plots, which are only based on model outcomes, such as loadings, weights and regression coefficients are available:

```
par(mfrow = c(2, 1))
plotRegcoeffs(m)
plotWeights(m)
```





PLS Discriminant Analysis

PLS Discriminant Analysis (PLS-DA) is a discrimination method based on PLS regression. To some extent, the idea of PLS-DA is similar to logistic regression — we use PLS for a dummy response variable, y, which is equal to +1 for objects belonging to a class, and -1 for those that do not (in some implementations it can also be 1 and 0 correspondingly). Then a conventional PLS regression model is calibrated and validated, which means that all methods and plots, you already used in PLS, can be used for PLS-DA models and results as well.

The extra step in PLS-DA is, actually, classification, which is based on thresholding of predicted y-values. If the predicted value is above 0, a corresponding object is considered as a member of a class and if not — as a stranger. In *mdatools* this is done automatically using methods `plsda()` and `plsdares()`, which inherit all `pls()` and `plsres()` methods. Plus they have something extra to represent classification results, which you have already read about in the chapter devoted to SIMCA. If you have not, it makes sense to do this first, to make the understanding of PLS-DA implementation easier.

In this chapter we will describe shortly how PLS-DA implementation works. All examples are based on the well-known Iris dataset, which will be split into two subsets — calibration (75 objects, 25 for each class) and validation (another 75 objects). Two PLS-DA models will be built — one only for *virginica* class and one for all three classes.

Calibration of PLS-DA model

Calibration of PLS-DA model is very similar to conventional PLS with one difference — you need to provide information about class membership of each object instead of a matrix or a vector with response values. This can be done in two different ways. If you have multiple classes, it is always recommended to provide your class membership data as a *factor* with predefined labels or a vector with class names as text values. The labels/values in this case will be used as class names. It is also acceptable to use numbers as labels but it will make interpretation of results less readable and can possibly cause problems with performance statistics calculations. So use names!

It is very important that you use the same labels/names for e.g. calibration and test set, because this is the way the model will identify which class an object came from. And if you have e.g. a typo in a label value, model will assume that the corresponding object is a stranger.

So let's prepare our data.

```
data(iris)

cal.ind = c(1:25, 51:75, 101:125)
val.ind = c(26:50, 76:100, 126:150)

Xc = iris[cal.ind, 1:4]
Xv = iris[val.ind, 1:4]
```

```
cc.all = iris[cal.ind, 5]
cv.all = iris[val.ind, 5]
```

In this case, the fifth column of dataset *Iris* is already a factor, otherwise we have to make it as a factor explicitly. Let's check if it is indeed correct.

```
show(cc.all)
```

```
##  [1] setosa     setosa     setosa     setosa     setosa     setosa
##  [7] setosa     setosa     setosa     setosa     setosa     setosa
## [13] setosa     setosa     setosa     setosa     setosa     setosa
## [19] setosa     setosa     setosa     setosa     setosa     setosa
## [25] setosa     versicolor versicolor versicolor versicolor versicolor
## [31] versicolor versicolor versicolor versicolor versicolor versicolor
## [37] versicolor versicolor versicolor versicolor versicolor versicolor
## [43] versicolor versicolor versicolor versicolor versicolor versicolor
## [49] versicolor versicolor virginica  virginica  virginica  virginica
## [55] virginica  virginica  virginica  virginica  virginica  virginica
## [61] virginica  virginica  virginica  virginica  virginica  virginica
## [67] virginica  virginica  virginica  virginica  virginica  virginica
## [73] virginica  virginica  virginica
## Levels: setosa versicolor virginica
```

However, for the model with just one class, *virginica*, we need to prepare the class variable in a different way. In this case it is enough to provide a vector with logical values, where `TRUE` will correspond to a member and `FALSE` to a non-member of the class. Here is an example of how to do it (we will make two — one for calibration and one for validation subsets).

```
cc.vir = cc.all == "virginica"
cv.vir = cv.all == "virginica"
show(cc.vir)
```

```
##  [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [13] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [25] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [37] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [49] FALSE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
## [61]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
## [73]  TRUE  TRUE  TRUE
```

Now we can calibrate the models:

```
m.all = plsda(Xc, cc.all, 3, cv = 1)
m.vir = plsda(Xc, cc.vir, 3, cv = 1, classname = "virginica")
```

You could notice one important difference. In case when parameter `c` is a vector with logical values you also need to provide a name of the class. If you do not do this, a default name will be used, but it may cause problems when you e.g. validate your model using test set where class membership is a factor as we have in this example.

Let's look at the summary for each of the model. As you can see below, summary for multi class PLS-DA simply shows one set of results for each class. The performance statistics include explained X and Y variance (cumulative), values for confusion matrix (true positives, false positives, true negatives, false negatives) as well as specificity, sensitivity and accuracy values.

```
summary(m.all)
```

```
##
```

```
## PLS-DA model (class plsda) summary
## -----------------------------------
## Info:
## Number of selected components: 1
## Cross-validation: full (leave one out)
##
## Class #1 (setosa)
##     X cumexpvar Y cumexpvar TP FP TN FN Spec. Sens. Accuracy
## Cal       91.97       46.36 25  0 50  0     1     1        1
## Cv           NA          NA 25  0 50  0     1     1        1
##
## Class #2 (versicolor)
##     X cumexpvar Y cumexpvar TP FP TN FN Spec. Sens. Accuracy
## Cal       91.97       46.36  0  0 50 25  1.00     0    0.667
## Cv           NA          NA  0  3 47 25  0.94     0    0.627
##
## Class #3 (virginica)
##     X cumexpvar Y cumexpvar TP FP TN FN Spec. Sens. Accuracy
## Cal       91.97       46.36 24  5 45  1  0.90  0.96    0.920
## Cv           NA          NA 24  6 44  1  0.88  0.96    0.907
```

Dealing with the multi-class PLS-DA model is similar to dealing with PLS2 models, when you have several y-variables. Every time you want to show a plot or results for a particular class, just provide the class number using parameter `nc`. For example this is how to show summary only for the third class (*virginica*).

```
summary(m.all, nc = 3)
```

```
##
## PLS-DA model (class plsda) summary
## -----------------------------------
## Info:
## Number of selected components: 1
## Cross-validation: full (leave one out)
##
## Class #3 (virginica)
##     X cumexpvar Y cumexpvar TP FP TN FN Spec. Sens. Accuracy
## Cal       91.97       46.36 24  5 45  1  0.90  0.96    0.920
## Cv           NA          NA 24  6 44  1  0.88  0.96    0.907
```

You can also show statistics only for calibration or only for cross-validation parts, in this case you will see details about contribution of every component to the model.

```
summary(m.all$calres, nc = 3)
```

```
##
## PLS-DA results (class plsdares) summary:
## Number of selected components: 1
##
## Class #3 (virginica):
##        X expvar X cumexpvar Y expvar Y cumexpvar TP FP TN FN Spec. Sens.
## Comp 1   91.969      91.969   46.356      46.356 24  5 45  1  0.90  0.96
## Comp 2    5.500      97.468    6.878      53.233 21  5 45  4  0.90  0.84
## Comp 3    2.186      99.654    4.816      58.049 24  3 47  1  0.94  0.96
##        Accuracy
## Comp 1    0.920
## Comp 2    0.880
```

```
## Comp 3    0.947
```

For one class models, the behaviour is actually similar, but there will be always one set of results — for the corresponding class. Here is the summary.

```
summary(m.vir)
```

```
##
## PLS-DA model (class plsda) summary
## -----------------------------------
## Info:
## Number of selected components: 3
## Cross-validation: full (leave one out)
##
## Class #1 (virginica)
##     X cumexpvar Y cumexpvar TP FP TN FN Spec. Sens. Accuracy
## Cal       98.53       61.31 24  3 47  1  0.94  0.96    0.947
## Cv           NA          NA 24  3 47  1  0.94  0.96    0.947
```

Like in SIMCA you can also get a confusion matrix for a particular result. Here is an example for multiple classes model.

```
getConfusionMatrix(m.all$calres)
```

```
##            setosa versicolor virginica None
## setosa         25          0         0    0
## versicolor      0          0         5   20
## virginica       0          0        24    1
```

And for the one-class model.

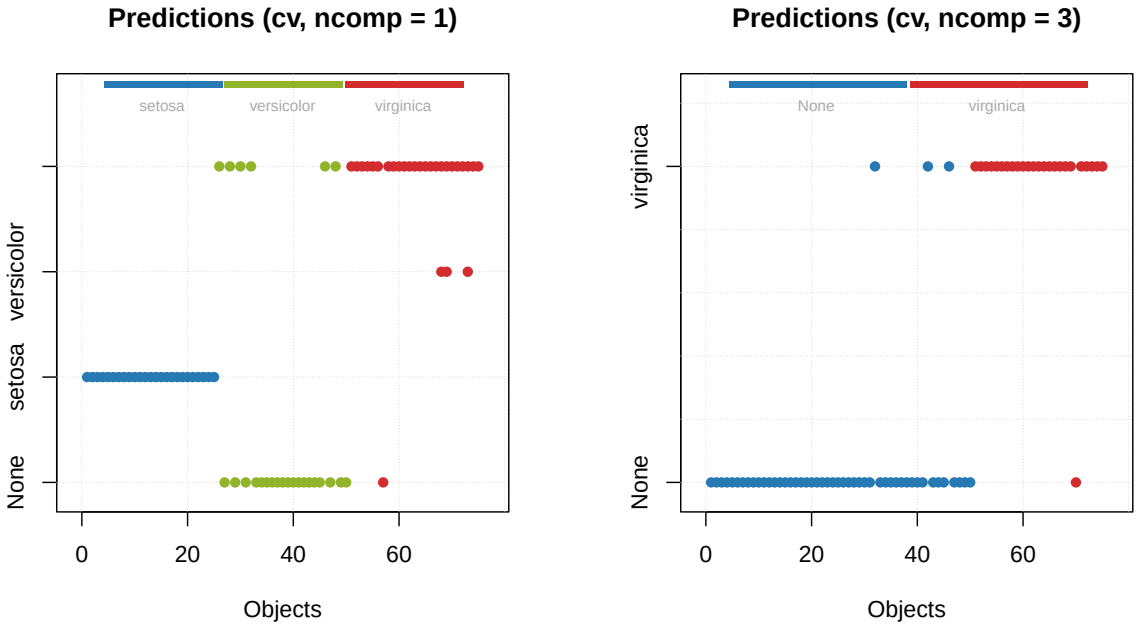
```
getConfusionMatrix(m.vir$calres)
```

```
##           virginica None
## virginica        24    1
## None              3   47
```

Classification plots

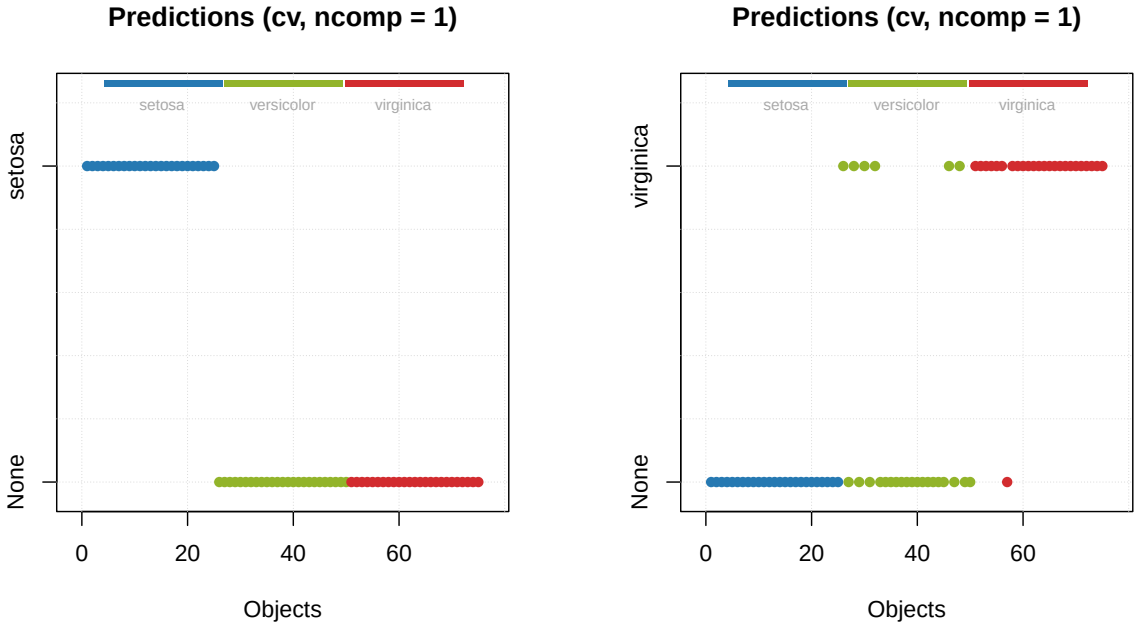
Most of the plots for visualisation of classification results described in the SIMCA chapter can also be used for PLS-DA models and results. Let's start with classification plots. By default it is shown for cross-validation results (we change position of the legend so it does not hide the points). You can clearly spot for example three false positives and one false negatives in the one-class PLS-DA model for virginica.

```
par(mfrow = c(1, 2))
plotPredictions(m.all)
plotPredictions(m.vir)
```



In case of multiple classes model you can select which class to show the predictions for.

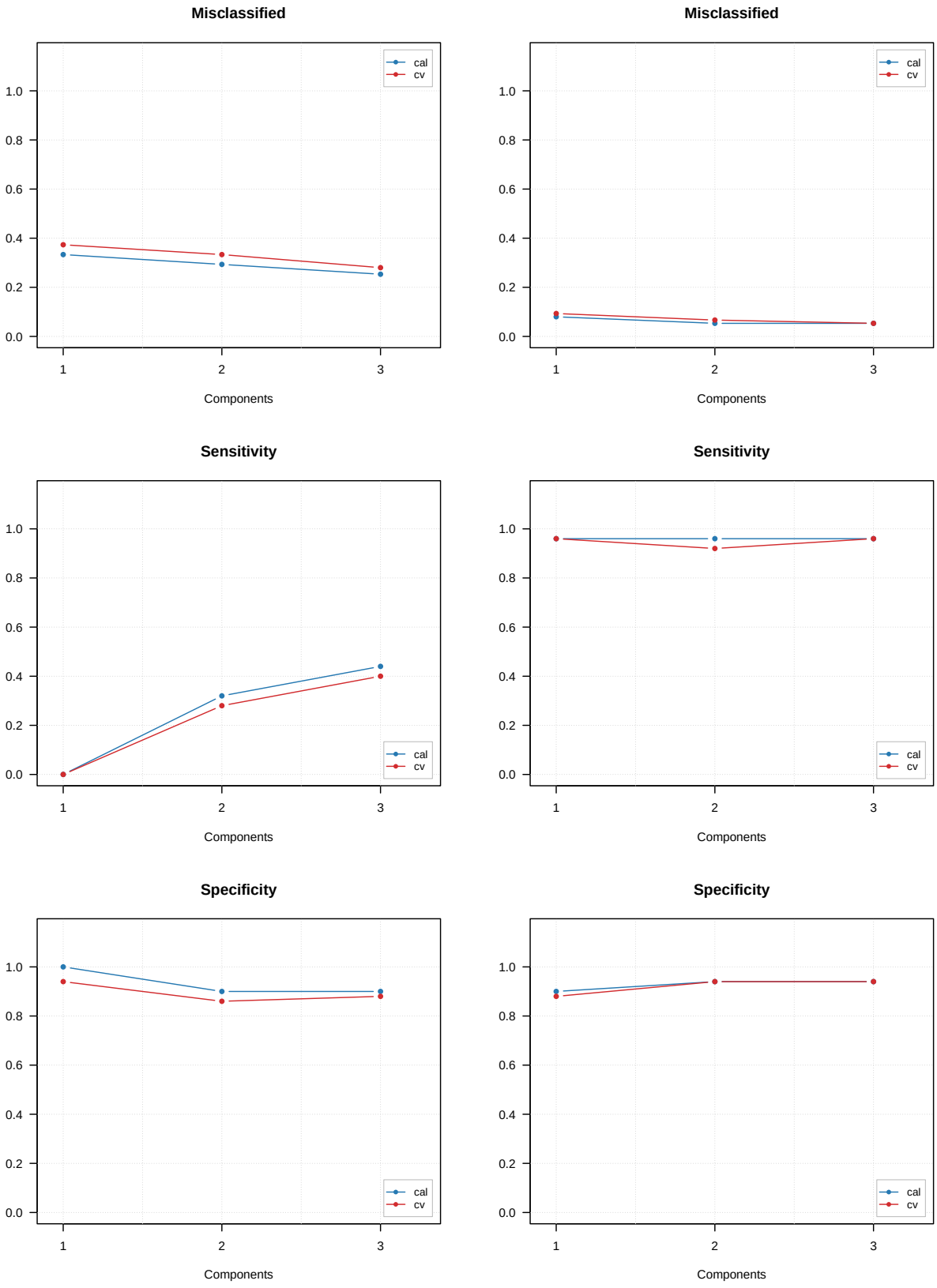
```
par(mfrow = c(1, 2))
plotPredictions(m.all, nc = 1)
plotPredictions(m.all, nc = 3)
```



Performance plots

As in SIMCA you can show how sensitivity, specificity and total amount of misclassified samples depending on the number of components by using corresponding plots. In case of multiple-classes model you can also provide a class number to show the plot for (by default package will show the plot for overall statistic computed for all classes).

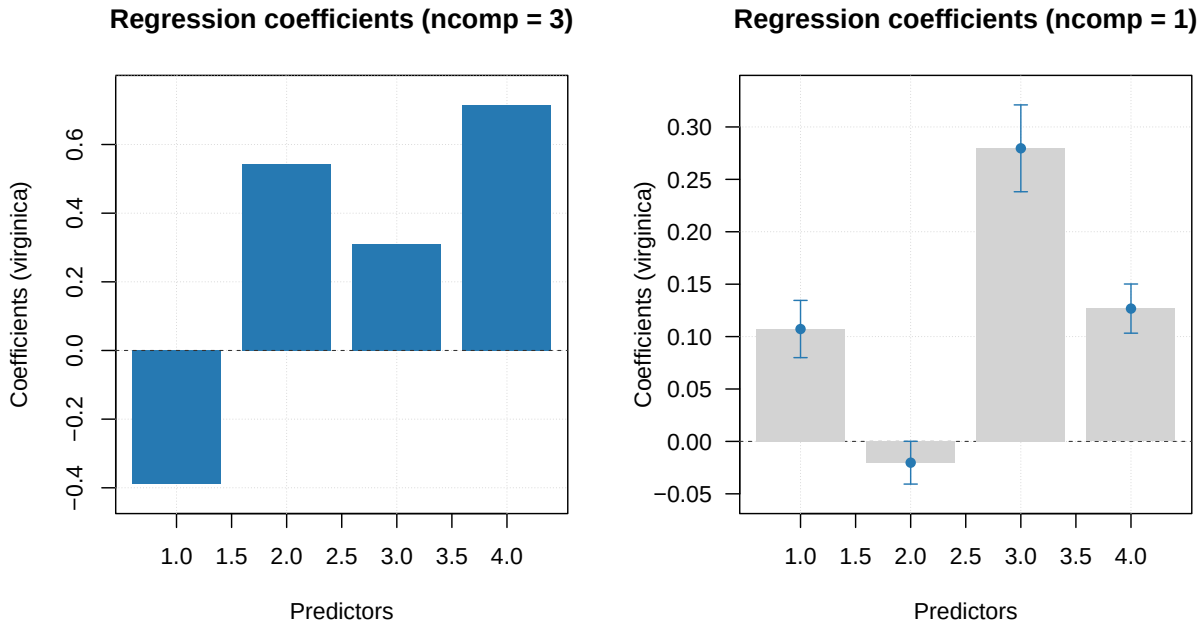
```
par(mfrow = c(3, 2))
plotMisclassified(m.all, nc = 2)
plotMisclassified(m.vir)
plotSensitivity(m.all, nc = 2)
plotSensitivity(m.vir)
plotSpecificity(m.all, nc = 2)
plotSpecificity(m.vir)
```



As usual, you can also change type of plot to line or scatter-line, change colors, etc. All PLS regression plots,

including RMSE, X and Y variance, etc. will also work smoothly with PLS-DA models or results. You can also show regression coefficients like in the example below. To show regression coefficients for particular class you need to provide its number using parameter `ny` (the reason we use `ny` and not `nc` here is that this plot is inherited from PLS model). So in our example both plots show regression coefficients for prediction of virginica objects (left obtained using multiple class model and right — using one class model).

```
par(mfrow = c(1, 2))
plotRegcoeffs(m.all, ncomp = 3, ny = 3)
plotRegcoeffs(m.vir, ncomp = 1, show.ci = TRUE)
```



Predictions for new data

Again very similar to PLS — just use method `predict()` and provide at least matrix or data frame with predictors (which should contain the same number of variables/columns). For test set validation you can also provide class reference information similar to what you have used for calibration of PLS-DA models.

In case of multiple class model, the reference values should be provided as a factor or vector with class names as text values. Here is an example.

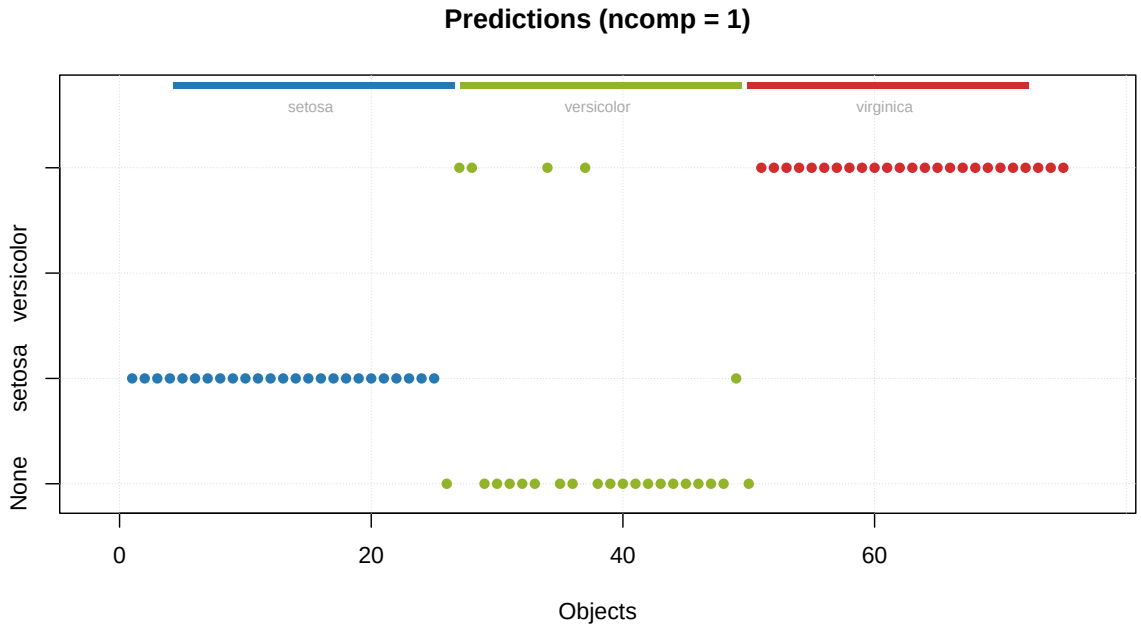
```
res = predict(m.all, Xv, cv.all)
summary(res)
```

```
##
## PLS-DA results (class plsdares) summary:
## Number of selected components: 1
##
## Class #1 (setosa):
##        X expvar X cumexpvar Y expvar Y cumexpvar TP FP TN FN Spec. Sens.
## Comp 1   92.925      92.925   47.013      47.013 25  1 49  0  0.98     1
## Comp 2    4.560      97.484   10.373      57.386 25  0 50  0  1.00     1
## Comp 3    1.789      99.274    1.588      58.974 25  0 50  0  1.00     1
##        Accuracy
## Comp 1    0.987
```

```
## Comp 2    1.000
## Comp 3    1.000
##
##
## Class #2 (versicolor):
##        X expvar X cumexpvar Y expvar Y cumexpvar TP FP TN FN Spec. Sens.
## Comp 1   92.925      92.925   47.013      47.013  0  0 50 25  1.00   0.0
## Comp 2    4.560      97.484   10.373      57.386 10  4 46 15  0.92   0.4
## Comp 3    1.789      99.274    1.588      58.974 10  6 44 15  0.88   0.4
##        Accuracy
## Comp 1    0.667
## Comp 2    0.747
## Comp 3    0.720
##
##
## Class #3 (virginica):
##        X expvar X cumexpvar Y expvar Y cumexpvar TP FP TN FN Spec. Sens.
## Comp 1   92.925      92.925   47.013      47.013 25  4 46  0  0.92  1.00
## Comp 2    4.560      97.484   10.373      57.386 25  4 46  0  0.92  1.00
## Comp 3    1.789      99.274    1.588      58.974 24  4 46  1  0.92  0.96
##        Accuracy
## Comp 1    0.947
## Comp 2    0.947
## Comp 3    0.933
```

And the corresponding plot with predictions.

```
par(mfrow = c(1, 1))
plotPredictions(res)
```



If vector with reference class values contains names of classes the model knows nothing about, they will simply be considered as members of none of the known classes ("None").

In case of one-class model, the reference values can be either factor/vector with names or logical values, like the ones used for calibration of the model. Here is an example for each of the cases.

```
res21 = predict(m.vir, Xv, cv.all)
summary(res21)
```

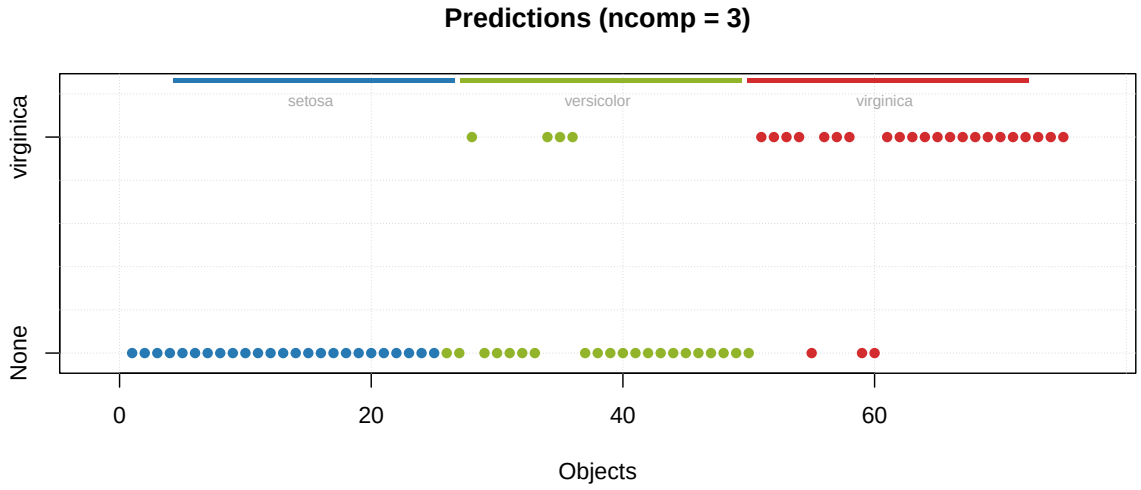
```
##
## PLS-DA results (class plsdares) summary:
## Number of selected components: 3
##
## Class #1 (virginica):
##        X expvar X cumexpvar Y expvar Y cumexpvar TP FP TN FN Spec. Sens.
## Comp 1   92.949      92.949   53.963      53.963 25  4 46  0  0.92  1.00
## Comp 2    1.624      94.573    6.097      60.059 24  4 46  1  0.92  0.96
## Comp 3    2.702      97.275   -0.151      59.908 22  4 46  3  0.92  0.88
##        Accuracy
## Comp 1    0.947
## Comp 2    0.933
## Comp 3    0.907
```

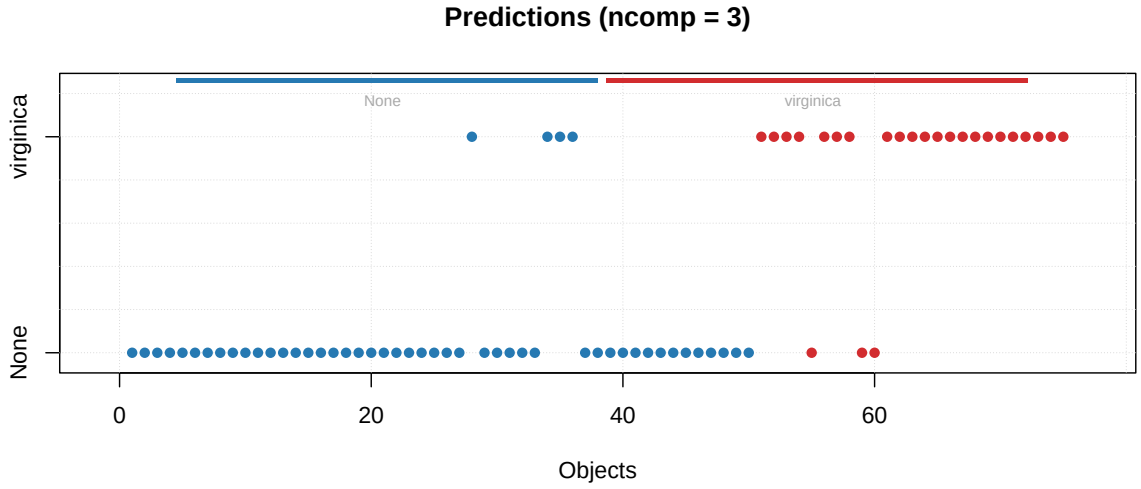
```
res22 = predict(m.vir, Xv, cv.vir)
summary(res22)
```

```
##
## PLS-DA results (class plsdares) summary:
## Number of selected components: 3
##
## Class #1 (virginica):
##        X expvar X cumexpvar Y expvar Y cumexpvar TP FP TN FN Spec. Sens.
## Comp 1   92.949      92.949   53.963      53.963 25  4 46  0  0.92  1.00
## Comp 2    1.624      94.573    6.097      60.059 24  4 46  1  0.92  0.96
## Comp 3    2.702      97.275   -0.151      59.908 22  4 46  3  0.92  0.88
##        Accuracy
## Comp 1    0.947
## Comp 2    0.933
## Comp 3    0.907
```

As you can see, statistically results are identical. However, the predictions plot will look a bit different for these two cases, as you can see below.

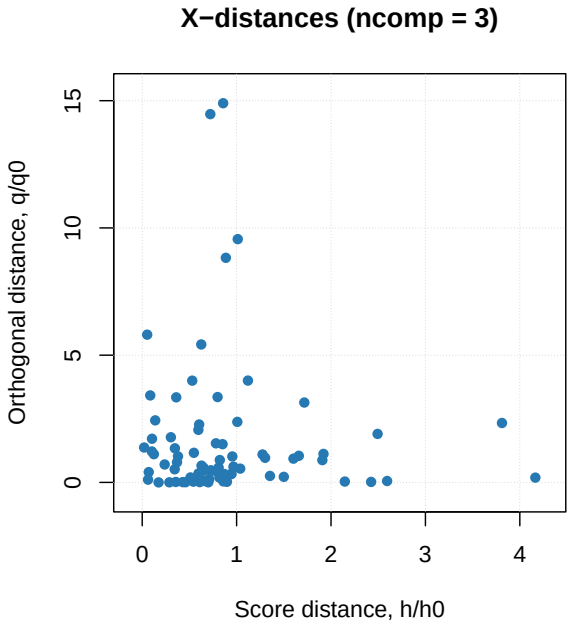
```
par(mfrow = c(2, 1))
plotPredictions(res21)
plotPredictions(res22)
```

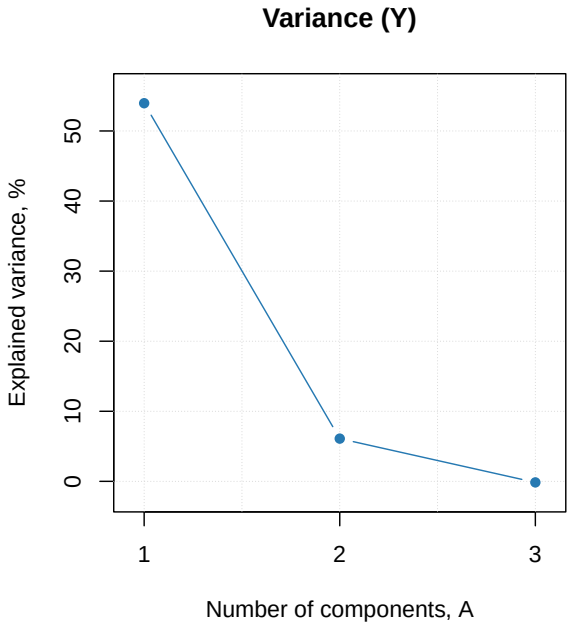




And because `predict()` returns an object with results you can also use most of the plots available for PLS regression results. In the last example below you will find plots for X-distance and Y-variance.

```
par(mfrow = c(1, 2))
plotXResiduals(res21)
plotYVariance(res22)
```


X−distances (ncomp = 3)
Orthogonal distance, q/q0
0
5
10
15
0
1
2
3
4
Score distance, h/h0


Variance (Y)
Explained variance, %
0
10
20
30
40
50
1
2
3
Number of components, A

Interval PLS

Interval PLS (iPLS) is a variable selection method used mostly for spectroscopic data proposed by Lars Noergaard *et al.* in 2000. The main idea is to try different parts (or intervals — hence the name) of spectra and their combinations to find the most relevant for prediction of a response variable. There are two algorithms — forward and backward.

Forward iPLS

The forward approach works as follows

1. Split spectral data into N intervals
2. Create an empty vector with selected intervals
3. Create a model where intervals in the vector (already selected) are combined with one of the rest. If combination improves the model, add this new interval to the vector.
4. Repeat previous step until there are no improvements.

Apparently, when nothing is selected, on step 3 the algorithm simply creates a model for every interval available. As you can see, iPLS is based on iterative procedure. If you have 20 intervals you need to create 20 models on the first step, 19 models on the second step and so on.

In *mdatools*, you can run iPLS selection by using function `ipls()`. You have to specify matrix with predictors (`x`) and responses (`y`), number of components in global model (`glob.ncomp`), and define the intervals. In addition to that, you can also specify parameters of cross-validation (`cv`) as well as criterion for selection of optimal number of components (`ncomp.selcrit`) for local models similar to PLS. Forward iPLS is used as a default algorithm.

Intervals can be defined using one of the following three parameters: number of intervals (`int.num`), width of an interval (`int.width`) or specify start and end of each interval as a two-column matrix with variable indices (`int.limits`). In addition to that, you can specify maximum number of component for local models (`int.ncomp`) and maximum number of iterations — so method will stop regardless if you have improvements or not (`int.niter`). By default the maximum number of iterations is limited to 30.

Here is an example of applying iPLS for Simdata (for concentration of second chemical component, C2) using 15 intervals.

```
data(simdata)
X = simdata$spectra.c
y = simdata$conc.c[, 2, drop = FALSE]

m = ipls(X, y, glob.ncomp = 4, int.num = 15)
```

```
## Model with all intervals: RMSE = 0.027572, nLV = 3

## Iteration   1/ 15... selected interval   6 (RMSE = 0.030405, nLV = 3)
## Iteration   2/ 15... selected interval  11 (RMSE = 0.027882, nLV = 3)
## Iteration   3/ 15... selected interval   1 (RMSE = 0.027509, nLV = 3)
## Iteration   4/ 15... selected interval   2 (RMSE = 0.027243, nLV = 2)
```

```
## Iteration   5/ 15... selected interval  13 (RMSE = 0.027210, nLV = 2)
## Iteration   6/ 15... selected interval  12 (RMSE = 0.027201, nLV = 2)
## Iteration   7/ 15... no improvements, stop.
```

As you can see, by default method shows information for every step in the console. Use parameter `silent = TRUE` to change this behaviour.

From the example above we can see that the global model had RMSECV = 0.027625 with 3 components (by default the method uses systematic cross-validation, "Venetian blinds", with 10 segments). Creating local models with individual intervals gave the best performance with RMSECV = 0.029830 (interval #6). Combination of the interval #6 with interval #1 (next step) gave RMSECV = 0.027809. The small improvement was obtained for adding 3 more intervals giving final result with RMSECV = 0.027246 with 5 selected intervals in total (6, 1, 11, 3, 12).

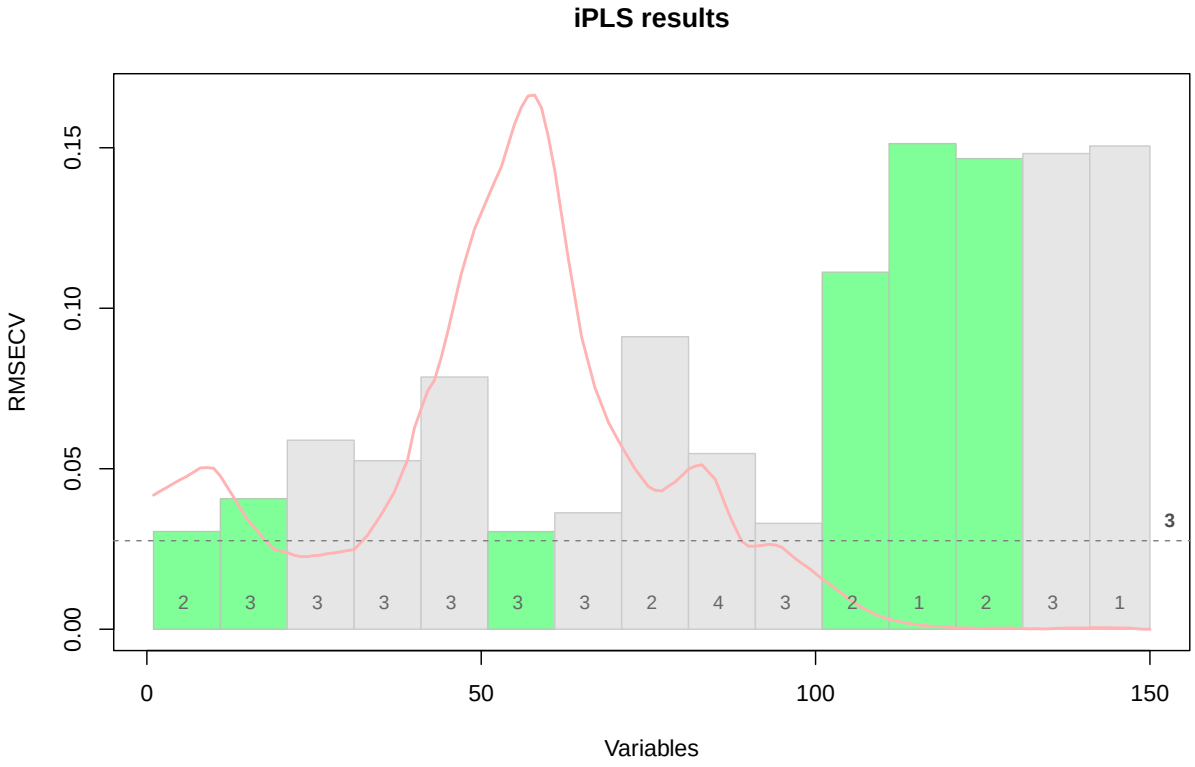
There are several ways to explore the iPLS results both graphically and numerically. First of all, summary will show full information about the selected intervals, including index of variables on both ends:

```
summary(m)
```

```
##
## iPLS variable selection results
##   Method: forward
##   Validation: venetian blinds with 10 segments
##   Number of intervals: 15
##   Number of selected intervals: 6
##   RMSECV for global model: 0.027572 (3 LVs)
##   RMSECV for optimized model: 0.027201 (2 LVs)
##
## Summary for selection procedure:
##    n start end selected nComp       RMSE    R2
## 1  0     1 150    FALSE     3 0.02757234 0.965
## 2  6    51  60     TRUE     3 0.03040477 0.958
## 3 11   101 110     TRUE     3 0.02788242 0.965
## 4  1     1  10     TRUE     3 0.02750915 0.966
## 5  2    11  20     TRUE     2 0.02724335 0.966
## 6 13   121 130     TRUE     2 0.02720961 0.966
## 7 12   111 120     TRUE     2 0.02720089 0.966
```

Also you can see the first step (performance of individual models) and the selected interval by using function `plot()` for the whole model:

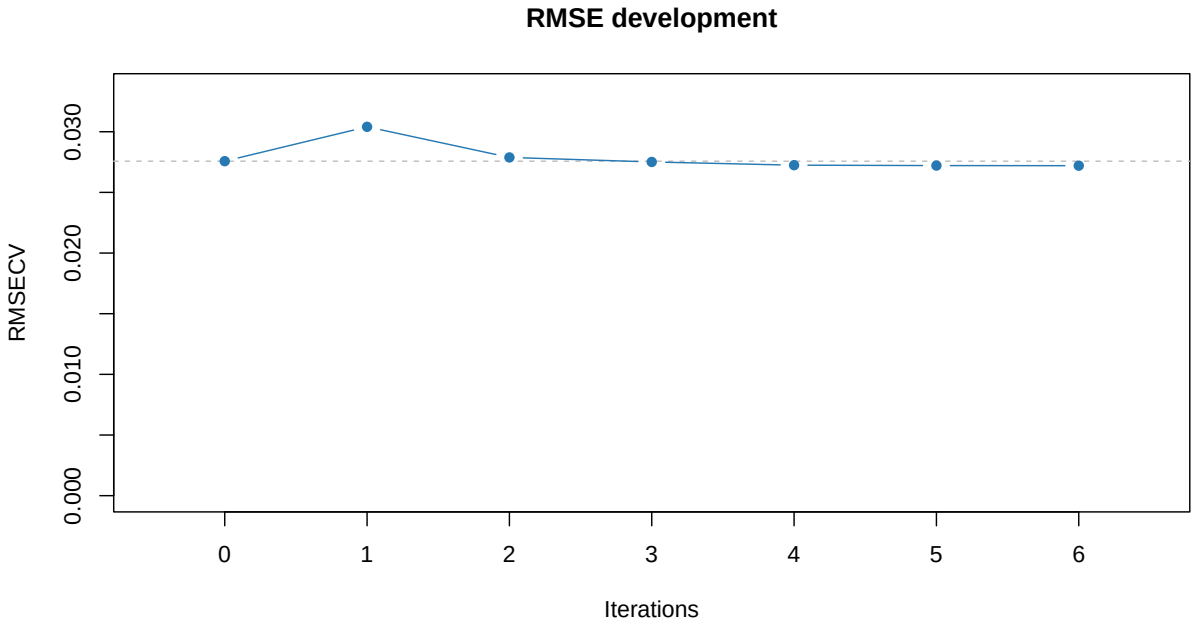
```
plot(m)
```



The red curve here is the average spectrum. Bars are intervals, height of each bar corresponds to the RMSECV value obtained for local model made using variables from this interval as predictors (first iteration). The number inside each bar is the number of PLS components used in the local model. Green color shows intervals which have been selected at the end of the procedure and dashed line shows error for the global model

You can also see the improvements of RMSECV with iPLS iterations graphically:

```
plotRMSE(m)
```



Dashed line in this case shows RMSECV value for the global model with all intervals included.

Finally, you can get both selected intervals and corresponding indices of variables (all of them not just interval limits) as follows:

```
show(m$int.selected)
```

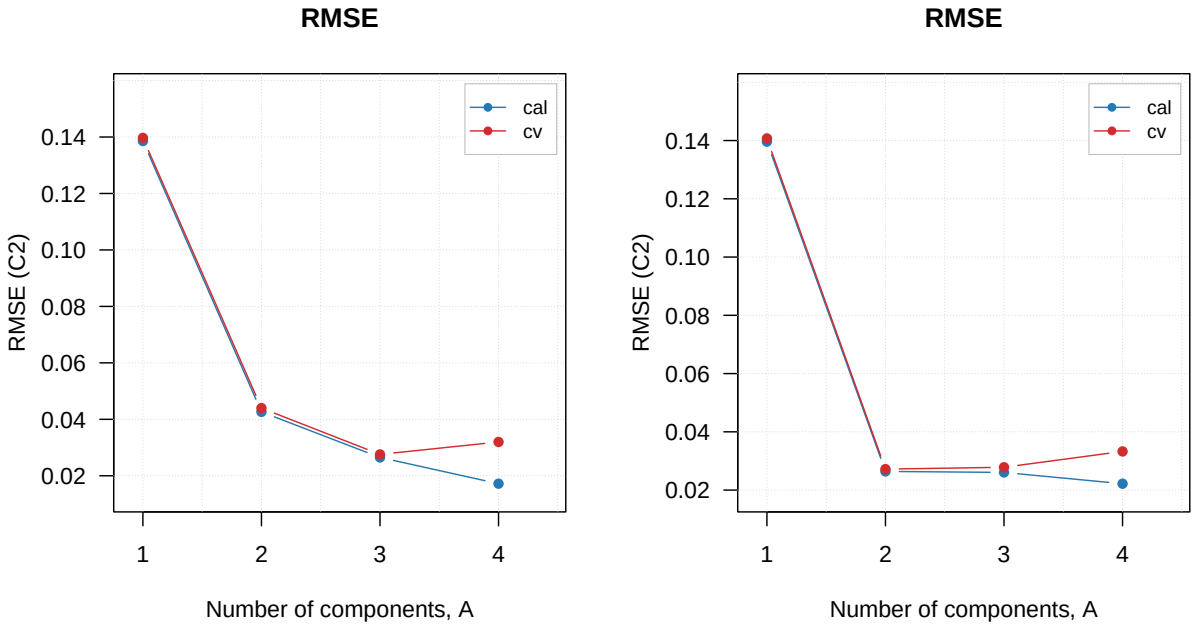
```
## [1]  6 11  1  2 13 12
```

```
show(m$var.selected)
```

```
##  [1]  51  52  53  54  55  56  57  58  59  60 101 102 103 104 105 106 107 108 109
## [20] 110   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
## [39]  19  20 121 122 123 124 125 126 127 128 129 130 111 112 113 114 115 116 117
## [58] 118 119 120
```

Also the model object contains the initial global model for full data (`m$gm`) and final optimized PLS model made using only variables from the selected intervals, `m$om`. In the following example I compare RMSE plots for both:

```
par(mfrow = c(1, 2))
plotRMSE(m$gm)
plotRMSE(m$om)
```



Running full procedure

By default the iPLS procedure will stop when no improvement is observed. This means if RMSECV value for the next iteration is not smaller than the RMSECV value for the current iteration. However, you can change this behavior if you set a logical parameter `full = TRUE`. In this case the procedure will continue until the maximum number of iterations is reached.

Here are some examples. First of all let's run the same example as in the previous section with 15 intervals:

```
data(simdata)
X = simdata$spectra.c
y = simdata$conc.c[, 2, drop = FALSE]

m = ipls(X, y, glob.ncomp = 4, int.num = 15)
```

```
## Model with all intervals: RMSE = 0.027572, nLV = 3

## Iteration   1/ 15... selected interval   6 (RMSE = 0.030405, nLV = 3)
## Iteration   2/ 15... selected interval  11 (RMSE = 0.027882, nLV = 3)
## Iteration   3/ 15... selected interval   1 (RMSE = 0.027509, nLV = 3)
## Iteration   4/ 15... selected interval   2 (RMSE = 0.027243, nLV = 2)
## Iteration   5/ 15... selected interval  13 (RMSE = 0.027210, nLV = 2)
## Iteration   6/ 15... selected interval  12 (RMSE = 0.027201, nLV = 2)
## Iteration   7/ 15... no improvements, stop.
```

As you can see, the procedure stops at iteration #6 because no improvement is observed. And if you look at statistics, indeed only 5 intervals are selected:

```
summary(m)
```

```
##
## iPLS variable selection results
##   Method: forward
##   Validation: venetian blinds with 10 segments
```

```
##   Number of intervals: 15
##   Number of selected intervals: 6
##   RMSECV for global model: 0.027572 (3 LVs)
##   RMSECV for optimized model: 0.027201 (2 LVs)
##
## Summary for selection procedure:
##    n start end selected nComp       RMSE    R2
## 1  0     1 150    FALSE     3 0.02757234 0.965
## 2  6    51  60     TRUE     3 0.03040477 0.958
## 3 11   101 110     TRUE     3 0.02788242 0.965
## 4  1     1  10     TRUE     3 0.02750915 0.966
## 5  2    11  20     TRUE     2 0.02724335 0.966
## 6 13   121 130     TRUE     2 0.02720961 0.966
## 7 12   111 120     TRUE     2 0.02720089 0.966
```

```
show(m$int.selected)
```

```
## [1]  6 11  1  2 13 12
```

Now let's do the same but with parameter `full = TRUE`.

```
m = ipls(X, y, glob.ncomp = 4, int.num = 15, full = TRUE)
```

```
## Model with all intervals: RMSE = 0.027572, nLV = 3
```

```
## Iteration   1/ 15... selected interval   6 (RMSE = 0.030405, nLV = 3)
## Iteration   2/ 15... selected interval  11 (RMSE = 0.027882, nLV = 3)
## Iteration   3/ 15... selected interval   1 (RMSE = 0.027509, nLV = 3)
## Iteration   4/ 15... selected interval   2 (RMSE = 0.027243, nLV = 2)
## Iteration   5/ 15... selected interval  13 (RMSE = 0.027210, nLV = 2)
## Iteration   6/ 15... selected interval  12 (RMSE = 0.027201, nLV = 2)
## Iteration   7/ 15... selected interval   8 (RMSE = 0.027219, nLV = 2)
## Iteration   8/ 15... selected interval  14 (RMSE = 0.027245, nLV = 2)
## Iteration   9/ 15... selected interval  15 (RMSE = 0.027331, nLV = 2)
## Iteration  10/ 15... selected interval   4 (RMSE = 0.027443, nLV = 3)
## Iteration  11/ 15... selected interval   3 (RMSE = 0.027391, nLV = 3)
## Iteration  12/ 15... selected interval   9 (RMSE = 0.027353, nLV = 3)
## Iteration  13/ 15... selected interval   5 (RMSE = 0.027378, nLV = 3)
## Iteration  14/ 15... selected interval   7 (RMSE = 0.027487, nLV = 3)
## Iteration  15/ 15... selected interval  10 (RMSE = 0.027572, nLV = 3)
```

Now it runs 15 iterations, because by default this is the largest number of iterations in this case (all possible intervals).

In case of full procedure, the selection of intervals and corresponding variables is done by finding a global minimum of RMSECV. As you can see from the output above, the first local minimum (RMSECV = 0.027246) was indeed at the 5th iteration, after which the procedure stopped in our previous example. However, when we use `full = TRUE` it continues and the global minimum is observed at iteration #8 (RMSECV = 0.027216).

Therefore the number of selected intervals will be 8:

```
summary(m)
```

```
##
## iPLS variable selection results
##   Method: forward
##   Validation: venetian blinds with 10 segments
##   Number of intervals: 15
##   Number of selected intervals: 6
```

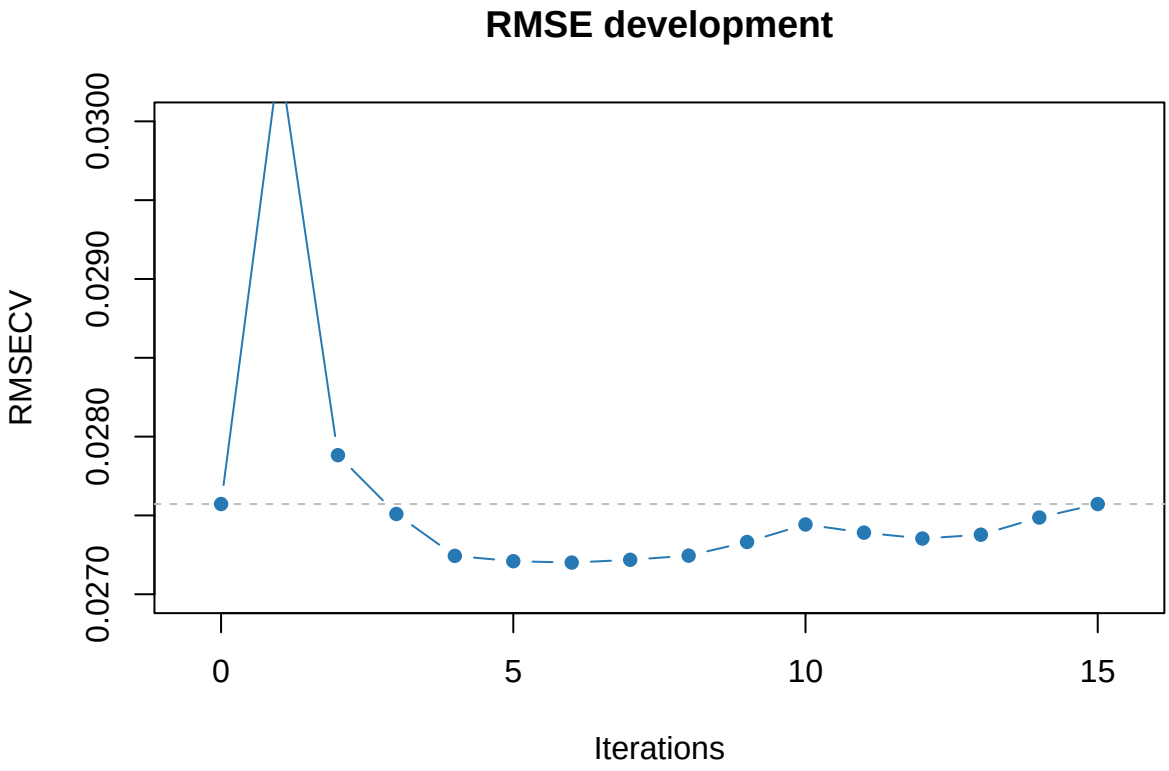
```
##   RMSECV for global model: 0.027572 (3 LVs)
##   RMSECV for optimized model: 0.027201 (2 LVs)
##
## Summary for selection procedure:
##     n start end selected nComp       RMSE    R2
## 1   0     1 150    FALSE     3 0.02757234 0.965
## 2   6    51  60     TRUE     3 0.03040477 0.958
## 3  11   101 110     TRUE     3 0.02788242 0.965
## 4   1     1  10     TRUE     3 0.02750915 0.966
## 5   2    11  20     TRUE     2 0.02724335 0.966
## 6  13   121 130     TRUE     2 0.02720961 0.966
## 7  12   111 120     TRUE     2 0.02720089 0.966
## 8   8    71  80    FALSE     2 0.02721858 0.966
## 9  14   131 140    FALSE     2 0.02724466 0.966
## 10 15   141 150    FALSE     2 0.02733144 0.966
## 11  4    31  40    FALSE     3 0.02744254 0.966
## 12  3    21  30    FALSE     3 0.02739080 0.966
## 13  9    81  90    FALSE     3 0.02735341 0.966
## 14  5    41  50    FALSE     3 0.02737763 0.966
## 15  7    61  70    FALSE     3 0.02748661 0.966
## 16 10    91 100    FALSE     3 0.02757234 0.965
```

```
show(m$int.selected)
```

```
## [1]  6 11  1  2 13 12
```

You can also see this on RMSECV vs iterations plot:

```
plotRMSE(m, ylim = c(0.027, 0.030))
```



The first minimum is clearly observed at 5th iteration and the global minimum at 8th. Of course, in this particular case this effect is most probably random, however there are cases where running full iPLS procedure is useful.

If the number of intervals is large, by default, the maximum number of iterations `ipls()` will try, is limited by 30. You can change this by specifying an additional parameter, `iter.niter`, for example `iter.niter = 100`.

Using test set for validation

Although iPLS was developed for using with cross-validation, sometimes, especially if the dataset is large, it can give very large computational time. In this case you can provide test set for validation instead. The syntax and the parameter names are similar to test validation in `pls()`. Here is an example:

```
data(simdata)
X = simdata$spectra.c
y = simdata$conc.c[, 2, drop = FALSE]

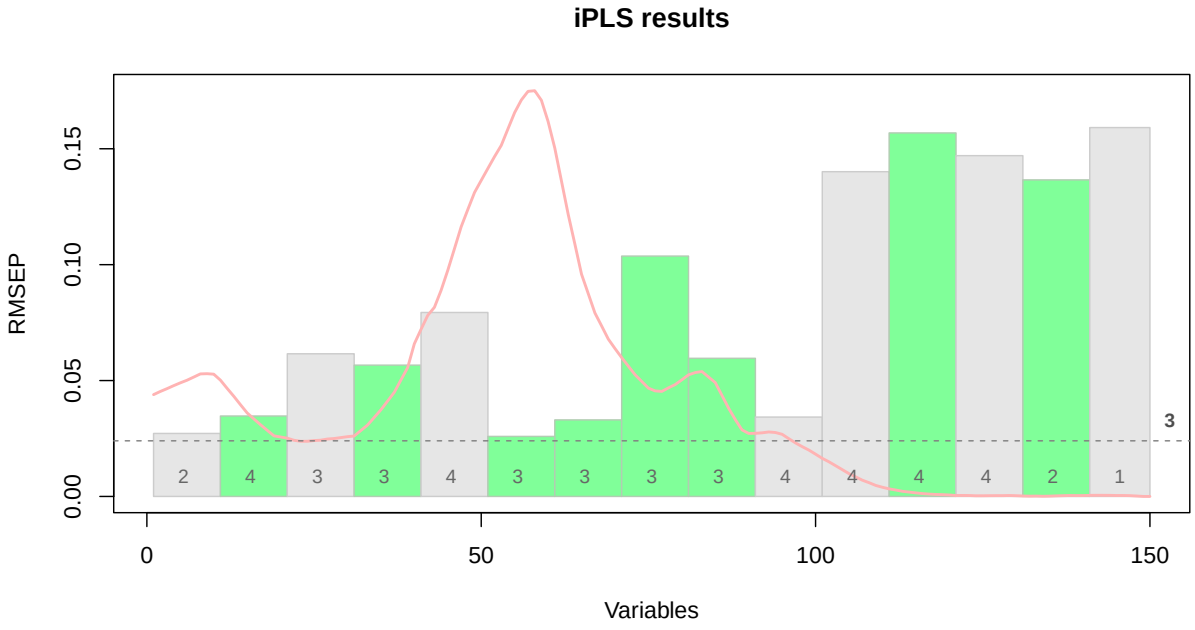
X.t = simdata$spectra.t
y.t = simdata$conc.t[, 2, drop = FALSE]
m = ipls(X, y, glob.ncomp = 4, int.num = 15, x.test = X.t, y.test = y.t)
```

```
## Model with all intervals: RMSE = 0.024003, nLV = 3

## Iteration   1/ 15... selected interval   6 (RMSE = 0.025899, nLV = 3)
## Iteration   2/ 15... selected interval   7 (RMSE = 0.023865, nLV = 3)
## Iteration   3/ 15... selected interval  12 (RMSE = 0.023390, nLV = 3)
## Iteration   4/ 15... selected interval   2 (RMSE = 0.022984, nLV = 3)
```

```
## Iteration   5/ 15... selected interval   4 (RMSE = 0.022741, nLV = 3)
## Iteration   6/ 15... selected interval   9 (RMSE = 0.022551, nLV = 3)
## Iteration   7/ 15... selected interval   8 (RMSE = 0.022474, nLV = 3)
## Iteration   8/ 15... selected interval  14 (RMSE = 0.022428, nLV = 3)
## Iteration   9/ 15... no improvements, stop.
```

```
plot(m)
```



Backward iPLS

In backward iPLS, instead of selecting best intervals we do the opposite — get rid of the worst. So, at the first step, we try to remove every interval from the data to see if it gives any improvement. If it does, we keep it as excluded and then try to remove another one. The process continues until no improvement is observed.

To use the backward method simply specify parameter `method = "backward"` when you call `ipls()`. The rest, including plots and statistics, is the same.

Multivariate Curve Resolution

Multivariate Curve Resolution (MCR) is a group of methods that can be used to solve the curve resolution problem in spectroscopy, which, in its general form, can be defined as follows. Let's say we have a mixture of A chemical components (e.g. ribose, fructose and lactose). Every individual component is usually called a *pure* component. Every pure component i has a spectrum (IR, NIR, Raman, etc.), which can be represented as a column vector $\mathbf{s}_i$, with size $J \times 1$, where J is a number of values in each spectrum (corresponding to the number of wavelengths, wavenumbers, chemical shifts, etc.).

According to the Beer-Lambert law, if you mix the components into one mixture and take a spectrum of this mixture, the spectrum will be just a linear combination of the spectra of the pure components. This can be written as follows:

$$\mathbf{d} = c_1\mathbf{s}_1^T + c_2\mathbf{s}_2^T + \cdots + c_A\mathbf{s}_A^T$$

In this equation, $\mathbf{d}$ is a vector of spectral values representing the spectrum of the mixture $(1 \times J)$, $c_1, c_2, ..., c_A$ are concentrations of the pure components in the mixture and $\mathbf{s}_1, \mathbf{s}_2, ..., \mathbf{s}_A$ are the spectra of the pure components. If we combine the concentration values into a $1 \times A$ row-vector $\mathbf{c} = [c_1, c_2, ..., c_A]$ then the equation can be written in a more compact form:

$$\mathbf{d} = \mathbf{c}\mathbf{S}^T$$

Where $\mathbf{S}$ is a $J \times A$ matrix containing spectra of all pure components as columns.

Apparently, if we have more than one mixture and concentrations of the pure components vary, we can combine all concentration values into a matrix $\mathbf{C}$, where every row will correspond to a particular mixture. In this case we can write the equation as follows:

$$\mathbf{D} = \mathbf{C}\mathbf{S}^T$$

The task of the MCR methods is to get $\mathbf{C}$ and $\mathbf{S}$ by knowing $\mathbf{D}$, so we sort of resolve the mixtures into individual components and their concentrations. This is not a trivial task as the expression above does not have a unique solution. For example, one of the solutions is what PCA gives, but scores do not correspond to the real concentration values, nor do loadings represent the spectra of the pure components.

In fact, it is impossible to get $\mathbf{C}$ and $\mathbf{S}$ precisely, what we get is an estimate, which can be denoted as $\hat{\mathbf{C}}$ and $\hat{\mathbf{S}}$. In this case we can rewrite the equation as:

$$\mathbf{D} = \hat{\mathbf{C}}\hat{\mathbf{S}}^T + \mathbf{E}$$

Where $\mathbf{E}$ is a matrix with residuals.

So there are many different methods and tricks which help to get a decent solution in this case. In *mdatools*, starting from v. 0.11.0, there are two MCR methods available — based on the purity approach (`mcrpure()`),

also known as SIMPLISMA, and, based on constrained alternating least squares (`mcrals()`). This chapter explains how to use both for practical tasks.

More information about the MCR methods in general can be found in this book.

Starting from v. 0.11.0 the *mdatools* package contains additional dataset, `carbs`, which has three objects: `carbs$S` is a matrix (1401 × 3) of Raman spectra of three carbohydrates: fructose, lactose, and ribose; `carbs$D` contains 21 simulated spectra of their mixtures and `carbs$C` contains concentrations used to create the mixtures. The mixture spectra also contain some random noise, which is uniformly distributed between 0 and 3% of maximum intensity. The spectra of the pure components were taken from publicly available SPECARB library created by S.B. Engelsen. This dataset will be mainly used in this chapter to show how the implemented MCR methods work.

Purity based

The purity based approach implemented in *mdatools* was proposed by Willem Windig and co-authors in 2005 as an alternative to classical SIMPLISMA method. The general idea of the approach is to find variables (wavelength, wavenumbers, etc.) in $\mathbf{D}$, which are influenced mostly by one chemical component. Such variables are called *pure variables*. If we identify pure variable for each of the components, then we can solve the MCR problem by using the ordinary least squares method:

$$\hat{\mathbf{S}} = \mathbf{D}^T\mathbf{D}_R(\mathbf{D}_R^T\mathbf{D})^{-1}$$

$$\hat{\mathbf{C}} = \mathbf{D}\hat{\mathbf{S}}(\hat{\mathbf{S}}^T\hat{\mathbf{S}})^{-1}$$

Here $\hat{\mathbf{S}}$ and $\hat{\mathbf{C}}$ are the spectra and concentrations (contributions) of the pure components estimated by this method. The matrix $\mathbf{D}_R$ is a reduced version of $\mathbf{D}$ where only pure variables are kept (one for each pure component, so this matrix has a dimension $A \times A$). So the question is how to find the pure variables?

Windig and co-authors proposed to do it by computing angles between the spectral variables. For the first component angle between all variables and a vector of ones is computed and first pure variable is selected as the one having the largest angle. Then, angles between the first selected pure variable and the rest are computed and, again, the variable with the largest angle is selected as the purest. This continues until pure variables for all components are identified.

To reduce the influence of noisy variables, a correction factor is computed for each variable as follows:

$$\mathbf{n} = \frac{\mathbf{m}}{\mathbf{m} + offset \times \max(\mathbf{m})}$$

Here $\mathbf{m}$ is a mean spectrum computed for the original data, $\mathbf{D}$, and the $offset$ is a tuning parameter defined by a user. Usually a value between 0.001 and 0.05 is a good choice for the offset. More details about the method can be found in the above mentioned paper.

An example

Function `mcrpure()` implements the purity based method in *mdatools*. It has two mandatory arguments — matrix with the original spectra and number of pure components — as well as several extra parameters. The most important one is the offset, which by default is 0.05.

The code below shows how to resolve the spectra from the `carbs` dataset and show summary information:

```
data(carbs)
m = mcrpure(carbs$D, ncomp = 3)
summary(m)
```

```
##
## Summary for MCR Purity case (class mcrpure)
##        Expvar Cumexpvar Varindex Purity
## Comp 1  61.94     61.94      782 36.038
## Comp 2  24.96     86.91     1245 56.703
## Comp 3  12.65     99.56     1059 46.941
```

The summary shows explained variance, cumulative explained variance, index of pure variable and its purity value for each of the three components. For example, in case of the first component, the purest variable is located in column 782 and its purity (in this case an angle between this variable and a vector of ones) is equal to 36.038 degrees. As one can see, current solution explains 99.6% of the total variance.

Any MCR method in *mdatools* contains the resolved spectra (as `m$resspec`) and contributions (as `m$rescont`) as well as some extra data, e.g. purity spectra for each component, etc. You can see a list of all objects available by simply printing the object with *mcrpure* results:

```
show(m)
```

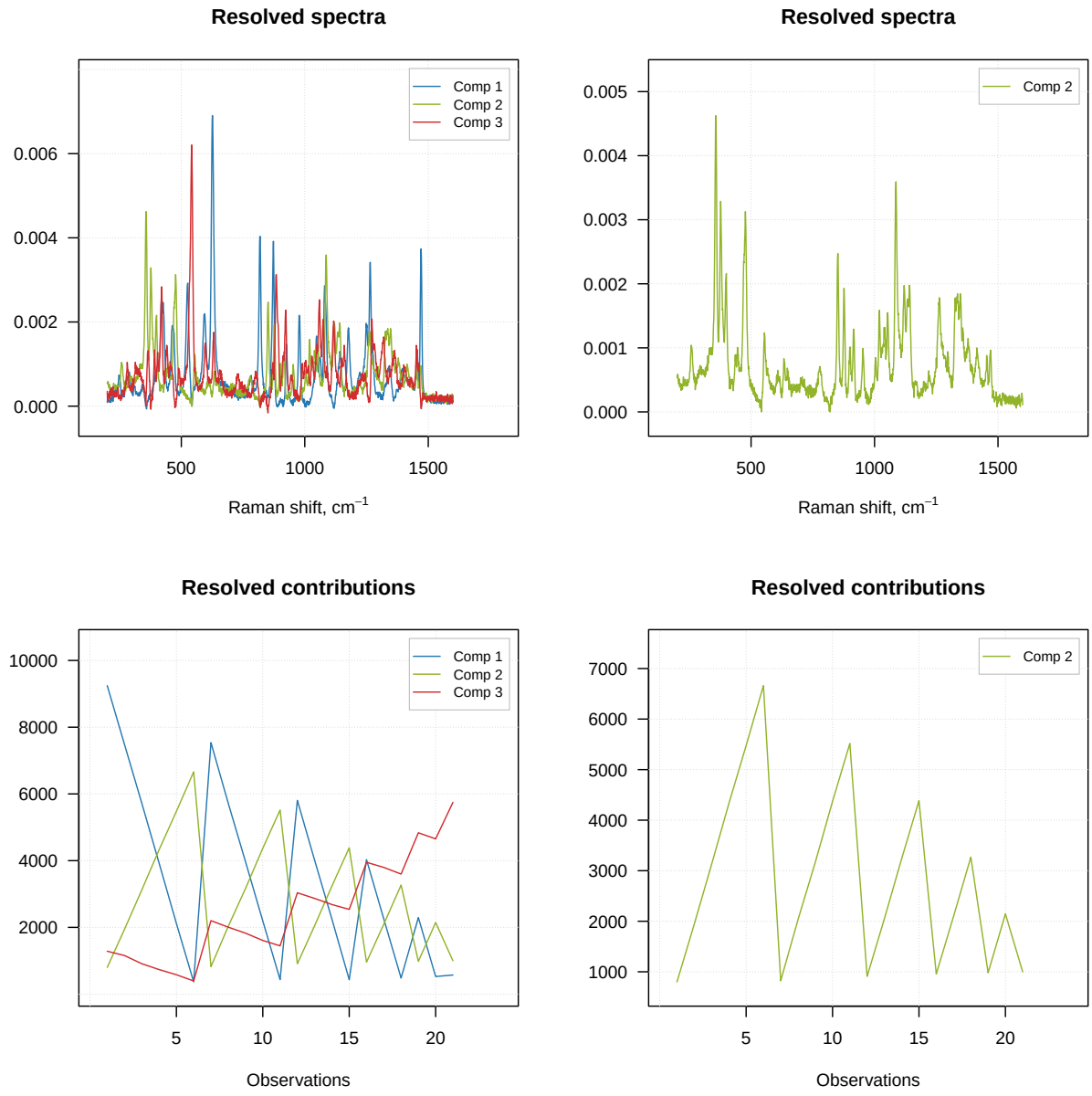
```
##
## MCR Purity unmixing case (class mcrpure)
##
##
## Call:
## mcrpure(x = carbs$D, ncomp = 3)
##
## Major model fields:
##  $ncomp - number of calculated components
##  $resspec - matrix with resolved spectra
##  $rescont - matrix with resolved contributions
##  $purityspec - matrix with purity spectra
##  $purevars - vector with indices of pure variables
##  $purevals - purity values for the selected pure variables
##  $expvar - vector with explained variance
##  $cumexpvar - vector with cumulative explained variance
##  $offset - offset value used to compute the purity
##  $info - case info provided by user
```

The reason we use word *contributions* instead of *concentrations* is that the method does not give the real concentrations, measured in the units of interest. The resolved values are rather in arbitrary units and can be later scaled/re-scaled accordingly.

The result object also has several graphical methods, which let the user plot the resolved spectra (`plotSpectra()`) and the resolved contributions (`plotContributions()`). The example below demonstrates how to show the plots for all three components as well as for a selected one.

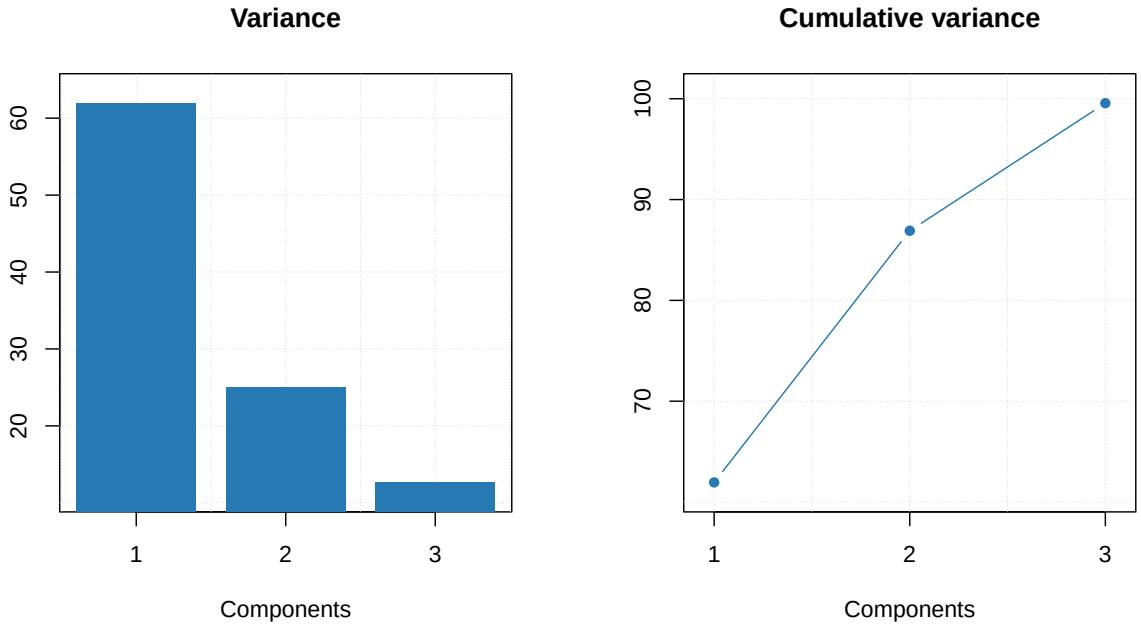
```
par(mfcol = c(2, 2))
plotSpectra(m)
plotContributions(m)

plotSpectra(m, comp = 2)
plotContributions(m, comp = 2)
```



As you can see, when you select particular components the plot preserves their color. You can also create the explained variance plot (both individual as well as cumulative) as it is shown below.

```
par(mfcol = c(1, 2))
plotVariance(m)
plotCumVariance(m)
```



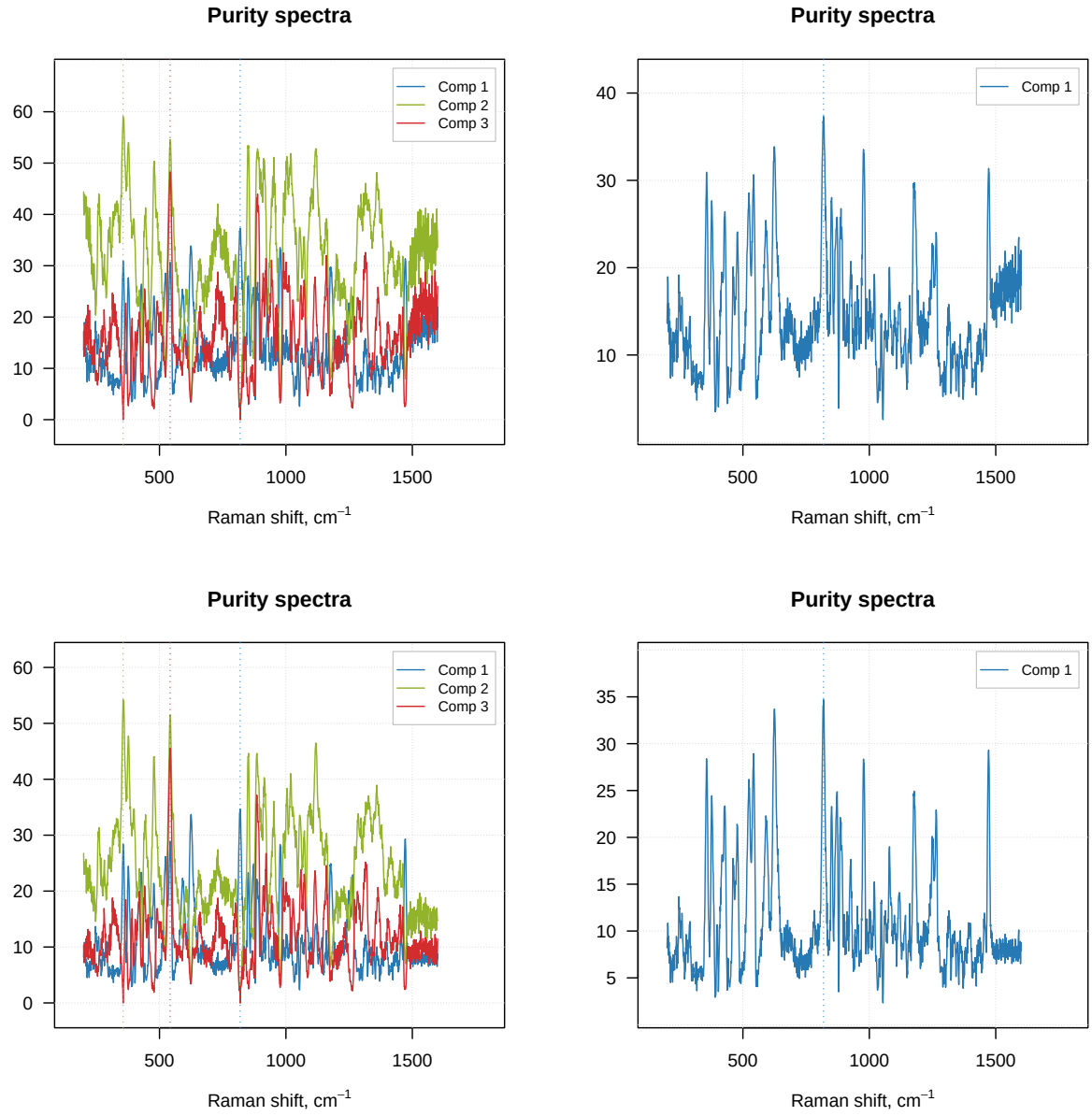
All plot parameters are similar to what you have used for other methods (e.g. PCA, PLS, etc), so you can change type of plot, colors, etc. in a similar way.

Purity values and spectra

The purity spectra and the purity values for each component are quite important and investigation of the spectra can be very useful. For example, the code below creates two solutions using different offset values and shows purity spectra for each solution. The plots on the left side show purity spectra for all three components, while the plots on the right side show purity only for the first component.

```
m1 = mcrpure(carbs$D, ncomp = 3, offset = 0.01)
m2 = mcrpure(carbs$D, ncomp = 3, offset = 0.10)

par(mfrow = c(2, 2))
plotPuritySpectra(m1)
plotPuritySpectra(m1, comp = 1)
plotPuritySpectra(m2)
plotPuritySpectra(m2, comp = 1)
```



The vertical dashed lines on the purity spectra plot show the selected pure variables. As we can see, they indeed correspond to the largest values in the corresponding purity spectrum (so have the highest purity). We can also notice that when offset is small (top plots, 1%) the purity spectra look quite noisy. On the other hand, using too large offset can lead to a selection of less pure variables, so this parameter should be selected with caution. It is always a good idea to vary the parameter and investigate all plots before the final decision.

Since dataset `carbs` contains spectra of pure components as well, we can compare the resolved spectra with the original ones as shown in the example below. To do that, we normalize both sets of spectra to a unit length to avoid a scaling problem. The original spectra are shown as red and thick curves while the resolved spectra are black and thin.

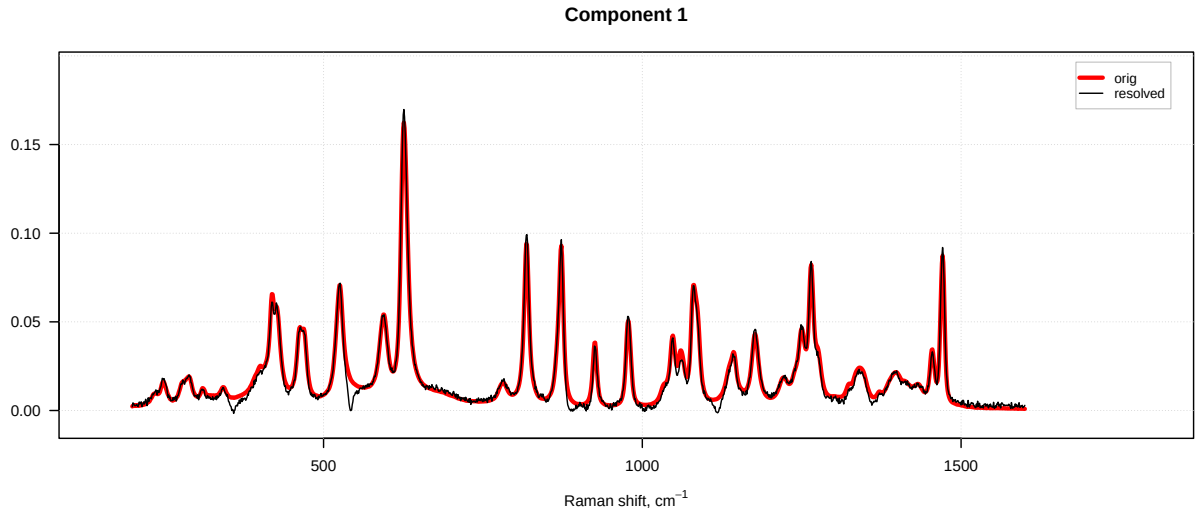
```
# apply purity method
m = mcrpure(carbs$D, ncomp = 3, offset = 0.05)

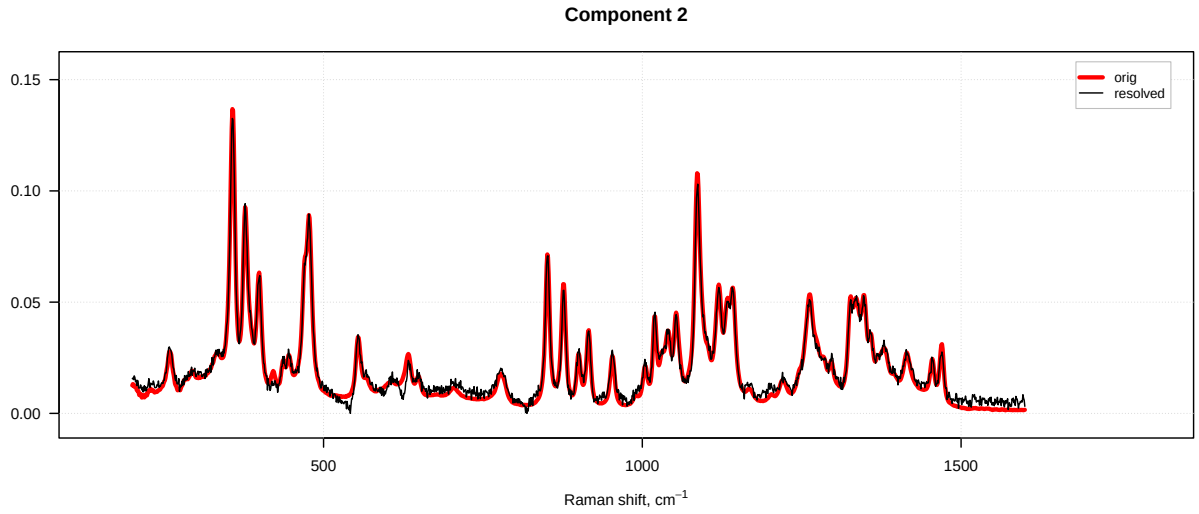
# get spectra, transpose and normalize to unit area
```

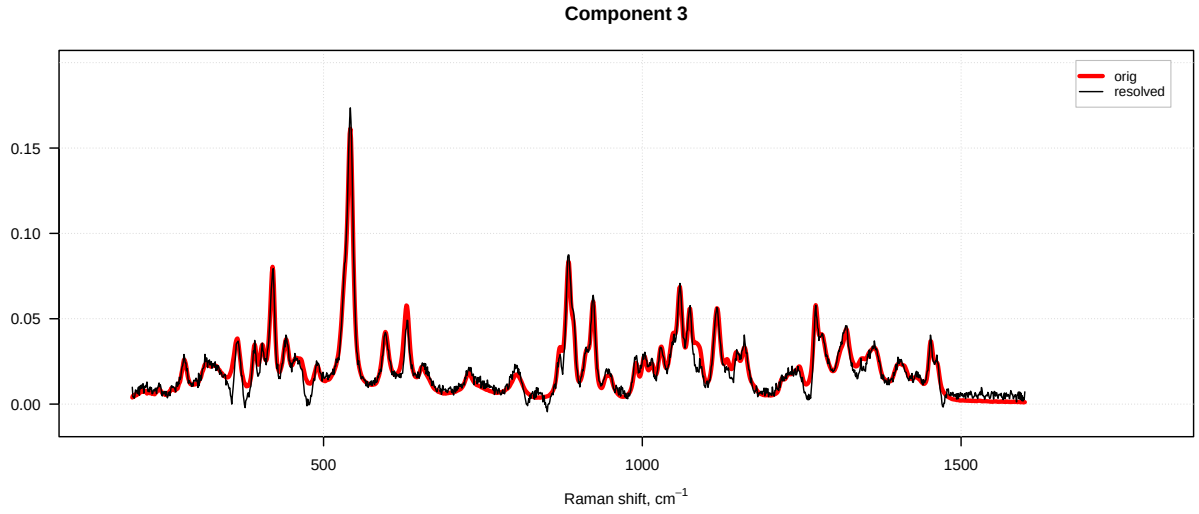
```
S      = prep.norm(mda.t(carbs$S), "length")
S.hat = prep.norm(mda.t(m$resspec), "length")

# define color and line width for the spectral curves
col = c("red", "black")
lwd = c(3, 1)

# show the plots separately for each component and each result object
par(mfrow = c(3, 1))
for (a in 1:3) {
   s = mda.subset(S, a)
   s.hat = mda.subset(S.hat, a)
   mdaplotg(list(orig = s, resolved = s.hat), type = "l", col = col, lwd = lwd,
      main = paste0("Component ", a))
}
```







In the code we use `mda.t()` and `mda.subset()` instead of just `t` and `subset()` to preserve values for Raman

shift and axis titles which are defined as attributes of the matrices with spectra, you can read more details in section about Attributes and factors.

As one can see, the quality of resolving is quite good in both cases. However, in case with offset = 10%, one can also notice large artifacts for example for the third component, which looks like a sort of negative peaks. You can try and play with the offset parameter and see how it influences the quality of the resolution.

Predictions

It is also possible to predict concentrations for one or several spectra based on already resolved data. In the code chunk below we predict the concentration values using the matrix with spectra of the pure components. Then we scale the predicted values, so they sum up to one and show the results.

```
c = predict(m, mda.t(carbs$S))
c = c / apply(c, 2, sum)
show(c)
```

```
##              Comp 1     Comp 2      Comp 3
## fructose 0.94329239 0.03701313 0.094968338
## lactose  0.02396884 0.86686908 0.003781956
## ribose   0.06152230 0.09344879 0.847020106
```

In ideal case we should see an identity matrix. However in this case, we got, for example, $[0.943, 0.024, 0.062]$ for the first component instead of expected $[1, 0, 0]$. The results are a bit worse for the second and third components.

Tuning the offset parameter

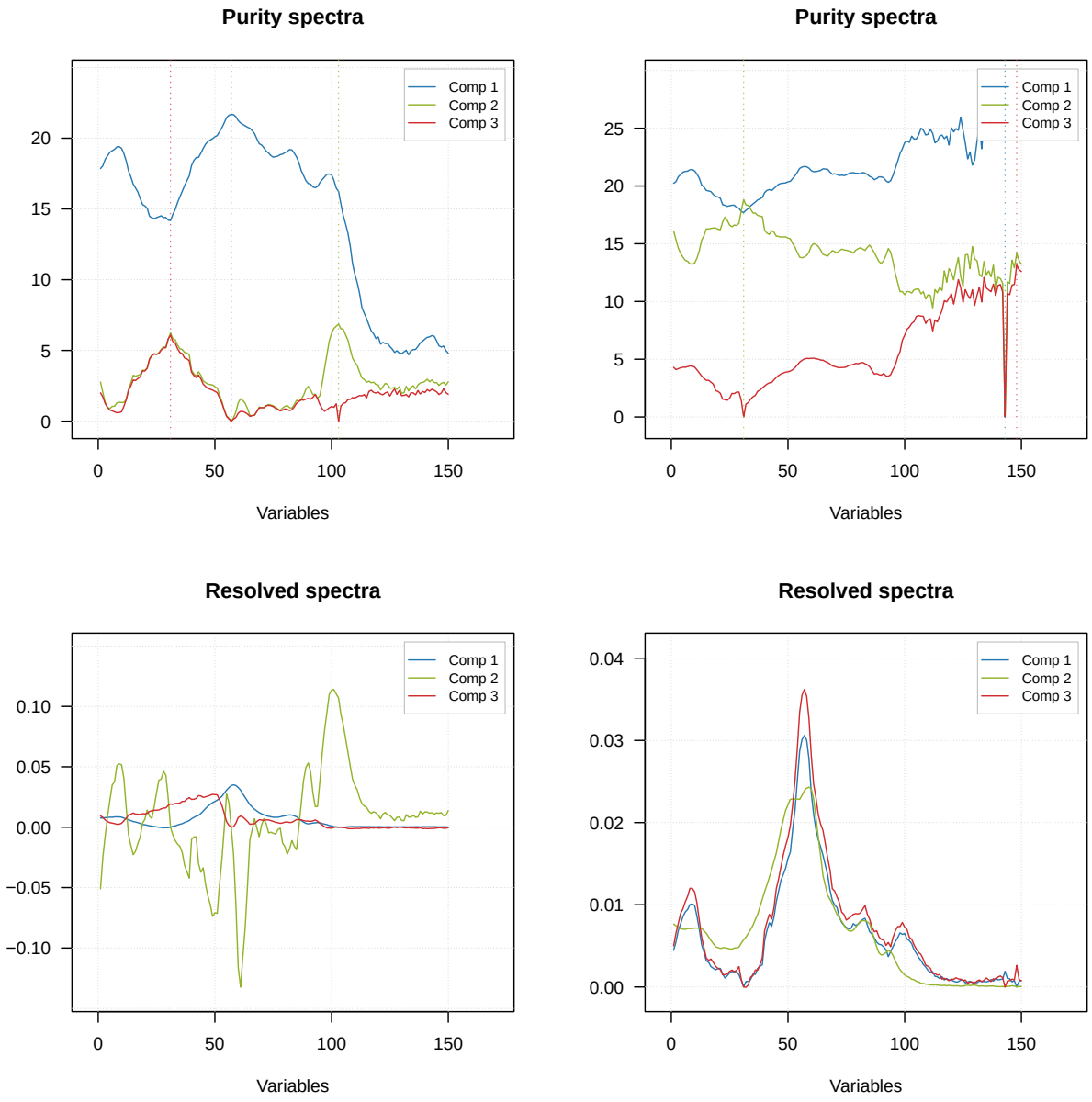
One of the ways to improve the result of resolution based on the purity approach is to tune the offset. The code below shows the result of applying `mcrpure()` to the *Simdata*, which consists of 150 UV/Vis spectra with very broad peaks.

```
data(simdata)
D <- simdata$spectra.c

m1 <- mcrpure(D, ncomp = 3)
m2 <- mcrpure(D, ncomp = 3, offset = 0.001)


par(mfrow = c(2, 2))
plotPuritySpectra(m1)
plotPuritySpectra(m2)

plotSpectra(m1)
plotSpectra(m2)
```



Using the default settings (left plots) does not allow to get any meaningful solutions (you can notice, for example, negative peaks). However, tuning the offset value results in a very good outcome — the resolved spectra are very similar to the original ones (original spectra are not shown on the plots).

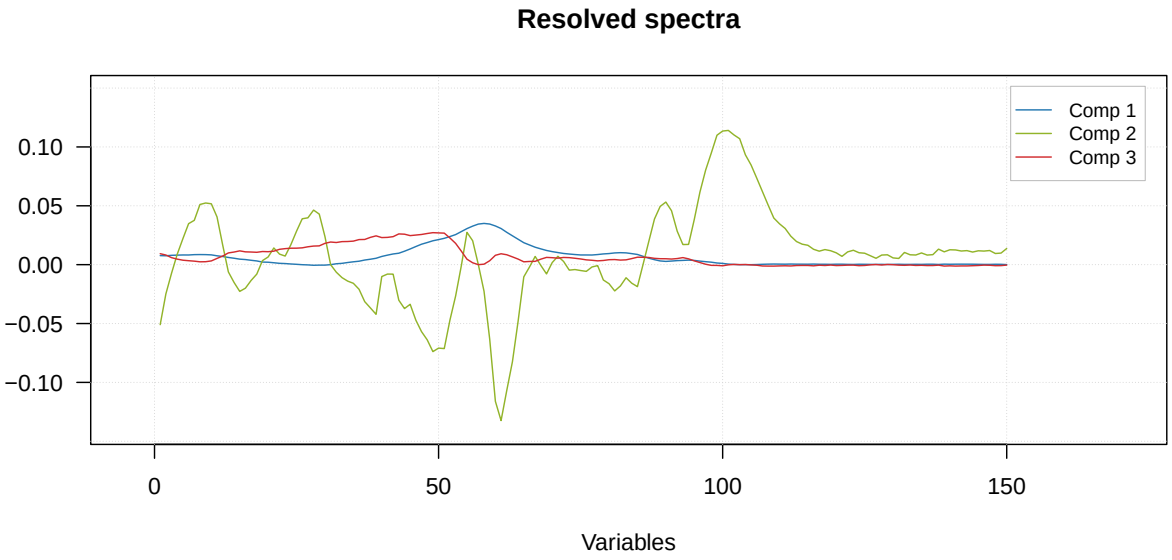
Providing indices of pure variables

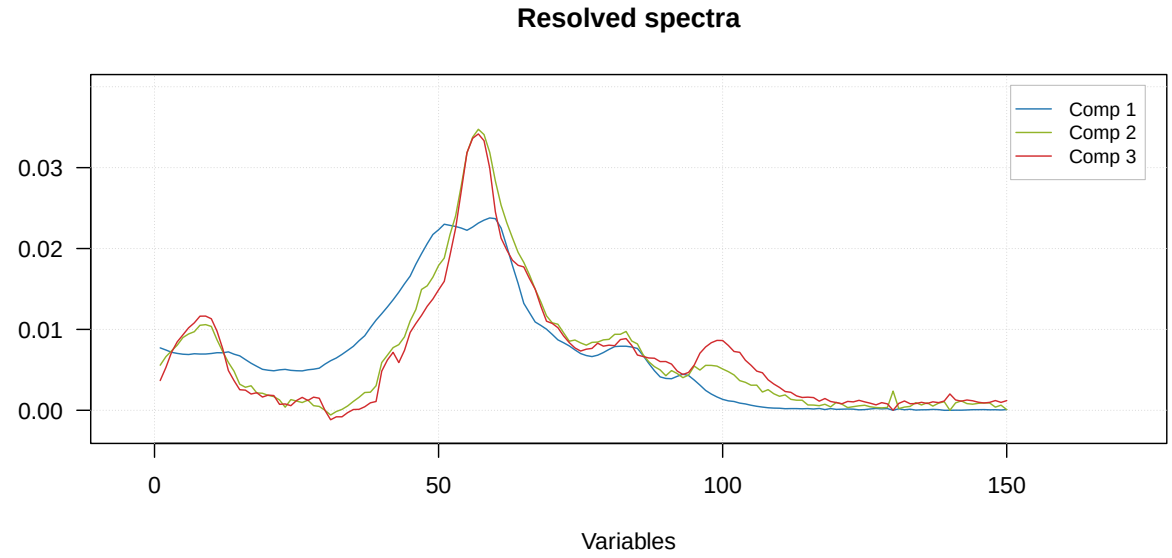
Finally you can provide the indices of pure variables if you know them a priory. In this case the method will skip the first step and try to resolve the spectra based on the provided values, as shown in the example below.

```
data(simdata)
D <- simdata$spectra.c

m1 <- mcrpure(D, ncomp = 3)
m2 <- mcrpure(D, ncomp = 3, purevars = c(30, 130, 140))
```

```
par(mfrow = c(2, 1))
plotSpectra(m1)
plotSpectra(m2)
```






```
summary(m1)
```

```
##
## Summary for MCR Purity case (class mcrpure)
##        Expvar Cumexpvar Varindex Purity
## Comp 1  94.75     94.75       57 21.687
## Comp 2  -0.36     94.39      103  6.857
## Comp 3   5.61    100.00       31  6.073
```

```
summary(m2)
```

```
##
## Summary for MCR Purity case (class mcrpure)
##        Expvar Cumexpvar Varindex Purity
## Comp 1  89.69     89.69       30 14.210
## Comp 2   6.15     95.85      130  2.452
## Comp 3   4.15     99.99      140  0.787
```

Alternating least squares

The alternating least squares allows to get $\hat{\mathbf{C}}$ and $\hat{\mathbf{S}}$ by using iterative algorithm, when the ordinary least squares is applied consequently, first to resolve the concentrations by knowing spectra and then to resolve the spectra by using the resolved concentrations:

$$\hat{\mathbf{C}} = \mathbf{D}\hat{\mathbf{S}}(\hat{\mathbf{S}}^T\hat{\mathbf{S}})^{-1}$$

$$\hat{\mathbf{S}} = \mathbf{D}^T\hat{\mathbf{C}}(\hat{\mathbf{C}}^T\hat{\mathbf{C}})^{-1}$$

These two steps continue until a convergence criteria is met. Apparently, there are several issues with the algorithm, that have to be clarified. First of all, to run the first iteration we need to have values for the matrix $\hat{\mathbf{S}}$. This is what is called *initial estimates*. In *mdatools* the initial estimates for the pure component spectra generated automatically as a matrix with random values taken from uniform distribution (between 0 and 1). The matrix $(\hat{\mathbf{S}}^T\hat{\mathbf{S}})$ should not be singular and using random values will ensure this.

On the other hand, using random values does not always provide reproducible results, so some alternatives can be considered. Therefore user can provide any pre-computed values for $\hat{\mathbf{S}}$ using parameter `spec.ini` as will also be shown below.

The second issue is that using ordinary least squares (OLS) method for the iterations will result in a solution with negative values, which is not physically meaningful. To tackle this problem we either need to constrain the solution, for example by setting all negative values to zero, after each iteration, or by using non-negative least squares for solving the equations. In *mdatools* both options are available.

Finally, as we mentioned before, curve resolution problem does not have a unique solution. To narrow the range of feasible solutions down, and also to move them towards the right one, we can apply different constraints. Some of the constraints are implemented and available for user, some will be implemented later. However, user can easily provide a manual constraint function as it will be shown below.

We will take all these issues gradually and let's start with non-negativity and constraints.

Non-negativity

To tackle the non-negativity problem, in *mdatools* you can choose one of the following options:

1. Use standard OLS solver and apply non-negativity constraint by setting all negative values to zero.
2. Use non-negative least squares solver (NNLS), e.g. proposed by Lawsen and Hanson.
3. Use its faster version — Fast Combinatorial NNLS (FC-NNLS) proposed by Benthem and Keenan.
4. Use your own solver

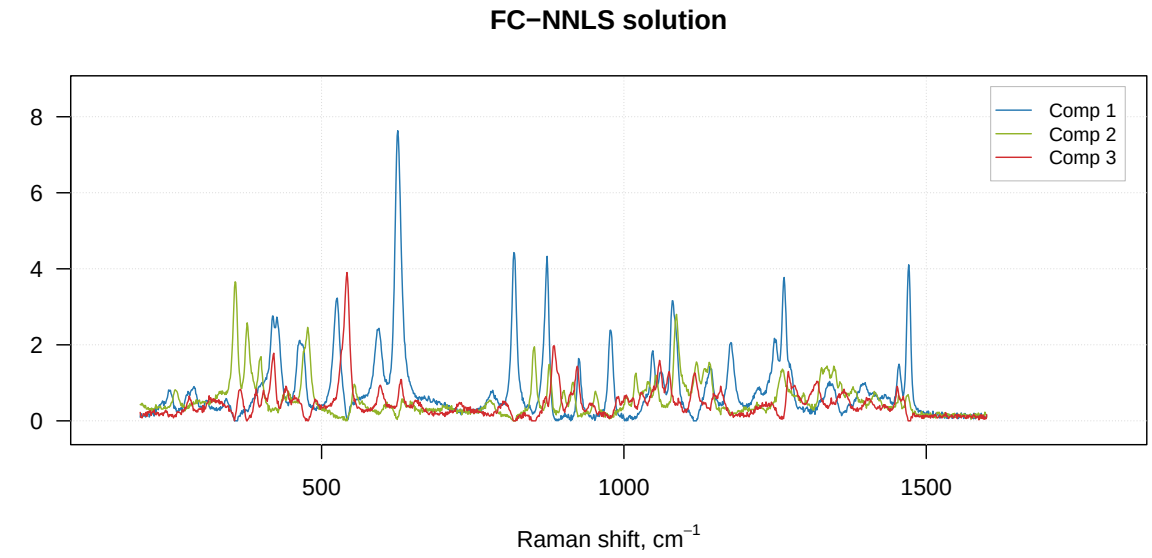
The default option is nr. 3 and this is what we would recommend to use. It gives a solution identical to nr. 2 however is much faster in case of multivariate data. You can also provide a function, which implements your own solver. The solver can be changed both for spectra and for contributions, you can even use different solvers for each. The code below shows how to apply MCR-ALS using OLS and FC-NNLS solvers without any constraints.

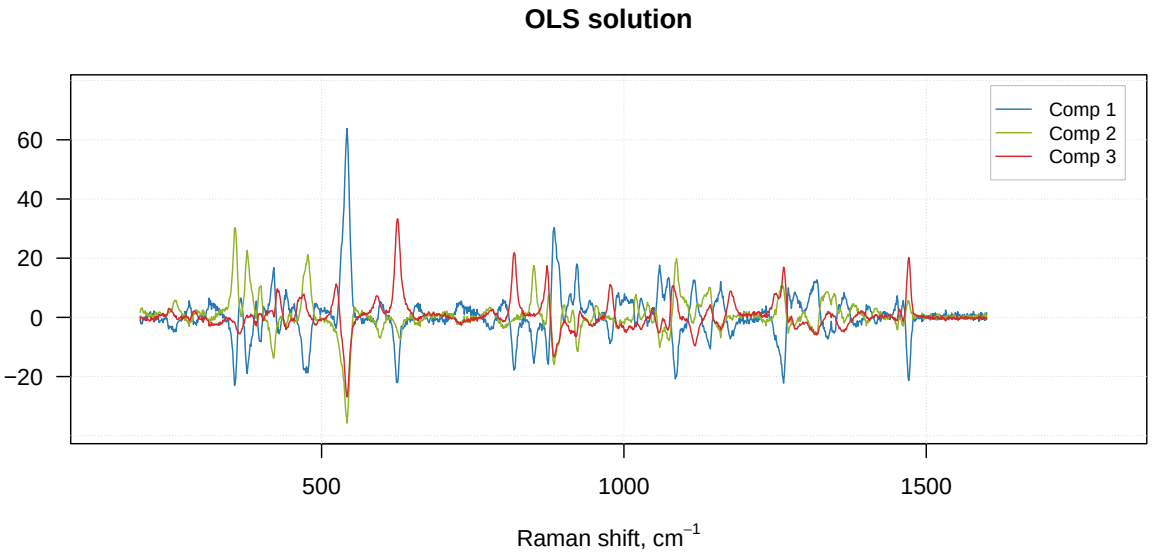
```
data(carbs)

# apply MCR-ALS with default solver (FC-NNLS)
m1 = mcrals(carbs$D, ncomp = 3)

# apply MCR-ALS with OLS solver
m2 = mcrals(carbs$D, ncomp = 3, spec.solver = mcrals.ols, cont.solver = mcrals.ols)

# show the resolved spectra
par(mfrow = c(2, 1))
plotSpectra(m1, main = "FC-NNLS solution")
plotSpectra(m2, main = "OLS solution")
```





As you can see, the OLS solution without non-negativity constraint contains negative values. Once again, if

you do not want to experiment with negative values, using the default option (FCNNLS) will be the best.

Constraints

Constraints are small functions that can be applied separately to current estimates of spectra or contributions on every step of the algorithm in order to improve the solution. A constraint function takes the matrix with spectra or contributions as an input, does something with the values and returns the matrix of the same size as an output. For example, the simplest constraint is non-negativity, which sets all negative values in the matrix to zeros.

In *mdatools* constraints can be combined together using lists. Spectra and contributions should have separate sets of constrains. In the example below we show how to use some of the built in constraints:

```
# define constraints for contributions
cc = list(
   constraint("norm", params = list(type = "sum"))
)

# define constraints for spectra
sc = list(
   constraint("angle", params = list(weight = 0.05)),
   constraint("norm", params = list(type = "length"))
)

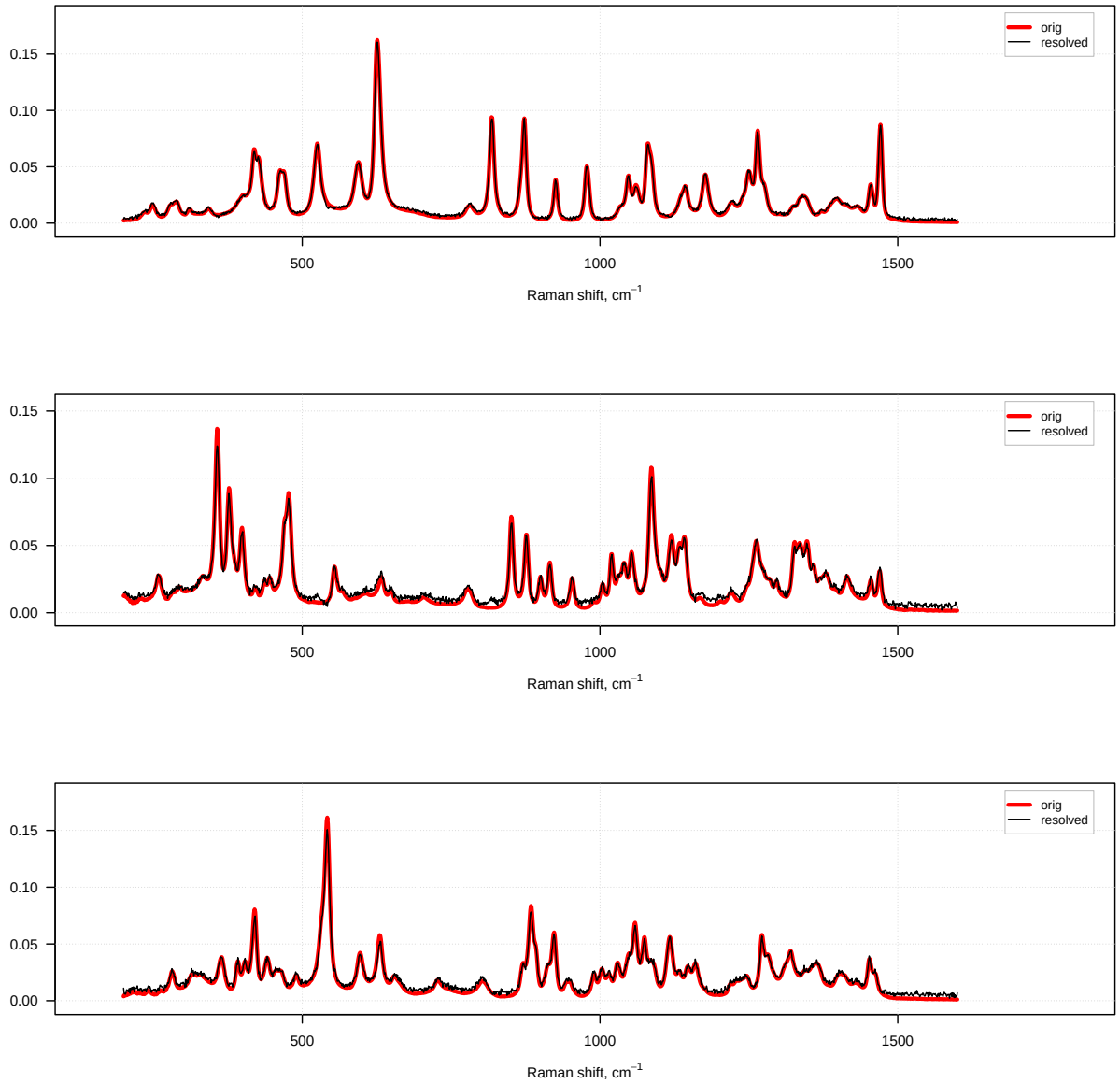
# run MCR-ALS with the constraints
set.seed(6)
m = mcrals(carbs$D, ncomp = 3, spec.constraints = sc, cont.constraints = cc)

# normalzie the original spectra to unit length
S = prep.norm(mda.t(carbs$S), "length")

# get resolved spectra and transpose them
S.hat = mda.t(m$resspec)

# define color and line width for the spectral curves
col = c("red", "black")
lwd = c(3, 1)

# show the plots separately for each component and each result object
par(mfrow = c(3, 1))
for (a in 1:3) {
   s = mda.subset(S, a)
   s.hat = mda.subset(S.hat, a)
   mdaplotg(list(orig = s, resolved = s.hat), type = "l", col = col, lwd = lwd)
}
```



As you can see, we use one constraint to normalize the contributions, so they sum up to one on every step. And we use two constraints for the spectra — first angle constraint, which reduces contrast among the spectra and increases contrast among the resolved concentration profiles. Then we normalize the spectra to the unit length. The constraints are applied to the spectra or contributions on every iteration and in the same order as they are defined in the list.

You may have noticed the following instruction: `set.seed(6)` in the code block above. This is needed to make results more reproducible. Since the initial estimates for the resolved spectra are obtained using random number generator, the final result can be different from run to run. To avoid this we seed the generator with number 6, so the final result will be the same regardless how many times we run this code.

The list of available constraints and their parameters can be shown by running `constraints.list()`:

```
constraints.list()
```

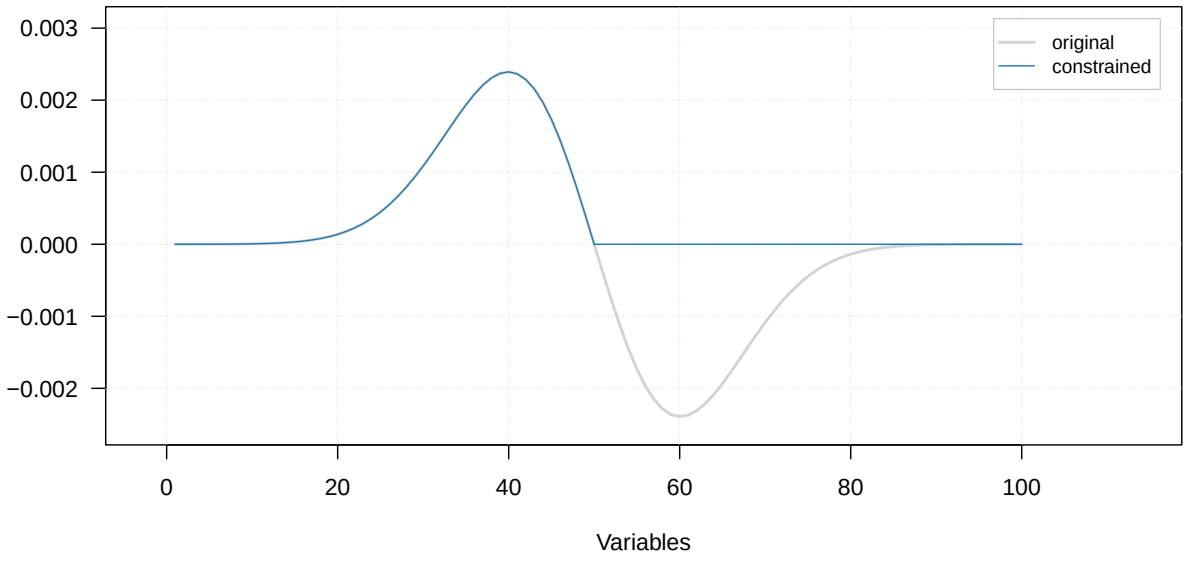
```
##
```

```
## List of constraints available for mcrals():
##
##
##  Non-negativity (sets negative values to zero)
##  ---------------
##  name: 'nonneg'
##  no parameters required
##
##
##  Unimodality (forces contribution or spectral profile to have a single maximum)
##  ---------------
##  name: 'unimod'
##  parameters:
##   'tol': tolerance (between 0 and 1)
##
##
##  Closure (forces contributions or spectral profiles sum up to constant value)
##  ---------------
##  name: 'closure'
##  parameters:
##   'sum': value, the data rows should sum up to
##
##
##  Normalization (normalize spectra or contributions)
##  ---------------
##  name: 'norm'
##  parameters:
##   'type': type of normalization: 'length', 'area' or 'sum'
##
##
##  Angle (increases contrast among resolved spectra or contributions)
##  ---------------
##  name: 'angle'
##  parameters:
##   'weight': how much of mean will be added: between 0 and 1
```

In the next few subsections you will find a short description of the currently implemented constraints with some illustrations, as well as instructions how to make a user defined constraint.

Non-negativity constraint

The non-negativity constraint simply takes a signal (spectra or contributions for a particular component) and set all negative values to zeros. A plot below demonstrates how the constraint works, the gray line represents the original signal, the blue — signal after applying the constraint.



It does not have any parameters, so to add the constraint to the list simply use `constraint("nonneg")`, e.g.:

```
cc <- list(
   constraint("nonneg")
)
```

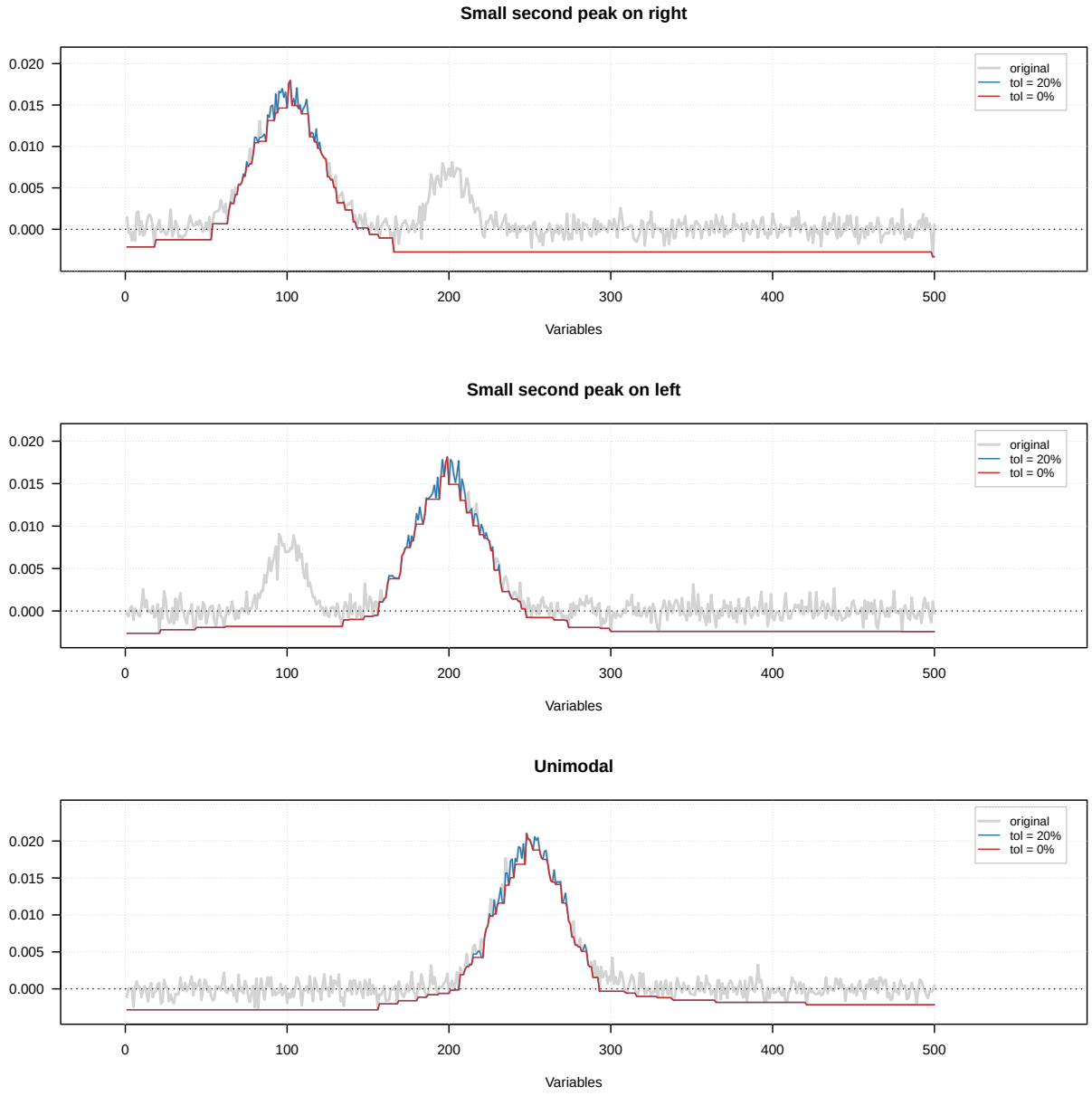
Unimodality constraint

The unimodality constraint is used to force signals having only one peak (maximum). This can be particularly useful for constraining contribution profiles, e.g. when spectra came from a reaction and it is a priori known that concentration developing of components has one peak.

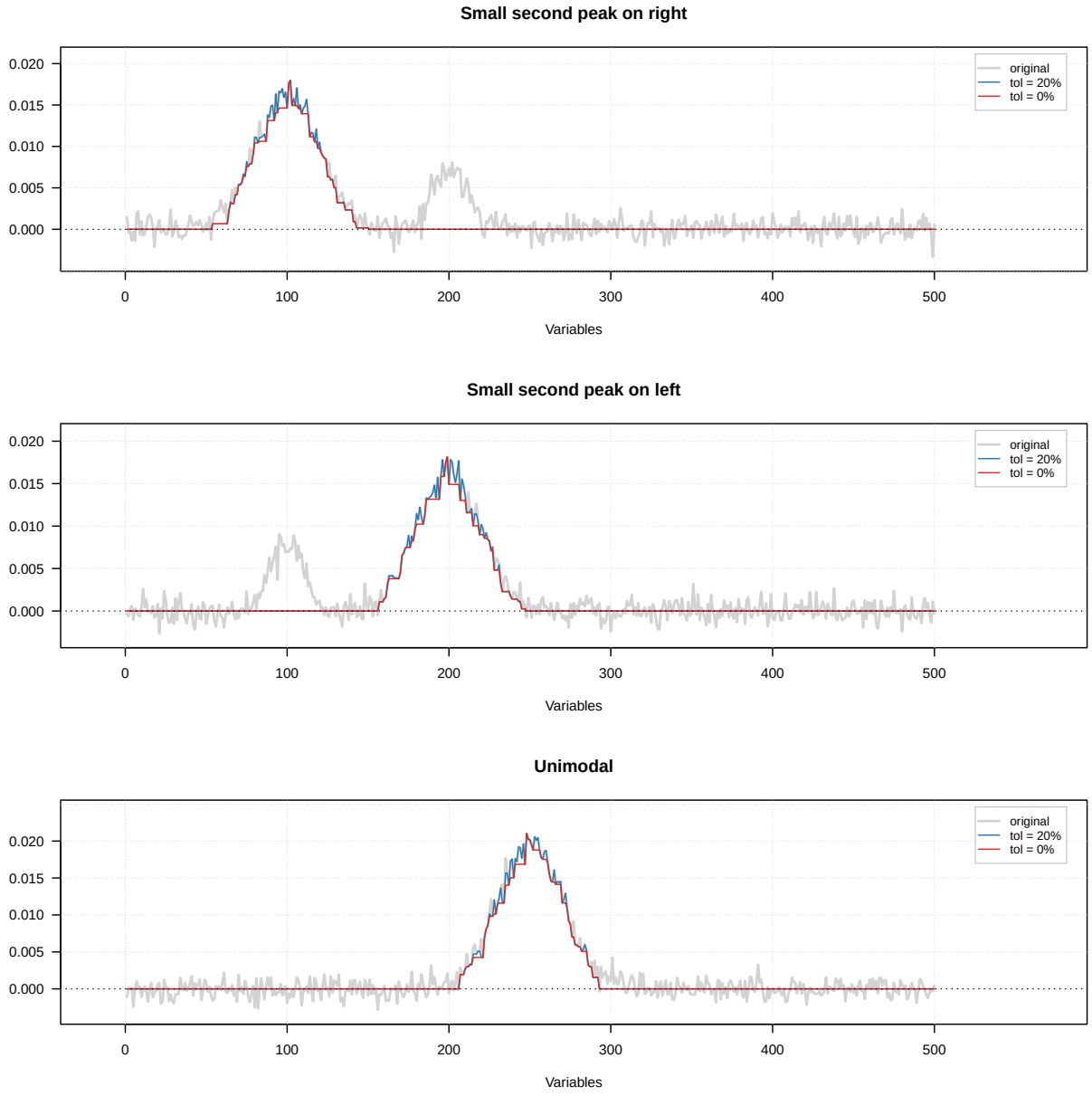
The constraint works as follows. First it finds a global maximum in the signal. Then it goes along each side from the peak and checks if intensity of next point in the signal does not exceed the value of the previous one. If it does, constraint will replace the current value by the previous one.

It is also possible to define a tolerance — value between 0 and 1, which allows taking into account small fluctuation of the intensity due to e.g. noise. Thus if tolerance is 0.05, then only when intensity of current point exceeds the intensity of the previous point by more than 5% it will trigger the constraint.

The figure below shows how the constraint works using three signals — one with small extra peak on the left, one with small extra peak on the right and one unimodal signal. A small amount of noise was added to all signals. There are one original signal (gray) and two constrained signals on the plot. The red one shows the result for 0% tolerance and the blue one shows the result for 20% tolerance.



As we can see, because of the noise, the constrained profiles have negative values. In this case it makes sense to combine the unimodality and non-negativity constrains. The result of this combination is shown below.



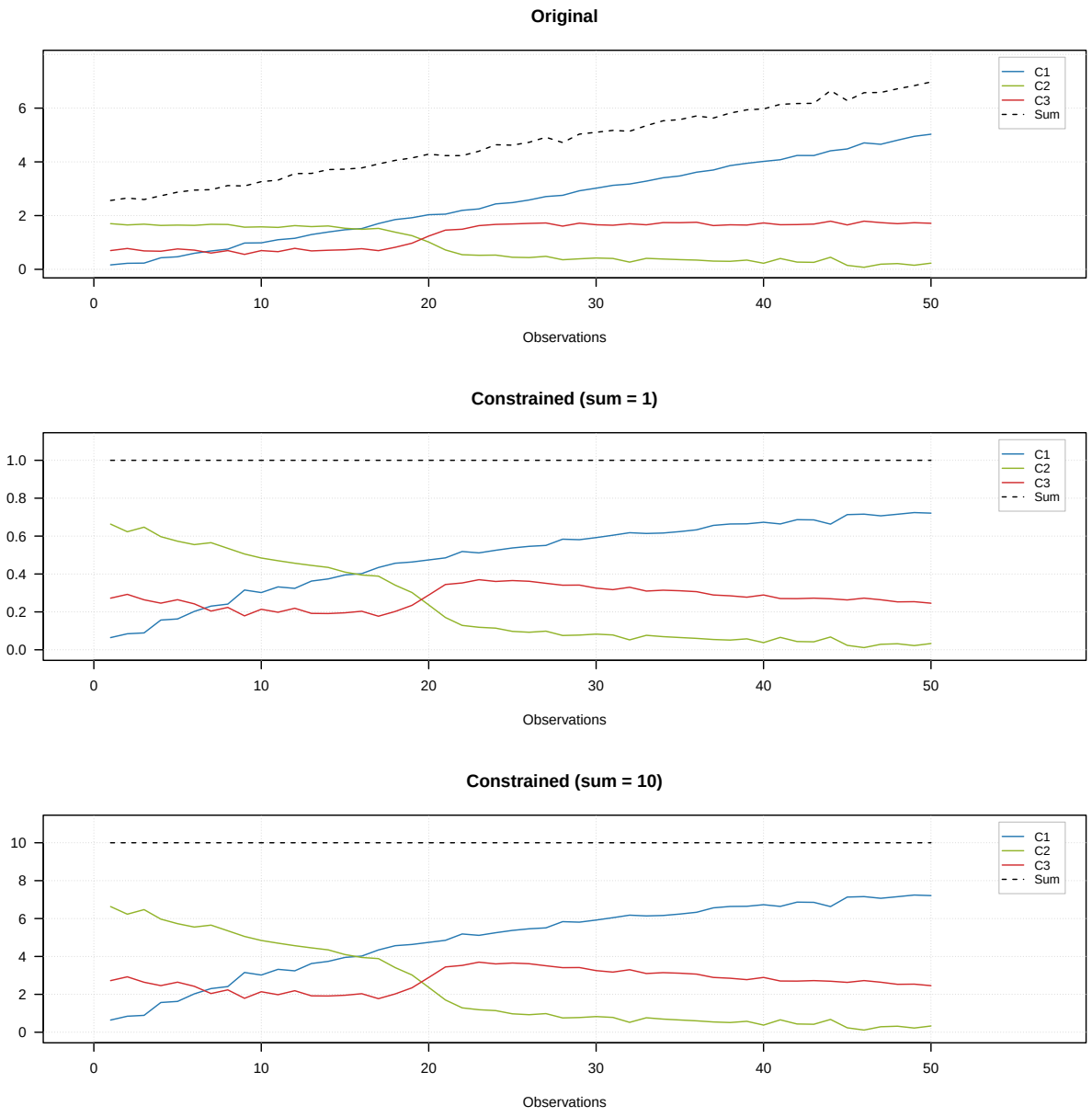
In order to add the constraint use `constraint("unimod")` for zero tolerance or `constraint("unimod", params = list(tol = 0.05))` for 5% tolerance. In the code chunk below you see how to create a set of unimodality and non-negativity constrains e.g. for contribution profiles

```
cc <- list(
   constraint("unimod", params = list(tol = 0.05)),
   constraint("nonneg")
)
```

Closure constraint

Closure constraint, like unimodality, is mostly applicable to the contribution profiles and aims to preserve the mass balance — so the sum of the concentrations of pure components is constant. From that point of view the closure constraint can be thought of as normalization made along the observations (or wavelength/wavenumbers in case of spectra) instead of along the components.

The figure below shows how it works using contribution profiles of three components. The colored lines show the profiles and the black dashed line shows the sum of contributions for a given measurements.



The top plot shows the original profiles. The middle plot shows the constrained profiles, where the closure constraint, which limits sum of the contributions to one was applied. The bottom plot shows results for the constraint limiting the sum to 10.

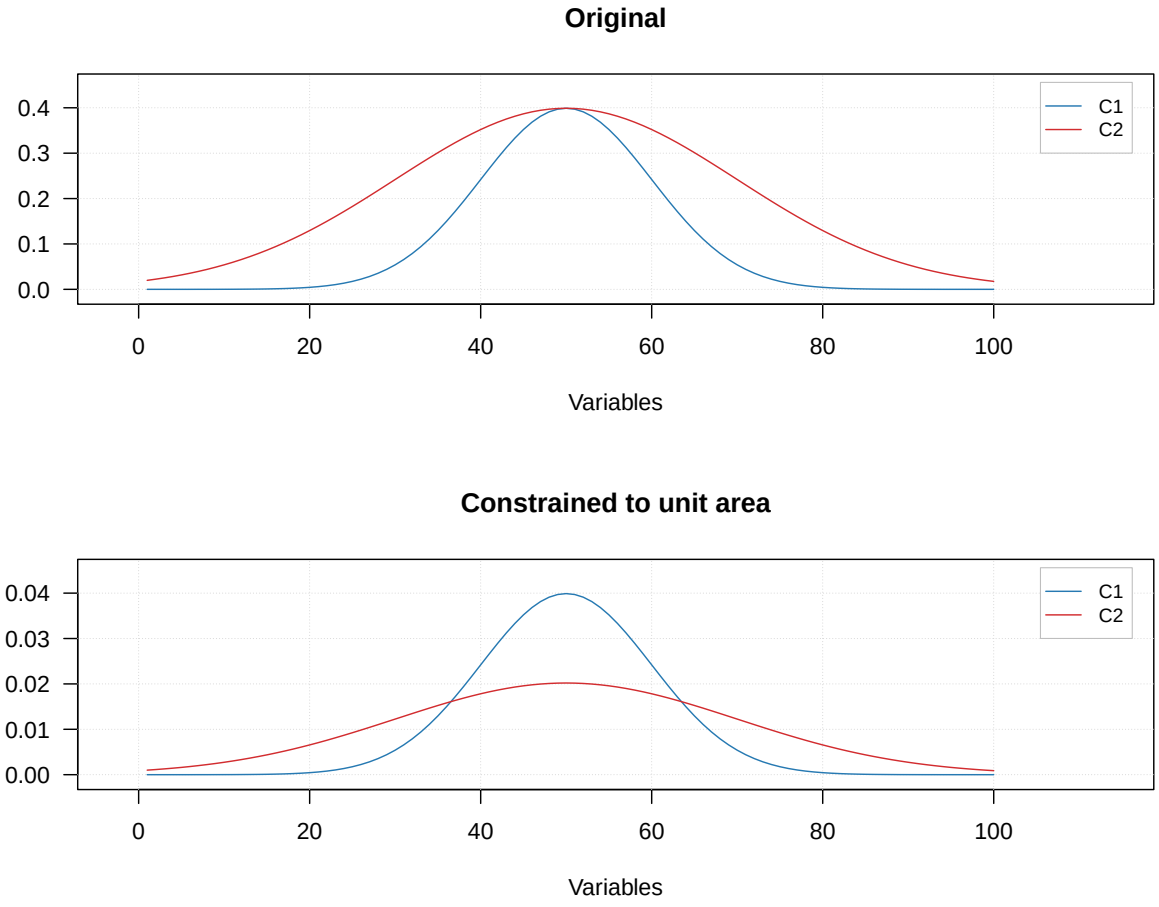
As you can see in the last two plots, the dashed line show the constant desired value for all measurements. In order to add the constraint use `constraint("closure")` for unit sum or `constraint("closure", params = list(sum = 10))` for e.g. 10.

Normalization constraint

Normalization constraint aims at making spectra or contribution profiles of individual components normalized. The normalization can be made by unit length, area or sum. For example, in case of sum normalization, sum

of all values of each spectrum or contribution profile will be equal to one.

An example below shows two original signals and the result of normalization of the signals to the unit area (so area under each curve is equal to one).



The constraint can be added using `constraint("norm", params = list(type = "area"))`. The value for parameter `type` can be `"area"`, `"length"`, or `"sum"`

Angle constraint

Angle constraint allows to change a contrast among the contribution profiles or resolved spectra. The idea was proposed by Windig *et al.* and all details can be found in this paper.

The constraint works as follows. First, mean spectrum or mean contribution profile are computed using the original data, **D**. Then, on every iteration, a portion of the mean signal (e.g. 5%) is added, for example, to the resolved contributions. This will lead to smaller contrast among the contribution profiles but, at the same time, will increase the contrast for the resolved spectra. And vice versa, to increase the contrast among the resolved contribution profiles, the constraint should be applied to the spectra.

The constraint can be added using `constraint("angle")`. By default the portion of mean to be added to the signals is 5% (0.05). To change this value to, e.g. 10%, use `constraint("angle", params = list(weight = 0.10))`.

User defined constraints

We plan to implement more constraints gradually. However you do not need to wait and can use your own constraints, if necessary. This section shows how to do it step by step and how to combine user defined constraint with the implemented ones.

Let's say, we want to add a random noise to our resolved spectra on each iteration (sounds very stupid, but let's use it just for fun in this example). So, first we need to create the constraint function. It should have two mandatory arguments: `x` which will take current estimates of spectra or contributions and `d` which is original data used for resolving. There can be any number of optional arguments, in our case we will use argument `noise.level` which will define a level of noise in percent of maximum value in `x`:

Here is the function:

```
# constraint function which adds random noise from normal distribution
myConstraintFunction <- function(x, d, noise.level = 0.01) {
   nr <- nrow(x)
   nc <- ncol(x)
   noise <- rnorm(nr * nc, mean = 0, sd = max(x) * noise.level)

   return(x + matrix(noise, nr, nc))
}
```

As you can see we use normal distribution for generating the noise, with zero mean and standard deviation equal to the 1% of maximum value by default. Load this function to the global environment by running this code.

Next we need to create a constraint object, which will use this function, and set the required level of noise, like it is shown here (we take 2% instead of the default value of 1%):

```
# create a constraint with noise level of 2%
myConstraint = list(method = myConstraintFunction, params = list(noise.level = 0.02))
class(myConstraint) = "constraint"
```

Next we need to define sets of constraints for contributions and for spectra by using lists, like we did for built in constraints, and add our manual constraint object to the list:

```
# constraints for contributions
cc = list(
   constraint("norm", params = list(type = "sum"))
)

# constraints for spectra
sc = list(
   myConstraint,
   constraint("norm", params = list(type = "length"))
)
```

Finally, we just run the MCR-ALS algorithm and check the results. I will again seed the random number generator to make results reproducible.

```
# run mcrals
set.seed(6)
m <- mcrals(carbs$D, ncomp = 3, cont.constraints = cc, spec.constraints = sc)

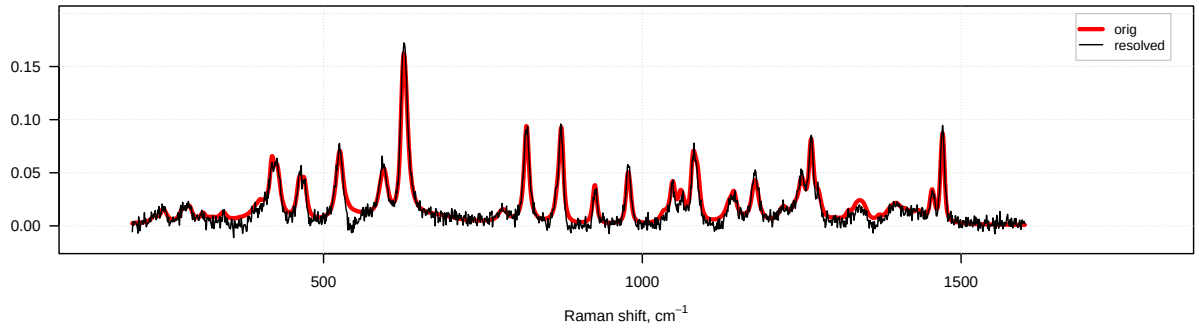
# normalize the original spectra to unit length
S = prep.norm(mda.t(carbs$S), "length")

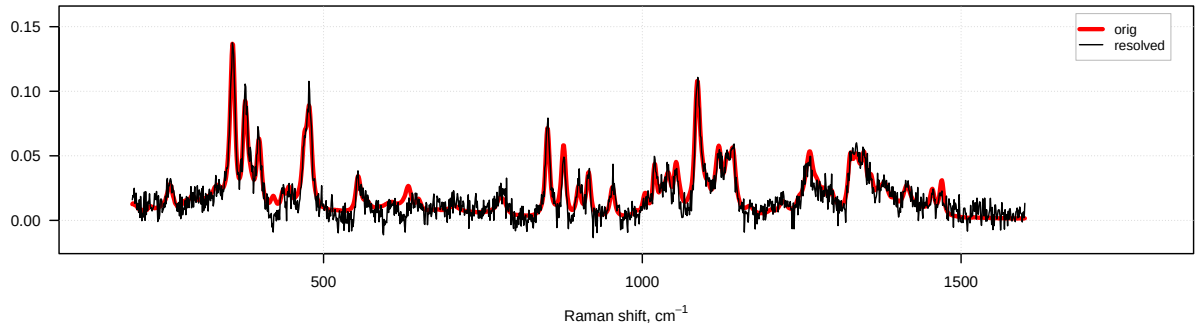
# get the resolved spectra and transpose them
```

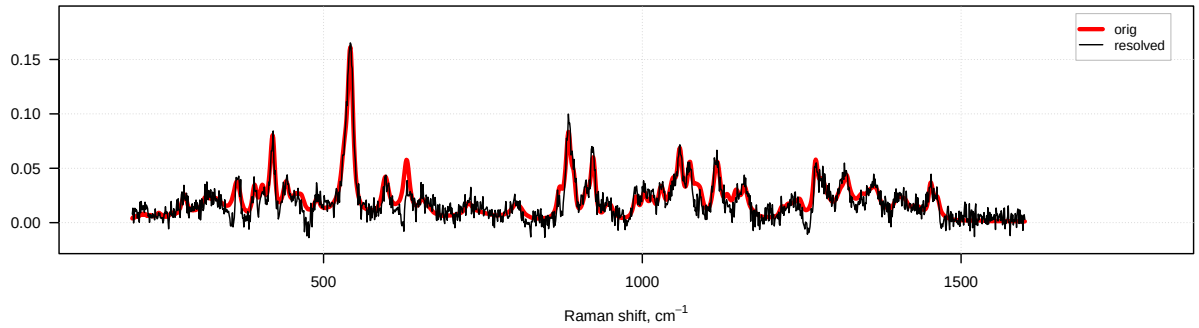
```
S.hat = mda.t(m$resspec)

# define color and line width for the spectral curves
col = c("red", "black")
lwd = c(3, 1)

# show the plots separately for each component and each result object
par(mfrow = c(3, 1))
for (a in 1:3) {
   s = mda.subset(S, a)
   s.hat = mda.subset(S.hat, a)
   mdaplotg(list(orig = s, resolved = s.hat), type = "l", col = col, lwd = lwd)
}
```







As you can see, even with this rather strange constraint the method works, although the resolved spectra are quite noisy. The presence of negative values in the resolved spectra is because we use normal distribution and the constraint is applied after the solver, which provides non-negative solution.

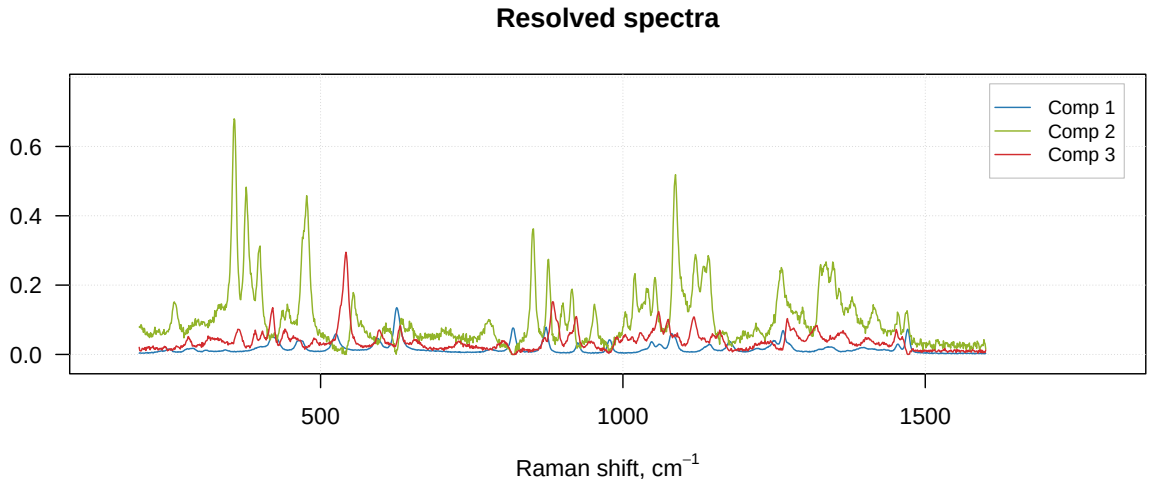
Initial estimates

As it was mentioned above, one can provide any initial estimates for the spectra as a matrix with size $J \times A$, where A is the number of components and J is number of variables (columns) in the original data. However, the matrix with provided values should not be singular, otherwise the algorithm will fail.

In the example below we provide absolute values of PCA loadings as the initial estimates. No constraints are provided in this case for the sake of simplicity.

```
m.pca = pca(carbs$D, ncomp = 3, center = FALSE)
m = mcrals(carbs$D, ncomp = 3, spec.ini = abs(m.pca$loadings))

plotSpectra(m)
```



Forcing contribution and spectral values

Sometimes we know the exact concentration of components or their absence for given measurements. For example we may know the concentration of ingredients at the beginning of a reaction or measure concentration of reaction products at the end. Same for the spectra, e.g. we know wavenumbers or wavelength where no peaks are expected for a particular component. In order to provide this information we can let `mcrals()` force these values to be equal to zero. Actually it can be any number, but since the resolved contributions and spectral intensities are relative, you need to be careful with providing the non-zero values.

In order to use this possibility you need to provide values for two parameters: `cont.forced` for the contributions and `spec.forced` for the spectra. For example if we resolve spectra and contributions for three component system and we do not expect presence of third component at the beginning of a reaction, we can set the values as follows:

```
cont.forced = matrix(NA, nrow(D), ncomp)
cont.forced[1, 3] = 0
```

The first line in this chunk creates a matrix and fills it with missing (`NA`) values. The matrix has the same dimension as future matrix with resolved contributions — same number of rows as the matrix with mixtures, `D`, and same number of columns as the expected number of pure components.

After that, we set a value at the first row and the third column to zero. Which means "force concentration of the third component for the first observation to zero". Inside the ALS loop, on every iteration the `mcrals()` method will replace the resolved concentration with the provided value.

Here is an example, based on the carbs data we used before. In this case I add the pure spectra on top of the matrix with mixtures. And since these are spectra of pure components, I know that the spectrum for the first observation does not contain any of C2 and C3, the second observation does not contain C1 and C3 and, finally, the third observation does not contain C1 and C2. Here is how to do this:

```
ncomp <- 3
Dplus <- mda.rbind(mda.t(carbs$S), carbs$D)

cont.forced = matrix(NA, nrow(Dplus), ncomp)
cont.forced[1, ] = c(NA, 0, 0)
cont.forced[2, ] = c(0, NA, 0)
cont.forced[3, ] = c(0, 0, NA)

m <- mcrals(Dplus, ncomp = ncomp, cont.forced = cont.forced)
```

Here are the resolved contributions:

```
show(head(m$rescont))
```

```
##         Comp 1     Comp 2     Comp 3
## [1,] 7.6462723  0.0000000 0.11344608
## [2,] 0.1331184 13.1168977 0.09604429
## [3,] 0.1492307  0.0000000 7.61280985
## [4,] 7.8180191  0.6992354 0.64461538
## [5,] 6.3207464  3.2922424 0.70466107
## [6,] 4.8315353  5.9818678 0.60008529
```

As one can see, the ones we forced to be zero, are either zeros or just very small numbers. Forcing the spectral values works in a similar way.

Overview of resolving process and results

The `mcrals()` also computes the explained variance, which can be shown using `plotVariance()` and `plotCumVariance()` functions. It also has `predict()` method which works similar to `mcrpure`. Some examples are shown below:

```
# define constraints for contributions
cc = list(
   constraint("norm", params = list(type = "sum"))
)

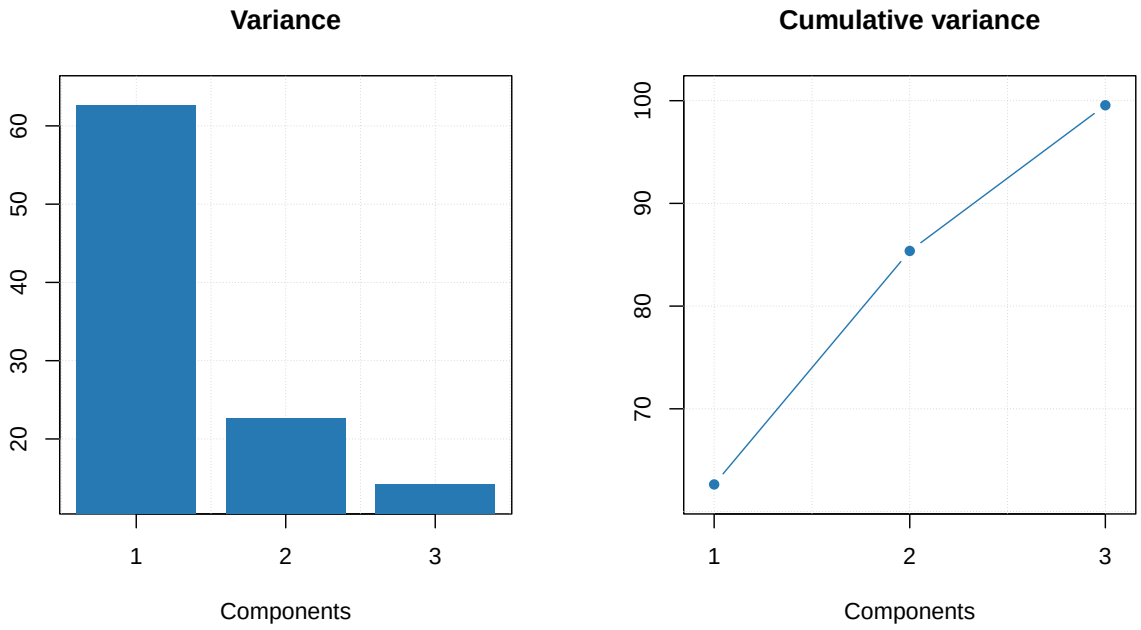
# define constraints for spectra
sc = list(
   constraint("angle", params = list(weight = 0.05)),
   constraint("norm", params = list(type = "length"))
)

# run MCR-ALS with the constraints
set.seed(6)
m = mcrals(carbs$D, ncomp = 3, spec.constraints = sc, cont.constraints = cc)

# show summary
summary(m)
```

```
##
## Summary for MCR ALS case (class mcrals)
##
## Constraints for spectra:
##  -  angle
##  -  norm
##
## Constraints for contributions:
##  -  norm
##
##        Expvar Cumexpvar
## Comp 1  62.64     62.64
## Comp 2  22.73     85.37
## Comp 3  14.19     99.56
```

```
# make variance plots
par(mfrow = c(1, 2))
plotVariance(m)
plotCumVariance(m)
```



Since MCR-ALS is iterative algorithm, it should stop when convergence criteria is met. There are two ways to stop MCR-ALS iterations in *mdatools*. First is a tolerance (`tol`) parameter, which tells a minimum difference in explained variance to make the algorithm continue. For example, if explained variance on previous step is 0.95467 and on the current step it is 0.95468, the difference is 0.00001 and the algorithm will continue if the tolerance is smaller than this value.

Another way to limit the iterations is to define the maximum number by using parameter `max.iter`. By default is is set to 100. The `mcrals()` has a verbose mode which shows improvements in each iteration as it is demonstrated in the example below (we use the same constraint as in the previous chunk of code)

```
m = mcrals(carbs$D, ncomp = 3, spec.constraints = sc, cont.constraints = cc, verbose = TRUE)
```

```
## Starting iterations (max.niter = 100):
```

```
## Iteration    1, R2 = 0.000000
## Iteration    2, R2 = 0.957402
## Iteration    3, R2 = 0.976722
## Iteration    4, R2 = 0.987305
## Iteration    5, R2 = 0.992613
## Iteration    6, R2 = 0.994525
## Iteration    7, R2 = 0.995165
## Iteration    8, R2 = 0.995385
## Iteration    9, R2 = 0.995449
## Iteration   10, R2 = 0.995453
## No more improvements.
```

SIMCA classification

This chapter is about more general `simca` method, which can do both Data Driven SIMCA and other SIMCA implementations. For DD-SIMCA it is recommended to use the new `ddsimca` method, the full description can be found here.

However, if you want to use the other SIMCA implementations you may find this text useful.

SIMCA (Soft Independent Modelling of Class Analogy) is a simple but efficient one-class classification method mainly based on PCA. The general idea is to create a PCA model using only samples/objects belonging to a class and classify new objects based on how well the model can fit them. The decision is made using the two distances we discussed in detail in the PCA chapter — orthogonal and score distances and corresponding critical limits.

Critical limits computed for both distances (or their combination) are used to cut-off the strangers (extreme objects) and accept class members with a pre-defined expected ratio of false negatives (α). If a data driven approach (either classic/moments or robust) is used to compute the critical limits, then the method is called DD-SIMCA (Data Driven SIMCA), which is implemented separately.

The classification performance can be assessed using the number of true/false positives and negatives and statistics, showing the ability of a classification model to recognize class members (*sensitivity* or true positive rate) and how good the model is for identifying strangers (*specificity* or true negative rate). In addition to that, model also calculates the percentage of misclassified objects. All statistics are calculated for calibration and validation (if any) results, but one must be aware that specificity cannot be computed without objects not belonging to the class and, therefore, calibration and cross-validation results in SIMCA do not have specificity values.

It must also be noted that any SIMCA model is also a PCA model object and any SIMCA result is also a PCA result object, therefore all plots, methods, statistics, available for PCA, can be used for SIMCA model and result objects as well.

Calibration and validation

The model calibration is similar to PCA, but there are several additional arguments, which are important for classification. First of all, it is a class name, which is a second mandatory argument. The class name is a string that can be used later e.g. for identifying class members for testing. The second important argument is the level of significance, `alpha`. This parameter is used for calculation of statistical limits and can be considered as probability for false negatives. The default value is 0.05. Finally the parameter `lim.type` allows to select the method for computing critical limits for the distances, as it is described in the PCA chapter. By default `lim.type = "ddmoments"` (data driven, based on classical moments) in PCA.

In this chapter as well as for describing other classification methods we will use a famous Iris dataset, available in R. The dataset includes 150 measurements of three Iris species: *Setosa*, *Virginica* and *Versicola*. The measurements are length and width of petals and sepals in cm. Use `?iris` for more details.

Let's get the data and split it to calibration and test sets.

```
data(iris)
head(iris)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1          5.1         3.5          1.4         0.2  setosa
## 2          4.9         3.0          1.4         0.2  setosa
## 3          4.7         3.2          1.3         0.2  setosa
## 4          4.6         3.1          1.5         0.2  setosa
## 5          5.0         3.6          1.4         0.2  setosa
## 6          5.4         3.9          1.7         0.4  setosa
```

```
# generate indices for calibration set
idx = seq(1, nrow(iris), by = 2)

# split the values
Xc = iris[idx, 1:4]
cc = iris[idx, 5]

Xt = iris[-idx, 1:4]
ct = iris[-idx, 5]
```

Now, because for calibration we need only objects belonging to a class, we will split `Xc` into three matrices — one for each species. The data is ordered by the species, so it can be done relatively easily by taking every 25 rows.

```
X.set = Xc[1:25, ]
X.ver = Xc[26:50, ]
X.vir = Xc[51:75, ]
```

Let's start with creating a model for class *Versicolor* and exploring available statistics and plots. In this case default values for method and significance level to compute the critical limits (`lim.type = "ddmoments"` and `alpha = 0.05`) are used.

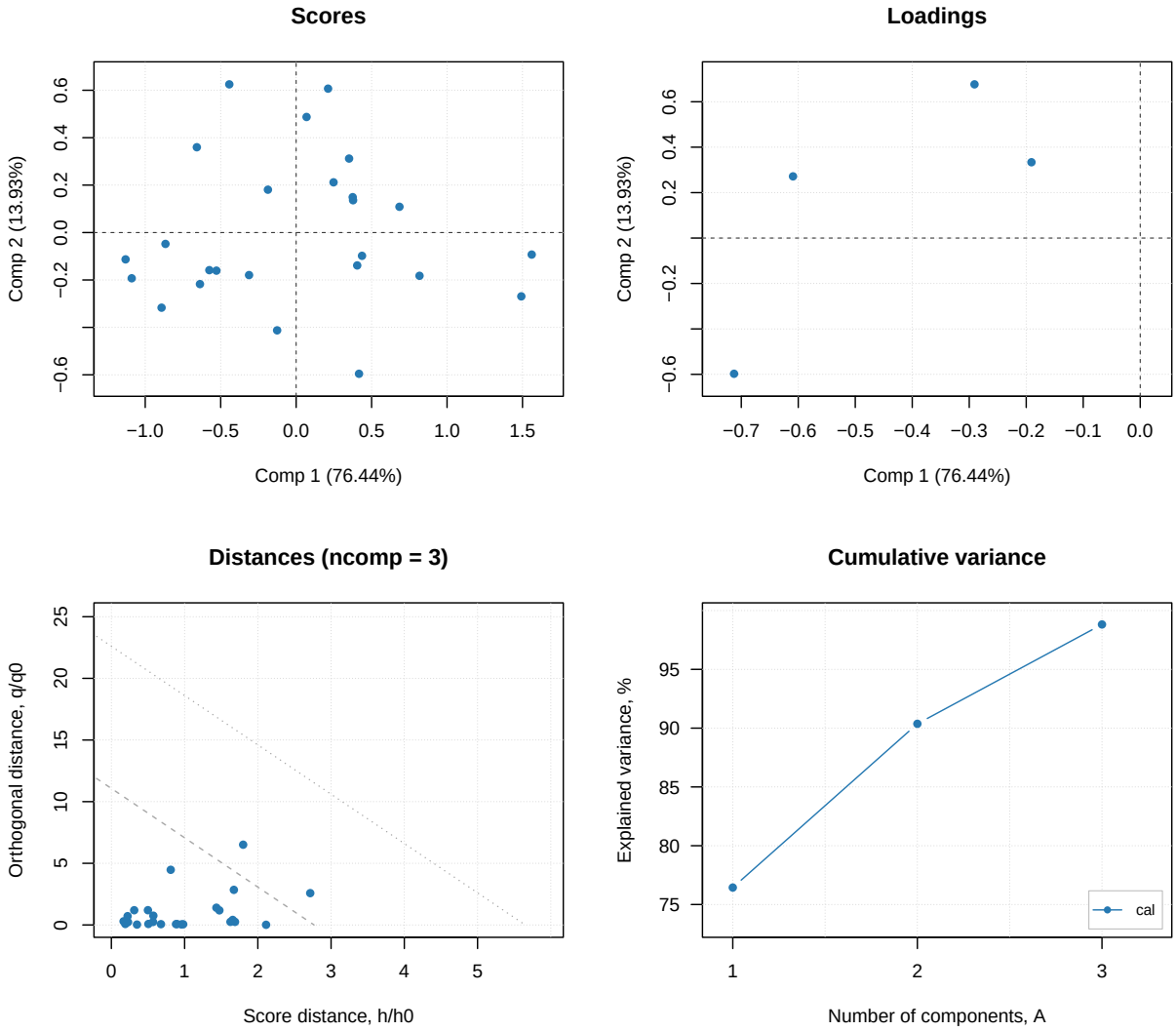
```
m = simca(X.ver, "versicolor", ncomp = 3)
summary(m)
```

```
##
## SIMCA model for class 'versicolor' summary
##
##
## Number of components: 3
## Type of limits: ddmoments
## Alpha: 0.05
## Gamma: 0.01
##
##     Expvar Cumexpvar TP FP TN FN Spec. Sens. Accuracy
## Cal   8.45     98.82 23  0  0  2    NA  0.92     0.92
```

The summary output shows (in addition to explained and cumulative explained variance) number of true positives, false positives, true negatives, false negatives as well as specificity, sensitivity and accuracy of classification. All statistics are shown for each available result object (in this case only calibration) and only for optimal number of components (in this case 3).

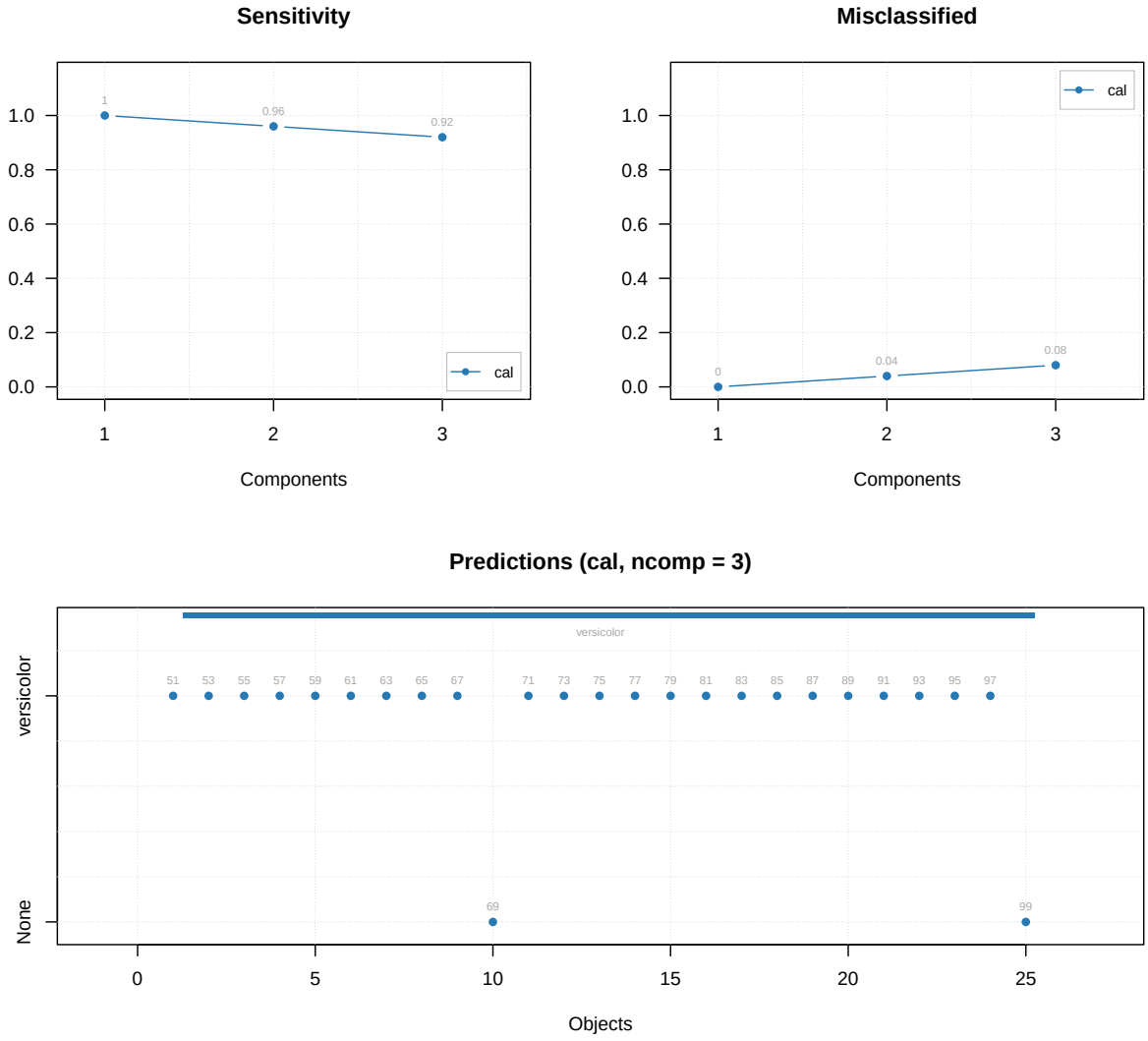
The summary plot looks very similar to what we have seen for PCA.

```
plot(m)
```



In addition to standard PCA plots, SIMCA model (as well as any other classification model, e.g. PLS-DA) can show plot for sensitivity and ratio of misclassified values depending on the number of components. Plus a prediction plot, which shows classification results for each object. See the example below.

```
layout(matrix(c(1, 3, 2, 3), ncol = 2))
plotSensitivity(m, show.labels = TRUE)
plotMisclassified(m, show.labels = TRUE)
plotPredictions(m, show.labels = TRUE)
```



Validation

Because SIMCA is based on PCA, you can use any validation method described in PCA section. Just keep in mind that when cross-validation is used, only performance statistics will be computed (in this case classification performance). Therefore cross-validated result object will not contain scores, distances, explained variance etc. and corresponding plots will not be available.

Here I will show briefly an example based on Procrustes cross-validation. First we load the `pcv` package and create a PV-set for the target class (versicolor):

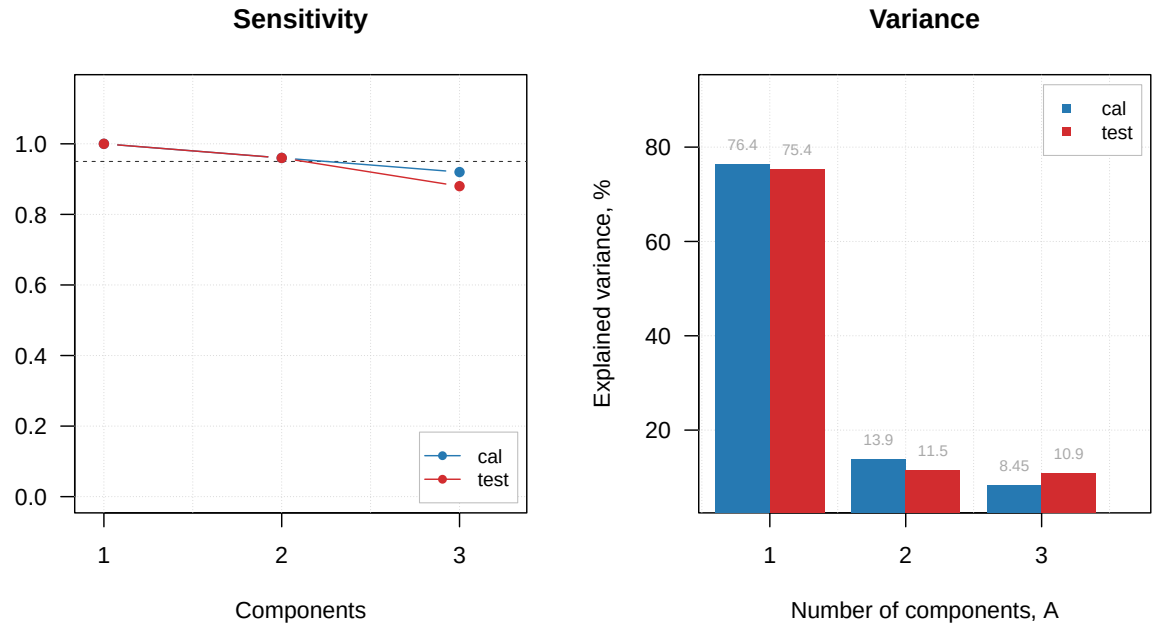
```
library(pcv)
Xpv = pcvpca(as.matrix(X.ver), 4, center = TRUE, scale = TRUE, cv = list("ven", 4))
```

Then we create a SIMCA model with PV-set as test set:

```
m = simca(X.ver, "versicolor", ncomp = 3, x.test = Xpv)
```

Let's look at the sensitivity and explained variance plots:

```
par(mfrow = c(1, 2))
plotSensitivity(m, show.line = c(NA, 0.95))
plotVariance(m, type = "h", show.labels = TRUE)
```



Based on the plot we can select 2 PCs as optimal number. We set this value and show the summary:

```
m = selectCompNum(m, 2)
summary(m)
```

```
##
## SIMCA model for class 'versicolor' summary
##
##
## Number of components: 2
## Type of limits: ddmoments
## Alpha: 0.05
## Gamma: 0.01
##
##      Expvar Cumexpvar TP FP TN FN Spec. Sens. Accuracy
## Cal   13.93     90.37 24  0  0  1    NA  0.96     0.96
## Test  11.49     86.87 24  0  0  1    NA  0.96     0.96
```

Predictions and testing the model

When model is calibrated and optimized, one can test it using a test set with known classes. In this case we will use objects from all three species and be able to see how well the model performs on strangers (and calculate the specificity). In order to do that we will provide both matrix with predictors, `Xt`, and a vector with names of the classes for corresponding objects/rows (`ct`). The values with known classes in this case can be:

- a vector with text values (names)
- a factor using the names as labels (also as a vector)
- a vector with logical values (`TRUE` for class members and `FALSE` for strangers)

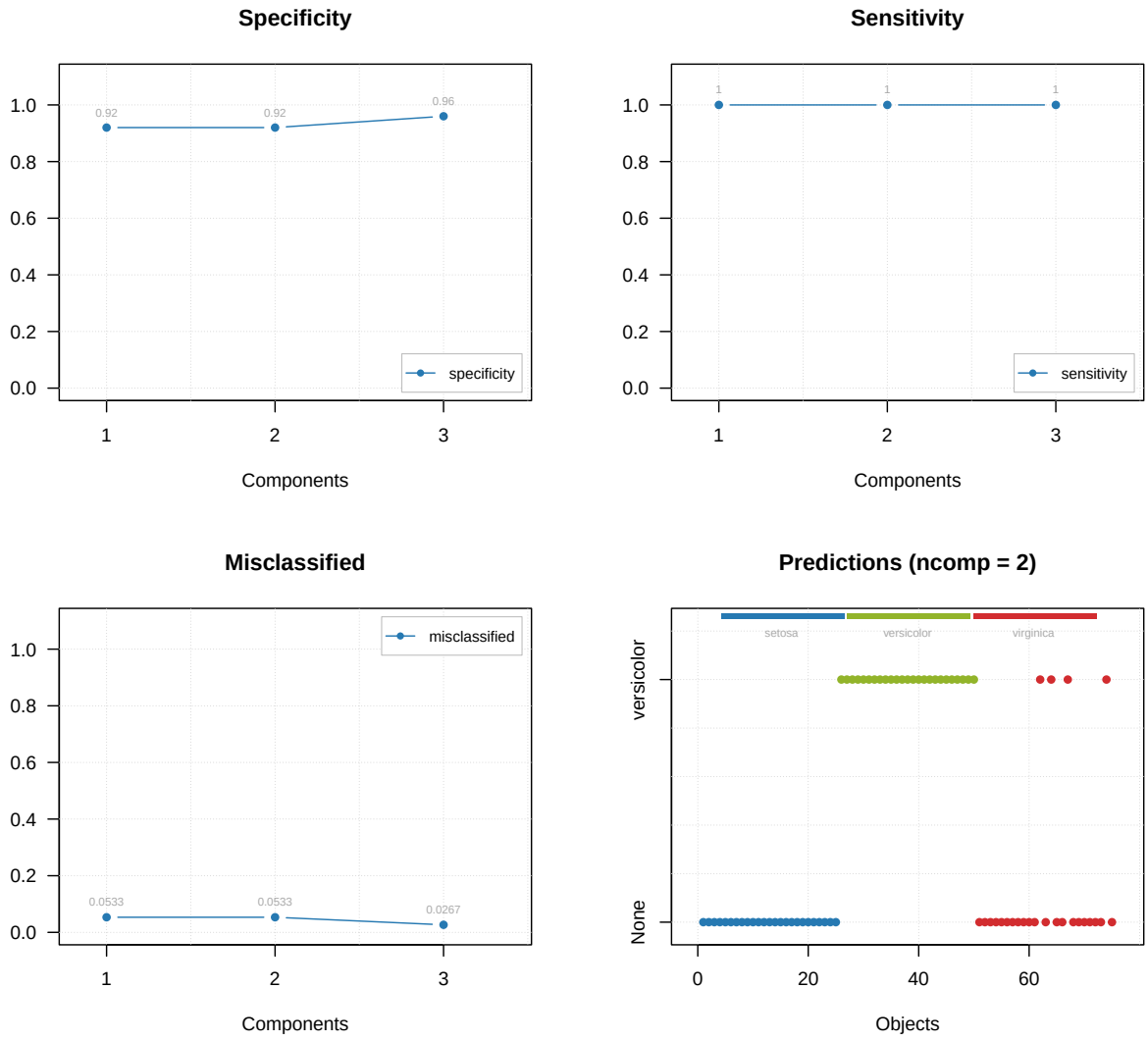
In our case we have a vector with text values, which will be automatically converted to a factor by the function `predict()`. Instead of creating a new model and providing the values as test set we will simply make predictions.

```
res = predict(m, Xt, ct)
summary(res)
```

```
##
## Summary for SIMCA one-class classification result
##
## Class name: versicolor
## Number of selected components: 2
##
##        Expvar Cumexpvar TP FP TN FN Spec. Sens. Accuracy
## Comp 1  64.27     64.27 25  4 46  0  0.92     1    0.947
## Comp 2   1.67     65.95 25  4 46  0  0.92     1    0.947
## Comp 3  32.45     98.40 25  2 48  0  0.96     1    0.973
```

In this case we can also see the specificity values and corresponding plot can be made, as shown below together with other plots.

```
par(mfrow = c(2, 2))
plotSpecificity(res, show.labels = TRUE)
plotSensitivity(res, show.labels = TRUE)
plotMisclassified(res, show.labels = TRUE)
plotPredictions(res)
```



As you can see, the prediction plot looks a bit different in this case. Because the test set has objects from several classes and the class belonging is known, this information is shown as color bar legend. For instance, in the example above we can see, that two *Virginica* objects were erroneously classified as members of *Versicolor*.

You can also show the predictions as a matrix with -1 and $+1$ using method `showPredictions()` or get the array with predicted class values directly as it is shown in the example below (for 10 rows in the middle of the data, different number of components and the first classification variable).

```
show(res$c.pred[45:55, 1:3, 1])
```

```
##     Comp 1 Comp 2 Comp 3
## 90       1      1      1
## 92       1      1      1
## 94       1      1      1
## 96       1      1      1
## 98       1      1      1
## 100      1      1      1
## 102     -1     -1     -1
## 104     -1     -1     -1
## 106     -1     -1     -1
```

```
## 108     -1     -1     -1
## 110     -1     -1     -1
```

You can also get and show the confusion matrix (rows correspond to real classes and columns to the predicted class) for an object with SIMCA results (as well as results obtained with any other classification method, e.g. PLS-DA).

```
show(getConfusionMatrix(res))
```

```
##            versicolor None
## versicolor         25    0
## setosa              0   25
## virginica           4   21
```

Class belonging probabilities

In addition to the array with predicted class, the object with SIMCA results also contains an array with class belongings probabilities. The probabilities are calculated depending on how close a particular object is to the critical limit border.

To compute the probability we use the theoretical distribution for score and orthogonal distances as when computing critical values (defined by the parameter `lim.type`). The distribution is used to calculate a p-value — chance to get object with given distance value or larger. The p-value is then compared with significance level, α, and the probability, π is calculated as follows:

$$\pi = 0.5(p/\alpha)$$

So if p-value is the same as significance level (which happens when object is lying exactly on the acceptance line) the probability is 0.5. If p-value is e.g. 0.04, $\pi = 0.4$, or 40%, and the object will be rejected as a stranger (here we assume that the $\alpha = 0.05$). If the p-value is e.g. 0.06, $\pi = 0.6$, or 60%, and the object will be accepted as a member of the class. If p-value is larger than $2 \times \alpha$ the probability is set to 1.

In case of rectangular acceptance area (`lim.type = "jm"` or `"chisq"`) the probability is computed separately for q and h values and the smallest of the two is taken. In case of triangular acceptance area (`lim.type = "ddmoments"` or `"ddrobust"`) the probability is calculated for a full distance, f.

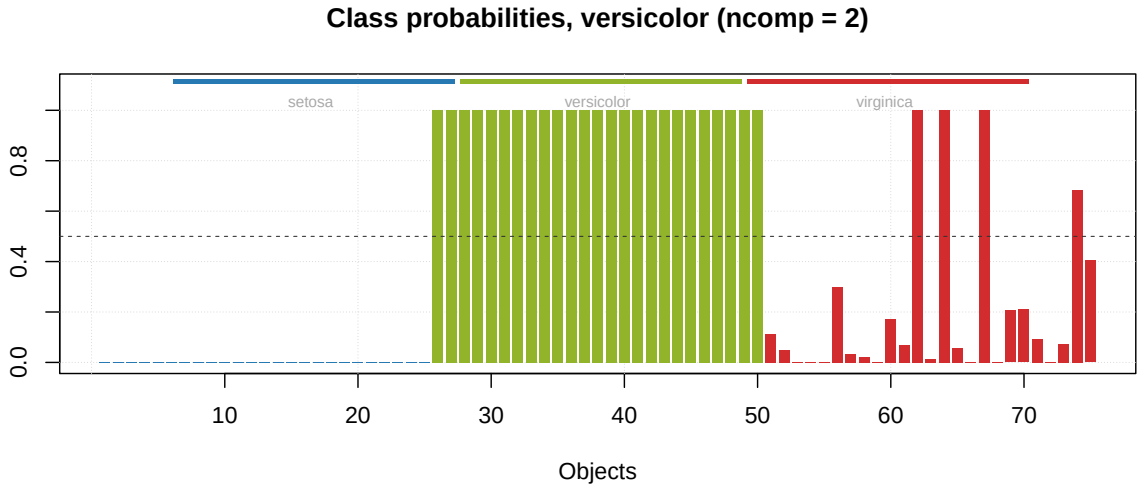
Here is how to show the probability values, that correspond to the predictions shown in the previous code chunk. I round the probability values to four decimals for better output.

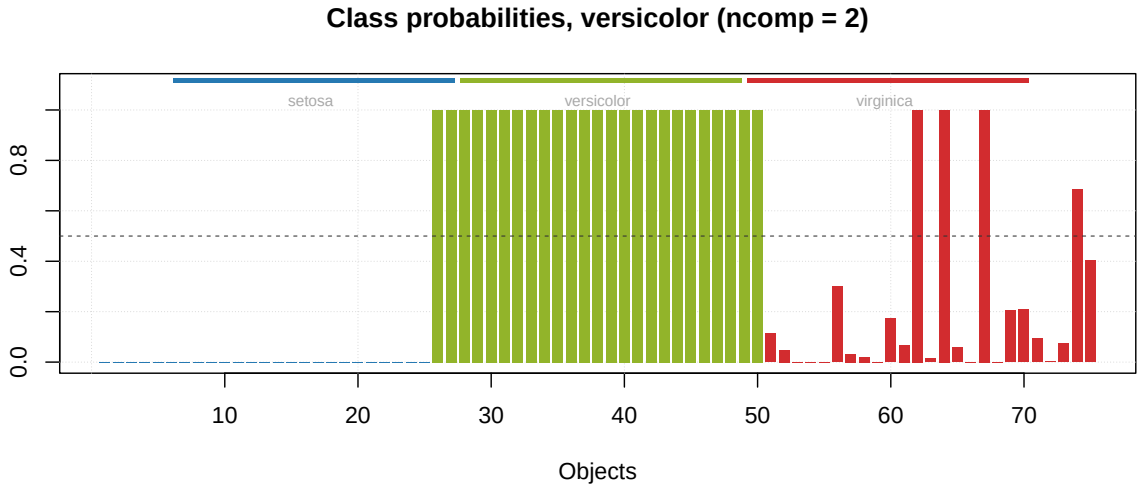
```
show(round(res$p.pred[45:55, 1:3, 1], 4))
```

```
##     Comp 1 Comp 2 Comp 3
## 90  1.0000 1.0000 1.0000
## 92  1.0000 1.0000 1.0000
## 94  1.0000 1.0000 1.0000
## 96  1.0000 1.0000 1.0000
## 98  1.0000 1.0000 1.0000
## 100 1.0000 1.0000 1.0000
## 102 0.0376 0.1121 0.0028
## 104 0.0285 0.0477 0.0216
## 106 0.0000 0.0000 0.0000
## 108 0.0006 0.0002 0.0000
## 110 0.0001 0.0000 0.0000
```

It is also possible to show the probability values as a plot with method `plotProbabilities()`:

```
par(mfrow = c(2, 1))
plotProbabilities(res, cgroup = ct)
plotProbabilities(res, ncomp = 2, cgroup = ct)
```





The plot can be shown for any SIMCA results (including e.g. calibration set or cross-validated results).

Multiclass classification

Several SIMCA models can be combined to a special object `simcam`, which is used to make a multiclass classification. Besides this, it also allows calculating distance between individual models and a *discrimination power* — importance of variables to discriminate between any two classes. Let's see how it works.

First we create three single-class SIMCA models with individual settings, such as number of optimal components and alpha.

```
m.set = simca(X.set, "setosa", 3, alpha = 0.01)
m.set = selectCompNum(m.set, 1)

m.vir = simca(X.vir, "virginica", 3)
```

```
m.vir = selectCompNum(m.vir, 2)

m.ver = simca(X.ver, "versicolor", 3)
m.ver = selectCompNum(m.ver, 1)
```

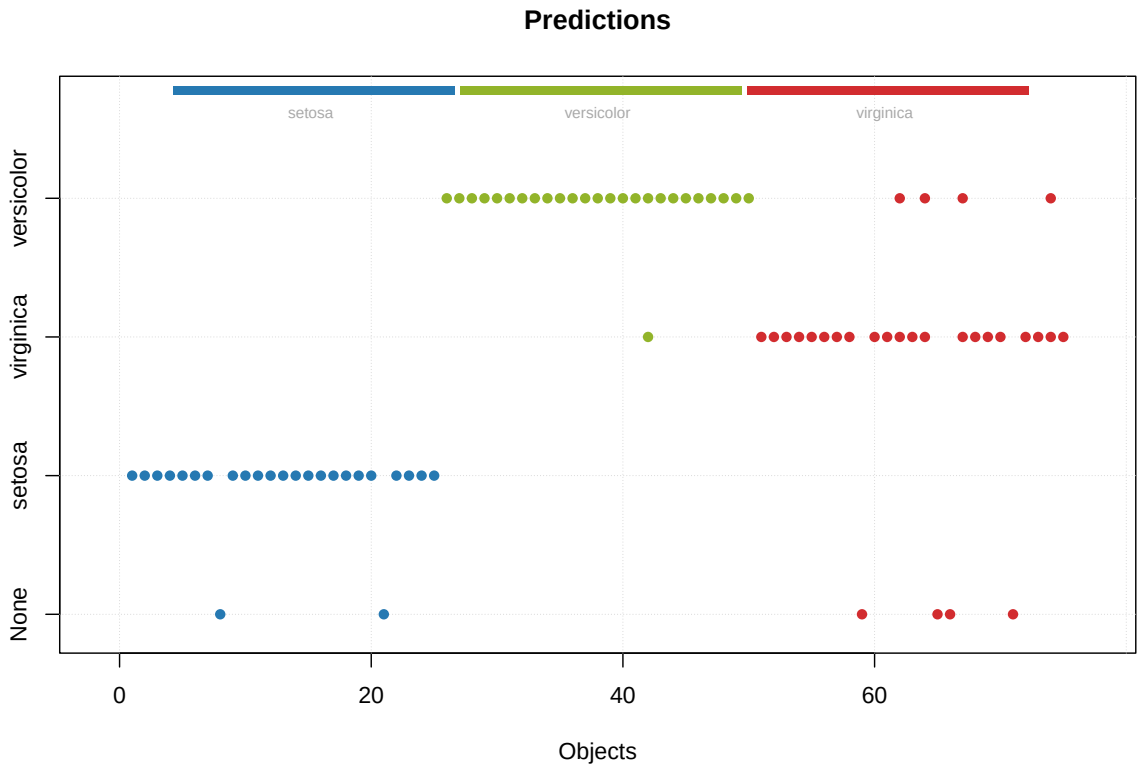
Then we combine the models into a `simcam` model object. Summary will show the performance on calibration set, which is a combination of calibration sets for each of the individual models

```
mm = simcam(list(m.set, m.vir, m.ver))
summary(mm)
```

```
##
## SIMCA multiple classes classification (class simcam)
##
## Number of classes: 3
## Info:
##
## Summary for calibration results
##            Ncomp TP FP TN FN Spec. Sens. Accuracy
## setosa         1 25  0 50  0  1.00  1.00     1.00
## virginica      2 22  3 47  3  0.94  0.88     0.92
## versicolor     1 25  3 47  0  0.94  1.00     0.96
```

Now we apply the combined model to the test set and look at the predictions.

```
res = predict(mm, Xt, ct)
plotPredictions(res)
```



In this case, the predictions are shown only for the number of components each model found optimal. The

names of classes along y-axis are the individual models. Similarly we can show the predicted values.

```
show(res$c.pred[20:30, 1, ])
```

```
##    setosa virginica versicolor
## 40      1        -1         -1
## 42     -1        -1         -1
## 44      1        -1         -1
## 46      1        -1         -1
## 48      1        -1         -1
## 50      1        -1         -1
## 52     -1        -1          1
## 54     -1        -1          1
## 56     -1        -1          1
## 58     -1        -1          1
## 60     -1        -1          1
```

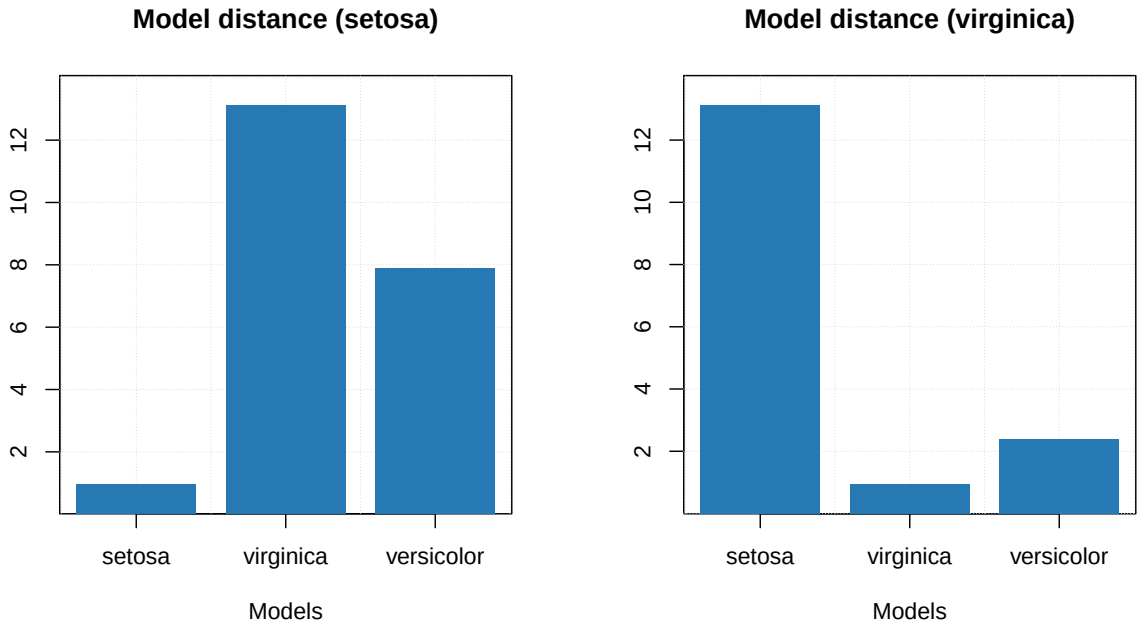
Method `getConfusionMatrix()` is also available in this case.

```
show(getConfusionMatrix(res))
```

```
##            setosa virginica versicolor None
## setosa         23         0          0    2
## virginica       0        21          4    4
## versicolor      0         1         25    0
```

There are three additional plots available for multiclass SIMCA model. First of all it is a distance between a selected model and the others.

```
par(mfrow = c(1, 2))
plotModelDistance(mm, 1)
plotModelDistance(mm, 2)
```

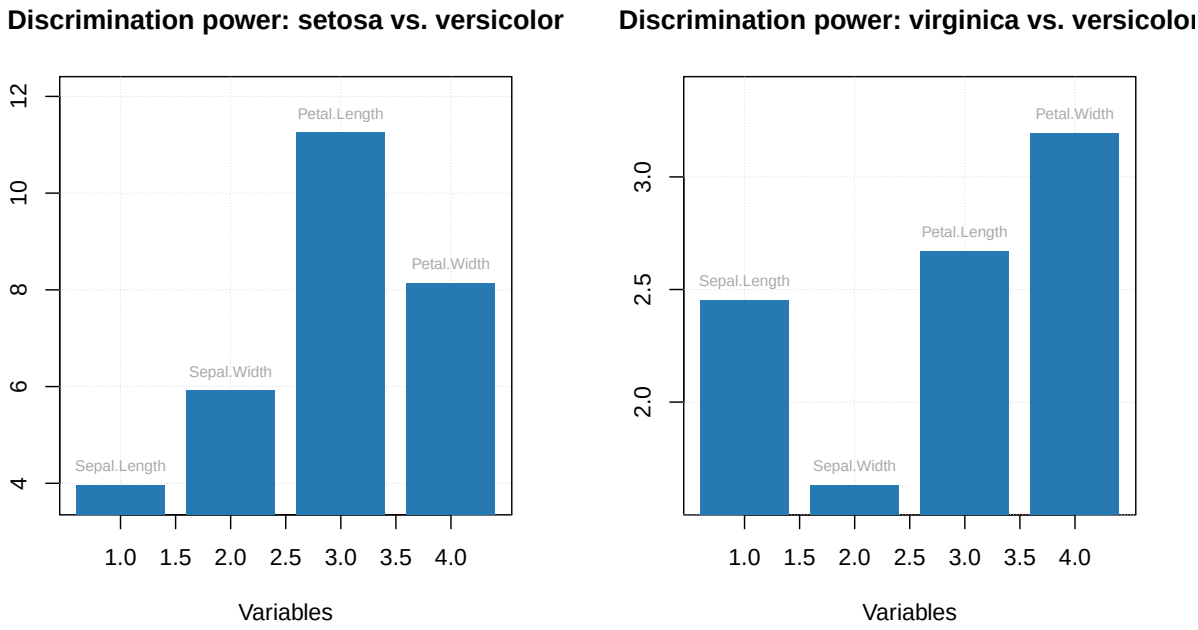


The plot shows not a real distance but rather a similarity between a selected model and the others as a ratio of residual variances. You can find more detailed description about how the model distance is calculated in

description of the method or in help for `plotModelDistance.simcam` function.

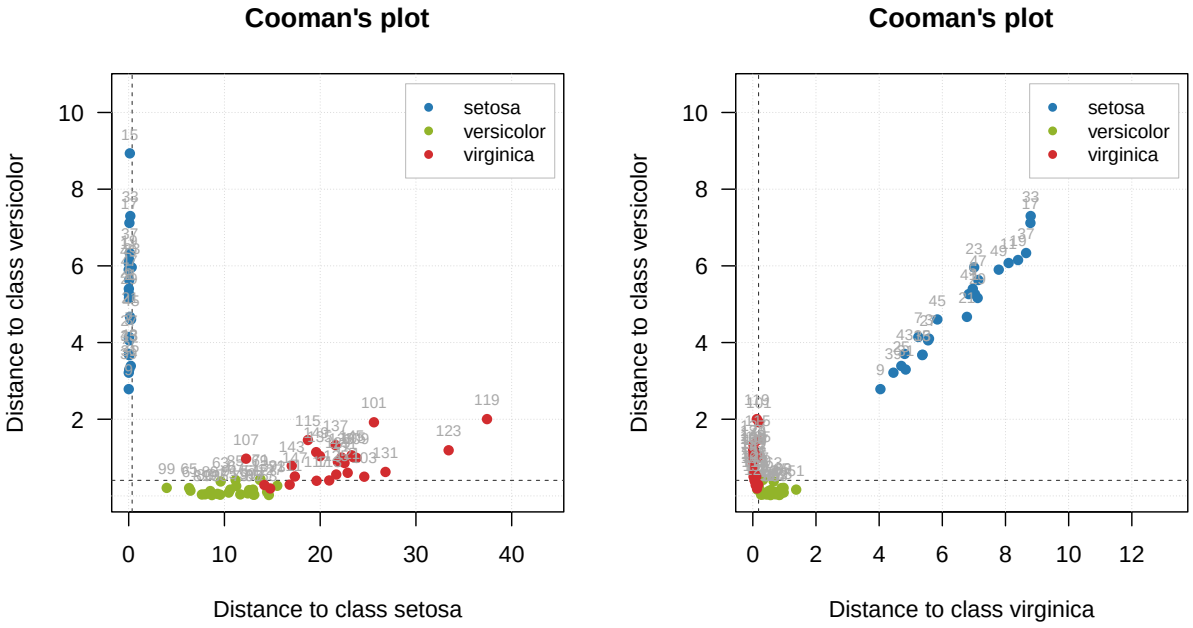
The second plot is a discrimination power, mentioned in the beginning of the section.

```
par(mfrow = c(1, 2))
plotDiscriminationPower(mm, c(1, 3), show.labels = TRUE)
plotDiscriminationPower(mm, c(2, 3), show.labels = TRUE)
```



And, finally, a Cooman's plot showing an orthogonal distance, q, from objects to two selected classes/models.

```
par(mfrow = c(1, 2))
plotCooman(mm, c(1, 3), show.labels = TRUE)
plotCooman(mm, c(2, 3), show.labels = TRUE)
```



The limits, shown as dashed lines, are computed using chi-square distribution but only for q values.